# MINA

# Product documentation

## Mediation, Integration and Automation

Version 1.0.3 - 104 activities, 11 groups, 103 functions and 12 shared connection types.

This manual and the searchable web guide share the same content. Activity tables reflect base editor contracts; selected schemas and provider modes can add fields or requirements. Read the Security section before shared or production deployment.

Use the PDF bookmarks or the linked table of contents to navigate. Content does not certify external providers, guarantee performance, or establish production security readiness.

# Contents


MINA documentation 12

Tasks, sub-tasks and event starters 13

Input mapping, schemas and expressions 14

Transformation contracts and validation 15

Transformation tests, templates and durable mediation 16

AI-assisted task and project development 17

Workspace, projects and samples 18

Start 19

Scheduler 21

Call Sub Task 23

End 25

Read File 27

Write File 30

List Files 33

Delete File 36

Rename File 38

Copy File 40

File Poller 42

FTP Get 44

FTP Put 47

FTP Delete 50

FTP Directory 53

FTP Change Directory 56

SFTP Get 59

SFTP Put 62

SFTP Delete 65

SFTP Directory 68

SFTP Change Directory 71

HTTP Listener 74

HTTP Request 77

Read Server-Sent Events 85

WebSocket Exchange 88

HTTP Write Preconditions 91

HTTP Response 94

REST API Receiver 97

REST API Invoke 101

SOAP Service 109

SOAP Request Reply 111

EMS Queue Receiver 114

EMS Topic Subscriber 118

EMS Queue Sender 122

EMS Topic Publisher 126

EMS Request Reply 130

EMS Reply 135

Get JMS Queue Message 140

JMS Receive Message 143

JMS Request Reply 146

JMS Send Message 150

Reply to JMS Message 154

Wait for JMS Request 158

Kafka Receive Message 161

Kafka Send Message 166

Kafka Get Messages 172

Pub/Sub Subscriber 178

Pub/Sub Publisher 181

Dynamic Connection 185

IDoc Acknowledgment 189

IDoc Confirmation 193

IDoc Converter 197

IDoc Listener 200

IDoc Parser 204

IDoc Reader 208

Post IDoc to SAP 211

IDoc Renderer 215

RFC BAPI Listener 218

Invoke RFC BAPI in SAP 222

Reply from RFC BAPI in SAP 226

Read Table 230

JDBC Create 234

JDBC Update 237

JDBC Query 241

JDBC Truncate 245

JDBC Delete 248

Stored Procedure 251

Dynamic SQL 254

Snowflake Insert 258

Snowflake Query 262

Snowflake Update 265

Snowflake Delete 268

Snowflake Bulk Load 271

AMQP Receive Message 276

AMQP Get Message 280

AMQP Send Message 285

AMQP Dead Letter Message 289

Parse XML 292

Render XML 294

Parse JSON 297

Render JSON 299

Parse Data 301

Render Data 305

Read Excel Workbook 308

Mapper 311

Transform 314

Log 316

Catch Exception 318

Throw Exception 320

Rethrow Exception 322

Confirm Message 323

Java Activity 325

Python Invoke 328

External Command 331

Assign 334

Checkpoint 336

Deduplicate Message 338

Message Receipt 341

Replay Messages 344

Aggregate Messages 346

Split Records 349

Sleep 351

Get Shared Variable 353

Set Shared Variable 355

If 357

While True 358

For Each 359

Iterate 361

Repeat Until True 363

Repeat on Error Until True 364

Scope 365

None 366

Critical Section 367

JDBC Transaction 368

Pick First 369

concat 370

trim 371

normalizeSpace 372

upperCase 373

lowerCase 374

capitalize 375

stringLength 376

substring 377

substringBefore 378

substringAfter 379

replace 380

matches 381

startsWith 382

endsWith 383

contains 384

compare 385

translate 386

padLeft 387

padRight 388

tokenize 389

currentDate 390

currentTime 391

currentDateTime 392

parseDate 393

formatDate 394

formatDateTime 395

addDays 396

addMonths 397

addHours 398

addMinutes 399

addSeconds 400

dateDifference 401

year 402

month 403

day 404

hour 405

minute 406

second 407

timezoneFromDateTime 408

equal 409

notEqual 410

greaterThan 411

lessThan 412

greaterOrEqual 413

lessOrEqual 414

add 415

subtract 416

multiply 417

divide 418

mod 419

number 420

integer 421

round 422

roundHalfToEven 423

floor 424

ceiling 425

abs 426

sqrt 427

power 428

modulo 429

clamp 430

min 431

max 432

sum 433

average 434

count 435

lookup 436

lookupTable 437

enrich 438

chunks 439

joinBy 440

group 441

top 442

distinctValues 443

sort 444

reverse 445

first 446

last 447

indexOf 448

subsequence 449

flatten 450

join 451

split 452

deepEqual 453

string 454

boolean 455

jsonParse 456

jsonRender 457

render-xml 458

coalesce 459

default 460

exists 461

empty 462

not 463

ifThenElse 464

uuid 465

base64Encode 466

base64Decode 467

hexEncode 468

hexDecode 469

urlEncode 470

urlDecode 471

hash 472

JDBC shared connection 473

SNOWFLAKE shared connection 477

AMQP shared connection 481

HTTP shared connection 485

FTP shared connection 499

SFTP shared connection 500

EMS shared connection 502

JMS shared connection 505

KAFKA shared connection 508

PUBSUB shared connection 511

SAP shared connection 513

SAP_TID shared connection 516

Environment property defaults 517

MINA documentation site and PDF 551

XML JSON and Data parse and render activities 554

Transformation and mediation capabilities 557

Activity debugging and testing 561

MINA Studio debugging 563

Execution analytics 565

Data and file utilities 566

Runtime logging 568

MINA Control Plane and Data Plane: one-stop guide 569

Python archive modes 578

MINA build and distribution 580

Vendor driver installation 584

MINA environment variables 586

SAP ECC integration runtime guide 590

EMS/JMS sender reuse in Run mode 593

Kafka publishing performance 594

REST APIs and shared HTTP connections 595

HTTP and REST activity verification 606

MINA security review - 2026-09-22 610

Web dependency security update 614

# MINA documentation

Getting started

## Overview

Mediation, Integration and Automation. This installed reference covers Studio, activities, tasks, groups, mapping functions, shared connections, runtime, deployment and Control Plane. It is available offline and contains no project data or credentials.

Activity fields are extracted from the Studio editor contracts. A shared field marked optional may still be needed for a selected mode or provider. Dynamic schemas are project-specific; the guide explains how Studio derives them rather than inventing their fields.

Use the tree or search to choose a topic. Activity pages have Configuration, Input, Output, Advanced and Errors tabs. Download the complete PDF for the same reference in a portable format.

Scope: this is a product reference, not a claim of provider certification or production security readiness. Read the Security section before shared or production deployment.

# Tasks, sub-tasks and event starters

Studio

## Configuration

New projects open with a callable Subtask containing Start and End. Add a Starter Task for a configured event such as an HTTP listener or scheduler. Give each task a unique name. Studio replaces spaces in object names with hyphens. Start and End define the Subtask input and output boundaries. Call Sub Task selects a target and may resolve a dynamic override at runtime. Keep an explicit fallback when packaging must discover a called task.

Create activities on the canvas and connect transitions. Conditions use input, environment properties and earlier activity output. Validate Task and Validate Project before Run or Debug. Nested calls are shown in the debug process/activity tree.

## Input

Declare the task input using an inline schema or a project schema. Call Sub Task maps into that interface. Event starters receive data from their configured system; they do not require mapping an inbound event from a preceding activity. Real providers still require valid connection and listener configuration.

## Output

Map the End activity to the task output interface. A waiting Call Sub Task publishes the called result. Spawn without waiting does not provide the synchronous business result. Project-selected schemas determine actual child fields.

## Advanced

Run and ordinary Debug keep continuous starters ready for events until stopped. Activity Testing opens a paused session without starting event listeners. Debug stepping and breakpoints affect the active job; test-bench Run once executes only the selected activity, not a complete process.

## Errors

Connect specific error paths or use Catch and Rethrow. Unhandled sub-task errors propagate to the caller. Missing task references, broken mappings, unavailable providers and malformed group graphs require correction before reliable execution.

# Input mapping, schemas and expressions

Studio

## Mapping workflow

Select an activity and open Input. Drag Data fields, Functions or Constants onto a target, or type an expression. The expanded input editor provides more space and applies edits only after Apply mappings. For typed suggestions enter a dot in a data expression, or press Ctrl+Space; Enter or Tab accepts a suggestion.

```
${input.customer.id}
${properties.routing.destination}
${Previous-Activity.result}
${vars.currentElement}
```

Use the Data tree for the actual saved reference names. A constant string is different from an expression; keep datatype conversion and quoting appropriate for the target. Structural schema nodes are populated through child fields; supported union/scalar targets accept whole values.

Repeating targets support For Each and For Each Group. Conditional mappings support If, When/Otherwise and Choose. String literals require matching single or double quotes; numeric and boolean literals are unquoted. Invalid mappings appear dark red. Enter validates the field, keeps focus on an invalid value, and advances to the next field after success. Conditions support parentheses, not, and, or, comparisons and function calls; quoted text is preserved. For example: (${input.amount} > 10 and contains(${input.status}, "ready")) or false.

Project functions use named parameters such as $value in their definitions, and custom:normalize(${input.name}) when called. Nested project functions work in Mapper, activity Input mappings, Map & Test and both Python archive formats. Calls must supply the declared argument count; missing definitions and excessive recursion produce errors. Built-in function pages list signatures and insertion templates, not guaranteed outputs for every input datatype.

JDBC named parameters such as :orderId create Input parameters.orderId after SQL analysis. Set the parameter datatype before mapping. SAP IDoc fields derive from fetched metadata; XML/JSON/flat data fields derive from the chosen schema. Call Sub Task fields derive from the selected task interface. Refresh metadata and validate after changing contracts.

# Transformation contracts and validation

Mapping

## Supported contracts

Mapper uses the current selected project schema. JSON schema trees resolve local references and nullable scalar types; recursive structures are bounded. Runtime contract checks support local references, required fields, scalar types, enum/const, additional properties, allOf/anyOf/oneOf/not, array size and uniqueness, tuple items, string length, numeric bounds and multipleOf. Patterns and patternProperties use the bounded regex dependency. Common string formats, conditional contracts, property names, dependencies and contains counts are validated. Remote references and complete JSON Schema conformance remain outside the supported subset.

JSON Parse returns the parsed value directly. Duplicate-key policy supports First wins, Last wins or Error. JSON Render includes all mapped root fields and supports removing null properties. String constants use quotes; numbers, booleans and JSON object/array constants retain their types in Studio and both Python archive formats.

SFTP uses system or configured known-hosts files and verifies host keys by default. Private-key files, passphrases, SSH agent selection and connection timeouts are shared by Studio and Python archives. Explicitly disabling host-key verification permits unknown hosts.

# Transformation tests, templates and durable mediation

Mapping

## Saved tests and assets

Map & Test includes independently saved input/expected-output cases, field differences, named lookup tables, immutable exported template versions, local version history and contract baseline comparisons. Template imports clear project-specific schema IDs. Both Python archives include run_tests.py and transform_jsonl.py; the latter transforms JSON Lines without executing connectors.

Use lookupTable(value, name, fallback), lookup(value, table, fallback), enrich(record, fields), joinBy(left, right, leftKey, rightKey) and chunks(records, size). Limits bound evaluation steps, execution duration and output size; workflow cancellation signals evaluation checkpoints. Input target search and collapse/expand controls help navigate large schemas.

## Durable mediation

General provides Deduplicate Message, Message Receipt, Replay Messages, Aggregate Messages and Split Records. Use the same explicit namespace and durable SQLite file for cooperating activities. Claims return fencing tokens; complete after successful work, fail on the error path and acknowledge the broker last. Replay lists failures or deliberately requeues one stable key. Purge removes expired completed records and retains failed/pending work.

Aggregation requires a stable message ID, a correlation/window key and expected count. Duplicate IDs do not increase the count; forward only when ready=true. A timed-out group needs explicit flush to emit partial records. Use a new correlation/window key for a new group. Local SQLite coordinates processes on one host; use persistent storage in containers. External side effects must be idempotent because these activities do not guarantee exactly-once external writes.

Payload limit is 1 MB, group limit is 64 MB and state capacity defaults to 256 MB. JSONL defaults to 16 MB per batch and 1 MB per output record. Pattern matching times out at 100 ms. These are bounded portable capabilities, not a distributed durable workflow engine or complete schema standards implementation.

# AI-assisted task and project development

Studio

## Workflow

AI Build accepts a requirement for the current Task or Complete Project. Review the design preview and generated definition before Apply. Generation alone does not deploy or run the application. Without a configured model key, the local blueprint builder is used.

Do not enter credentials or confidential payloads into AI prompts. The current prompt instruction to exclude credentials is not an enforced redaction boundary. Validate and test generated designs, and review the security report before enabling shared access.

# Workspace, projects and samples

Studio

## Workspace

Use the project tree to manage Tasks, schemas, resources and environment properties. The palette and searchable activity picker add activities to the canvas. Select and move activities, edit their display names, and configure transition conditions. Use Window for the XML/JSON viewers and comparison workspace.

Project export/import preserves editable design artifacts. Deployment export packages an executable application; these are different workflows. Installed samples open as editable project copies. Samples marked connection setup need your own provider configuration and credentials.

Themes include Plain Classic and Plain Studio with light/dark variants. Layout panes can be shown or hidden from menus. The status/footer area exposes background work and execution state.



# Start

Activities / Starters & Tasks

## Overview

Defines the task input boundary.

For Sub Tasks, the Start schema is the callable input contract. Starter Tasks normally use one external event activity instead.

Activity type: start; operation: start.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The Task input schema replaces the generic payload boundary when configured.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Starter payload<br>payload | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Output

The Task input schema replaces the generic payload boundary when configured.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Initial task payload<br>payload | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Scheduler

Activities / Starters & Tasks

## Overview

Runs Scheduler on the task execution path.

Activity type: timer; operation: schedule.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Schedule mode<br>scheduleMode | select<br>Configuration setting | Scheduler configuration. Select date/time, interval or a five-field cron schedule as applicable; runOnceOnLocalStart triggers an immediate local test execution.<br>Choices: dateTime, cron |
| Scheduled date and time<br>scheduledDateTime | text<br>Configuration setting | Scheduler configuration. Select date/time, interval or a five-field cron schedule as applicable; runOnceOnLocalStart triggers an immediate local test execution. |
| Cron expression<br>cronExpression | text<br>Configuration setting | Scheduler configuration. Select date/time, interval or a five-field cron schedule as applicable; runOnceOnLocalStart triggers an immediate local test execution. |
| Time zone<br>timezone | select<br>Configuration setting | Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems.<br>Choices: local, UTC |
| Repeat after first run<br>repeatEnabled | boolean<br>Configuration setting | Scheduler configuration. Select date/time, interval or a five-field cron schedule as applicable; runOnceOnLocalStart triggers an immediate local test execution. |
| Interval<br>interval | number<br>Configuration setting | Polling/scheduling interval for repeated execution. Size the interval to expected arrival rate and processing duration; select the unit where the activity exposes one. |
| Unit<br>unit | select<br>Configuration setting | Sleep duration and its selected unit, or actual reported sleep in milliseconds. Sleeping pauses this branch, not unrelated parallel branches.<br>Choices: seconds, minutes, hours, days |
| Run once immediately when started locally<br>runOnceOnLocalStart | boolean<br>Configuration setting | Scheduler configuration. Select date/time, interval or a five-field cron schedule as applicable; runOnceOnLocalStart triggers an immediate local test execution. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Scheduled time<br>scheduledTime | dateTime<br>Optional | Scheduler output: planned fire time, observed fire time or event sequence. Compare scheduledTime with actualTime when measuring trigger delay. |
| Actual fire time<br>actualTime | dateTime<br>Optional | Scheduler output: planned fire time, observed fire time or event sequence. Compare scheduledTime with actualTime when measuring trigger delay. |
| Sequence<br>sequence | integer<br>Optional | Scheduler output: planned fire time, observed fire time or event sequence. Compare scheduledTime with actualTime when measuring trigger delay. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

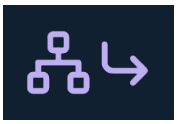
This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| SCHEDULER | The schedule expression or interval is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Call Sub Task

Activities / Starters & Tasks

## Overview

Invokes a reusable Sub Task and waits for its result.

Mapped values become the Sub Task Start input. Dynamic Sub Task ID/name is resolved per invocation; the selected Sub Task is its design-time schema and runtime fallback. The Sub Task End output returns to this activity.

Activity type: call_task; operation: call.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Sub Task<br>taskId | task<br>Required | Select the Sub Task to invoke. Its Start interface defines the caller input and its End interface defines the returned value. |
| Dynamic Sub Task ID/name expression<br>dynamicTaskId | text<br>Configuration setting | Resolved for every invocation. Use a literal Sub Task ID/name or an expression such as ${properties.routing.targetTask}; the selected Sub Task remains the fallback. |
| Spawn without waiting<br>spawn | boolean<br>Configuration setting | Run the selected Sub Task without waiting for its result. Leave disabled when subsequent activities depend on that result. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected Sub Task Start and End schemas replace generic payload/result fields.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Sub Task input<br>payload | object<br>Required | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |
| Dynamic Sub Task ID/name expression<br>dynamicTaskId | string<br>Optional | Resolved for every invocation. Use a literal Sub Task ID/name or an expression such as ${properties.routing.targetTask}; the selected Sub Task remains the fallback. |
| Spawn without waiting<br>spawn | boolean<br>Optional | Run the selected Sub Task without waiting for its result. Leave disabled when subsequent activities depend on that result. |

## Output

The selected Sub Task Start and End schemas replace generic payload/result fields.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Invoked task ID<br>taskId | string<br>Optional | Select the Sub Task to invoke. Its Start interface defines the caller input and its End interface defines the returned value. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Sub Task output<br>output | object<br>Optional | Post-operation action or returned output. Use the activity-specific operation/contract to determine whether the value is content, metadata or a completion result. |
| Spawn status<br>spawned | boolean<br>Optional | Indicates that the Sub Task was started asynchronously; it does not report completion of the child work. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| TASK_NOT_FOUND | The selected or dynamic Sub Task does not exist. |
| SUBTASK_ERROR | The invoked Sub Task returned an error. |
| VALIDATION | Required input or mapped data is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx




# End

Activities / Starters & Tasks

## Overview

Defines the task output boundary.

Mapped values become the final task result. For a Sub Task, this result is returned to its Call Sub Task activity.

Activity type: end; operation: end.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The Task output schema determines the result mapping.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Task result<br>result | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Output

The Task output schema determines the result mapping.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Task output<br>result | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Read File

Activities / File

## Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: read.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Filename / directory<br>path | text<br>Configuration setting | Absolute paths and project-property expressions are supported. |
| Include timestamp metadata<br>includeTimestamp | boolean<br>Configuration setting | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| Read as<br>readAs | select<br>Configuration setting | Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract.<br>Choices: Text, Binary |
| Exclude file content<br>excludeFileContent | boolean<br>Configuration setting | Omit file content from polling output when only file metadata is needed. Downstream Read File can fetch content explicitly. |
| Text encoding<br>encoding | text<br>Configuration setting | Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Filename<br>path | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Text encoding<br>encoding | string<br>Optional | Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters. |
| Include timestamp metadata<br>includeTimestamp | boolean<br>Optional | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| Read as<br>readAs | string<br>Optional | Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Exclude file content<br>excludeFileContent | boolean<br>Optional | Omit file content from polling output when only file metadata is needed. Downstream Read File can fetch content explicitly. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Text file content<br>textContent | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Binary file content<br>binaryContent | base64<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| Full filename<br>fileInfo.fullName | string<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |
| Filename<br>fileInfo.fileName | string<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |
| Location<br>fileInfo.location | string<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |
| File type<br>fileInfo.type | string<br>Optional | Type reported for this value or selected for this operation. Use the corresponding schema datatype and operation-specific choices. |
| Read protected<br>fileInfo.readProtected | boolean<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |
| Write protected<br>fileInfo.writeProtected | boolean<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |
| Size<br>fileInfo.size | integer<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |
| Last modified<br>fileInfo.lastModified | dateTime<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| FILE_NOT_FOUND | The configured source path does not exist. |
| FILE_IO | The runtime cannot read, write, move, or delete the file. |
| PERMISSION | The runtime lacks filesystem permission. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Write File

Activities / File

## Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: write.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Filename / directory path | text<br>Configuration setting | Absolute paths and project-property expressions are supported. |
| Include timestamp metadata<br>includeTimestamp | boolean<br>Configuration setting | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| Write as<br>writeAs | select<br>Configuration setting | Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract.<br>Choices: Text, Binary |
| Text encoding<br>encoding | text<br>Configuration setting | Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters. |
| Append to existing file<br>append | boolean<br>Configuration setting | Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved. |
| Overwrite existing file<br>overwrite | boolean<br>Configuration setting | Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved. |
| Create non-existing directories<br>createDirectories | boolean<br>Configuration setting | Create missing parent directories for the output path. The runtime account still needs directory creation/write permission. |
| Add line separator<br>addLineSeparator | boolean<br>Configuration setting | Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record. |
| Compression<br>compression | select<br>Configuration setting | Compression selection for file or staging content. Match decompression settings at the receiving operation; compression changes the wire bytes.<br>Choices: None, GZip |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Filename<br>path | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Text content<br>textContent | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Binary content<br>binaryContent | base64<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| Text encoding<br>encoding | string<br>Optional | Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters. |
| Add line separator<br>addLineSeparator | boolean<br>Optional | Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record. |
| Include timestamp metadata<br>includeTimestamp | boolean<br>Optional | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| Write as<br>writeAs | string<br>Optional | Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract. |
| Append to existing file<br>append | boolean<br>Optional | Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved. |
| Overwrite existing file<br>overwrite | boolean<br>Optional | Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved. |
| Create non-existing directories<br>createDirectories | boolean<br>Optional | Create missing parent directories for the output path. The runtime account still needs directory creation/write permission. |
| Compression<br>compression | string<br>Optional | Compression selection for file or staging content. Match decompression settings at the receiving operation; compression changes the wire bytes. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Affected path<br>path | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Operation result<br>success | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| FILE_NOT_FOUND | The configured source path does not exist. |
| FILE_IO | The runtime cannot read, write, move, or delete the file. |
| PERMISSION | The runtime lacks filesystem permission. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# List Files

Activities / File

## Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: list.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Filename / directory<br>path | text<br>Configuration setting | Absolute paths and project-property expressions are supported. |
| Include timestamp metadata<br>includeTimestamp | boolean<br>Configuration setting | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| File name pattern / wildcard<br>pattern | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Recursive<br>recursive | boolean<br>Configuration setting | Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees. |
| List<br>listType | select<br>Configuration setting | File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned.<br>Choices: Files and Directories, Only Files, Only Directories |
| Sort by<br>sortBy | select<br>Configuration setting | File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned.<br>Choices: Name, Size, Last Modified |
| Sort order<br>sortOrder | select<br>Configuration setting | File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned.<br>Choices: Ascending, Descending |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Directory / wildcard<br>path | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Recursive<br>recursive | boolean<br>Optional | Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees. |
| Include timestamp metadata<br>includeTimestamp | boolean<br>Optional | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| File name pattern / wildcard<br>pattern | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| List<br>listType | string<br>Optional | File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned. |
| Sort by<br>sortBy | string<br>Optional | File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned. |
| Sort order<br>sortOrder | string<br>Optional | File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Matched file information<br>files | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| File count<br>count | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Detected change<br>eventType | string<br>Optional | Type of file event selected/reported by the poller, such as a created or changed file. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| FILE_NOT_FOUND | The configured source path does not exist. |
| FILE_IO | The runtime cannot read, write, move, or delete the file. |
| PERMISSION | The runtime lacks filesystem permission. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Delete File

Activities / File

## Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: delete.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Filename / directory<br>path | text<br>Configuration setting | Absolute paths and project-property expressions are supported. |
| Include timestamp metadata<br>includeTimestamp | boolean<br>Configuration setting | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| Remove non-empty directory recursively<br>recursive | boolean<br>Configuration setting | Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees. |
| Ignore missing file<br>ignoreMissing | boolean<br>Configuration setting | Allow a missing file/path where supported instead of faulting. Use only when absence is an expected business outcome. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Filename / directory<br>path | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Recursive<br>recursive | boolean<br>Optional | Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees. |
| Include timestamp metadata<br>includeTimestamp | boolean<br>Optional | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| Ignore missing file<br>ignoreMissing | boolean<br>Optional | Allow a missing file/path where supported instead of faulting. Use only when absence is an expected business outcome. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Affected path<br>path | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Operation result<br>success | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| FILE_NOT_FOUND | The configured source path does not exist. |
| FILE_IO | The runtime cannot read, write, move, or delete the file. |
| PERMISSION | The runtime lacks filesystem permission. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Rename File

Activities / File

## Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: rename.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Filename / directory<br>path | text<br>Configuration setting | Absolute paths and project-property expressions are supported. |
| Include timestamp metadata<br>includeTimestamp | boolean<br>Configuration setting | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| To filename<br>destination | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Overwrite existing file<br>overwrite | boolean<br>Configuration setting | Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved. |
| Create non-existing directories<br>createDirectories | boolean<br>Configuration setting | Create missing parent directories for the output path. The runtime account still needs directory creation/write permission. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| From filename<br>path | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| To filename<br>destination | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Include timestamp metadata<br>includeTimestamp | boolean<br>Optional | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| Overwrite existing file<br>overwrite | boolean<br>Optional | Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Create non-existing directories<br>createDirectories | boolean<br>Optional | Create missing parent directories for the output path. The runtime account still needs directory creation/write permission. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Affected path<br>path | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Operation result<br>success | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| FILE_NOT_FOUND | The configured source path does not exist. |
| FILE_IO | The runtime cannot read, write, move, or delete the file. |
| PERMISSION | The runtime lacks filesystem permission. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Copy File

Activities / File

## Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: copy.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Filename / directory path | text<br>Configuration setting | Absolute paths and project-property expressions are supported. |
| Include timestamp metadata includeTimestamp | boolean<br>Configuration setting | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| Destination filename / directory destination | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Overwrite existing file overwrite | boolean<br>Configuration setting | Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved. |
| Create non-existing directories createDirectories | boolean<br>Configuration setting | Create missing parent directories for the output path. The runtime account still needs directory creation/write permission. |
| Preserve file attributes preserveAttributes | boolean<br>Configuration setting | Request preservation of supported file attributes during the operation. Availability depends on filesystem and runtime permissions. |
| Copy directory recursively recursive | boolean<br>Configuration setting | Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Source filename / directory path | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Destination filename / directory destination | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Recursive<br>recursive | boolean<br>Optional | Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees. |
| Include timestamp metadata<br>includeTimestamp | boolean<br>Optional | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| Overwrite existing file<br>overwrite | boolean<br>Optional | Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved. |
| Create non-existing directories<br>createDirectories | boolean<br>Optional | Create missing parent directories for the output path. The runtime account still needs directory creation/write permission. |
| Preserve file attributes<br>preserveAttributes | boolean<br>Optional | Request preservation of supported file attributes during the operation. Availability depends on filesystem and runtime permissions. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Affected path<br>path | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Operation result<br>success | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| FILE_NOT_FOUND | The configured source path does not exist. |
| FILE_IO | The runtime cannot read, write, move, or delete the file. |
| PERMISSION | The runtime lacks filesystem permission. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# File Poller

Activities / File

## Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: poll.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Filename / directory<br>path | text<br>Configuration setting | Absolute paths and project-property expressions are supported. |
| Include timestamp metadata<br>includeTimestamp | boolean<br>Configuration setting | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content. |
| File name pattern / wildcard<br>pattern | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Change type<br>eventType | select<br>Configuration setting | Type of file event selected/reported by the poller, such as a created or changed file.<br>Choices: Created, Modified, Deleted, Any |
| Poll interval (seconds)<br>pollInterval | number<br>Configuration setting | Polling/scheduling interval for repeated execution. Size the interval to expected arrival rate and processing duration; select the unit where the activity exposes one. |
| Include existing files<br>includeExisting | boolean<br>Configuration setting | Include files already present when polling begins, rather than only later arrivals. Test restart behavior to avoid unintended reprocessing. |
| Recursive<br>recursive | boolean<br>Configuration setting | Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees. |
| After processing<br>postAction | select<br>Configuration setting | Post-operation action or returned output. Use the activity-specific operation/contract to determine whether the value is content, metadata or a completion result.<br>Choices: None, Delete, Move |
| Move processed file to<br>moveTo | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Matched file information<br>files | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| File count<br>count | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Detected change<br>eventType | string<br>Optional | Type of file event selected/reported by the poller, such as a created or changed file. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| FILE_NOT_FOUND | The configured source path does not exist. |
| FILE_IO | The runtime cannot read, write, move, or delete the file. |
| PERMISSION | The runtime lacks filesystem permission. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# FTP Get

Activities / FTP

## Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: get.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: ftp |
| Remote path<br>remotePath | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Configuration setting | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Configuration setting | FTP connection/transfer socket timeout in seconds. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Remote path<br>remotePath | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Optional | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Optional | FTP connection/transfer socket timeout in seconds. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Downloaded content<br>contentBase64 | base64<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| Size<br>size | integer<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| AUTHENTICATION | Remote authentication failed. |
| REMOTE_NOT_FOUND | The remote path does not exist. |
| TRANSFER | The remote transfer failed. |
| CONNECTIVITY | The configured shared connection is unavailable. |
| RETRY_EXHAUSTED | The reconnection or retry policy was exhausted. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# FTP Put

Activities / FTP

## Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: put.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: ftp |
| Remote path<br>remotePath | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Configuration setting | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Configuration setting | FTP connection/transfer socket timeout in seconds. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Content<br>content | binary\|string<br>Required | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Remote path<br>remotePath | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Binary transfer<br>binary | boolean<br>Optional | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Optional | FTP connection/transfer socket timeout in seconds. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Remote path<br>remotePath | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Operation result<br>success | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
|---|---|
| AUTHENTICATION | Remote authentication failed. |
| REMOTE_NOT_FOUND | The remote path does not exist. |
| TRANSFER | The remote transfer failed. |
| CONNECTIVITY | The configured shared connection is unavailable. |
| RETRY_EXHAUSTED | The reconnection or retry policy was exhausted. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# FTP Delete

Activities / FTP

## Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: delete.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: ftp |
| Remote path<br>remotePath | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Configuration setting | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Configuration setting | FTP connection/transfer socket timeout in seconds. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Remote path<br>remotePath | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Optional | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Optional | FTP connection/transfer socket timeout in seconds. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Remote path<br>remotePath | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Operation result<br>success | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| AUTHENTICATION | Remote authentication failed. |
| REMOTE_NOT_FOUND | The remote path does not exist. |
| TRANSFER | The remote transfer failed. |
| CONNECTIVITY | The configured shared connection is unavailable. |
| RETRY_EXHAUSTED | The reconnection or retry policy was exhausted. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# FTP Directory

Activities / FTP

## Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: dir.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: ftp |
| Remote path<br>remotePath | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Configuration setting | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Configuration setting | FTP connection/transfer socket timeout in seconds. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Remote path<br>remotePath | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Optional | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Optional | FTP connection/transfer socket timeout in seconds. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Directory entries<br>entries | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Current directory<br>directory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| AUTHENTICATION | Remote authentication failed. |
| REMOTE_NOT_FOUND | The remote path does not exist. |
| TRANSFER | The remote transfer failed. |
| CONNECTIVITY | The configured shared connection is unavailable. |
| RETRY_EXHAUSTED | The reconnection or retry policy was exhausted. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# FTP Change Directory

Activities / FTP

## Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: change_dir.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: ftp |
| Target directory<br>remotePath | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Configuration setting | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Configuration setting | FTP connection/transfer socket timeout in seconds. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Remote path<br>remotePath | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Optional | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Optional | FTP connection/transfer socket timeout in seconds. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Remote path<br>remotePath | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Operation result<br>success | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| AUTHENTICATION | Remote authentication failed. |
| REMOTE_NOT_FOUND | The remote path does not exist. |
| TRANSFER | The remote transfer failed. |
| CONNECTIVITY | The configured shared connection is unavailable. |
| RETRY_EXHAUSTED | The reconnection or retry policy was exhausted. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# SFTP Get

Activities / SFTP

## Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: get.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: sftp |
| Remote path<br>remotePath | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Configuration setting | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Configuration setting | SSH connection timeout in seconds. |
| Verify host key<br>verifyHostKey | boolean<br>Configuration setting | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Remote path<br>remotePath | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Binary transfer<br>binary | boolean<br>Optional | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Optional | SSH connection timeout in seconds. |
| Verify host key<br>verifyHostKey | boolean<br>Optional | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Downloaded content<br>contentBase64 | base64<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| Size<br>size | integer<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

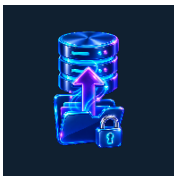
Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| AUTHENTICATION | Remote authentication failed. |
| REMOTE_NOT_FOUND | The remote path does not exist. |
| TRANSFER | The remote transfer failed. |
| CONNECTIVITY | The configured shared connection is unavailable. |
| RETRY_EXHAUSTED | The reconnection or retry policy was exhausted. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# SFTP Put

Activities / SFTP

## Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: put.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: sftp |
| Remote path<br>remotePath | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Configuration setting | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Configuration setting | SSH connection timeout in seconds. |
| Verify host key<br>verifyHostKey | boolean<br>Configuration setting | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Content<br>content | binary\|string<br>Required | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Remote path<br>remotePath | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Working directory<br>workingDirectory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Optional | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Optional | SSH connection timeout in seconds. |
| Verify host key<br>verifyHostKey | boolean<br>Optional | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Remote path<br>remotePath | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Operation result<br>success | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

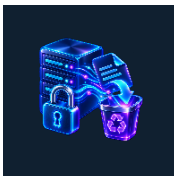
Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| AUTHENTICATION | Remote authentication failed. |
| REMOTE_NOT_FOUND | The remote path does not exist. |
| TRANSFER | The remote transfer failed. |
| CONNECTIVITY | The configured shared connection is unavailable. |
| RETRY_EXHAUSTED | The reconnection or retry policy was exhausted. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# SFTP Delete

Activities / SFTP

## Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: delete.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: sftp |
| Remote path<br>remotePath | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Configuration setting | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Configuration setting | SSH connection timeout in seconds. |
| Verify host key<br>verifyHostKey | boolean<br>Configuration setting | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Remote path<br>remotePath | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Binary transfer<br>binary | boolean<br>Optional | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Optional | SSH connection timeout in seconds. |
| Verify host key<br>verifyHostKey | boolean<br>Optional | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Remote path<br>remotePath | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Operation result<br>success | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| AUTHENTICATION | Remote authentication failed. |
| REMOTE_NOT_FOUND | The remote path does not exist. |
| TRANSFER | The remote transfer failed. |
| CONNECTIVITY | The configured shared connection is unavailable. |
| RETRY_EXHAUSTED | The reconnection or retry policy was exhausted. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# SFTP Directory

Activities / SFTP

## Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: dir.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: sftp |
| Remote path<br>remotePath | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Configuration setting | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Configuration setting | SSH connection timeout in seconds. |
| Verify host key<br>verifyHostKey | boolean<br>Configuration setting | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Remote path<br>remotePath | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Binary transfer<br>binary | boolean<br>Optional | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Optional | SSH connection timeout in seconds. |
| Verify host key<br>verifyHostKey | boolean<br>Optional | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Directory entries<br>entries | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Current directory<br>directory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

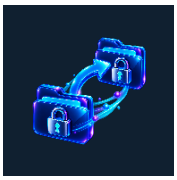
Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| AUTHENTICATION | Remote authentication failed. |
| REMOTE_NOT_FOUND | The remote path does not exist. |
| TRANSFER | The remote transfer failed. |
| CONNECTIVITY | The configured shared connection is unavailable. |
| RETRY_EXHAUSTED | The reconnection or retry policy was exhausted. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# SFTP Change Directory

Activities / SFTP

## Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: change_dir.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: sftp |
| Target directory<br>remotePath | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Binary transfer<br>binary | boolean<br>Configuration setting | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Configuration setting | SSH connection timeout in seconds. |
| Verify host key<br>verifyHostKey | boolean<br>Configuration setting | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Remote path<br>remotePath | string<br>Required | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Working directory<br>workingDirectory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Binary transfer<br>binary | boolean<br>Optional | Transfer/read bytes without text decoding when enabled. Map binary content separately from decoded text. |
| Timeout seconds<br>timeout | number<br>Optional | SSH connection timeout in seconds. |
| Verify host key<br>verifyHostKey | boolean<br>Optional | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Remote path<br>remotePath | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Operation result<br>success | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| AUTHENTICATION | Remote authentication failed. |
| REMOTE_NOT_FOUND | The remote path does not exist. |
| TRANSFER | The remote transfer failed. |
| CONNECTIVITY | The configured shared connection is unavailable. |
| RETRY_EXHAUSTED | The reconnection or retry policy was exhausted. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# HTTP Listener

Activities / HTTP & APIs

## Overview

Starts a Starter Task when an inbound HTTP request matches its configured method and path.

Choose the server connection, methods, route, content types, and schema references when creating the task. Setup adds HTTP Send Response for the reply. Run or Debug shows the live endpoint, Swagger link, and Postman download when API documentation is enabled. JSON request fields appear under body. Parsed XSD fields appear under xmlBody, with the original XML under body.

Activity type: http_listener; operation: listen.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| HTTP shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: http |
| Listener path<br>path | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Allowed HTTP methods<br>methods | methods<br>Configuration setting | Select every HTTP method accepted by this listener. |
| Request parameters<br>requestParameters | requestParameters<br>Configuration setting | Declare typed URI parameters, query parameters, and custom headers. |
| Request content types<br>requestContentTypes | mediaTypes<br>Configuration setting | Select the accepted request media types. A selected XSD applies to XML and a JSON Schema applies to JSON when several formats are selected. |
| Response content types<br>responseContentTypes | mediaTypes<br>Configuration setting | Select the offered response media types. With multiple selections, the Accept header selects the format; the first selected type is the default. |
| Validate request schema<br>validateRequest | boolean<br>Configuration setting | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| Request schema<br>requestSchema | schema<br>Configuration setting | Optional contract for the incoming request body. Select a project schema or browse for an XSD or JSON Schema file. |
| Validate response schema<br>validateResponse | boolean<br>Configuration setting | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| Response schema<br>responseSchema | schema<br>Configuration setting | Optional contract for the response body. Select or browse for a schema to make its fields available in HTTP Send Response. |
| Request execution timeout (seconds)<br>activityTimeoutSeconds | number<br>Configuration setting | Request execution timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Request body<br>body | any<br>Optional | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Base64 request bytes<br>binaryContent | base64<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| Temporary uploaded file<br>fileName | string<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |
| Uploaded byte count<br>size | integer<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |
| All received header pairs<br>headerPairs | array<br>Optional | Complete ordered received header name/value pairs, preserving repeated headers. Use this collection when every wire header occurrence is needed. |
| Client IP<br>clientIp | string<br>Optional | Client IP used for allowlists and throttling. Uses the transport peer unless the shared connection trusts the forwarding proxy. |
| HTTP trace ID<br>traceId | string<br>Optional | Trace identifier from a valid incoming traceparent, or one generated by the listener. Present when HTTP trace context is enabled. |
| Request headers<br>headers | object<br>Optional | Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings. |
| Query parameters<br>query | object<br>Optional | HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text. |
| Path parameters<br>pathParameters | object<br>Optional | Values substituted into endpoint path placeholders. Provide a value for each required placeholder in the request URI or selected operation. |
| HTTP method<br>method | string<br>Optional | HTTP verb or invoked method name, according to this activity. For HTTP select the verb required by the API, such as GET, POST or PUT. |

## Advanced

| Field / key | Type / required | Details |
|---|---|---|
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| HTTP_BAD_REQUEST | The inbound HTTP request is invalid. |
| HTTP_UNAUTHORIZED | Listener authentication rejected the request. |
| HTTP_NOT_FOUND | No matching listener resource exists. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# HTTP Request

Activities / HTTP & APIs

## Overview

Sends an outbound HTTP request and exposes the response to downstream activities.

Select a shared HTTP client connection. In Input -> RestInputRequest, map the URI, method, body, and query parameters under Config. Choose CustomHeaders -> Add header to map each name/value pair independently, or use For Each for a collection. Set Content-Type and Accept under Headers.

Activity type: http; operation: request.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| HTTP shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: http |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| RestInputRequest<br>RestInputRequest | object<br>Required | Root request structure. Map endpoint and request settings under Config, HTTP headers under Headers and multipart content under mimeEnvelopeElement. |
| Config<br>RestInputRequest.Config | object<br>Optional | Request endpoint and processing settings. Map host, port, requestURI, Method and RequestBody here; shared connection settings provide connection-level defaults. |
| host<br>RestInputRequest.Config.host | string<br>Optional | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. |
| port<br>RestInputRequest.Config.port | integer<br>Optional | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. |
| scheme<br>RestInputRequest.Config.scheme | string<br>Optional | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. |
| requestURI<br>RestInputRequest.Config.requestURI | string<br>Optional | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Method<br>RestInputRequest.Config.Method | string<br>Optional | HTTP verb or invoked method name, according to this activity. For HTTP select the verb required by the API, such as GET, POST or PUT. |
| RequestBody<br>RestInputRequest.Config.RequestBody | any<br>Optional | HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests. |
| bodyFormat<br>RestInputRequest.Config.bodyFormat | string<br>Optional | HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests. |
| QueryParameters<br>RestInputRequest.Config.QueryParameters | object<br>Optional | HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text. |
| URIParameters<br>RestInputRequest.Config.URIParameters | object<br>Optional | Values substituted into endpoint path placeholders. Provide a value for each required placeholder in the request URI or selected operation. |
| QueryString<br>RestInputRequest.Config.QueryString | string<br>Optional | HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text. |
| FilePath<br>RestInputRequest.Config.FilePath | string<br>Optional | File path on the runtime host. Ensure the runtime account can read the file and that the path exists in the deployment environment. |
| timeout<br>RestInputRequest.Config.timeout | integer<br>Optional | HTTP timeout in milliseconds. Distinguish connect/read timeouts in the shared connection from the whole-activity timeout. |
| activityTimeoutSeconds<br>RestInputRequest.Config.activityTimeoutSeconds | number<br>Optional | activityTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| followRedirects<br>RestInputRequest.Config.followRedirects | boolean<br>Optional | Follow HTTP redirect responses. Disable when the flow must inspect or reject redirects explicitly. |
| raiseForStatus<br>RestInputRequest.Config.raiseForStatus | boolean<br>Optional | HTTP success policy. raiseForStatus faults responses outside the accepted success status codes; successStatusCodes defines that accepted set. |
| streamRequest<br>RestInputRequest.Config.streamRequest | boolean<br>Optional | Stream MIME attachment content from files in bounded chunks instead of buffering the complete envelope. File body uploads stream automatically. |
| responseFile<br>RestInputRequest.Config.responseFile | string<br>Optional | Runtime filename for a streamed download. A temporary file is atomically published only after successful completion; body returns fileName and size. Response schema validation requires buffered mode. |
| overwriteResponseFile<br>RestInputRequest.Config.overwriteResponseFile | boolean<br>Optional | Permit replacing an existing download destination after a successful transfer. Disabled by default to protect existing files. |
| responseBodyMode<br>RestInputRequest.Config.responseBodyMode | string<br>Optional | Set binary to preserve the response bytes as the body. File mode uses responseFile; structured output also exposes base64 binaryContent. |
| multipartPartHeaders<br>RestInputRequest.Config.multipartPartHeaders | object<br>Optional | Object keyed by multipart field name. Map only additional headers declared by that field in the OpenAPI encoding. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| callbackUrl<br>RestInputRequest.Config.callbackUrl | string<br>Optional | Callback target URL supplied to a generated callback subtask. The destination must match a configured HTTPS origin. |
| openApiDocument<br>RestInputRequest.Config.openApiDocument | any<br>Optional | OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification. |
| openApiReferenceDocuments<br>RestInputRequest.Config.openApiReferenceDocuments | any<br>Optional | URI-to-document JSON object supplying external API reference documents without a network fetch. Each URI identifies its JSON or YAML contract document. |
| requestSchema<br>RestInputRequest.Config.requestSchema | any<br>Optional | Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities. |
| responseSchema<br>RestInputRequest.Config.responseSchema | any<br>Optional | Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities. |
| openApiBaseUrl<br>RestInputRequest.Config.openApiBaseUrl | string<br>Optional | Absolute contract document URL used to resolve relative external references. It does not change the REST service endpoint. |
| openApiAllowedRefOrigins<br>RestInputRequest.Config.openApiAllowedRefOrigins | string<br>Optional | Explicit HTTPS origins permitted for external contract retrieval. Empty disables network reference retrieval. Redirects and implicit file access are forbidden; downloaded documents have size/count bounds. |
| operationId<br>RestInputRequest.Config.operationId | string<br>Optional | OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification. |
| validateRequest<br>RestInputRequest.Config.validateRequest | boolean<br>Optional | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| validateResponse<br>RestInputRequest.Config.validateResponse | boolean<br>Optional | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| requestEntityProcessing<br>RestInputRequest.Config.requestEntityProcessing | string<br>Optional | HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests. |
| sendBodyMode<br>RestInputRequest.Config.sendBodyMode | string<br>Optional | HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests. |
| successStatusCodes<br>RestInputRequest.Config.successStatusCodes | string<br>Optional | HTTP success policy. raiseForStatus faults responses outside the accepted success status codes; successStatusCodes defines that accepted set. |
| Headers<br>RestInputRequest.Headers | object<br>Optional | HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers. |
| accept<br>RestInputRequest.Headers.accept | string<br>Optional | HTTP Accept header listing response media types the caller can process, for example application/json. |
| content-type<br>RestInputRequest.Headers.content-type | string<br>Optional | MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Accept-Charset<br>RestInputRequest.Headers.Accept-Charset | string<br>Optional | HTTP character sets acceptable in the response. Keep body encoding and declared media type consistent. |
| Accept-Encoding<br>RestInputRequest.Headers.Accept-Encoding | string<br>Optional | HTTP content encodings accepted by the caller or applied to the response, such as gzip. This is different from text character encoding. |
| Cookie<br>RestInputRequest.Headers.Cookie | string<br>Optional | HTTP cookie header. Cookie values can contain session credentials; avoid logging them. |
| pragma<br>RestInputRequest.Headers.pragma | string<br>Optional | HTTP Pragma header used for legacy cache directives, for example no-cache. |
| CustomHeaders<br>RestInputRequest.CustomHeaders | object[]<br>Optional | Repeating request headers. Use Add header to map each name/value pair independently, or map a collection with For Each. |
| name<br>RestInputRequest.CustomHeaders.name | string<br>Optional | Custom HTTP header name, for example X-Correlation-ID. Provide one name/value pair per repeating entry. |
| value<br>RestInputRequest.CustomHeaders.value | string<br>Optional | Value of the custom HTTP header named in this entry. Map the required string without embedding confidential tokens in logs. |
| mimeEnvelopeElement<br>RestInputRequest.mimeEnvelopeElement | object<br>Optional | Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation. |
| mimePart<br>RestInputRequest.mimeEnvelopeElement.mimePart | object[]<br>Optional | Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation. |
| mimeHeaders<br>RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders | object<br>Optional | Headers for this multipart part, including media type, content disposition and optional content ID. |
| content-type<br>RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-type | string<br>Optional | MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body. |
| content-transfer-encoding<br>RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-transfer-encoding | string<br>Optional | Transfer-encoding metadata for this MIME part. Ensure it matches the supplied content representation. |
| content-id<br>RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-id | string<br>Optional | Identifier of a multipart part, used when the message refers to an attachment by content ID. |
| content-disposition<br>RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-disposition | string<br>Optional | Multipart disposition, for example form-data; name="document"; filename="document.xml". Quote parameter values containing spaces. |
| binaryContent<br>RestInputRequest.mimeEnvelopeElement.mimePart.binaryContent | string<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| textContent<br>RestInputRequest.mimeEnvelopeElement.mimePart.textContent | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| fileName<br>RestInputRequest.mimeEnvelopeElement.mimePart.fileName | string<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| RestOutputResponse<br>RestOutputResponse | object<br>Required | Complete HTTP response envelope containing statusLine, headers, decoded body and optional text/binary/multipart content. |
| statusLine<br>RestOutputResponse.statusLine | object<br>Optional | HTTP protocol version, numeric response status and reason phrase returned by the server. |
| httpVersion<br>RestOutputResponse.statusLine.httpVersion | string<br>Optional | HTTP protocol version reported for the response, for example HTTP/1.1. |
| statusCode<br>RestOutputResponse.statusLine.statusCode | integer<br>Optional | Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success. |
| reasonPhrase<br>RestOutputResponse.statusLine.reasonPhrase | string<br>Optional | Human-readable HTTP status description. Use statusCode for programmatic decisions. |
| body<br>RestOutputResponse.body | any<br>Optional | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| asciiContent<br>RestOutputResponse.asciiContent | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| binaryContent<br>RestOutputResponse.binaryContent | string<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| Headers<br>RestOutputResponse.Headers | object<br>Optional | HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers. |
| Allow<br>RestOutputResponse.Headers.Allow | string[]<br>Optional | HTTP response methods allowed by the endpoint, normally returned with method-not-allowed responses. |
| Content-Type<br>RestOutputResponse.Headers.Content-Type | string<br>Optional | MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body. |
| Content-Length<br>RestOutputResponse.Headers.Content-Length | string<br>Optional | HTTP content length in bytes. Use the actual encoded body length; it is not a character count. |
| Content-Encoding<br>RestOutputResponse.Headers.Content-Encoding | string<br>Optional | HTTP content encodings accepted by the caller or applied to the response, such as gzip. This is different from text character encoding. |
| Date<br>RestOutputResponse.Headers.Date | string<br>Optional | Time reported by the operation/provider. Preserve the original representation when tracing; epoch values and formatted date/time strings need explicit conversion. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Location<br>RestOutputResponse.Headers.Location | string<br>Optional | HTTP redirect or newly created resource URL returned by the server. |
| Set-Cookie<br>RestOutputResponse.Headers.Set-Cookie | string[]<br>Optional | HTTP cookie header. Cookie values can contain session credentials; avoid logging them. |
| Pragma<br>RestOutputResponse.Headers.Pragma | string<br>Optional | HTTP Pragma header used for legacy cache directives, for example no-cache. |
| DynamicHeaders<br>RestOutputResponse.Headers.DynamicHeaders | object[]<br>Optional | Repeating custom HTTP headers. Add one name/value entry for each header that is not represented by a fixed field. |
| name<br>RestOutputResponse.Headers.DynamicHeaders.name | string<br>Optional | Custom HTTP header name, for example X-Correlation-ID. Provide one name/value pair per repeating entry. |
| value<br>RestOutputResponse.Headers.DynamicHeaders.value | string<br>Optional | Value of the custom HTTP header named in this entry. Map the required string without embedding confidential tokens in logs. |
| mimeEnvelopeElement<br>RestOutputResponse.mimeEnvelopeElement | object<br>Optional | Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation. |
| mimePart<br>RestOutputResponse.mimeEnvelopeElement.mimePart | object[]<br>Optional | Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation. |
| mimeHeaders<br>RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders | object<br>Optional | Headers for this multipart part, including media type, content disposition and optional content ID. |
| content-type<br>RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-type | string<br>Optional | MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body. |
| content-transfer-encoding<br>RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-transfer-encoding | string<br>Optional | Transfer-encoding metadata for this MIME part. Ensure it matches the supplied content representation. |
| content-id<br>RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-id | string<br>Optional | Identifier of a multipart part, used when the message refers to an attachment by content ID. |
| content-disposition<br>RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-disposition | string<br>Optional | Multipart disposition, for example form-data; name="document"; filename="document.xml". Quote parameter values containing spaces. |
| binaryContent<br>RestOutputResponse.mimeEnvelopeElement.mimePart.binaryContent | string<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| textContent<br>RestOutputResponse.mimeEnvelopeElement.mimePart.textContent | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| fileName<br>RestOutputResponse.mimeEnvelopeElement.mimePart.fileName | string<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| HTTP_CONNECTIVITY | The HTTP endpoint could not be reached. |
| HTTP_TIMEOUT | The request exceeded its timeout. |
| HTTP_CLIENT_ERROR | The endpoint returned a 4xx response. |
| HTTP_SERVER_ERROR | The endpoint returned a 5xx response. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Read Server-Sent Events

Activities / HTTP & APIs

## Overview

Reads a bounded batch from a Server-Sent Events endpoint.

Map an HTTP or HTTPS URL, maximum events and optional Last-Event-ID. The activity sends Accept: text/event-stream and closes after the requested event count, end of stream or timeout.

Activity type: http; operation: sse_read.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| HTTP shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: http |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| RestInputRequest<br>RestInputRequest | object<br>Required | Root request structure. Map endpoint and request settings under Config, HTTP headers under Headers and multipart content under mimeEnvelopeElement. |
| Config<br>RestInputRequest.Config | object<br>Optional | Request endpoint and processing settings. Map host, port, requestURI, Method and RequestBody here; shared connection settings provide connection-level defaults. |
| requestURI<br>RestInputRequest.Config.requestURI | string<br>Optional | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. |
| QueryParameters<br>RestInputRequest.Config.QueryParameters | object<br>Optional | HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text. |
| URIParameters<br>RestInputRequest.Config.URIParameters | object<br>Optional | Values substituted into endpoint path placeholders. Provide a value for each required placeholder in the request URI or selected operation. |
| timeout<br>RestInputRequest.Config.timeout | integer<br>Optional | HTTP timeout in milliseconds. Distinguish connect/read timeouts in the shared connection from the whole-activity timeout. |
| activityTimeoutSeconds<br>RestInputRequest.Config.activityTimeoutSeconds | number<br>Optional | activityTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Headers<br>RestInputRequest.Headers | object<br>Optional | HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers. |
| accept<br>RestInputRequest.Headers.accept | string<br>Optional | HTTP Accept header listing response media types the caller can process, for example application/json. |
| content-type<br>RestInputRequest.Headers.content-type | string<br>Optional | MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body. |
| Accept-Charset<br>RestInputRequest.Headers.Accept-Charset | string<br>Optional | HTTP character sets acceptable in the response. Keep body encoding and declared media type consistent. |
| Accept-Encoding<br>RestInputRequest.Headers.Accept-Encoding | string<br>Optional | HTTP content encodings accepted by the caller or applied to the response, such as gzip. This is different from text character encoding. |
| Cookie<br>RestInputRequest.Headers.Cookie | string<br>Optional | HTTP cookie header. Cookie values can contain session credentials; avoid logging them. |
| pragma<br>RestInputRequest.Headers.pragma | string<br>Optional | HTTP Pragma header used for legacy cache directives, for example no-cache. |
| CustomHeaders<br>RestInputRequest.CustomHeaders | object[]<br>Optional | Repeating request headers. Use Add header to map each name/value pair independently, or map a collection with For Each. |
| name<br>RestInputRequest.CustomHeaders.name | string<br>Optional | Custom HTTP header name, for example X-Correlation-ID. Provide one name/value pair per repeating entry. |
| value<br>RestInputRequest.CustomHeaders.value | string<br>Optional | Value of the custom HTTP header named in this entry. Map the required string without embedding confidential tokens in logs. |
| Maximum events<br>RestInputRequest.Config.maxEvents | integer<br>Optional | Maximum number of complete events or WebSocket messages to read before closing the connection. |
| Last event ID<br>RestInputRequest.Config.lastEventId | string<br>Optional | Optional Last-Event-ID sent when reconnecting to an SSE source. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Received events or messages<br>events | object[]<br>Optional | Ordered events or messages read before the configured limit. SSE returns event type, data and ID. WebSocket messages include type and data, with binary data in base64. |
| Number received<br>eventCount | integer<br>Optional | Number of events or messages returned by this activity. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| HTTP_CONNECTIVITY | The HTTP endpoint could not be reached. |
| HTTP_TIMEOUT | The request exceeded its timeout. |
| HTTP_CLIENT_ERROR | The endpoint returned a 4xx response. |
| HTTP_SERVER_ERROR | The endpoint returned a 5xx response. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# WebSocket Exchange

Activities / HTTP & APIs

## Overview

Exchanges messages with a WebSocket server.

Map a ws or wss URL, an optional outbound message and the maximum replies. Received binary messages are base64. The activity closes its connection when it finishes.

Activity type: http; operation: websocket_exchange.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| HTTP shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: http |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| RestInputRequest<br>RestInputRequest | object<br>Required | Root request structure. Map endpoint and request settings under Config, HTTP headers under Headers and multipart content under mimeEnvelopeElement. |
| Config<br>RestInputRequest.Config | object<br>Optional | Request endpoint and processing settings. Map host, port, requestURI, Method and RequestBody here; shared connection settings provide connection-level defaults. |
| requestURI<br>RestInputRequest.Config.requestURI | string<br>Optional | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. |
| RequestBody<br>RestInputRequest.Config.RequestBody | any<br>Optional | HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests. |
| QueryParameters<br>RestInputRequest.Config.QueryParameters | object<br>Optional | HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text. |
| URIParameters<br>RestInputRequest.Config.URIParameters | object<br>Optional | Values substituted into endpoint path placeholders. Provide a value for each required placeholder in the request URI or selected operation. |
| timeout<br>RestInputRequest.Config.timeout | integer<br>Optional | HTTP timeout in milliseconds. Distinguish connect/read timeouts in the shared connection from the whole-activity timeout. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| activityTimeoutSeconds<br>RestInputRequest.Config.activityTimeoutSeconds | number<br>Optional | activityTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| Headers<br>RestInputRequest.Headers | object<br>Optional | HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers. |
| accept<br>RestInputRequest.Headers.accept | string<br>Optional | HTTP Accept header listing response media types the caller can process, for example application/json. |
| content-type<br>RestInputRequest.Headers.content-type | string<br>Optional | MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body. |
| Accept-Charset<br>RestInputRequest.Headers.Accept-Charset | string<br>Optional | HTTP character sets acceptable in the response. Keep body encoding and declared media type consistent. |
| Accept-Encoding<br>RestInputRequest.Headers.Accept-Encoding | string<br>Optional | HTTP content encodings accepted by the caller or applied to the response, such as gzip. This is different from text character encoding. |
| Cookie<br>RestInputRequest.Headers.Cookie | string<br>Optional | HTTP cookie header. Cookie values can contain session credentials; avoid logging them. |
| pragma<br>RestInputRequest.Headers.pragma | string<br>Optional | HTTP Pragma header used for legacy cache directives, for example no-cache. |
| CustomHeaders<br>RestInputRequest.CustomHeaders | object[]<br>Optional | Repeating request headers. Use Add header to map each name/value pair independently, or map a collection with For Each. |
| name<br>RestInputRequest.CustomHeaders.name | string<br>Optional | Custom HTTP header name, for example X-Correlation-ID. Provide one name/value pair per repeating entry. |
| value<br>RestInputRequest.CustomHeaders.value | string<br>Optional | Value of the custom HTTP header named in this entry. Map the required string without embedding confidential tokens in logs. |
| Maximum events<br>RestInputRequest.Config.maxEvents | integer<br>Optional | Maximum number of complete events or WebSocket messages to read before closing the connection. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Received events or messages<br>events | object[]<br>Optional | Ordered events or messages read before the configured limit. SSE returns event type, data and ID. WebSocket messages include type and data, with binary data in base64. |
| Number received<br>eventCount | integer<br>Optional | Number of events or messages returned by this activity. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| HTTP_CONNECTIVITY | The HTTP endpoint could not be reached. |
| HTTP_TIMEOUT | The request exceeded its timeout. |
| HTTP_CLIENT_ERROR | The endpoint returned a 4xx response. |
| HTTP_SERVER_ERROR | The endpoint returned a 5xx response. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# HTTP Write Preconditions

Activities / HTTP & APIs

## Overview

Evaluates request validators before a write.

Map inbound request headers and the current ETag, Last-Modified and existence. Branch on allowed, send 412 or 428 on rejection, and guard the actual write atomically in the datastore.

Activity type: http; operation: write_precondition.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| RestInputRequest<br>RestInputRequest | object<br>Required | Root request structure. Map endpoint and request settings under Config, HTTP headers under Headers and multipart content under mimeEnvelopeElement. |
| Config<br>RestInputRequest.Config | object<br>Optional | Request endpoint and processing settings. Map host, port, requestURI, Method and RequestBody here; shared connection settings provide connection-level defaults. |
| Headers<br>RestInputRequest.Headers | object<br>Optional | HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers. |
| Current ETag<br>RestInputRequest.Config.currentETag | string<br>Optional | Current datastore entity tag, including the double quotes used in HTTP validators. |
| Current Last-Modified<br>RestInputRequest.Config.currentLastModified | string<br>Optional | Current datastore Last-Modified value in HTTP date format. |
| Representation exists<br>RestInputRequest.Config.representationExists | boolean<br>Optional | Whether the resource already exists when evaluating a write precondition. |
| Require a precondition<br>RestInputRequest.Config.requireWritePrecondition | boolean<br>Optional | Return 428 when the caller supplies no write precondition. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Write allowed<br>allowed | boolean<br>Optional | Whether the write precondition passed. Execute the guarded write only when true. |
| Suggested HTTP status<br>statusCode | integer<br>Optional | Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| HTTP_CONNECTIVITY | The HTTP endpoint could not be reached. |
| HTTP_TIMEOUT | The request exceeded its timeout. |
| HTTP_CLIENT_ERROR | The endpoint returned a 4xx response. |

| Fault | Description |
| --- | --- |
| HTTP_SERVER_ERROR | The endpoint returned a 5xx response. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# HTTP Response

Activities / HTTP & APIs

## Overview

Completes the active inbound HTTP exchange.

Place HTTP Send Response on the same execution path as the listener or receiver. In Input, optional HTTPHeaders contains Content-Type, complex StatusLine and repeating Cookies. Map StatusLine.httpVersion, StatusLine.statusCode and StatusLine.reasonPhrase independently. Native HTTP/1 listeners send the mapped version (1.0 or 1.1) and custom reason phrase, including an empty phrase. HTTP/1.0 responses use Content-Length and close the connection. HTTP/2 and HTTP/3 require a matching negotiated version and no nonempty reason phrase; unsupported overrides fail before sending headers. HTTP/1 fallback on an HTTP/2 or HTTP/3 listener does not support custom reason phrases. Unmapped fields use existing defaults, and existing full status-line strings still work. Mapped settings remain visible in statusLine execution output. DynamicHTTPHeaders contains repeating Headers with headerName and headerValue. Dynamic headers override standard/legacy headers; duplicate names remain separate header lines, including Set-Cookie. Legacy statusCode, contentType and headers mappings still work. If the endpoint declares a response schema, its fields are available for body mapping. An explicit Content-Type must be offered by the endpoint and acceptable to the caller.

Activity type: http_response; operation: response.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Status code<br>statusCode | number<br>Configuration setting | Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success. |
| Response content type<br>contentType | mediaType<br>Configuration setting | Choose a common media type, enter a custom value, or use Automatic to use the listener default. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| HTTPHeaders<br>HTTPHeaders | object<br>Optional | Optional standard response headers and status. Unmapped fields keep the existing response defaults. |
| Content-Type<br>HTTPHeaders.Content-Type | string<br>Optional | Response media type. Overrides the legacy contentType mapping when supplied. |
| StatusLine<br>HTTPHeaders.StatusLine | object<br>Optional | Optional response status structure. Map httpVersion, statusCode and reasonPhrase independently, or map an entire object. Existing status-line string mappings remain supported. |
| httpVersion<br>HTTPHeaders.StatusLine.httpVersion | string<br>Optional | Native HTTP/1 listeners send the selected HTTP/1.0 or HTTP/1.1 version. HTTP/1.0 closes the connection. On HTTP/2 or HTTP/3, the value must match the negotiated protocol; incompatible overrides fail. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| statusCode<br>HTTPHeaders.StatusLine.statusCode | integer<br>Optional | Numeric response status from 100 to 599, for example 201 without quotes. Overrides the legacy statusCode when mapped. Hosted final responses require 200-599. |
| reasonPhrase<br>HTTPHeaders.StatusLine.reasonPhrase | string<br>Optional | Custom wire reason phrase on native HTTP/1 listeners, including an empty string. Omit for the standard phrase. HTTP/2 and HTTP/3 have no reason phrase; use an empty value or omit it. Unsupported transports and control characters fail validation. |
| Cookies<br>HTTPHeaders.Cookies | string[]<br>Optional | One or more complete Set-Cookie values, such as session=abc; Path=/; HttpOnly. Each value is sent as its own Set-Cookie header. |
| DynamicHTTPHeaders<br>DynamicHTTPHeaders | object<br>Optional | Optional custom response headers. Repeat Headers for each headerName/headerValue pair. |
| Headers<br>DynamicHTTPHeaders.Headers | object[]<br>Optional | Add or duplicate a header entry, map a collection, or use For Each. Dynamic values override standard/legacy headers; repeated names emit separate header lines. Set-Cookie appends to Cookies. |
| headerName<br>DynamicHTTPHeaders.Headers.headerName | string<br>Optional | HTTP header name, for example X-Correlation-ID. Required when the entry has a value. |
| headerValue<br>DynamicHTTPHeaders.Headers.headerValue | string<br>Optional | HTTP header value. Empty strings are allowed; line breaks and control characters are rejected. |
| Response headers<br>headers | object<br>Optional | Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings. |
| Response body<br>body | any<br>Optional | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Base64 response bytes<br>binaryContent | base64<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| Response file to stream<br>fileName | string<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |
| Status code<br>statusCode | number<br>Optional | Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success. |
| Response content type<br>contentType | string<br>Optional | Choose a common media type, enter a custom value, or use Automatic to use the listener default. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Sent status code<br>statusCode | integer<br>Optional | Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success. |
| Mapped status line<br>statusLine | object<br>Optional | Mapped status settings retained for debugging. Supported version/reason overrides affect the native HTTP/1 wire response; transport compatibility is checked before sending. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| httpVersion<br>statusLine.httpVersion | string<br>Optional | Mapped protocol metadata, normalized to an HTTP/ version token. |
| statusCode<br>statusLine.statusCode | integer<br>Optional | Effective numeric response status after applying optional StatusLine overrides. |
| reasonPhrase<br>statusLine.reasonPhrase | string<br>Optional | Mapped reason phrase, sent on native HTTP/1 listeners. Nonempty reason phrases are unsupported on HTTP/2 and HTTP/3. |
| Response sent<br>sent | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| HTTP_RESPONSE | The listener response could not be written. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# REST API Receiver

Activities / HTTP & APIs

## Overview

Starts a task for a matching REST method and route.

In Request parameters, declare URI path placeholders, query values, and custom headers. Select a type and mark required entries. Invalid or missing required values return HTTP 400. Output provides pathParameters, query, headers with lowercase names, and body or xmlBody. Use HTTP Send Response to reply. Run or Debug exposes the live endpoint and, when enabled, Swagger and Postman.

Activity type: rest; operation: receiver.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| OpenAPI 3 / Swagger 2 JSON or YAML contract<br>openApiDocument | textarea<br>Configuration setting | OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification. |
| Contract base URL for relative references<br>openApiBaseUrl | text<br>Configuration setting | Absolute contract document URL used to resolve relative external references. It does not change the REST service endpoint. |
| Supplied external reference documents (URI/document JSON)<br>openApiReferenceDocuments | textarea<br>Configuration setting | URI-to-document JSON object supplying external API reference documents without a network fetch. Each URI identifies its JSON or YAML contract document. |
| Allowed external reference HTTPS origins<br>openApiAllowedRefOrigins | text<br>Configuration setting | Explicit HTTPS origins permitted for external contract retrieval. Empty disables network reference retrieval. Redirects and implicit file access are forbidden; downloaded documents have size/count bounds. |
| Contract operation ID<br>operationId | text<br>Configuration setting | OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification. |
| HTTP shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: http |
| Resource path<br>path | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Allowed HTTP methods<br>methods | methods<br>Configuration setting | Select every HTTP method exposed by this REST resource. |
| Request parameters<br>requestParameters | requestParameters<br>Configuration setting | Declare typed URI parameters, query parameters, and custom headers. |
| Request content types<br>requestContentTypes | mediaTypes<br>Configuration setting | Select one or more accepted media types. With an OpenAPI contract, select media types declared by the operation or leave all unchecked to use the contract. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Response content types<br>responseContentTypes | mediaTypes<br>Configuration setting | Select the offered response media types. With multiple selections, the Accept header selects the format; the first selected type is the default. |
| Request execution timeout (seconds)<br>activityTimeoutSeconds | number<br>Configuration setting | Request execution timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| Validate request schema<br>validateRequest | boolean<br>Configuration setting | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| Request schema<br>requestSchema | schema<br>Configuration setting | Optional contract for the incoming request body. Select a project schema or browse for an XSD or JSON Schema file. |
| Validate response schema<br>validateResponse | boolean<br>Configuration setting | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| Response schema<br>responseSchema | schema<br>Configuration setting | Optional contract for the response body. Select or browse for a schema to make its fields available in HTTP Send Response. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| OpenAPI 3 / Swagger 2 JSON or YAML contract<br>openApiDocument | string<br>Optional | OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification. |
| Contract base URL for relative references<br>openApiBaseUrl | string<br>Optional | Absolute contract document URL used to resolve relative external references. It does not change the REST service endpoint. |
| Supplied external reference documents (URI/document JSON)<br>openApiReferenceDocuments | string<br>Optional | URI-to-document JSON object supplying external API reference documents without a network fetch. Each URI identifies its JSON or YAML contract document. |
| Allowed external reference HTTPS origins<br>openApiAllowedRefOrigins | string<br>Optional | Explicit HTTPS origins permitted for external contract retrieval. Empty disables network reference retrieval. Redirects and implicit file access are forbidden; downloaded documents have size/count bounds. |
| Contract operation ID<br>operationId | string<br>Optional | OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification. |
| Resource path<br>path | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Allowed HTTP methods<br>methods | string<br>Optional | Select every HTTP method exposed by this REST resource. |
| Request parameters<br>requestParameters | string<br>Optional | Declare typed URI parameters, query parameters, and custom headers. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Request content types<br>requestContentTypes | string<br>Optional | Select one or more accepted media types. With an OpenAPI contract, select media types declared by the operation or leave all unchecked to use the contract. |
| Response content types<br>responseContentTypes | string<br>Optional | Select the offered response media types. With multiple selections, the Accept header selects the format; the first selected type is the default. |
| Request execution timeout (seconds)<br>activityTimeoutSeconds | number<br>Optional | Request execution timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| Validate request schema<br>validateRequest | boolean<br>Optional | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| Request schema<br>requestSchema | string<br>Optional | Optional contract for the incoming request body. Select a project schema or browse for an XSD or JSON Schema file. |
| Validate response schema<br>validateResponse | boolean<br>Optional | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| Response schema<br>responseSchema | string<br>Optional | Optional contract for the response body. Select or browse for a schema to make its fields available in HTTP Send Response. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Request entity<br>body | any<br>Optional | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Base64 request bytes<br>binaryContent | base64<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| Temporary uploaded file<br>fileName | string<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |
| Uploaded byte count<br>size | integer<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |
| All received header pairs<br>headerPairs | array<br>Optional | Complete ordered received header name/value pairs, preserving repeated headers. Use this collection when every wire header occurrence is needed. |
| Client IP<br>clientIp | string<br>Optional | Client IP used for allowlists and throttling. Uses the transport peer unless the shared connection trusts the forwarding proxy. |
| HTTP trace ID<br>traceId | string<br>Optional | Trace identifier from a valid incoming traceparent, or one generated by the listener. Present when HTTP trace context is enabled. |
| Headers<br>headers | object<br>Optional | Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Query parameters<br>query | object<br>Optional | HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text. |
| URI parameters<br>pathParameters | object<br>Optional | Values substituted into endpoint path placeholders. Provide a value for each required placeholder in the request URI or selected operation. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| REST_VALIDATION | The request does not conform to its API/schema contract. |
| REST_UNSUPPORTED_MEDIA | The request media type is unsupported. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# REST API Invoke

Activities / HTTP & APIs

## Overview

Calls a remote REST API through a shared HTTP client connection.

In Input -> RestInputRequest.Config, map the URI and query parameters, request body, and optional OpenAPI contract and operation ID. Use CustomHeaders -> Add header for repeating name/value pairs, or use For Each for a collection. RestOutputResponse exposes the status, headers, and body.

Activity type: rest; operation: invoke.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| HTTP shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: http |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| RestInputRequest<br>RestInputRequest | object<br>Required | Root request structure. Map endpoint and request settings under Config, HTTP headers under Headers and multipart content under mimeEnvelopeElement. |
| Config<br>RestInputRequest.Config | object<br>Optional | Request endpoint and processing settings. Map host, port, requestURI, Method and RequestBody here; shared connection settings provide connection-level defaults. |
| host<br>RestInputRequest.Config.host | string<br>Optional | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. |
| port<br>RestInputRequest.Config.port | integer<br>Optional | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. |
| scheme<br>RestInputRequest.Config.scheme | string<br>Optional | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https. |
| requestURI<br>RestInputRequest.Config.requestURI | string<br>Optional | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Method<br>RestInputRequest.Config.Method | string<br>Optional | HTTP verb or invoked method name, according to this activity. For HTTP select the verb required by the API, such as GET, POST or PUT. |
| RequestBody<br>RestInputRequest.Config.RequestBody | any<br>Optional | HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests. |
| bodyFormat<br>RestInputRequest.Config.bodyFormat | string<br>Optional | HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests. |
| QueryParameters<br>RestInputRequest.Config.QueryParameters | object<br>Optional | HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text. |
| URIParameters<br>RestInputRequest.Config.URIParameters | object<br>Optional | Values substituted into endpoint path placeholders. Provide a value for each required placeholder in the request URI or selected operation. |
| QueryString<br>RestInputRequest.Config.QueryString | string<br>Optional | HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text. |
| FilePath<br>RestInputRequest.Config.FilePath | string<br>Optional | File path on the runtime host. Ensure the runtime account can read the file and that the path exists in the deployment environment. |
| timeout<br>RestInputRequest.Config.timeout | integer<br>Optional | HTTP timeout in milliseconds. activityTimeoutSeconds separately bounds the overall activity. |
| activityTimeoutSeconds<br>RestInputRequest.Config.activityTimeoutSeconds | number<br>Optional | activityTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| followRedirects<br>RestInputRequest.Config.followRedirects | boolean<br>Optional | Follow HTTP redirect responses. Disable when the flow must inspect or reject redirects explicitly. |
| raiseForStatus<br>RestInputRequest.Config.raiseForStatus | boolean<br>Optional | HTTP success policy. raiseForStatus faults responses outside the accepted success status codes; successStatusCodes defines that accepted set. |
| streamRequest<br>RestInputRequest.Config.streamRequest | boolean<br>Optional | Stream MIME attachment content from files in bounded chunks instead of buffering the complete envelope. File body uploads stream automatically. |
| responseFile<br>RestInputRequest.Config.responseFile | string<br>Optional | Runtime filename for a streamed download. A temporary file is atomically published only after successful completion; body returns fileName and size. Response schema validation requires buffered mode. |
| overwriteResponseFile<br>RestInputRequest.Config.overwriteResponseFile | boolean<br>Optional | Permit replacing an existing download destination after a successful transfer. Disabled by default to protect existing files. |
| responseBodyMode<br>RestInputRequest.Config.responseBodyMode | string<br>Optional | Set binary to preserve the response bytes as the body. File mode uses responseFile; structured output also exposes base64 binaryContent. |
| multipartPartHeaders<br>RestInputRequest.Config.multipartPartHeaders | object<br>Optional | Object keyed by multipart field name. Map only additional headers declared by that field in the OpenAPI encoding. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| callbackUrl<br>RestInputRequest.Config.callbackUrl | string<br>Optional | Callback target URL supplied to a generated callback subtask. The destination must match a configured HTTPS origin. |
| openApiDocument<br>RestInputRequest.Config.openApiDocument | any<br>Optional | OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification. |
| openApiReferenceDocuments<br>RestInputRequest.Config.openApiReferenceDocuments | any<br>Optional | URI-to-document JSON object supplying external API reference documents without a network fetch. Each URI identifies its JSON or YAML contract document. |
| requestSchema<br>RestInputRequest.Config.requestSchema | any<br>Optional | Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities. |
| responseSchema<br>RestInputRequest.Config.responseSchema | any<br>Optional | Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities. |
| openApiBaseUrl<br>RestInputRequest.Config.openApiBaseUrl | string<br>Optional | Absolute contract document URL used to resolve relative external references. It does not change the REST service endpoint. |
| openApiAllowedRefOrigins<br>RestInputRequest.Config.openApiAllowedRefOrigins | string<br>Optional | Explicit HTTPS origins permitted for external contract retrieval. Empty disables network reference retrieval. Redirects and implicit file access are forbidden; downloaded documents have size/count bounds. |
| operationId<br>RestInputRequest.Config.operationId | string<br>Optional | OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification. |
| validateRequest<br>RestInputRequest.Config.validateRequest | boolean<br>Optional | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| validateResponse<br>RestInputRequest.Config.validateResponse | boolean<br>Optional | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| requestEntityProcessing<br>RestInputRequest.Config.requestEntityProcessing | string<br>Optional | HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests. |
| sendBodyMode<br>RestInputRequest.Config.sendBodyMode | string<br>Optional | HTTP request entity configuration. RequestBody supplies content; bodyFormat selects its encoding/serialization and sendBodyMode controls body transmission. Use multipart parts for multipart requests. |
| successStatusCodes<br>RestInputRequest.Config.successStatusCodes | string<br>Optional | HTTP success policy. raiseForStatus faults responses outside the accepted success status codes; successStatusCodes defines that accepted set. |
| Headers<br>RestInputRequest.Headers | object<br>Optional | HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers. |
| accept<br>RestInputRequest.Headers.accept | string<br>Optional | HTTP Accept header listing response media types the caller can process, for example application/json. |
| content-type<br>RestInputRequest.Headers.content-type | string<br>Optional | MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Accept-Charset<br>RestInputRequest.Headers.Accept-Charset | string<br>Optional | HTTP character sets acceptable in the response. Keep body encoding and declared media type consistent. |
| Accept-Encoding<br>RestInputRequest.Headers.Accept-Encoding | string<br>Optional | HTTP content encodings accepted by the caller or applied to the response, such as gzip. This is different from text character encoding. |
| Cookie<br>RestInputRequest.Headers.Cookie | string<br>Optional | HTTP cookie header. Cookie values can contain session credentials; avoid logging them. |
| pragma<br>RestInputRequest.Headers.pragma | string<br>Optional | HTTP Pragma header used for legacy cache directives, for example no-cache. |
| CustomHeaders<br>RestInputRequest.CustomHeaders | object[]<br>Optional | Repeating request headers. Use Add header to map each name/value pair independently, or map a collection with For Each. |
| name<br>RestInputRequest.CustomHeaders.name | string<br>Optional | Custom HTTP header name, for example X-Correlation-ID. Provide one name/value pair per repeating entry. |
| value<br>RestInputRequest.CustomHeaders.value | string<br>Optional | Value of the custom HTTP header named in this entry. Map the required string without embedding confidential tokens in logs. |
| mimeEnvelopeElement<br>RestInputRequest.mimeEnvelopeElement | object<br>Optional | Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation. |
| mimePart<br>RestInputRequest.mimeEnvelopeElement.mimePart | object[]<br>Optional | Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation. |
| mimeHeaders<br>RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders | object<br>Optional | Headers for this multipart part, including media type, content disposition and optional content ID. |
| content-type<br>RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-type | string<br>Optional | MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body. |
| content-transfer-encoding<br>RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-transfer-encoding | string<br>Optional | Transfer-encoding metadata for this MIME part. Ensure it matches the supplied content representation. |
| content-id<br>RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-id | string<br>Optional | Identifier of a multipart part, used when the message refers to an attachment by content ID. |
| content-disposition<br>RestInputRequest.mimeEnvelopeElement.mimePart.mimeHeaders.content-disposition | string<br>Optional | Multipart disposition, for example form-data; name="document"; filename="document.xml". Quote parameter values containing spaces. |
| binaryContent<br>RestInputRequest.mimeEnvelopeElement.mimePart.binaryContent | string<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| textContent<br>RestInputRequest.mimeEnvelopeElement.mimePart.textContent | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| fileName<br>RestInputRequest.mimeEnvelopeElement.mimePart.fileName | string<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| RestOutputResponse<br>RestOutputResponse | object<br>Required | Complete HTTP response envelope containing statusLine, headers, decoded body and optional text/binary/multipart content. |
| statusLine<br>RestOutputResponse.statusLine | object<br>Optional | HTTP protocol version, numeric response status and reason phrase returned by the server. |
| httpVersion<br>RestOutputResponse.statusLine.httpVersion | string<br>Optional | HTTP protocol version reported for the response, for example HTTP/1.1. |
| statusCode<br>RestOutputResponse.statusLine.statusCode | integer<br>Optional | Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success. |
| reasonPhrase<br>RestOutputResponse.statusLine.reasonPhrase | string<br>Optional | Human-readable HTTP status description. Use statusCode for programmatic decisions. |
| body<br>RestOutputResponse.body | any<br>Optional | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| asciiContent<br>RestOutputResponse.asciiContent | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| binaryContent<br>RestOutputResponse.binaryContent | string<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| Headers<br>RestOutputResponse.Headers | object<br>Optional | HTTP header container. On requests, use CustomHeaders for additional headers. On responses, DynamicHeaders lists the received headers. |
| Allow<br>RestOutputResponse.Headers.Allow | string[]<br>Optional | HTTP response methods allowed by the endpoint, normally returned with method-not-allowed responses. |
| Content-Type<br>RestOutputResponse.Headers.Content-Type | string<br>Optional | MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body. |
| Content-Length<br>RestOutputResponse.Headers.Content-Length | string<br>Optional | HTTP content length in bytes. Use the actual encoded body length; it is not a character count. |
| Content-Encoding<br>RestOutputResponse.Headers.Content-Encoding | string<br>Optional | HTTP content encodings accepted by the caller or applied to the response, such as gzip. This is different from text character encoding. |
| Date<br>RestOutputResponse.Headers.Date | string<br>Optional | Time reported by the operation/provider. Preserve the original representation when tracing; epoch values and formatted date/time strings need explicit conversion. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Location<br>RestOutputResponse.Headers.Location | string<br>Optional | HTTP redirect or newly created resource URL returned by the server. |
| Set-Cookie<br>RestOutputResponse.Headers.Set-Cookie | string[]<br>Optional | HTTP cookie header. Cookie values can contain session credentials; avoid logging them. |
| Pragma<br>RestOutputResponse.Headers.Pragma | string<br>Optional | HTTP Pragma header used for legacy cache directives, for example no-cache. |
| DynamicHeaders<br>RestOutputResponse.Headers.DynamicHeaders | object[]<br>Optional | Repeating custom HTTP headers. Add one name/value entry for each header that is not represented by a fixed field. |
| name<br>RestOutputResponse.Headers.DynamicHeaders.name | string<br>Optional | Custom HTTP header name, for example X-Correlation-ID. Provide one name/value pair per repeating entry. |
| value<br>RestOutputResponse.Headers.DynamicHeaders.value | string<br>Optional | Value of the custom HTTP header named in this entry. Map the required string without embedding confidential tokens in logs. |
| mimeEnvelopeElement<br>RestOutputResponse.mimeEnvelopeElement | object<br>Optional | Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation. |
| mimePart<br>RestOutputResponse.mimeEnvelopeElement.mimePart | object[]<br>Optional | Multipart content container. Supply one part per attachment/body component, with part headers and one supported content representation. |
| mimeHeaders<br>RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders | object<br>Optional | Headers for this multipart part, including media type, content disposition and optional content ID. |
| content-type<br>RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-type | string<br>Optional | MIME media type describing the content, for example application/json, application/xml or text/plain. Match it to the serialized body. |
| content-transfer-encoding<br>RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-transfer-encoding | string<br>Optional | Transfer-encoding metadata for this MIME part. Ensure it matches the supplied content representation. |
| content-id<br>RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-id | string<br>Optional | Identifier of a multipart part, used when the message refers to an attachment by content ID. |
| content-disposition<br>RestOutputResponse.mimeEnvelopeElement.mimePart.mimeHeaders.content-disposition | string<br>Optional | Multipart disposition, for example form-data; name="document"; filename="document.xml". Quote parameter values containing spaces. |
| binaryContent<br>RestOutputResponse.mimeEnvelopeElement.mimePart.binaryContent | string<br>Optional | Binary content represented by the activity contract, commonly base64 for JSON-visible fields. Use binary mode and matching encoding when forwarding bytes. |
| textContent<br>RestOutputResponse.mimeEnvelopeElement.mimePart.textContent | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| fileName<br>RestOutputResponse.mimeEnvelopeElement.mimePart.fileName | string<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| HTTP_CONNECTIVITY | The HTTP endpoint could not be reached. |
| HTTP_TIMEOUT | The request exceeded its timeout. |
| HTTP_CLIENT_ERROR | The endpoint returned a 4xx response. |
| HTTP_SERVER_ERROR | The endpoint returned a 5xx response. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# SOAP Service

Activities / HTTP & APIs

## Overview

Hosts a SOAP service or invokes a request/reply web service.

Service mode publishes the configured contract and begins the task for a matching SOAP request. Request/reply mode uses the selected HTTP connection and WSDL metadata for outbound invocation.

Activity type: soap; operation: service.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| HTTP shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: http |
| Service path<br>path | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Service name<br>serviceName | text<br>Configuration setting | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. |
| Operation name<br>operationName | text<br>Configuration setting | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. |
| WSDL resource<br>wsdl | text<br>Configuration setting | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Service path<br>path | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Service name<br>serviceName | string<br>Optional | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. |
| Operation name<br>operationName | string<br>Optional | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| WSDL resource<br>wsdl | string<br>Optional | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| SOAP envelope<br>envelope | xml<br>Optional | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. |
| SOAP/HTTP headers<br>headers | object<br>Optional | Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings. |
| Resolved operation<br>operation | string<br>Optional | Select the operation performed by this activity. The operation determines the available configuration, input contract and output contract. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| SOAP_FAULT | The request generated a SOAP fault. |
| WSDL_VALIDATION | The message does not conform to the WSDL. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# SOAP Request Reply

Activities / HTTP & APIs

## Overview

Hosts a SOAP service or invokes a request/reply web service.

Service mode publishes the configured contract and begins the task for a matching SOAP request. Request/reply mode uses the selected HTTP connection and WSDL metadata for outbound invocation.

Activity type: soap; operation: request_reply.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| HTTP shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: http |
| Service endpoint<br>url | text<br>Configuration setting | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. |
| SOAP action<br>soapAction | text<br>Configuration setting | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. |
| WSDL resource<br>wsdl | text<br>Configuration setting | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. |
| Timeout seconds<br>timeout | number<br>Configuration setting | SOAP HTTP transport timeout; configure connection/read waits on the selected HTTP shared connection. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| SOAP envelope<br>envelope | xml<br>Required | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. |
| SOAP headers<br>headers | object<br>Optional | Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings. |
| Service endpoint<br>url | string<br>Optional | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| SOAP action<br>soapAction | string<br>Optional | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. |
| WSDL resource<br>wsdl | string<br>Optional | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request. |
| Timeout seconds<br>timeout | number<br>Optional | SOAP HTTP transport timeout; configure connection/read waits on the selected HTTP shared connection. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| HTTP status<br>statusCode | integer<br>Optional | Numeric HTTP status. Apply success/error routing according to the endpoint contract rather than treating every response as success. |
| SOAP response envelope<br>body | xml<br>Optional | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Response headers<br>headers | object<br>Optional | Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

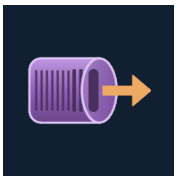
Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| SOAP_FAULT | The remote service returned a SOAP fault. |
| WSDL_VALIDATION | The request or response violates the WSDL. |
| CONNECTIVITY | The configured shared connection is unavailable. |
| RETRY_EXHAUSTED | The reconnection or retry policy was exhausted. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# EMS Queue Receiver

Activities / EMS

## Overview

### *Receive a queue event*

Use as a task starter for work arriving on a queue; no upstream message-body input is required.

### *Prerequisites*

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: queue_receiver.

## Configuration

### *Configuration procedure*

Select EMS resource, browse the destination property, and configure selector and receive wait. Client acknowledgement exposes a handle, but acknowledgement timing depends on the runtime path. Session count and flow limit can be mapped from properties; neither is a proven bridge concurrency control.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| EMS connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: ems |
| Messaging style<br>messagingStyle | select<br>Configuration setting | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.<br>Choices: Queue, Topic, Generic |
| Destination<br>queue | text<br>Configuration setting | Queue name, not the broker URL. Browse an environment property to avoid hard-coding environment-specific destinations. |
| Message type<br>messageType | select<br>Configuration setting | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.<br>Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text |
| JMS message selector<br>messageSelector | text<br>Configuration setting | Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Acknowledge mode<br>acknowledgeMode | select<br>Configuration setting | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.<br>Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional |
| Session count<br>maxSessions | number<br>Configuration setting | Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it. |
| Flow limit<br>flowLimit | number<br>Configuration setting | Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism. |
| Receive timeout (ms)<br>receiveTimeout | number<br>Configuration setting | Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
queue = ${properties.messaging.inboundQueue}
receiveTimeout = 30000
Downstream message = ${EMS-Receive.body}
Confirm Message ackId = ${EMS-Receive.ackId}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Messaging style<br>messagingStyle | string<br>Optional | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. |
| Destination<br>queue | string<br>Optional | Queue name, not the broker URL. Browse an environment property to avoid hard-coding environment-specific destinations. |
| Message type<br>messageType | string<br>Optional | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. |
| JMS message selector<br>messageSelector | string<br>Optional | Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body. |
| Acknowledge mode<br>acknowledgeMode | string<br>Optional | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. |
| Session count<br>maxSessions | number<br>Optional | Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it. |
| Flow limit<br>flowLimit | number<br>Optional | Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism. |
| Receive timeout (ms)<br>receiveTimeout | number<br>Optional | Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault. |

## Output

### *Illustrative runtime output*

```
{"body":{"id":"A100"},"headers":{"JMSMessageID":"<provider-id>"},"properties":{},"ackId":"<handle>","co
unt":1}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| JMS message body<br>body | object\|string\|binary<br>Optional | Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string. |
| JMS standard headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| JMS application properties<br>properties | object<br>Optional | Application-defined message properties returned by the broker. Standard JMS headers are returned separately. |
| Client acknowledgement handle<br>ackId | string<br>Optional | Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart. |

## Advanced

### *Delivery, transactions and performance*

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

### *Implemented behavior and limitations*

Important engine limitation: the one-shot receive bridge acknowledges client-mode delivery before closing, then the engine returns a local confirmation token. Confirm Message cannot undo that early provider acknowledgement. The direct Python listener uses a different persistent transport; do not transfer its guarantees to Studio/engine mode.

| Field / key | Type / required | Details |
|---|---|---|
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

### *Troubleshooting procedure*

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

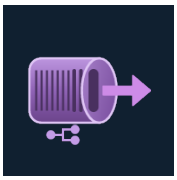
### *Acceptance test*

Acceptance: publish while Run is active, verify automatic flow start, stop during processing and check broker redelivery. Do not approve loss-sensitive processing until acknowledgement timing is verified.

| Fault | Description |
| --- | --- |
| JMSInvalidInputException | The JMS activity input is invalid. |
| JMSMessageCreateException | The configured JMS message could not be created. |
| JMSSessionCreateException | A JMS session could not be created. |
| JMSReceiveException | The JMS message could not be received. |
| ActivityTimedOutException | The JMS operation reached its configured timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# EMS Topic Subscriber

Activities / EMS

## Overview

### *Receive a topic event*

Use as a starter for topic messages. Topic delivery differs from queue competing-consumer behavior.

### *Prerequisites*

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: topic_subscriber.

## Configuration

### *Configuration procedure*

Select the EMS resource and topic property. Set selector only when necessary. The UI offers durable/shared options, but the native engine uses a regular consumer and does not wire these subscription constructors.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| EMS connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: ems |
| Messaging style<br>messagingStyle | select<br>Configuration setting | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.<br>Choices: Queue, Topic, Generic |
| Destination<br>topic | text<br>Configuration setting | Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons. |
| Message type<br>messageType | select<br>Configuration setting | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.<br>Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text |
| JMS message selector<br>messageSelector | text<br>Configuration setting | Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Acknowledge mode<br>acknowledgeMode | select<br>Configuration setting | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.<br>Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional |
| Session count<br>maxSessions | number<br>Configuration setting | Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it. |
| Flow limit<br>flowLimit | number<br>Configuration setting | Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism. |
| Receive timeout (ms)<br>receiveTimeout | number<br>Configuration setting | Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault. |
| Durable subscriber<br>durable | boolean<br>Configuration setting | Editor option not wired to a durable subscriber constructor in the native engine bridge. Offline topic retention cannot be assumed. |
| Durable subscription name<br>subscriptionName | text<br>Configuration setting | Intended durable subscription identity; current native engine consumer does not use it to create a durable subscription. |
| Shared subscription<br>sharedSubscription | boolean<br>Configuration setting | Editor option not wired to a shared-subscription constructor in the current native engine bridge. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
topic = ${properties.messaging.eventsTopic}
Downstream message = ${EMS-Topic.body}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Messaging style<br>messagingStyle | string<br>Optional | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. |
| Destination<br>topic | string<br>Optional | Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons. |
| Message type<br>messageType | string<br>Optional | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. |
| JMS message selector<br>messageSelector | string<br>Optional | Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body. |
| Acknowledge mode<br>acknowledgeMode | string<br>Optional | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Session count<br>maxSessions | number<br>Optional | Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it. |
| Flow limit<br>flowLimit | number<br>Optional | Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism. |
| Receive timeout (ms)<br>receiveTimeout | number<br>Optional | Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault. |
| Durable subscriber<br>durable | boolean<br>Optional | Editor option not wired to a durable subscriber constructor in the native engine bridge. Offline topic retention cannot be assumed. |
| Durable subscription name<br>subscriptionName | string<br>Optional | Intended durable subscription identity; current native engine consumer does not use it to create a durable subscription. |
| Shared subscription<br>sharedSubscription | boolean<br>Optional | Editor option not wired to a shared-subscription constructor in the current native engine bridge. |

## Output

### *Illustrative runtime output*

```
{"body":"event text","headers":{"JMSMessageID":"<provider-id>"},"properties":{},"count":1}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message body<br>body | object\|string\|binary<br>Optional | Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string. |
| JMS standard headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| JMS application properties<br>properties | object<br>Optional | Application-defined message properties returned by the broker. Standard JMS headers are returned separately. |
| Client acknowledgement handle<br>ackId | string<br>Optional | Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart. |

## Advanced

### *Delivery, transactions and performance*

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

### *Implemented behavior and limitations*

Do not assume messages published while the subscriber is offline are retained. durable, subscriptionName and sharedSubscription are editor-only on the current native engine path. Client acknowledgement has the same early-ack limitation as Queue Receiver.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

### *Troubleshooting procedure*

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

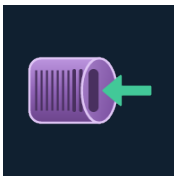
### *Acceptance test*

Acceptance: test online delivery and offline publication separately. Record observed retention and redelivery rather than assuming durable subscription behavior.

| Fault | Description |
| --- | --- |
| JMSInvalidInputException | The JMS activity input is invalid. |
| JMSMessageCreateException | The configured JMS message could not be created. |
| JMSSessionCreateException | A JMS session could not be created. |
| JMSReceiveException | The JMS message could not be received. |
| ActivityTimedOutException | The JMS operation reached its configured timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# EMS Queue Sender

Activities / EMS

## Overview

### *Send a queue message*

Use inside a task to send its mapped body to a queue.

### *Prerequisites*

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: send.

## Configuration

### *Configuration procedure*

Select the EMS connection and set queue/destination explicitly. Map message, optional correlationId and application properties. The palette uses operation send; its current base editor contract omits some sender controls, so the supplemental field guide below describes runtime keys as well as editor fields.

| Runtime setting | Meaning |
|---|---|
| destination | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| message | Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload. |
| deliveryMode | Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. |
| priority | JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent. |
| expiration | Message time-to-live in milliseconds; zero means no producer-specified expiry. |
| correlationId | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| replyTo | Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank. |
| dynamicProperties | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| EMS connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: ems |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.order}
destination = ${properties.messaging.outboundQueue}
correlationId = ${input.order.id}
deliveryMode = Persistent
priority = 4
expiration = 0
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message body<br>message | object\|string\|binary<br>Required | Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload. |
| Dynamic destination override<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Dynamic JMS properties<br>dynamicProperties | object<br>Optional | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |
| JMS headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |

## Output

### *Illustrative runtime output*

```
{"messageId":"<provider-id>","destination":"orders.out","published":true,"senderReused":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message ID<br>messageId | string<br>Optional | Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message. |
| Resolved destination<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS timestamp<br>timestamp | dateTime<br>Optional | Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation. |

## Advanced

### *Delivery, transactions and performance*

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

### *Implemented behavior and limitations*

Native send creates a text message; object bodies are JSON-encoded. Message-type, disable-message-ID and disable-timestamp editor options are not fully implemented by the bridge. A warm persistent sender can be faster than the first send; senderReused is diagnostic, not a delivery guarantee.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

### *Troubleshooting procedure*

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

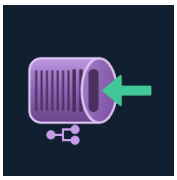
### *Acceptance test*

Acceptance: browse the test queue independently and verify text body, correlation and properties; compare first-send and warm-send timing under the same delivery policy.

| Fault | Description |
| --- | --- |
| JMSInvalidInputException | The JMS activity input is invalid. |
| JMSMessageCreateException | The configured JMS message could not be created. |
| JMSSessionCreateException | A JMS session could not be created. |
| JMSSendException | The JMS message could not be sent. |
| ActivityTimedOutException | The JMS operation reached its configured timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# EMS Topic Publisher

Activities / EMS

## Overview

### *Publish a topic message*

Use inside a task to broadcast a mapped body to the configured topic.

### *Prerequisites*

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: publish.

## Configuration

### *Configuration procedure*

Select the EMS connection, topic/destination property and message mapping. Operation publish selects topic semantics. Its current base editor contract omits some sender controls; use the supplemental runtime-key guide below.

| Runtime setting | Meaning |
|---|---|
| destination | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| message | Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload. |
| deliveryMode | Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. |
| priority | JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent. |
| expiration | Message time-to-live in milliseconds; zero means no producer-specified expiry. |
| correlationId | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| replyTo | Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank. |
| dynamicProperties | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| EMS connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: ems |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.event}
destination = ${properties.messaging.eventsTopic}
deliveryMode = Persistent
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message body<br>message | object\|string\|binary<br>Required | Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload. |
| Dynamic destination override<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Dynamic JMS properties<br>dynamicProperties | object<br>Optional | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |
| JMS headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |

## Output

### *Illustrative runtime output*

```
{"messageId":"<provider-id>","destination":"events","published":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message ID<br>messageId | string<br>Optional | Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message. |
| Resolved destination<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| JMS timestamp<br>timestamp | dateTime<br>Optional | Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation. |

## Advanced

### *Delivery, transactions and performance*

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

### *Implemented behavior and limitations*

A successful topic publish does not establish delivery to every intended subscriber. Offline retention depends on a genuinely durable provider subscription; the native engine subscriber options do not implement that guarantee. Native text-message restrictions match Queue Sender.

| Field / key | Type / required | Details |
|---|---|---|
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

### *Troubleshooting procedure*

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

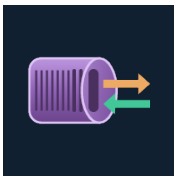
### *Acceptance test*

Acceptance: attach two independent online subscribers, publish a unique event and check each body. Test offline behavior separately.

| Fault | Description |
| --- | --- |
| JMSInvalidInputException | The JMS activity input is invalid. |
| JMSMessageCreateException | The configured JMS message could not be created. |
| JMSSessionCreateException | A JMS session could not be created. |
| JMSSendException | The JMS message could not be sent. |
| ActivityTimedOutException | The JMS operation reached its configured timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# EMS Request Reply

Activities / EMS

## Overview

### *Send a request and await a reply*

Use when the task needs a synchronous response from another JMS application.

### *Prerequisites*

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: request_reply.

## Configuration

### *Configuration procedure*

Set the request destination and mapped message. Leave replyTo blank for a temporary reply queue. A fixed reply queue requires external correlation isolation; the current bridge does not install a correlation selector on that queue.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| EMS connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: ems |
| Messaging style<br>messagingStyle | select<br>Configuration setting | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.<br>Choices: Queue, Topic, Generic |
| Destination<br>destination | text<br>Configuration setting | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Message type<br>messageType | select<br>Configuration setting | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.<br>Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text |
| Delivery mode<br>deliveryMode | select<br>Configuration setting | Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.<br>Choices: Persistent, Non-Persistent |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Priority (0-9)<br>priority | number<br>Configuration setting | JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent. |
| Expiration (ms)<br>expiration | number<br>Configuration setting | Message time-to-live in milliseconds; zero means no producer-specified expiry. |
| JMS correlation ID<br>correlationId | text<br>Configuration setting | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Reply-to destination<br>replyTo | text<br>Configuration setting | Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank. |
| Disable JMS message ID<br>disableMessageId | boolean<br>Configuration setting | Editor option not forwarded to the current native sender; do not assume identifiers are disabled. |
| Disable JMS timestamp<br>disableTimestamp | boolean<br>Configuration setting | Editor option not forwarded to the current native sender; do not assume timestamps are disabled. |
| JMS application / dynamic properties (JSON)<br>dynamicProperties | textarea<br>Configuration setting | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |
| Request timeout (ms)<br>requestTimeout | number<br>Configuration setting | Editor request timeout; current engine bridge reads receiveTimeout for its actual wait, not this field. Verify the effective wait in your deployed archive. |
| Use temporary reply destination<br>temporaryReplyDestination | boolean<br>Configuration setting | Editor flag; the current native request bridge chooses a temporary queue based on replyTo being blank, not this flag. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.request}
destination = ${properties.messaging.requestQueue}
replyTo = <leave blank for a temporary queue>
correlationId = ${input.request.id}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message body<br>message | object\|string\|binary<br>Required | Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload. |
| Dynamic destination override<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Dynamic JMS properties<br>dynamicProperties | object<br>Optional | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |
| JMS headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Messaging style<br>messagingStyle | string<br>Optional | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. |
| Message type<br>messageType | string<br>Optional | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. |
| Delivery mode<br>deliveryMode | string<br>Optional | Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. |
| Priority (0-9)<br>priority | number<br>Optional | JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent. |
| Expiration (ms)<br>expiration | number<br>Optional | Message time-to-live in milliseconds; zero means no producer-specified expiry. |
| JMS correlation ID<br>correlationId | string<br>Optional | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Reply-to destination<br>replyTo | string<br>Optional | Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank. |
| Disable JMS message ID<br>disableMessageId | boolean<br>Optional | Editor option not forwarded to the current native sender; do not assume identifiers are disabled. |
| Disable JMS timestamp<br>disableTimestamp | boolean<br>Optional | Editor option not forwarded to the current native sender; do not assume timestamps are disabled. |
| Request timeout (ms)<br>requestTimeout | number<br>Optional | Editor request timeout; current engine bridge reads receiveTimeout for its actual wait, not this field. Verify the effective wait in your deployed archive. |
| Use temporary reply destination<br>temporaryReplyDestination | boolean<br>Optional | Editor flag; the current native request bridge chooses a temporary queue based on replyTo being blank, not this flag. |

## Output

### *Illustrative runtime output*

```
{"received":true,"body":{"accepted":true},"headers":{},"properties":{},"count":1}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Reply message body<br>body | object\|string\|binary<br>Optional | Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string. |
| Reply JMS headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| Reply properties<br>properties | object<br>Optional | Application-defined message properties returned by the broker. Standard JMS headers are returned separately. |

## Advanced

### *Delivery, transactions and performance*

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

### *Implemented behavior and limitations*

The engine wrapper uses receiveTimeout (default 30000 ms), not the displayed requestTimeout, for the actual reply wait. The temporaryReplyDestination flag is not consumed; blank replyTo controls temporary queue creation. Concurrent callers sharing a fixed reply queue can receive the wrong reply. A timeout can return received=false/count=0 rather than throw.

| Field / key | Type / required | Details |
|---|---|---|
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

### *Troubleshooting procedure*

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

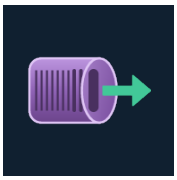
### *Acceptance test*

Acceptance: test a successful reply, no reply, and concurrent requests. Use temporary reply queues until shared-destination correlation is qualified.

| Fault | Description |
| --- | --- |
| JMSInvalidInputException | The JMS activity input is invalid. |
| JMSMessageCreateException | The configured JMS message could not be created. |
| JMSSessionCreateException | A JMS session could not be created. |
| JMSSendException | The JMS message could not be sent. |
| ActivityTimedOutException | The JMS operation reached its configured timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# EMS Reply

Activities / EMS

## Overview

### *Send a response to a JMS request*

Use after receiving a request whose reply destination and correlation information are available.

### *Prerequisites*

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: reply.

## Configuration

### *Configuration procedure*

Map the response body and explicit reply destination. For the engine path this is an outgoing send; it does not automatically derive destination from a listener context. Map the request correlation ID, or request message ID if that is the agreed protocol.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| EMS connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: ems |
| Messaging style<br>messagingStyle | select<br>Configuration setting | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.<br>Choices: Queue, Topic, Generic |
| Reply destination<br>destination | text<br>Configuration setting | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Message type<br>messageType | select<br>Configuration setting | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.<br>Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text |
| Request correlation ID<br>correlationId | text<br>Configuration setting | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Delivery mode<br>deliveryMode | select<br>Configuration setting | Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.<br>Choices: Persistent, Non-Persistent |
| Priority (0-9)<br>priority | number<br>Configuration setting | JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent. |
| Expiration (ms)<br>expiration | number<br>Configuration setting | Message time-to-live in milliseconds; zero means no producer-specified expiry. |
| JMS correlation ID<br>correlationId | text<br>Configuration setting | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Reply-to destination<br>replyTo | text<br>Configuration setting | Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank. |
| Disable JMS message ID<br>disableMessageId | boolean<br>Configuration setting | Editor option not forwarded to the current native sender; do not assume identifiers are disabled. |
| Disable JMS timestamp<br>disableTimestamp | boolean<br>Configuration setting | Editor option not forwarded to the current native sender; do not assume timestamps are disabled. |
| JMS application / dynamic properties (JSON)<br>dynamicProperties | textarea<br>Configuration setting | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.response}
destination = ${properties.messaging.replyQueue}
correlationId = ${EMS-Receive.headers.JMSCorrelationID}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message body<br>message | object\|string\|binary<br>Required | Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload. |
| Dynamic destination override<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Dynamic JMS properties<br>dynamicProperties | object<br>Optional | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |
| JMS headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| Messaging style<br>messagingStyle | string<br>Optional | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Message type<br>messageType | string<br>Optional | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. |
| Request correlation ID<br>correlationId | string<br>Optional | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Delivery mode<br>deliveryMode | string<br>Optional | Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. |
| Priority (0-9)<br>priority | number<br>Optional | JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent. |
| Expiration (ms)<br>expiration | number<br>Optional | Message time-to-live in milliseconds; zero means no producer-specified expiry. |
| JMS correlation ID<br>correlationId | string<br>Optional | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Reply-to destination<br>replyTo | string<br>Optional | Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank. |
| Disable JMS message ID<br>disableMessageId | boolean<br>Optional | Editor option not forwarded to the current native sender; do not assume identifiers are disabled. |
| Disable JMS timestamp<br>disableTimestamp | boolean<br>Optional | Editor option not forwarded to the current native sender; do not assume timestamps are disabled. |

## Output

### *Illustrative runtime output*

```
{"messageId":"<provider-id>","destination":"replies","published":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message ID<br>messageId | string<br>Optional | Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message. |
| Resolved destination<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| JMS timestamp<br>timestamp | dateTime<br>Optional | Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation. |

## Advanced

### *Delivery, transactions and performance*

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and

string application properties are the supported send path. Do not treat all editor choices as implemented native features.Retry after an ambiguous timeout can duplicate a message.

### *Implemented behavior and limitations*

The request JMSReplyTo value must be usable as a destination name before mapping it. Do not assume the generic headers object sets native JMS correlation/reply headers; use the dedicated correlationId and destination fields. A reply does not prove the requester received it.

| Field / key | Type / required | Details |
|---|---|---|
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

### *Troubleshooting procedure*

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

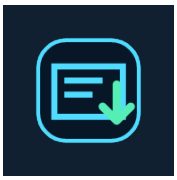
### *Acceptance test*

Acceptance: send a unique correlation ID through a requester/responder pair and verify it on the response.

| Fault | Description |
| --- | --- |
| JMSInvalidInputException | The JMS activity input is invalid. |
| JMSMessageCreateException | The configured JMS message could not be created. |
| JMSSessionCreateException | A JMS session could not be created. |
| JMSSendException | The JMS message could not be sent. |
| ActivityTimedOutException | The JMS operation reached its configured timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# Get JMS Queue Message

Activities / JMS

## Overview

Runs Get JMS Queue Message on the task execution path.

Activity type: jms; operation: get_queue_message.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: jms |
| Queue name<br>queue | text<br>Configuration setting | Queue name, not the broker URL. Browse an environment property to avoid hard-coding environment-specific destinations. |
| Message type<br>messageType | select<br>Configuration setting | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.<br>Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text |
| JMS message selector<br>messageSelector | text<br>Configuration setting | Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body. |
| Acknowledge mode<br>acknowledgeMode | select<br>Configuration setting | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.<br>Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional |
| Receive timeout (ms)<br>receiveTimeout | number<br>Configuration setting | Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Queue name<br>queue | string<br>Optional | Queue name, not the broker URL. Browse an environment property to avoid hard-coding environment-specific destinations. |
| Message type<br>messageType | string<br>Optional | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message selector<br>messageSelector | string<br>Optional | Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body. |
| Acknowledge mode<br>acknowledgeMode | string<br>Optional | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. |
| Receive timeout (ms)<br>receiveTimeout | number<br>Optional | Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message body<br>body | object\|string\|binary<br>Optional | Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string. |
| JMS standard headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| JMS application properties<br>properties | object<br>Optional | Application-defined message properties returned by the broker. Standard JMS headers are returned separately. |
| Client acknowledgement handle<br>ackId | string<br>Optional | Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| JMSInvalidInputException | The JMS activity input is invalid. |
| JMSMessageCreateException | The configured JMS message could not be created. |
| JMSSessionCreateException | A JMS session could not be created. |
| JMSReceiveException | The JMS message could not be received. |
| ActivityTimedOutException | The JMS operation reached its configured timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

# JMS Receive Message

Activities / JMS

## Overview

Runs JMS Receive Message on the task execution path.

Activity type: jms; operation: receive_message.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: jms |
| Messaging style<br>messagingStyle | select<br>Configuration setting | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.<br>Choices: Queue, Topic, Generic |
| Destination<br>destination | text<br>Configuration setting | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Message type<br>messageType | select<br>Configuration setting | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.<br>Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text |
| JMS message selector<br>messageSelector | text<br>Configuration setting | Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body. |
| Acknowledge mode<br>acknowledgeMode | select<br>Configuration setting | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.<br>Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional |
| Maximum sessions<br>maxSessions | number<br>Configuration setting | Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it. |
| Flow limit<br>flowLimit | number<br>Configuration setting | Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism. |
| Receive timeout (ms)<br>receiveTimeout | number<br>Configuration setting | Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault. |
| Durable subscription<br>durable | boolean<br>Configuration setting | Editor option not wired to a durable subscriber constructor in the native engine bridge. Offline topic retention cannot be assumed. |
| Durable subscription name<br>subscriptionName | text<br>Configuration setting | Intended durable subscription identity; current native engine consumer does not use it to create a durable subscription. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Shared subscription<br>sharedSubscription | boolean<br>Configuration setting | Editor option not wired to a shared-subscription constructor in the current native engine bridge. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Messaging style<br>messagingStyle | string<br>Optional | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. |
| Destination<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Message type<br>messageType | string<br>Optional | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. |
| JMS message selector<br>messageSelector | string<br>Optional | Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body. |
| Acknowledge mode<br>acknowledgeMode | string<br>Optional | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. |
| Maximum sessions<br>maxSessions | number<br>Optional | Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it. |
| Flow limit<br>flowLimit | number<br>Optional | Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism. |
| Receive timeout (ms)<br>receiveTimeout | number<br>Optional | Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault. |
| Durable subscription<br>durable | boolean<br>Optional | Editor option not wired to a durable subscriber constructor in the native engine bridge. Offline topic retention cannot be assumed. |
| Durable subscription name<br>subscriptionName | string<br>Optional | Intended durable subscription identity; current native engine consumer does not use it to create a durable subscription. |
| Shared subscription<br>sharedSubscription | boolean<br>Optional | Editor option not wired to a shared-subscription constructor in the current native engine bridge. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message body<br>body | object\|string\|binary<br>Optional | Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS standard headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| JMS application properties<br>properties | object<br>Optional | Application-defined message properties returned by the broker. Standard JMS headers are returned separately. |
| Client acknowledgement handle<br>ackId | string<br>Optional | Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| JMSInvalidInputException | The JMS activity input is invalid. |
| JMSMessageCreateException | The configured JMS message could not be created. |
| JMSSessionCreateException | A JMS session could not be created. |
| JMSReceiveException | The JMS message could not be received. |
| ActivityTimedOutException | The JMS operation reached its configured timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# JMS Request Reply

Activities / JMS

## Overview

Runs JMS Request Reply on the task execution path.

Activity type: jms; operation: request_reply.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| JMS connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: jms |
| Messaging style<br>messagingStyle | select<br>Configuration setting | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.<br>Choices: Queue, Topic, Generic |
| Destination<br>destination | text<br>Configuration setting | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Message type<br>messageType | select<br>Configuration setting | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.<br>Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text |
| Delivery mode<br>deliveryMode | select<br>Configuration setting | Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.<br>Choices: Persistent, Non-Persistent |
| Priority (0-9)<br>priority | number<br>Configuration setting | JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent. |
| Expiration (ms)<br>expiration | number<br>Configuration setting | Message time-to-live in milliseconds; zero means no producer-specified expiry. |
| JMS correlation ID<br>correlationId | text<br>Configuration setting | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Reply-to destination<br>replyTo | text<br>Configuration setting | Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank. |
| Disable JMS message ID<br>disableMessageId | boolean<br>Configuration setting | Editor option not forwarded to the current native sender; do not assume identifiers are disabled. |
| Disable JMS timestamp<br>disableTimestamp | boolean<br>Configuration setting | Editor option not forwarded to the current native sender; do not assume timestamps are disabled. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS application / dynamic properties (JSON)<br>dynamicProperties | textarea<br>Configuration setting | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |
| Request timeout (ms)<br>requestTimeout | number<br>Configuration setting | Editor request timeout; current engine bridge reads receiveTimeout for its actual wait, not this field. Verify the effective wait in your deployed archive. |
| Use temporary reply destination<br>temporaryReplyDestination | boolean<br>Configuration setting | Editor flag; the current native request bridge chooses a temporary queue based on replyTo being blank, not this flag. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message body<br>message | object\|string\|binary<br>Required | Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload. |
| Dynamic destination override<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Dynamic JMS properties<br>dynamicProperties | object<br>Optional | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |
| JMS headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| Messaging style<br>messagingStyle | string<br>Optional | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. |
| Message type<br>messageType | string<br>Optional | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. |
| Delivery mode<br>deliveryMode | string<br>Optional | Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. |
| Priority (0-9)<br>priority | number<br>Optional | JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent. |
| Expiration (ms)<br>expiration | number<br>Optional | Message time-to-live in milliseconds; zero means no producer-specified expiry. |
| JMS correlation ID<br>correlationId | string<br>Optional | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Reply-to destination<br>replyTo | string<br>Optional | Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank. |
| Disable JMS message ID<br>disableMessageId | boolean<br>Optional | Editor option not forwarded to the current native sender; do not assume identifiers are disabled. |
| Disable JMS timestamp<br>disableTimestamp | boolean<br>Optional | Editor option not forwarded to the current native sender; do not assume timestamps are disabled. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Request timeout (ms)<br>requestTimeout | number<br>Optional | Editor request timeout; current engine bridge reads receiveTimeout for its actual wait, not this field. Verify the effective wait in your deployed archive. |
| Use temporary reply destination<br>temporaryReplyDestination | boolean<br>Optional | Editor flag; the current native request bridge chooses a temporary queue based on replyTo being blank, not this flag. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Reply message body<br>body | object\|string\|binary<br>Optional | Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string. |
| Reply JMS headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| Reply properties<br>properties | object<br>Optional | Application-defined message properties returned by the broker. Standard JMS headers are returned separately. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| JMSInvalidInputException | The JMS activity input is invalid. |
| JMSMessageCreateException | The configured JMS message could not be created. |
| JMSSessionCreateException | A JMS session could not be created. |
| JMSSendException | The JMS message could not be sent. |
| ActivityTimedOutException | The JMS operation reached its configured timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# JMS Send Message

Activities / JMS

## Overview

Runs JMS Send Message on the task execution path.

Activity type: jms; operation: send_message.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| JMS connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: jms |
| Messaging style<br>messagingStyle | select<br>Configuration setting | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.<br>Choices: Queue, Topic, Generic |
| Destination<br>destination | text<br>Configuration setting | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Message type<br>messageType | select<br>Configuration setting | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.<br>Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text |
| Delivery mode<br>deliveryMode | select<br>Configuration setting | Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.<br>Choices: Persistent, Non-Persistent |
| Priority (0-9)<br>priority | number<br>Configuration setting | JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent. |
| Expiration (ms)<br>expiration | number<br>Configuration setting | Message time-to-live in milliseconds; zero means no producer-specified expiry. |
| JMS correlation ID<br>correlationId | text<br>Configuration setting | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Reply-to destination<br>replyTo | text<br>Configuration setting | Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank. |
| Disable JMS message ID<br>disableMessageId | boolean<br>Configuration setting | Editor option not forwarded to the current native sender; do not assume identifiers are disabled. |
| Disable JMS timestamp<br>disableTimestamp | boolean<br>Configuration setting | Editor option not forwarded to the current native sender; do not assume timestamps are disabled. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS application / dynamic properties (JSON)<br>dynamicProperties | textarea<br>Configuration setting | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message body<br>message | object\|string\|binary<br>Required | Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload. |
| Dynamic destination override<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Dynamic JMS properties<br>dynamicProperties | object<br>Optional | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |
| JMS headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| Messaging style<br>messagingStyle | string<br>Optional | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. |
| Message type<br>messageType | string<br>Optional | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. |
| Delivery mode<br>deliveryMode | string<br>Optional | Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. |
| Priority (0-9)<br>priority | number<br>Optional | JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent. |
| Expiration (ms)<br>expiration | number<br>Optional | Message time-to-live in milliseconds; zero means no producer-specified expiry. |
| JMS correlation ID<br>correlationId | string<br>Optional | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Reply-to destination<br>replyTo | string<br>Optional | Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank. |
| Disable JMS message ID<br>disableMessageId | boolean<br>Optional | Editor option not forwarded to the current native sender; do not assume identifiers are disabled. |
| Disable JMS timestamp<br>disableTimestamp | boolean<br>Optional | Editor option not forwarded to the current native sender; do not assume timestamps are disabled. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message ID<br>messageId | string<br>Optional | Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message. |
| Resolved destination<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| JMS timestamp<br>timestamp | dateTime<br>Optional | Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

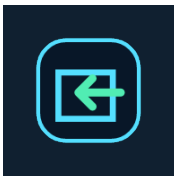
## Errors

| Fault | Description |
| --- | --- |
| JMSInvalidInputException | The JMS activity input is invalid. |

| Fault | Description |
| --- | --- |
| JMSMessageCreateException | The configured JMS message could not be created. |
| JMSSessionCreateException | A JMS session could not be created. |
| JMSSendException | The JMS message could not be sent. |
| ActivityTimedOutException | The JMS operation reached its configured timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Reply to JMS Message

Activities / JMS

## Overview

Runs Reply to JMS Message on the task execution path.

Activity type: jms; operation: reply_message.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| JMS connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: jms |
| Messaging style<br>messagingStyle | select<br>Configuration setting | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.<br>Choices: Queue, Topic, Generic |
| Reply destination<br>destination | text<br>Configuration setting | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Message type<br>messageType | select<br>Configuration setting | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.<br>Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text |
| Request correlation ID<br>correlationId | text<br>Configuration setting | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Delivery mode<br>deliveryMode | select<br>Configuration setting | Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.<br>Choices: Persistent, Non-Persistent |
| Priority (0-9)<br>priority | number<br>Configuration setting | JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent. |
| Expiration (ms)<br>expiration | number<br>Configuration setting | Message time-to-live in milliseconds; zero means no producer-specified expiry. |
| JMS correlation ID<br>correlationId | text<br>Configuration setting | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Reply-to destination<br>replyTo | text<br>Configuration setting | Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank. |
| Disable JMS message ID<br>disableMessageId | boolean<br>Configuration setting | Editor option not forwarded to the current native sender; do not assume identifiers are disabled. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Disable JMS timestamp<br>disableTimestamp | boolean<br>Configuration setting | Editor option not forwarded to the current native sender; do not assume timestamps are disabled. |
| JMS application / dynamic properties (JSON)<br>dynamicProperties | textarea<br>Configuration setting | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message body<br>message | object\|string\|binary<br>Required | Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload. |
| Dynamic destination override<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Dynamic JMS properties<br>dynamicProperties | object<br>Optional | Object/JSON object of application properties. Current messaging code stringifies outgoing property values. |
| JMS headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| Messaging style<br>messagingStyle | string<br>Optional | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. |
| Message type<br>messageType | string<br>Optional | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. |
| Request correlation ID<br>correlationId | string<br>Optional | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Delivery mode<br>deliveryMode | string<br>Optional | Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. |
| Priority (0-9)<br>priority | number<br>Optional | JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent. |
| Expiration (ms)<br>expiration | number<br>Optional | Message time-to-live in milliseconds; zero means no producer-specified expiry. |
| JMS correlation ID<br>correlationId | string<br>Optional | Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback. |
| Reply-to destination<br>replyTo | string<br>Optional | Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank. |
| Disable JMS message ID<br>disableMessageId | boolean<br>Optional | Editor option not forwarded to the current native sender; do not assume identifiers are disabled. |
| Disable JMS timestamp<br>disableTimestamp | boolean<br>Optional | Editor option not forwarded to the current native sender; do not assume timestamps are disabled. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message ID<br>messageId | string<br>Optional | Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message. |
| Resolved destination<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| JMS timestamp<br>timestamp | dateTime<br>Optional | Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| JMSInvalidInputException | The JMS activity input is invalid. |
| JMSMessageCreateException | The configured JMS message could not be created. |
| JMSSessionCreateException | A JMS session could not be created. |
| JMSSendException | The JMS message could not be sent. |
| ActivityTimedOutException | The JMS operation reached its configured timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Wait for JMS Request

Activities / JMS

## Overview

Runs Wait for JMS Request on the task execution path.

Activity type: jms; operation: wait_request.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| JMS connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: jms |
| Messaging style<br>messagingStyle | select<br>Configuration setting | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.<br>Choices: Queue, Topic, Generic |
| Destination<br>destination | text<br>Configuration setting | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Message type<br>messageType | select<br>Configuration setting | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.<br>Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text |
| JMS message selector<br>messageSelector | text<br>Configuration setting | Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body. |
| Acknowledge mode<br>acknowledgeMode | select<br>Configuration setting | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.<br>Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional |
| Receive timeout (ms)<br>receiveTimeout | number<br>Configuration setting | Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Messaging style<br>messagingStyle | string<br>Optional | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | sender into a topic publisher. |
| Destination<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Message type<br>messageType | string<br>Optional | Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. |
| JMS message selector<br>messageSelector | string<br>Optional | Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body. |
| Acknowledge mode<br>acknowledgeMode | string<br>Optional | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. |
| Receive timeout (ms)<br>receiveTimeout | number<br>Optional | Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JMS message body<br>body | object\|string\|binary<br>Optional | Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string. |
| JMS standard headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| JMS application properties<br>properties | object<br>Optional | Application-defined message properties returned by the broker. Standard JMS headers are returned separately. |
| Client acknowledgement handle<br>ackId | string<br>Optional | Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| JMSInvalidInputException | The JMS activity input is invalid. |
| JMSMessageCreateException | The configured JMS message could not be created. |
| JMSSessionCreateException | A JMS session could not be created. |

| Fault | Description |
| --- | --- |
| JMSReceiveException | The JMS message could not be received. |
| ActivityTimedOutException | The JMS operation reached its configured timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Kafka Receive Message

Activities / Kafka

## Overview

### *Start work from Kafka records*

Use as an event starter. Run/Debug registers continuous starter processing; one received batch becomes flow input. Pauses/breakpoints can delay downstream work.

### *Prerequisites*

Install the matching confluent-kafka package on the execution host. Configure a Kafka shared connection with reachable bootstrap servers and the required TLS/SASL credentials. Verify advertised broker addresses are reachable from that host; a successful TCP connection to one bootstrap address is not sufficient. These pages describe the Studio/engine runtime; qualify exported direct Python separately.

Produces, receives, or commits Apache Kafka records.

Broker security comes from the shared Kafka connection. Topic, key, headers, value, partitions, offsets, and acknowledgement handles are available for hierarchical mapping.

Activity type: kafka; operation: receive.

## Configuration

### *Configuration procedure*

Set topic and groupId explicitly. Select deserializers matching the producer. For Manual mode disable enableAutoCommit and map acknowledgement handles only after processing succeeds. Use a separate test group to avoid consuming production offsets.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Kafka connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: kafka |
| Consumer group ID<br>groupId | text<br>Configuration setting | Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups. |
| Topic names<br>topic | text<br>Configuration setting | Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons. |
| Assign custom partitions<br>assignCustomPartition | boolean<br>Configuration setting | Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior. |
| Partition IDs / ranges<br>partitionIds | text<br>Configuration setting | Comma/semicolon-separated partition IDs or inclusive ranges used for explicit assignment. |
| Consumer count<br>consumerCount | number<br>Configuration setting | Exposed by the editor; the engine receive path creates one consumer per call and does not implement a consumer pool from this field. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Key deserializer<br>keyDeserializer | select<br>Configuration setting | Incoming key decoding; Byte Array exposes base64 text, JSON parses UTF-8 JSON, String decodes text.<br>Choices: String, Byte Array, Avro Schema, JSON |
| Value deserializer<br>valueDeserializer | select<br>Configuration setting | Incoming record-value decoding. Invalid JSON raises a decode error; Avro is not implemented by this engine path.<br>Choices: String, Byte Array, Avro Schema, JSON |
| Use Schema Registry<br>useRegistry | boolean<br>Configuration setting | Editor registry flag; setting this does not activate an Avro serializer in the current engine. |
| Schema Registry URL<br>schemaRegistryUrl | text<br>Configuration setting | Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL. |
| Key schema subject : version<br>keySchemaSubject | text<br>Configuration setting | Intended registry key subject/version; not resolved by the current engine serializer. |
| Value schema subject : version<br>valueSchemaSubject | text<br>Configuration setting | Intended registry value subject/version; not resolved by the current engine serializer. |
| Acknowledgement mode<br>acknowledgeMode | select<br>Configuration setting | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.<br>Choices: Auto, Manual |
| Enable auto commit<br>enableAutoCommit | boolean<br>Configuration setting | Native periodic offset commit toggle. For manual acknowledgement keep this disabled; otherwise offsets can advance before downstream work finishes. |
| Auto offset reset<br>autoOffsetReset | select<br>Configuration setting | Policy used when no valid committed offset exists, commonly earliest or latest. It does not rewind a group that already has committed offsets. |
| Fetch minimum bytes<br>fetchMinBytes | number<br>Configuration setting | Minimum fetch bytes requested from the broker. Larger values can add latency at low traffic. |
| Maximum poll records<br>maxPollRecords | number<br>Configuration setting | Editor batch-size setting. The engine currently passes max.poll.records to its native consumer; this Java-style key may be rejected by the installed native client. Treat this as a known compatibility issue, not a guaranteed supported control. |
| Session timeout (ms)<br>sessionTimeoutMs | number<br>Configuration setting | Consumer failure-detection timeout in milliseconds; must satisfy broker limits. |
| Heartbeat interval (ms)<br>heartbeatIntervalMs | number<br>Configuration setting | Consumer heartbeat interval in milliseconds; coordinate with session timeout. |
| Additional consumer properties (JSON)<br>additionalProperties | textarea<br>Configuration setting | JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
topic = ${properties.messaging.ordersTopic}
groupId = ${properties.messaging.consumerGroup}
valueDeserializer = JSON
acknowledgeMode = Manual
enableAutoCommit = false
Downstream source = ${Kafka-Receive.messages}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Consumer group ID<br>groupId | string<br>Optional | Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups. |
| Topic names<br>topic | string<br>Optional | Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons. |
| Assign custom partitions<br>assignCustomPartition | boolean<br>Optional | Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior. |
| Partition IDs / ranges<br>partitionIds | string<br>Optional | Comma/semicolon-separated partition IDs or inclusive ranges used for explicit assignment. |
| Consumer count<br>consumerCount | number<br>Optional | Exposed by the editor; the engine receive path creates one consumer per call and does not implement a consumer pool from this field. |
| Key deserializer<br>keyDeserializer | string<br>Optional | Incoming key decoding; Byte Array exposes base64 text, JSON parses UTF-8 JSON, String decodes text. |
| Value deserializer<br>valueDeserializer | string<br>Optional | Incoming record-value decoding. Invalid JSON raises a decode error; Avro is not implemented by this engine path. |
| Use Schema Registry<br>useRegistry | boolean<br>Optional | Editor registry flag; setting this does not activate an Avro serializer in the current engine. |
| Schema Registry URL<br>schemaRegistryUrl | string<br>Optional | Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL. |
| Key schema subject : version<br>keySchemaSubject | string<br>Optional | Intended registry key subject/version; not resolved by the current engine serializer. |
| Value schema subject : version<br>valueSchemaSubject | string<br>Optional | Intended registry value subject/version; not resolved by the current engine serializer. |
| Acknowledgement mode<br>acknowledgeMode | string<br>Optional | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. |
| Enable auto commit<br>enableAutoCommit | boolean<br>Optional | Native periodic offset commit toggle. For manual acknowledgement keep this disabled; otherwise offsets can advance before downstream work finishes. |
| Auto offset reset<br>autoOffsetReset | string<br>Optional | Policy used when no valid committed offset exists, commonly earliest or latest. It does not rewind a group that already has committed offsets. |
| Fetch minimum bytes<br>fetchMinBytes | number<br>Optional | Minimum fetch bytes requested from the broker. Larger values can add latency at low traffic. |
| Maximum poll records<br>maxPollRecords | number<br>Optional | Editor batch-size setting. The engine currently passes max.poll.records to its native consumer; this Java-style key may be rejected by the installed native client. Treat this as a known compatibility issue, not a guaranteed supported control. |
| Session timeout (ms)<br>sessionTimeoutMs | number<br>Optional | Consumer failure-detection timeout in milliseconds; must satisfy broker limits. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Heartbeat interval (ms)<br>heartbeatIntervalMs | number<br>Optional | Consumer heartbeat interval in milliseconds; coordinate with session timeout. |
| Additional consumer properties (JSON)<br>additionalProperties | string<br>Optional | JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client. |

## Output

### *Illustrative runtime output*


```
{"messages":[{"data":{"id":"A100"},"key":"A100","topic":"orders","partition":0,"offset":42,"ackId":"<handle>"}],"count":1,"ackId":"<handle>","ackIds":["<handle>"]}
```


IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Kafka records<br>messages | array<br>Optional | Array of received records with data, key, topic, partition, offset, timestamp, headers and optional ackId. |
| Record count<br>count | integer<br>Optional | Number of records returned by this receive attempt. A zero count means no message was returned, not successful business processing. |
| Topic<br>topic | string<br>Optional | Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons. |
| Partition<br>partition | integer<br>Optional | Partition number on each returned message. The engine receive result nests it under messages[]. |
| Offset<br>offset | long<br>Optional | Kafka offset within one partition; it is not globally ordered across the topic. |
| Record timestamp<br>timestamp | dateTime<br>Optional | Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation. |
| Record headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| Manual acknowledgement handle<br>ackId | string<br>Optional | Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart. |

## Advanced

### *Delivery, transactions and performance*

Separate local enqueue, broker acknowledgement and downstream processing. Monitor background publisher errors and shutdown draining. Use stable business IDs and idempotent consumers when retries are possible. Producer reuse reduces warm-send overhead, but first connection, TLS, metadata and broker availability still affect latency; no fixed latency is guaranteed.

### *Implemented behavior and limitations*

Runtime records live under messages[], not the editor top-level topic/partition fields. consumerCount is not implemented as a pool in the engine receive call. max.poll.records is forwarded as a native property and may be rejected by confluent-kafka. Manual commits can advance a partition past earlier records, so preserve processing/commit order. Do not claim this receive path is production-qualified without a provider test.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

### *Troubleshooting procedure*

Connection/authentication failures: verify DNS, advertised listeners, TLS chain and ACLs on the runtime host. Serialization failures: compare actual payload type with the chosen serializer. Queue full/timeouts: inspect broker latency and producer buffer pressure before increasing limits. Consumer configuration errors may be caused by Java-style options passed to the native client. Capture the resolved non-secret configuration and the full runtime error.

### *Acceptance test*

Acceptance: send two distinguishable records, inspect messages[].data, test failure before confirmation and verify actual committed offsets and redelivery.

| Fault | Description |
| --- | --- |
| KafkaConnectionException | The Kafka connection or broker is unavailable. |
| KafkaSerializationException | The record key or value could not be serialized/deserialized. |
| KafkaProducerException | The producer could not publish the record. |
| KafkaConsumerException | The consumer could not fetch or acknowledge records. |
| KafkaSchemaRegistryException | The Avro schema or registry subject could not be resolved. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# Kafka Send Message

Activities / Kafka

## Overview

### *Publish a record*

Use inside a running flow to enqueue or confirm one record on a Kafka topic.

### *Prerequisites*

Install the matching confluent-kafka package on the execution host. Configure a Kafka shared connection with reachable bootstrap servers and the required TLS/SASL credentials. Verify advertised broker addresses are reachable from that host; a successful TCP connection to one bootstrap address is not sufficient. These pages describe the Studio/engine runtime; qualify exported direct Python separately.

Produces, receives, or commits Apache Kafka records.

Broker security comes from the shared Kafka connection. Topic, key, headers, value, partitions, offsets, and acknowledgement handles are available for hierarchical mapping.

Activity type: kafka; operation: send.

## Configuration

### *Configuration procedure*

Select the Kafka resource; bind topic to an environment property. Select JSON for an object payload, or String for already-serialized text. Start with explicit acks and waitForDelivery settings. Partition selection is optional; use a stable key before forcing a partition.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Kafka connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: kafka |
| Topic name<br>topic | text<br>Configuration setting | Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons. |
| Assign custom partition<br>assignCustomPartition | boolean<br>Configuration setting | Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior. |
| Partition ID<br>partitionId | number<br>Configuration setting | Explicit Kafka partition when assignCustomPartition is enabled. It must exist in the target topic. |
| Key serializer<br>keySerializer | select<br>Configuration setting | Outgoing key conversion: String, Byte Array or JSON. Avro selection currently raises a registry-runtime error in the engine.<br>Choices: String, Byte Array, Avro Schema, JSON |
| Value serializer<br>valueSerializer | select<br>Configuration setting | Outgoing value conversion. Select JSON for a structured object; String converts using string representation and is not a JSON encoder.<br>Choices: String, Byte Array, Avro Schema, JSON |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Use Schema Registry<br>useRegistry | boolean<br>Configuration setting | Editor registry flag; setting this does not activate an Avro serializer in the current engine. |
| Schema Registry URL<br>schemaRegistryUrl | text<br>Configuration setting | Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL. |
| Key schema subject : version<br>keySchemaSubject | text<br>Configuration setting | Intended registry key subject/version; not resolved by the current engine serializer. |
| Value schema subject : version<br>valueSchemaSubject | text<br>Configuration setting | Intended registry value subject/version; not resolved by the current engine serializer. |
| Wait for broker confirmation<br>waitForDelivery | boolean<br>Configuration setting | False normally returns after local enqueue into a reused producer. True waits for delivery confirmation. Transactions and one-shot producers also force waiting. |
| Acknowledgements<br>acks | select<br>Configuration setting | Broker acknowledgement policy: 0, 1 or all. This differs from the waitForDelivery switch; acks=0 cannot establish broker durability.<br>Choices: 0, 1, all |
| Compression type<br>compressionType | select<br>Configuration setting | Client batch compression. Select a codec supported by the installed client and broker; compression trades CPU for network bandwidth.<br>Choices: none, gzip, snappy, lz4, zstd |
| Producer retries<br>retries | number<br>Configuration setting | Native producer retry setting. Separate from MINA Advanced retries; application retries can repeat a send after an ambiguous timeout. |
| Buffer memory (bytes)<br>bufferMemory | number<br>Configuration setting | Local producer queue budget in bytes, translated to queue.buffering.max.kbytes. A full queue applies backpressure/errors; it is not durable storage. |
| Batch size<br>batchSize | number<br>Configuration setting | Target producer batch size in bytes, default 16384 in the engine. Larger batches may improve throughput at the expense of latency. |
| Linger (ms)<br>lingerMs | number<br>Configuration setting | Maximum batching delay requested from the producer, default 0 in the engine. Increasing it intentionally adds buffering delay. |
| Maximum request size<br>maxRequestSize | number<br>Configuration setting | Maximum native message size setting, default 1048576 bytes. Broker/topic limits can be lower. |
| Transactional ID<br>transactionalId | text<br>Configuration setting | Enables native transaction initialization/begin/commit for a send and forces confirmation. Must be unique for concurrently active producers; it is not a multi-activity transaction group. |
| Enable idempotence<br>enableIdempotence | boolean<br>Configuration setting | Requests native producer idempotence. Must be compatible with other native settings and does not deduplicate arbitrary repeated business calls. |
| Additional producer properties (JSON)<br>additionalProperties | textarea<br>Configuration setting | JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.order}
key = ${input.order.id}
topic = ${properties.messaging.ordersTopic}
```

```
valueSerializer = JSON
waitForDelivery = true
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Kafka record value<br>message | object\|string\|binary<br>Required | Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload. |
| Kafka record key<br>key | string\|binary<br>Optional | Kafka record key. Use a stable business key when partition locality matters; preserve the matching serializer. |
| Kafka headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| Dynamic topic override<br>topic | string<br>Optional | Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons. |
| Dynamic partition<br>partitionId | integer<br>Optional | Explicit Kafka partition when assignCustomPartition is enabled. It must exist in the target topic. |
| Assign custom partition<br>assignCustomPartition | boolean<br>Optional | Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior. |
| Key serializer<br>keySerializer | string<br>Optional | Outgoing key conversion: String, Byte Array or JSON. Avro selection currently raises a registry-runtime error in the engine. |
| Value serializer<br>valueSerializer | string<br>Optional | Outgoing value conversion. Select JSON for a structured object; String converts using string representation and is not a JSON encoder. |
| Use Schema Registry<br>useRegistry | boolean<br>Optional | Editor registry flag; setting this does not activate an Avro serializer in the current engine. |
| Schema Registry URL<br>schemaRegistryUrl | string<br>Optional | Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL. |
| Key schema subject : version<br>keySchemaSubject | string<br>Optional | Intended registry key subject/version; not resolved by the current engine serializer. |
| Value schema subject : version<br>valueSchemaSubject | string<br>Optional | Intended registry value subject/version; not resolved by the current engine serializer. |
| Wait for broker confirmation<br>waitForDelivery | boolean<br>Optional | False normally returns after local enqueue into a reused producer. True waits for delivery confirmation. Transactions and one-shot producers also force waiting. |
| Acknowledgements<br>acks | string<br>Optional | Broker acknowledgement policy: 0, 1 or all. This differs from the waitForDelivery switch; acks=0 cannot establish broker durability. |
| Compression type<br>compressionType | string<br>Optional | Client batch compression. Select a codec supported by the installed client and broker; compression trades CPU for network bandwidth. |
| Producer retries<br>retries | number<br>Optional | Native producer retry setting. Separate from MINA Advanced retries; application retries can repeat a send after an ambiguous timeout. |
| Buffer memory (bytes)<br>bufferMemory | number<br>Optional | Local producer queue budget in bytes, translated to queue.buffering.max.kbytes. A full queue applies backpressure/errors; it is not durable storage. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Batch size<br>batchSize | number<br>Optional | Target producer batch size in bytes, default 16384 in the engine. Larger batches may improve throughput at the expense of latency. |
| Linger (ms)<br>lingerMs | number<br>Optional | Maximum batching delay requested from the producer, default 0 in the engine. Increasing it intentionally adds buffering delay. |
| Maximum request size<br>maxRequestSize | number<br>Optional | Maximum native message size setting, default 1048576 bytes. Broker/topic limits can be lower. |
| Transactional ID<br>transactionalId | string<br>Optional | Enables native transaction initialization/begin/commit for a send and forces confirmation. Must be unique for concurrently active producers; it is not a multi-activity transaction group. |
| Enable idempotence<br>enableIdempotence | boolean<br>Optional | Requests native producer idempotence. Must be compatible with other native settings and does not deduplicate arbitrary repeated business calls. |
| Additional producer properties (JSON)<br>additionalProperties | string<br>Optional | JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client. |

## Output

### *Illustrative runtime output*

```
{"messageId":"<id>","topic":"orders","queued":true,"deliveryConfirmed":true,"partition":0,"offset":42}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Generated record ID<br>messageId | string<br>Optional | Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message. |
| Topic<br>topic | string<br>Optional | Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons. |
| Accepted by the local Kafka producer<br>queued | boolean<br>Optional | True indicates local producer acceptance, not broker durability or downstream consumption. |
| Broker delivery confirmed before completion<br>deliveryConfirmed | boolean<br>Optional | True only when the delivery path has completed its confirmation before returning. Check this instead of inferring delivery from queued. |
| Publisher activity latency in milliseconds<br>publishLatencyMs | number<br>Optional | Publisher activity latency; buffered mode measures enqueue/overhead, not end-to-end broker or consumer latency. |
| Broker partition (confirmation mode)<br>partition | integer<br>Optional | Partition number on each returned message. The engine receive result nests it under messages[]. |
| Broker offset (confirmation mode)<br>offset | long<br>Optional | Kafka offset within one partition; it is not globally ordered across the topic. |
| Record timestamp<br>timestamp | dateTime<br>Optional | Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation. |

## Advanced

### *Delivery, transactions and performance*

Separate local enqueue, broker acknowledgement and downstream processing. Monitor background publisher errors and shutdown draining. Use stable business IDs and idempotent consumers when retries are possible. Producer reuse reduces warm-send overhead, but first connection, TLS, metadata and broker availability still affect latency; no fixed latency is guaranteed.

### *Implemented behavior and limitations*

With waitForDelivery=false, partition/offset can be absent. The generated ID may be local. Avro Schema selection currently raises a registry-runtime fault even when registry fields are populated. A transactionalId starts a transaction for the send; it does not span other activities.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

### *Troubleshooting procedure*

Connection/authentication failures: verify DNS, advertised listeners, TLS chain and ACLs on the runtime host. Serialization failures: compare actual payload type with the chosen serializer. Queue full/timeouts: inspect broker latency and producer buffer pressure before increasing limits. Consumer configuration errors may be caused by Java-style options passed to the

native client. Capture the resolved non-secret configuration and the full runtime error.

### *Acceptance test*

Acceptance: publish a known ID, verify the exact decoded value/key with an independent consumer, then test unavailable-broker behavior in both buffered and confirmed modes.

| Fault | Description |
| --- | --- |
| KafkaConnectionException | The Kafka connection or broker is unavailable. |
| KafkaSerializationException | The record key or value could not be serialized/deserialized. |
| KafkaProducerException | The producer could not publish the record. |
| KafkaConsumerException | The consumer could not fetch or acknowledge records. |
| KafkaSchemaRegistryException | The Avro schema or registry subject could not be resolved. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# Kafka Get Messages

Activities / Kafka

## Overview

### *Poll records during a flow*

Use after Start or another activity to perform a bounded receive attempt, rather than registering a new event starter.

### *Prerequisites*

Install the matching confluent-kafka package on the execution host. Configure a Kafka shared connection with reachable bootstrap servers and the required TLS/SASL credentials. Verify advertised broker addresses are reachable from that host; a successful TCP connection to one bootstrap address is not sufficient. These pages describe the Studio/engine runtime; qualify exported direct Python separately.

Produces, receives, or commits Apache Kafka records.

Broker security comes from the shared Kafka connection. Topic, key, headers, value, partitions, offsets, and acknowledgement handles are available for hierarchical mapping.

Activity type: kafka; operation: get.

## Configuration

### *Configuration procedure*

Select the consumer resource/group/topics, maxMessages and poll timeout in seconds. For explicit assignment set assignCustomPartition, partitionIds and seekPosition together. Use customOffset only for custom seek.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Kafka connection<br>resourceId | resource<br>Required | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: kafka |
| Consumer group ID<br>groupId | text<br>Configuration setting | Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups. |
| Topic names<br>topic | text<br>Configuration setting | Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons. |
| Assign custom partitions<br>assignCustomPartition | boolean<br>Configuration setting | Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior. |
| Partition IDs / ranges<br>partitionIds | text<br>Configuration setting | Comma/semicolon-separated partition IDs or inclusive ranges used for explicit assignment. |
| Consumer count<br>consumerCount | number<br>Configuration setting | Exposed by the editor; the engine receive path creates one consumer per call and does not implement a consumer pool from this field. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Key deserializer<br>keyDeserializer | select<br>Configuration setting | Incoming key decoding; Byte Array exposes base64 text, JSON parses UTF-8 JSON, String decodes text.<br>Choices: String, Byte Array, Avro Schema, JSON |
| Value deserializer<br>valueDeserializer | select<br>Configuration setting | Incoming record-value decoding. Invalid JSON raises a decode error; Avro is not implemented by this engine path.<br>Choices: String, Byte Array, Avro Schema, JSON |
| Use Schema Registry<br>useRegistry | boolean<br>Configuration setting | Editor registry flag; setting this does not activate an Avro serializer in the current engine. |
| Schema Registry URL<br>schemaRegistryUrl | text<br>Configuration setting | Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL. |
| Key schema subject : version<br>keySchemaSubject | text<br>Configuration setting | Intended registry key subject/version; not resolved by the current engine serializer. |
| Value schema subject : version<br>valueSchemaSubject | text<br>Configuration setting | Intended registry value subject/version; not resolved by the current engine serializer. |
| Acknowledgement mode<br>acknowledgeMode | select<br>Configuration setting | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.<br>Choices: Auto, Manual |
| Enable auto commit<br>enableAutoCommit | boolean<br>Configuration setting | Native periodic offset commit toggle. For manual acknowledgement keep this disabled; otherwise offsets can advance before downstream work finishes. |
| Auto offset reset<br>autoOffsetReset | select<br>Configuration setting | Policy used when no valid committed offset exists, commonly earliest or latest. It does not rewind a group that already has committed offsets. |
| Fetch minimum bytes<br>fetchMinBytes | number<br>Configuration setting | Minimum fetch bytes requested from the broker. Larger values can add latency at low traffic. |
| Maximum poll records<br>maxPollRecords | number<br>Configuration setting | Editor batch-size setting. The engine currently passes max.poll.records to its native consumer; this Java-style key may be rejected by the installed native client. Treat this as a known compatibility issue, not a guaranteed supported control. |
| Session timeout (ms)<br>sessionTimeoutMs | number<br>Configuration setting | Consumer failure-detection timeout in milliseconds; must satisfy broker limits. |
| Heartbeat interval (ms)<br>heartbeatIntervalMs | number<br>Configuration setting | Consumer heartbeat interval in milliseconds; coordinate with session timeout. |
| Additional consumer properties (JSON)<br>additionalProperties | textarea<br>Configuration setting | JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client. |
| Seek position<br>seekPosition | select<br>Configuration setting | Explicit-assignment start offset. The engine maps beginning to -2 and custom to the supplied offset; other selections currently use -1 (end). This is not a reliable resume-current implementation.<br>Choices: current, beginning, end, custom |
| Custom offset<br>customOffset | number<br>Configuration setting | Absolute offset used with custom seek and explicit partition assignment. |
| Maximum messages<br>maxMessages | number<br>Configuration setting | Maximum poll attempts/records requested by one Get call. Each poll can wait for the configured timeout; total latency can grow with this count. |
| Poll timeout (seconds)<br>timeout | number<br>Configuration setting | Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
topic = ${properties.messaging.ordersTopic}
maxMessages = 10
timeout = 1
assignCustomPartition = true
partitionIds = 0
seekPosition = custom
customOffset = 42
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dynamic topic override<br>topic | string<br>Optional | Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons. |
| Dynamic partition<br>partition | integer<br>Optional | Partition number on each returned message. The engine receive result nests it under messages[]. |
| Dynamic offset<br>offset | long<br>Optional | Kafka offset within one partition; it is not globally ordered across the topic. |
| Consumer group ID<br>groupId | string<br>Optional | Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups. |
| Assign custom partitions<br>assignCustomPartition | boolean<br>Optional | Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior. |
| Partition IDs / ranges<br>partitionIds | string<br>Optional | Comma/semicolon-separated partition IDs or inclusive ranges used for explicit assignment. |
| Consumer count<br>consumerCount | number<br>Optional | Exposed by the editor; the engine receive path creates one consumer per call and does not implement a consumer pool from this field. |
| Key deserializer<br>keyDeserializer | string<br>Optional | Incoming key decoding; Byte Array exposes base64 text, JSON parses UTF-8 JSON, String decodes text. |
| Value deserializer<br>valueDeserializer | string<br>Optional | Incoming record-value decoding. Invalid JSON raises a decode error; Avro is not implemented by this engine path. |
| Use Schema Registry<br>useRegistry | boolean<br>Optional | Editor registry flag; setting this does not activate an Avro serializer in the current engine. |
| Schema Registry URL<br>schemaRegistryUrl | string<br>Optional | Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL. |
| Key schema subject : version<br>keySchemaSubject | string<br>Optional | Intended registry key subject/version; not resolved by the current engine serializer. |
| Value schema subject : version<br>valueSchemaSubject | string<br>Optional | Intended registry value subject/version; not resolved by the current engine serializer. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Acknowledgement mode<br>acknowledgeMode | string<br>Optional | Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. |
| Enable auto commit<br>enableAutoCommit | boolean<br>Optional | Native periodic offset commit toggle. For manual acknowledgement keep this disabled; otherwise offsets can advance before downstream work finishes. |
| Auto offset reset<br>autoOffsetReset | string<br>Optional | Policy used when no valid committed offset exists, commonly earliest or latest. It does not rewind a group that already has committed offsets. |
| Fetch minimum bytes<br>fetchMinBytes | number<br>Optional | Minimum fetch bytes requested from the broker. Larger values can add latency at low traffic. |
| Maximum poll records<br>maxPollRecords | number<br>Optional | Editor batch-size setting. The engine currently passes max.poll.records to its native consumer; this Java-style key may be rejected by the installed native client. Treat this as a known compatibility issue, not a guaranteed supported control. |
| Session timeout (ms)<br>sessionTimeoutMs | number<br>Optional | Consumer failure-detection timeout in milliseconds; must satisfy broker limits. |
| Heartbeat interval (ms)<br>heartbeatIntervalMs | number<br>Optional | Consumer heartbeat interval in milliseconds; coordinate with session timeout. |
| Additional consumer properties (JSON)<br>additionalProperties | string<br>Optional | JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client. |
| Seek position<br>seekPosition | string<br>Optional | Explicit-assignment start offset. The engine maps beginning to -2 and custom to the supplied offset; other selections currently use -1 (end). This is not a reliable resume-current implementation. |
| Custom offset<br>customOffset | number<br>Optional | Absolute offset used with custom seek and explicit partition assignment. |
| Maximum messages<br>maxMessages | number<br>Optional | Maximum poll attempts/records requested by one Get call. Each poll can wait for the configured timeout; total latency can grow with this count. |
| Poll timeout (seconds)<br>timeout | number<br>Optional | Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately. |

## Output

### *Illustrative runtime output*

```
{"messages":[],"count":0,"ackId":null,"ackIds":[]}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Kafka records<br>messages | array<br>Optional | Array of received records with data, key, topic, partition, offset, timestamp, headers and optional ackId. |
| Record count<br>count | integer<br>Optional | Number of records returned by this receive attempt. A zero count means no message was returned, not successful business processing. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Topic<br>topic | string<br>Optional | Broker topic name. Resolve it from the active environment, for example ${properties.messaging.topic}. Consumer topic lists use semicolons. |
| Partition<br>partition | integer<br>Optional | Partition number on each returned message. The engine receive result nests it under messages[]. |
| Offset<br>offset | long<br>Optional | Kafka offset within one partition; it is not globally ordered across the topic. |
| Record timestamp<br>timestamp | dateTime<br>Optional | Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation. |
| Record headers<br>headers | object<br>Optional | Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support. |
| Manual acknowledgement handle<br>ackId | string<br>Optional | Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart. |

## Advanced

### *Delivery, transactions and performance*

Separate local enqueue, broker acknowledgement and downstream processing. Monitor background publisher errors and shutdown draining. Use stable business IDs and idempotent consumers when retries are possible. Producer reuse reduces warm-send overhead, but first connection, TLS, metadata and broker availability still affect latency; no fixed latency is guaranteed.

### *Implemented behavior and limitations*

Empty results are normal when no record is available. Each poll can consume the timeout, so the upper wait grows with maxMessages. The engine reads partitionIds/customOffset; the displayed dynamic partition/offset aliases are not equivalent overrides. Current seek maps to end in explicit assignment; do not use it as a resume guarantee. Consumer native-option and Avro limitations are the same as Receive.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

### *Troubleshooting procedure*

Connection/authentication failures: verify DNS, advertised listeners, TLS chain and ACLs on the runtime host. Serialization failures: compare actual payload type with the chosen serializer. Queue full/timeouts: inspect broker latency and producer buffer pressure before increasing limits. Consumer configuration errors may be caused by Java-style options passed to the native client. Capture the resolved non-secret configuration and the full runtime error.

### *Acceptance test*

Acceptance: seed a test partition with known offsets; check beginning/custom/end behavior and the zero-message branch.

| Fault | Description |
| --- | --- |
| KafkaConnectionException | The Kafka connection or broker is unavailable. |
| KafkaSerializationException | The record key or value could not be serialized/deserialized. |
| KafkaProducerException | The producer could not publish the record. |
| KafkaConsumerException | The consumer could not fetch or acknowledge records. |
| KafkaSchemaRegistryException | The Avro schema or registry subject could not be resolved. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# Pub/Sub Subscriber

Activities / GCP Pub/Sub

## Overview

Publishes, receives, or acknowledges Google Cloud Pub/Sub messages.

Project and credential defaults come from the shared connection. Message data, attributes, ordering keys, and acknowledgement handles remain available on the execution path.

Activity type: pubsub; operation: subscribe.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Google Pub/Sub connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: pubsub |
| Project ID override<br>projectId | text<br>Configuration setting | Google Cloud project containing the Pub/Sub topic or subscription. Supply the project ID, not its display name. |
| Subscription ID<br>subscription | text<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Maximum message fetch count<br>maxMessages | number<br>Configuration setting | Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity. |
| Acknowledge mode<br>acknowledgeMode | select<br>Configuration setting | Message acknowledgement policy. Manual/client modes require explicit confirmation after successful processing; automatic/provider modes follow connector behavior.<br>Choices: Auto, Client |
| Receive timeout (seconds)<br>receiveTimeout | number<br>Configuration setting | Receive timeout (seconds): bounded wait/delay in the connector-defined time unit. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| Sequence key expression<br>sequenceKey | text<br>Configuration setting | Stable ordering/sequence key used to group related messages where the selected provider supports ordered delivery. |
| Custom job ID expression<br>customJobId | text<br>Configuration setting | Optional business identifier for tracing the received event/job. Use a value that distinguishes separate business deliveries. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Project ID override<br>projectId | string<br>Optional | Google Cloud project containing the Pub/Sub topic or subscription. Supply the project ID, not its display name. |
| Subscription ID<br>subscription | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Maximum message fetch count<br>maxMessages | number<br>Optional | Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity. |
| Acknowledge mode<br>acknowledgeMode | string<br>Optional | Message acknowledgement policy. Manual/client modes require explicit confirmation after successful processing; automatic/provider modes follow connector behavior. |
| Receive timeout (seconds)<br>receiveTimeout | number<br>Optional | Receive timeout (seconds): bounded wait/delay in the connector-defined time unit. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| Sequence key expression<br>sequenceKey | string<br>Optional | Stable ordering/sequence key used to group related messages where the selected provider supports ordered delivery. |
| Custom job ID expression<br>customJobId | string<br>Optional | Optional business identifier for tracing the received event/job. Use a value that distinguishes separate business deliveries. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Google message ID<br>MessageID | string<br>Optional | Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection. |
| Publish time<br>PublishTime | dateTime<br>Optional | Time reported by the operation/provider. Preserve the original representation when tracing; epoch values and formatted date/time strings need explicit conversion. |
| Message data<br>Data | string<br>Optional | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Message attributes<br>Attributes | object<br>Optional | Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system. |
| Client acknowledgement handle<br>AckID | string<br>Optional | Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| GooglePubSubPluginException | The subscriber could not receive or acknowledge the message. |
| GooglePubSubConnectionException | The Google Pub/Sub connection is unavailable. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Pub/Sub Publisher

Activities / GCP Pub/Sub

## Overview

Publishes, receives, or acknowledges Google Cloud Pub/Sub messages.

Project and credential defaults come from the shared connection. Message data, attributes, ordering keys, and acknowledgement handles remain available on the execution path.

Activity type: pubsub; operation: publish.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Google Pub/Sub connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: pubsub |
| Project ID override<br>projectId | text<br>Configuration setting | Google Cloud project containing the Pub/Sub topic or subscription. Supply the project ID, not its display name. |
| Topic ID<br>topic | text<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Ordering key<br>orderingKey | text<br>Configuration setting | Stable ordering/sequence key used to group related messages where the selected provider supports ordered delivery. |
| Wait for Google confirmation<br>waitForDelivery | boolean<br>Configuration setting | Off queues through the persistent batched gRPC publisher for maximum throughput. Enable only when the flow requires Google's message ID. |
| Publish timeout (seconds)<br>publishTimeout | number<br>Configuration setting | Publish timeout (seconds): bounded wait/delay in the connector-defined time unit. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| Batch maximum messages<br>batchMaxMessages | number<br>Configuration setting | Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency. |
| Batch maximum bytes<br>batchMaxBytes | number<br>Configuration setting | Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency. |
| Batch delay threshold (ms)<br>batchDelayThresholdMilliseconds | number<br>Configuration setting | Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Message data<br>data | string\|binary<br>Required | Data or at least one attribute is required. |
| Message attributes<br>attributes | object<br>Optional | Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system. |
| Dynamic Topic ID<br>topic | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Project ID override<br>projectId | string<br>Optional | Google Cloud project containing the Pub/Sub topic or subscription. Supply the project ID, not its display name. |
| Ordering key<br>orderingKey | string<br>Optional | Stable ordering/sequence key used to group related messages where the selected provider supports ordered delivery. |
| Wait for Google confirmation<br>waitForDelivery | boolean<br>Optional | Off queues through the persistent batched gRPC publisher for maximum throughput. Enable only when the flow requires Google's message ID. |
| Publish timeout (seconds)<br>publishTimeout | number<br>Optional | Publish timeout (seconds): bounded wait/delay in the connector-defined time unit. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| Batch maximum messages<br>batchMaxMessages | number<br>Optional | Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency. |
| Batch maximum bytes<br>batchMaxBytes | number<br>Optional | Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency. |
| Batch delay threshold (ms)<br>batchDelayThresholdMilliseconds | number<br>Optional | Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Published topic name<br>TopicName | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Google or local queued message ID<br>MessageID | string<br>Optional | Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection. |
| Google message ID when confirmation is enabled<br>providerMessageId | string<br>Optional | Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection. |
| Accepted by local publisher<br>queued | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |
| Confirmed by Google<br>deliveryConfirmed | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Activity enqueue latency (ms)<br>publishLatencyMs | number<br>Optional | Measured publish latency in milliseconds. It measures the reported publish stage, not downstream consumer processing. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| GooglePublisherException | The message could not be published. |
| GooglePubSubConnectionException | The Google Pub/Sub connection is unavailable. |
| PublisherInvalidInputException | Publisher requires message data or at least one attribute. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Dynamic Connection

Activities / SAP ECC

## Overview

### *Open or close a stateful SAP session*

Use when multiple RFC/BAPI activities must share one JCo context and an explicit commit/rollback boundary.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: dynamic_connection.

## Configuration

### *Configuration procedure*

Select the SAP resource, connection mode and transactional setting. Bind secrets through properties. On the success path call Dynamic Connection again with terminateConnection=true and commitOnTerminate=true; on the error path terminate without commit.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| Connection type<br>connectionType | select<br>Configuration setting | Connection routing mode: direct application server, message-server logon group, SNC variants or WebSocket configuration. Supply the corresponding shared-connection fields.<br>Choices: dedicated, logongroup, snc, sncwithlogongroup, websocket |
| Transactional<br>transactional | boolean<br>Configuration setting | Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context. |
| Terminate connection<br>terminateConnection | boolean<br>Configuration setting | When true, closes the session identified by sessionID instead of creating a new session. Terminating a transactional session rolls back unless commitOnTerminate is enabled. |
| Commit before termination<br>commitOnTerminate | boolean<br>Configuration setting | Commit the live transactional session before closing it. Enable only on the success path; use rollback termination on the error path.<br>Shown when: (config) => !!config.terminateConnection |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Session idle timeout (ms)<br>contextTimeout | number<br>Configuration setting | Idle lifetime of a dynamic SAP session in milliseconds. Default is 600000, clamped to at least 1000; later calls renew the session lifetime. |
| Timeout (ms)<br>timeout | number<br>Configuration setting | Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
Open: transactional = true
Invoke RFC Input sessionID = ${SAP-Open.sessionID}
Close Input sessionID = ${SAP-Open.sessionID}
Close: terminateConnection = true
Close success only: commitOnTerminate = true
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Session ID (required only when terminating)<br>sessionID | string<br>Optional | Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart. |
| Transaction context timeout in milliseconds<br>contextTimeout | integer<br>Optional | Idle lifetime of a dynamic SAP session in milliseconds. Default is 600000, clamped to at least 1000; later calls renew the session lifetime. |
| dedicated, logongroup, snc, sncwithlogongroup, or websocket<br>connectionType | string<br>Optional | Connection routing mode: direct application server, message-server logon group, SNC variants or WebSocket configuration. Supply the corresponding shared-connection fields. |
| Application server host<br>applicationServerHost | string<br>Optional | Application-server hostname for direct connection mode; must resolve from the runtime host. |
| System number<br>systemNumber | string<br>Optional | SAP instance/system number for a direct application-server connection, normally a two-character string. |
| Message server host<br>messageServerHost | string<br>Optional | Message-server hostname for logon-group mode. |
| SAP system ID<br>systemId | string<br>Optional | SAP system ID used with message-server/logon-group routing. |
| SAP logon group<br>logonGroup | string<br>Optional | SAP logon-group name configured by the SAP administrator. |
| SAP client<br>client | string<br>Optional | SAP client identifier. Keep leading zeros; provide it as a string. |
| SAP user<br>username | string<br>Optional | SAP logon user with authorization for the selected RFCs and metadata APIs. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| SAP password<br>password | string<br>Optional | SAP logon secret. Bind a secret/environment property; do not put credentials into examples, source control or payload logs. |
| Logon language<br>language | string<br>Optional | SAP logon language key. |
| SAProuter string<br>sapRouter | string<br>Optional | Optional SAProuter routing string supplied by the network/SAP administrator. |
| SNC partner name<br>sncPartnerName | string<br>Optional | SNC partner identity expected by the target SAP system. |
| SNC library path<br>sncLibraryPath | string<br>Optional | Native SNC library path on the runtime host, matching OS and architecture. |
| SNC own name<br>sncMyName | string<br>Optional | Optional client SNC identity. |
| SNC quality of protection<br>sncQop | string<br>Optional | SNC quality-of-protection setting accepted by the target system. |
| Transactional<br>transactional | boolean<br>Optional | Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context. |
| Terminate connection<br>terminateConnection | boolean<br>Optional | When true, closes the session identified by sessionID instead of creating a new session. Terminating a transactional session rolls back unless commitOnTerminate is enabled. |
| Commit before termination<br>commitOnTerminate | boolean<br>Optional | Commit the live transactional session before closing it. Enable only on the success path; use rollback termination on the error path. |
| Timeout (ms)<br>timeout | number<br>Optional | Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately. |

## Output

### *Illustrative runtime output*

```
{"sessionID":"<live-session>","connected":true,"transactional":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dynamic SAP session ID<br>sessionID | string<br>Optional | Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart. |
| Connection status<br>connected | boolean<br>Optional | True when Dynamic Connection successfully opens and pings its JCo worker; it does not guarantee later calls will succeed. |
| Transaction mode<br>transactional | boolean<br>Optional | Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

Close returns terminated/committed rather than connected. Sessions are process-local and expire after idle timeout; default is 600000 ms. A failed transactional worker call attempts rollback and removes its session. Ensure every branch closes its session. Do not confuse this with a JDBC transaction group or distributed transaction.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

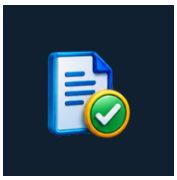
### *Acceptance test*

Acceptance: perform two reversible test changes in one session, first rollback and then commit; verify both outcomes directly in SAP.

| Fault | Description |
| --- | --- |
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# IDoc Acknowledgment

Activities / SAP ECC

## Overview

### *Call an SAP acknowledgement function*

Use only when the SAP team has defined an acknowledgement RFC and its required parameters.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_acknowledgment.

## Configuration

### *Configuration procedure*

Select the SAP resource and status intent. External mode requires functionName, but this activity editor does not currently expose that field. It must be present in resolved configuration/shared settings; otherwise execution fails. Status/successMessage/errorMessage alone do not implement an SAP acknowledgement protocol.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| SAP IDoc status<br>status | text<br>Configuration setting | Desired acknowledgement status metadata. Actual SAP status changes depend on the configured acknowledgement RFC and its parameters. |
| Success message<br>successMessage | text<br>Configuration setting | Editor text for acknowledgement intent; not automatically translated into an SAP RFC parameter by the native adapter. |
| Error message<br>errorMessage | text<br>Configuration setting | Editor error text; explicitly map the required acknowledgement RFC arguments rather than assuming this field changes SAP status. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
Resolved functionName = <approved acknowledgement RFC>
payload = ${input.acknowledgementArguments}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| SAP activity payload<br>payload | object<br>Required | Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape. |
| Dynamic connection session ID<br>sessionID | string<br>Optional | Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart. |
| SAP IDoc status<br>status | string<br>Optional | Desired acknowledgement status metadata. Actual SAP status changes depend on the configured acknowledgement RFC and its parameters. |
| Success message<br>successMessage | string<br>Optional | Editor text for acknowledgement intent; not automatically translated into an SAP RFC parameter by the native adapter. |
| Error message<br>errorMessage | string<br>Optional | Editor error text; explicitly map the required acknowledgement RFC arguments rather than assuming this field changes SAP status. |

## Output

### *Illustrative runtime output*

```
{"acknowledged":true,"status":"<configured-status>","idocNumber":"<supplied-number>"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| SAP response<br>result | object<br>Optional | Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result. |
| Completion status<br>successful | boolean<br>Optional | Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

The runtime invokes the configured RFC with mapped arguments and adds acknowledgement metadata. Check the RFC response before treating business acknowledgement as accepted. This is distinct from broker Confirm Message and JCo listener transport acknowledgement.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

### *Acceptance test*

Acceptance: verify the expected SAP status transition for a test IDoc and a deliberately rejected acknowledgement.

| Fault | Description |
| --- | --- |
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |

| Fault | Description |
| --- | --- |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# IDoc Confirmation

Activities / SAP ECC

## Overview

### *Confirm through a configured SAP RFC*

Use for the confirmation contract explicitly provided by the target SAP system.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_confirmation.

## Configuration

### *Configuration procedure*

Set functionName to the approved confirmation RFC and map its actual parameters. Select the SAP connection. confirmationMode and destination are descriptive/editor settings, not a generic implementation of every confirmation protocol.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| Confirmation mode<br>confirmationMode | select<br>Configuration setting | Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below. |
| Confirmation destination<br>destination | text<br>Configuration setting | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Confirmation RFC<br>functionName | text<br>Configuration setting | Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
functionName = <approved confirmation RFC>
payload = ${input.confirmationArguments}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| SAP activity payload<br>payload | object<br>Required | Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape. |
| Dynamic connection session ID<br>sessionID | string<br>Optional | Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart. |
| Confirmation mode<br>confirmationMode | string<br>Optional | Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below. |
| Confirmation destination<br>destination | string<br>Optional | Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default". |
| Confirmation RFC<br>functionName | string<br>Optional | Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata. |

## Output

### *Illustrative runtime output*

```
{"acknowledged":true,"status":"<configured-status>","idocNumber":"<supplied-number>"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| SAP response<br>result | object<br>Optional | Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result. |
| Completion status<br>successful | boolean<br>Optional | Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

No universal confirmation RFC name or parameter shape is assumed. The runtime requires functionName in external mode; a mock acknowledged result is not proof of a real SAP confirmation.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

### *Acceptance test*

Acceptance: inspect the configured RFC return and SAP document/transaction state, including duplicate confirmation behavior.

| Fault | Description |
| --- | --- |
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# IDoc Converter

Activities / SAP ECC

## Overview

### *Convert an IDoc representation*

Use inside a task to transform supported IDoc source content into the selected representation without posting it.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_converter.

## Configuration

### *Configuration procedure*

Select the saved IDoc basic type/extension metadata. Map the full IDoc source. XML output returns serialized XML; Raw returns the original source. Use IDoc Renderer when transport-ready control/data records are required.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| IDoc type fetched from SAP<br>idocType | idoc<br>Configuration setting | Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions. |
| Extension / CIM type<br>extensionType | text<br>Configuration setting | Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. |
| SAP release<br>release | text<br>Configuration setting | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. |
| IDoc output mode<br>idocOutputMode | select<br>Configuration setting | Parser: JSON or XML. Converter also exposes Raw; RAW returns the original source rather than manufacturing transport-ready EDI records.<br>Choices: XML, Raw |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
IDoc = ${SAP-Listener.payload}
SAPIDoc = ${SAP-Listener.controlRecord}
idocOutputMode = XML
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| SAP IDoc control record<br>SAPIDoc | object<br>Required | For parser input, control metadata or a structured IDoc. Parser output contains one canonical basic-type branch, for example SAPIDoc.ARTMAS05.IDOC. Legacy direct paths remain available for mapping compatibility but are not duplicated in serialized JSON. |
| Raw IDoc<br>IDoc | string\|object<br>Required | Raw IDoc input: XML text, supported structured control/data records or flat IDoc bytes/text. Do not map only one segment when the operation needs the complete document. |
| Extension / CIM type<br>extensionType | string<br>Optional | Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. |
| SAP release<br>release | string<br>Optional | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. |
| IDoc output mode<br>idocOutputMode | string<br>Optional | Parser: JSON or XML. Converter also exposes Raw; RAW returns the original source rather than manufacturing transport-ready EDI records. |

## Output

### *Illustrative runtime output*

```
{"format":"XML","IDocXML":"<IDoc>...</IDoc>","payload":"<IDoc>...</IDoc>","SAPIDoc":{}}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| Parsed IDoc JSON<br>SAPIDoc | object<br>Optional | For parser input, control metadata or a structured IDoc. Parser output contains one canonical basic-type branch, for example SAPIDoc.ARTMAS05.IDOC. Legacy direct paths remain available for mapping compatibility but are not duplicated in serialized JSON. |
| Output format<br>format | string<br>Optional | Output representation selected by the parser/converter: JSON, XML or RAW. Map the accompanying content field appropriate for that format. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound

workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection sessiononly within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

The converter shares the parser implementation. Its generic editor output can differ from the selected mode; XML also includes structured views, RAW preserves the supplied value. This activity is not an outbound SAP call.

| Field / key | Type / required | Details |
|---|---|---|
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

### *Acceptance test*

Acceptance: convert a small fetched-type fixture, then parse it back and compare control fields, segment values and array cardinality.

| Fault | Description |
|---|---|
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# IDoc Listener

Activities / SAP ECC

## Overview

### *Start a flow when an IDoc arrives*

Use as an event starter. Direct mode registers a JCo inbound server; broker mode consumes an already-published IDoc.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_listener.

## Configuration

### *Configuration procedure*

Configure SAP program ID/gateway and partner/RFC routing, fetch and save IDoc metadata, then run the task. NoMessaging uses direct JCo. For EMS/JMS/Kafka select the additional broker resource and destination; MINA does not automatically create the upstream SAP-to-broker bridge.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| Messaging source<br>messagingSource | select<br>Configuration setting | NoMessaging receives directly through JCo. EMS/JMS/Kafka modes consume an IDoc carried by a broker; an upstream SAP-to-broker producer is still required.<br>Choices: NoMessaging, EMS, JMS, Kafka |
| Messaging shared connection<br>messagingResourceId | resource<br>Configuration setting | Shared broker connection matching the chosen messaging source, in addition to the SAP metadata connection.<br>Shown when: (config) => !["", "NoMessaging"].includes(config.messagingSource \|\| "NoMessaging")<br>Connection types: |
| Queue or topic<br>messagingDestination | text<br>Configuration setting | Queue or topic carrying the complete IDoc document.<br>Shown when: (config) => !["", "NoMessaging"].includes(config.messagingSource \|\| "NoMessaging") |
| SAP TID Manager resource<br>tidManagerId | text<br>Configuration setting | Shared SAP transaction-ID store selection. Durable TID state supports duplicate detection; it does not make downstream external writes atomic.<br>Read-only |

| Field / key | Type / required | Details |
| --- | --- | --- |
| IDoc type fetched from SAP<br>idocType | idoc<br>Configuration setting | Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions. |
| Extension / CIM type<br>extensionType | text<br>Configuration setting | Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.<br>Read-only |
| SAP release<br>release | text<br>Configuration setting | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.<br>Read-only |
| Program ID<br>programId | text<br>Configuration setting | Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions.<br>Read-only |
| Invocation protocol<br>invocationProtocol | text<br>Configuration setting | Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode.<br>Read-only |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
messagingSource = NoMessaging
IDoc Parser IDoc = ${SAP-Listener.payload}
IDoc Parser SAPIDoc = ${SAP-Listener.controlRecord}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Messaging source<br>messagingSource | string<br>Optional | NoMessaging receives directly through JCo. EMS/JMS/Kafka modes consume an IDoc carried by a broker; an upstream SAP-to-broker producer is still required. |
| Queue or topic<br>messagingDestination | string<br>Optional | Queue or topic carrying the complete IDoc document. |
| SAP TID Manager resource<br>tidManagerId | string<br>Optional | Shared SAP transaction-ID store selection. Durable TID state supports duplicate detection; it does not make downstream external writes atomic. |
| Extension / CIM type<br>extensionType | string<br>Optional | Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. |
| SAP release<br>release | string<br>Optional | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. |
| Program ID<br>programId | string<br>Optional | Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions. |
| Invocation protocol<br>invocationProtocol | string<br>Optional | Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode. |

## Output

### *Illustrative runtime output*

```
{"SAPIDoc":{},"controlRecord":{"IDOCTYP":"<basic-type>"},"payload":"<IDoc>...</IDoc>","TID":"<transacti
on>"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| IDoc/RFC metadata<br>SAPIDoc | object<br>Optional | For parser input, control metadata or a structured IDoc. Parser output contains one canonical basic-type branch, for example SAPIDoc.ARTMAS05.IDOC. Legacy direct paths remain available for mapping compatibility but are not duplicated in serialized JSON. |
| SAP IDoc control record<br>controlRecord | object<br>Optional | EDI_DC40 control metadata describing sender, receiver, message/basic type and extension. Keep it with the associated data records. |
| Inbound SAP payload (XML)<br>payload | string<br>Optional | Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape. |
| Transaction ID<br>TID | string<br>Optional | SAP transaction identifier when provided by the transport. It is not the IDoc business document number and is not a JMS acknowledgement token. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

Direct delivery uses durable TID handling and listener completion; broker modes inherit the broker path acknowledgement limitations. Metadata arrays follow the saved schema. Do not acknowledge transport success before irreversible downstream work unless your recovery design tolerates it. Debug breakpoints can hold requests until the listener timeout.

| Field / key | Type / required | Details |
|---|---|---|
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

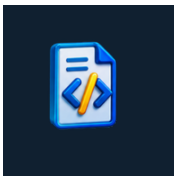
### *Acceptance test*

Acceptance: send a test IDoc while Run is active, confirm automatic execution without Step Into, then test duplicate TID, downstream error, stop/restart and redelivery.

| Fault | Description |
|---|---|
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# IDoc Parser

Activities / SAP ECC

## Overview

### *Decode an IDoc into mapped fields*

Use after a listener, file read or broker receive to expose control and segment fields in JSON/XML.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_parser.

## Configuration

### *Configuration procedure*

Fetch/save the matching basic type, extension and release. Choose JSON for field mappings or XML for serialized output. For flat data set the actual encoding/framing; keep strict invalid-byte handling unless replacement is intentional.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| IDoc source destination<br>sourceDestination | text<br>Configuration setting | Editor metadata describing the IDoc source. The native parser/reader does not use this value as a separate network destination; select the actual shared resource. |
| IDoc type fetched from SAP<br>idocType | idoc<br>Configuration setting | Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions. |
| Extension / CIM type<br>extensionType | text<br>Configuration setting | Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. |
| SAP release<br>release | text<br>Configuration setting | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. |
| Flat IDoc encoding<br>idocEncoding | text<br>Configuration setting | Character set used to decode flat records and SDATA byte offsets, for example UTF-8 or windows-1252. Must match the actual sender encoding. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Flat record delimiter<br>idocRecordDelimiter | text<br>Configuration setting | Explicit escaped record separator for flat input, such as \r\n. Use fixed-width/custom framing when segment data itself contains line breaks. |
| Invalid byte policy<br>invalidCharacterPolicy | select<br>Configuration setting | strict rejects invalid byte sequences; replace substitutes invalid characters and can lose data. Use strict when data fidelity is required.<br>Choices: strict, replace |
| Parser engine<br>parserEngine | select<br>Configuration setting | Built-in parses XML, structured and supported flat records. The optional licensed IDoc library path validates XML; it does not implement flat-byte parsing.<br>Choices: Built-in, SAP JIDocLib (when installed) |
| Reject mismatched IDoc type or extension<br>validateIdocType | boolean<br>Configuration setting | Checks available control-record basic type and configured extension against saved metadata. Missing metadata is not proof the payload was validated. |
| IDoc output format<br>idocOutputMode | select<br>Configuration setting | Parser: JSON or XML. Converter also exposes Raw; RAW returns the original source rather than manufacturing transport-ready EDI records.<br>Choices: JSON, XML |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
IDoc = ${SAP-Listener.payload}
SAPIDoc = ${SAP-Listener.controlRecord}
idocOutputMode = JSON
Next message = ${SAP-Parser.SAPIDoc}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
| --- | --- | --- |
| SAP IDoc control record<br>SAPIDoc | object<br>Required | For parser input, control metadata or a structured IDoc. Parser output contains one canonical basic-type branch, for example SAPIDoc.ARTMAS05.IDOC. Legacy direct paths remain available for mapping compatibility but are not duplicated in serialized JSON. |
| Raw IDoc<br>IDoc | string\|object<br>Required | Raw IDoc input: XML text, supported structured control/data records or flat IDoc bytes/text. Do not map only one segment when the operation needs the complete document. |
| IDoc source destination<br>sourceDestination | string<br>Optional | Editor metadata describing the IDoc source. The native parser/reader does not use this value as a separate network destination; select the actual shared resource. |
| Extension / CIM type<br>extensionType | string<br>Optional | Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. |
| SAP release<br>release | string<br>Optional | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Flat IDoc encoding<br>idocEncoding | string<br>Optional | Character set used to decode flat records and SDATA byte offsets, for example UTF-8 or windows-1252. Must match the actual sender encoding. |
| Flat record delimiter<br>idocRecordDelimiter | string<br>Optional | Explicit escaped record separator for flat input, such as \r\n. Use fixed-width/custom framing when segment data itself contains line breaks. |
| Invalid byte policy<br>invalidCharacterPolicy | string<br>Optional | strict rejects invalid byte sequences; replace substitutes invalid characters and can lose data. Use strict when data fidelity is required. |
| Parser engine<br>parserEngine | string<br>Optional | Built-in parses XML, structured and supported flat records. The optional licensed IDoc library path validates XML; it does not implement flat-byte parsing. |
| Reject mismatched IDoc type or extension<br>validateIdocType | boolean<br>Optional | Checks available control-record basic type and configured extension against saved metadata. Missing metadata is not proof the payload was validated. |
| IDoc output format<br>idocOutputMode | string<br>Optional | Parser: JSON or XML. Converter also exposes Raw; RAW returns the original source rather than manufacturing transport-ready EDI records. |

## Output

### *Illustrative runtime output*

```
{"format":"JSON","idocType":"<basic-type>","SAPIDoc":{"IDOC":{"EDI_DC40":{},"<repeating-segment>":[{"<field>":"value"}]}}}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Parsed IDoc JSON<br>SAPIDoc | object<br>Optional | For parser input, control metadata or a structured IDoc. Parser output contains one canonical basic-type branch, for example SAPIDoc.ARTMAS05.IDOC. Legacy direct paths remain available for mapping compatibility but are not duplicated in serialized JSON. |
| Output format<br>format | string<br>Optional | Output representation selected by the parser/converter: JSON, XML or RAW. Map the accompanying content field appropriate for that format. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

Illustrative shape only: actual paths come from your fetched schema. Repeating segments remain arrays even for one occurrence; optional absent fields are not minaated. The runtime also exposes the basic-type-named branch. The licensed IDoc-library option validates XML only; flat records require Built-in. Missing metadata can prevent SDATA expansion, and parser memory grows with document size.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

### *Acceptance test*

Acceptance: test one, multiple and absent repeating segments; malformed XML; mismatched type/extension; non-UTF-8 bytes; and segment data containing line breaks.

| Fault | Description |
| --- | --- |
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# IDoc Reader

Activities / SAP ECC

## Overview

### *Dispatch an IDoc using queued RFC*

Despite its label, the current runtime converts input to control/data tables and calls the inbound-IDoc RFC. It does not fetch an existing IDoc by document number.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_reader.

## Configuration

### *Configuration procedure*

Select metadata and SAP connection, provide a complete input IDoc, and configure queueName in resolved settings for qRFC ordering. Reader defaults to qRFC; the editor does not expose every dispatch setting.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| IDoc source destination<br>sourceDestination | text<br>Configuration setting | Editor metadata describing the IDoc source. The native parser/reader does not use this value as a separate network destination; select the actual shared resource. |
| IDoc type fetched from SAP<br>idocType | idoc<br>Configuration setting | Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions. |
| Extension / CIM type<br>extensionType | text<br>Configuration setting | Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. |
| SAP release<br>release | text<br>Configuration setting | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. |
| Confirmation mode<br>confirmationMode | text<br>Configuration setting | Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
payload = ${SAP-Renderer.rawIDoc}
Resolved queueName = ${properties.sap.outboundQueue}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| SAP activity payload<br>payload | object<br>Required | Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape. |
| Dynamic connection session ID<br>sessionID | string<br>Optional | Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart. |
| IDoc source destination<br>sourceDestination | string<br>Optional | Editor metadata describing the IDoc source. The native parser/reader does not use this value as a separate network destination; select the actual shared resource. |
| Extension / CIM type<br>extensionType | string<br>Optional | Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. |
| SAP release<br>release | string<br>Optional | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. |
| Confirmation mode<br>confirmationMode | string<br>Optional | Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below. |

## Output

### *Illustrative runtime output*

```
{"TID":"<transaction>","transactional":true,"transactionProtocol":"qrfc","confirmed":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| SAP response<br>result | object<br>Optional | Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result. |
| Completion status<br>successful | boolean<br>Optional | Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

Without queueName the bridge falls back to a transactional execution overload without the intended queue name. Treat this as a dispatch activity, not historical retrieval. Both EDI_DC40 control data and at least one EDI_DD40 segment are required.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

### *Acceptance test*

Acceptance: verify the queued outbound dispatch and ordering in SAP; do not test it against production expecting a read-only operation.

| Fault | Description |
| --- | --- |
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# Post IDoc to SAP

Activities / SAP ECC

## Overview

### *Post a business IDoc to SAP*

Use inside a flow to dispatch a complete IDoc using the selected SAP metadata and transport protocol.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: post_idoc.

## Configuration

### *Configuration procedure*

Select SAP resource and saved basic type/extension. Map XML, structured IDoc or rendered control/data envelope. Select tRFC (default) or qRFC and supply queueName for ordered dispatch.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| IDoc type fetched from SAP<br>idocType | idoc<br>Configuration setting | Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions. |
| Extension / CIM type<br>extensionType | text<br>Configuration setting | Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. |
| SAP release<br>release | text<br>Configuration setting | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. |
| IDoc delivery protocol<br>idocInputMode | select<br>Configuration setting | IDoc delivery protocol. Post defaults to tRFC; Reader defaults to qRFC. qRFC requires a real queue name for ordered dispatch.<br>Choices: tRFC, qRFC |
| IDoc payload format<br>inputFormat | select<br>Configuration setting | Describes the intended XML or raw input for posting; the native adapter converts the actual input using saved metadata.<br>Choices: XML, Raw |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Confirmation mode<br>confirmationMode | text<br>Configuration setting | Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below. |
| SAP queue name<br>queueName | text<br>Configuration setting | SAP outbound qRFC queue name. Without a queue name the bridge cannot establish the intended ordered queue execution. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
payload = ${SAP-Renderer.rawIDoc}
idocInputMode = tRFC
For qRFC: queueName = ${properties.sap.outboundQueue}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
| --- | --- | --- |
| SAP activity payload<br>payload | object<br>Required | Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape. |
| Dynamic connection session ID<br>sessionID | string<br>Optional | Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart. |
| Extension / CIM type<br>extensionType | string<br>Optional | Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. |
| SAP release<br>release | string<br>Optional | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. |
| IDoc delivery protocol<br>idocInputMode | string<br>Optional | IDoc delivery protocol. Post defaults to tRFC; Reader defaults to qRFC. qRFC requires a real queue name for ordered dispatch. |
| IDoc payload format<br>inputFormat | string<br>Optional | Describes the intended XML or raw input for posting; the native adapter converts the actual input using saved metadata. |
| Confirmation mode<br>confirmationMode | string<br>Optional | Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below. |
| SAP queue name<br>queueName | string<br>Optional | SAP outbound qRFC queue name. Without a queue name the bridge cannot establish the intended ordered queue execution. |

## Output

### *Illustrative runtime output*

```
{"TID":"<transaction>","transactional":true,"transactionProtocol":"trfc","confirmed":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
| --- | --- | --- |
| SAP response<br>result | object<br>Optional | Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result. |
| Completion status<br>successful | boolean<br>Optional | Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

The adapter builds IDOC_CONTROL_REC_40 and IDOC_DATA_REC_40 table parameters and calls IDOC_INBOUND_ASYNCHRONOUS unless functionName overrides it. confirmed indicates transport confirmation, not final SAP business posting status. Retrying can create another transport transaction; use a business-level duplicate strategy.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

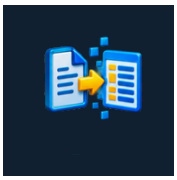
### *Acceptance test*

Acceptance: inspect SAP inbound document status as well as transport success; test a rejected business document and ambiguous network failure.

| Fault | Description |
| --- | --- |
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# IDoc Renderer

Activities / SAP ECC

## Overview

### *Build transport-ready control/data records*

Use to turn structured/XML IDoc data into the control/data envelope used by Post IDoc or Reader.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_renderer.

## Configuration

### *Configuration procedure*

Select the exact metadata and map the complete document. The saved field offsets, lengths and encoding govern SDATA construction. This operation renders locally; it does not post to SAP.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| IDoc type fetched from SAP<br>idocType | idoc<br>Configuration setting | Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions. |
| Extension / CIM type<br>extensionType | text<br>Configuration setting | Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. |
| SAP release<br>release | text<br>Configuration setting | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. |
| IDoc delivery protocol<br>idocInputMode | select<br>Configuration setting | IDoc delivery protocol. Post defaults to tRFC; Reader defaults to qRFC. qRFC requires a real queue name for ordered dispatch.<br>Choices: tRFC, qRFC |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
payload = ${SAP-Parser.SAPIDoc}
Post IDoc payload = ${SAP-Renderer.rawIDoc}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| SAP activity payload<br>payload | object<br>Required | Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape. |
| Dynamic connection session ID<br>sessionID | string<br>Optional | Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart. |
| Extension / CIM type<br>extensionType | string<br>Optional | Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. |
| SAP release<br>release | string<br>Optional | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. |
| IDoc delivery protocol<br>idocInputMode | string<br>Optional | IDoc delivery protocol. Post defaults to tRFC; Reader defaults to qRFC. qRFC requires a real queue name for ordered dispatch. |

## Output

### *Illustrative runtime output*

```
{"rawIDoc":{"control":{"IDOCTYP":"<basic-type>"},"data":[{"SEGNAM":"<segment>","SDATA":"<encoded-fields
>"}]},"format":"RAW"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| SAP response<br>result | object<br>Optional | Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result. |
| Completion status<br>successful | boolean<br>Optional | Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

rawIDoc is a structured control/data envelope, not necessarily a flat text file. The base editor advertises generic result/successful fields; use actual rawIDoc output. Ensure input fits the fetched field lengths and test encoding before dispatch.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

### *Acceptance test*

Acceptance: render and parse back a fixture with repeating segments, blank fields and non-ASCII text; compare every business field.

| Fault | Description |
| --- | --- |
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# RFC BAPI Listener

Activities / SAP ECC

## Overview

### *Start a flow for an inbound RFC/BAPI call*

Use as an event starter registered with the SAP gateway for a selected function.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: rfc_bapi_listener.

## Configuration

### *Configuration procedure*

Set functionName, programId and gateway settings. Provision the matching SAP destination and registration rules. Bound maximumConnections/maxPendingEvents and choose a reply timeout long enough for the flow. Fetch the function metadata.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| RFC / BAPI name<br>functionName | text<br>Configuration setting | Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata. |
| Invocation protocol<br>invocationProtocol | select<br>Configuration setting | Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode.<br>Choices: Request/Reply, tRFC, qRFC, bgRFC |
| Program ID<br>programId | text<br>Configuration setting | Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions. |
| Gateway host override<br>gatewayHost | text<br>Configuration setting | SAP gateway hostname override for inbound RFC registration. |
| Gateway service override<br>gatewayService | text<br>Configuration setting | Gateway service/port override for inbound RFC registration. |
| Maximum concurrent requests<br>maximumConnections | number<br>Configuration setting | Listener connection/concurrency limit. Size it with SAP gateway capacity and downstream processing, not just CPU count. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Maximum buffered requests<br>maxPendingEvents | number<br>Configuration setting | Bound on buffered listener events; prevents unbounded in-memory accumulation while downstream work is slow. |
| Reply / acknowledgement timeout (seconds)<br>ackTimeoutSeconds | number<br>Configuration setting | Maximum listener reply/acknowledgement wait in seconds. Set it above expected flow processing time and test timeout/redelivery behavior. |
| SAP TIDManager resource<br>tidManagerId | resource<br>Configuration setting | Shared SAP transaction-ID store selection. Durable TID state supports duplicate detection; it does not make downstream external writes atomic.<br>Connection type: sap_tid |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
Downstream input = ${SAP-RFC-Listener.imports}
Downstream tables = ${SAP-RFC-Listener.tables}
End response with Reply from RFC BAPI in SAP
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
| --- | --- | --- |
| RFC / BAPI name<br>functionName | string<br>Optional | Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata. |
| Invocation protocol<br>invocationProtocol | string<br>Optional | Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode. |
| Program ID<br>programId | string<br>Optional | Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions. |
| Gateway host override<br>gatewayHost | string<br>Optional | SAP gateway hostname override for inbound RFC registration. |
| Gateway service override<br>gatewayService | string<br>Optional | Gateway service/port override for inbound RFC registration. |
| Maximum concurrent requests<br>maximumConnections | number<br>Optional | Listener connection/concurrency limit. Size it with SAP gateway capacity and downstream processing, not just CPU count. |
| Maximum buffered requests<br>maxPendingEvents | number<br>Optional | Bound on buffered listener events; prevents unbounded in-memory accumulation while downstream work is slow. |
| Reply / acknowledgement timeout (seconds)<br>ackTimeoutSeconds | number<br>Optional | Maximum listener reply/acknowledgement wait in seconds. Set it above expected flow processing time and test timeout/redelivery behavior. |

## Output

### *Illustrative runtime output*

```
{"RfcRequest":{},"imports":{"<parameter>":"value"},"tables":{},"functionName":"<RFC>","TID":"<when-present>"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| RFC/BAPI request<br>RfcRequest | object<br>Optional | Inbound RFC request envelope. Parameter shapes follow the selected SAP function metadata. |
| RFC import parameters<br>imports | object<br>Optional | Inbound RFC import parameter object. |
| RFC table parameters<br>tables | object<br>Optional | Inbound RFC table parameter arrays. |
| Invoked RFC/BAPI<br>functionName | string<br>Optional | Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata. |
| Invocation protocol<br>invocationProtocol | string<br>Optional | Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode. |
| Transaction ID<br>TID | string<br>Optional | SAP transaction identifier when provided by the transport. It is not the IDoc business document number and is not a JMS acknowledgement token. |
| SAP connection ID<br>CPIC_ID | string<br>Optional | Connection identifier supplied by SAP transport when available. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

A request/reply caller waits for the flow response. Reply must execute within the originating transport context. bgRFC appears in the editor but there is no dedicated bgRFC native dispatch implementation to claim as supported.

| Field / key | Type / required | Details |
|---|---|---|
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension,

release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state beforeretrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the fullmessage and trace.

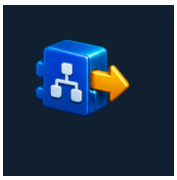
### *Acceptance test*

Acceptance: invoke from a test SAP caller, inspect imports/tables, return a known value and test timeout/error propagation and concurrent callers.

| Fault | Description |
| --- | --- |
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# Invoke RFC BAPI in SAP

Activities / SAP ECC

## Overview

### *Call a remote-enabled SAP function*

Use inside a flow to pass import/changing/table parameters to a remote-enabled function or BAPI.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: invoke_rfc_bapi.

## Configuration

### *Configuration procedure*

Select SAP resource and exact functionName. Match the fetched parameter types. For a multi-step logical unit of work map sessionID from Dynamic Connection and close the context explicitly.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| RFC / BAPI name<br>functionName | text<br>Configuration setting | Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata. |
| Invocation protocol<br>invocationProtocol | select<br>Configuration setting | Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode.<br>Choices: Request/Reply, tRFC, qRFC, bgRFC |
| SAP queue name<br>queueName | text<br>Configuration setting | SAP outbound qRFC queue name. Without a queue name the bridge cannot establish the intended ordered queue execution. |
| Transactional<br>transactional | boolean<br>Configuration setting | Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context. |
| Auto commit<br>autoCommit | boolean<br>Configuration setting | Requests BAPI_TRANSACTION_COMMIT for an eligible call. Calls inside an explicit dynamic context suppress per-activity commits until the context boundary. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Context end<br>contextEnd | boolean<br>Configuration setting | Ends the dynamic context after this call; autoCommit controls whether that explicit boundary commits. Do not reuse the session afterwards. |
| Timeout (ms)<br>timeout | number<br>Configuration setting | Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
functionName = <authorized remote function>
importParameters = ${input.rfcImports}
changingParameters = ${input.rfcChanging}
tableParameters = ${input.rfcTables}
sessionID = ${SAP-Open.sessionID}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
| --- | --- | --- |
| RFC import parameters<br>importParameters | object<br>Optional | Object keyed by exact RFC import parameter names; nested structures are flattened into named JCo parameters. |
| RFC changing parameters<br>changingParameters | object<br>Optional | Object keyed by RFC changing parameter names. Values are sent through the JCo changing-parameter list. |
| RFC table parameters<br>tableParameters | object<br>Optional | Object whose keys are RFC table parameter names and whose values are arrays of row objects, including single-row tables. |
| Dynamic connection session ID<br>sessionID | string<br>Optional | Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart. |
| RFC / BAPI name<br>functionName | string<br>Optional | Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata. |
| Invocation protocol<br>invocationProtocol | string<br>Optional | Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode. |
| SAP queue name<br>queueName | string<br>Optional | SAP outbound qRFC queue name. Without a queue name the bridge cannot establish the intended ordered queue execution. |
| Transactional<br>transactional | boolean<br>Optional | Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context. |
| Auto commit<br>autoCommit | boolean<br>Optional | Requests BAPI_TRANSACTION_COMMIT for an eligible call. Calls inside an explicit dynamic context suppress per-activity commits until the context boundary. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Context end<br>contextEnd | boolean<br>Optional | Ends the dynamic context after this call; autoCommit controls whether that explicit boundary commits. Do not reuse the session afterwards. |
| Timeout (ms)<br>timeout | number<br>Optional | Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately. |

## Output

### *Illustrative runtime output*

```
{"exports":{"<parameter>":"value"},"changing":{},"tables":{"RETURN":[{"TYPE":"S","MESSAGE":"<message>"}]
}}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
| --- | --- | --- |
| SAP response<br>result | object<br>Optional | Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result. |
| Completion status<br>successful | boolean<br>Optional | Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

RETURN is illustrative: not every function has that table. The runtime returns native exports/changing/tables rather than wrapping every result in result. Check business error fields explicitly. bgRFC is not a dedicated supported native path; worker timeout uses timeoutSeconds, so the editor timeout field alone should not be treated as a verified per-call deadline.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

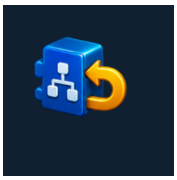
### *Acceptance test*

Acceptance: test scalar, nested and table parameters, business error returns, rollback/commit boundaries and a disconnected server.

| Fault | Description |
| --- | --- |
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# Reply from RFC BAPI in SAP

Activities / SAP ECC

## Overview

### *Complete the originating RFC request*

Use in a flow started by RFC BAPI Listener to send its output parameters back to the waiting SAP caller.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: reply_rfc_bapi.

## Configuration

### *Configuration procedure*

Map the response to the function output contract. The engine requires the original deliveryId/listenerKey transport context; functionName/listenerActivity alone cannot reply to a request from another job.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| RFC / BAPI name<br>functionName | text<br>Configuration setting | Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata. |
| RFC BAPI Listener activity<br>listenerActivity | text<br>Configuration setting | Design-time reference to the listener. Runtime reply routing uses the current listener transport context, not an arbitrary activity name. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
payload = ${input.rfcResponse}
Illustrative response: {"exports":{"RESULT":"OK"},"tables":{}}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
| --- | --- | --- |
| SAP activity payload<br>payload | object<br>Required | Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape. |
| Dynamic connection session ID<br>sessionID | string<br>Optional | Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart. |
| RFC / BAPI name<br>functionName | string<br>Optional | Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata. |
| RFC BAPI Listener activity<br>listenerActivity | string<br>Optional | Design-time reference to the listener. Runtime reply routing uses the current listener transport context, not an arbitrary activity name. |

## Output

### *Illustrative runtime output*

```
{"replied":true,"functionName":"<RFC>","response":{"exports":{"RESULT":"OK"}}}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
| --- | --- | --- |
| SAP response<br>result | object<br>Optional | Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result. |
| Completion status<br>successful | boolean<br>Optional | Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

Running this activity alone in external mode fails without a live listener context. Mock replied=true does not establish an SAP response. Reply within the configured timeout and avoid sending a second response on another branch.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

### *Acceptance test*

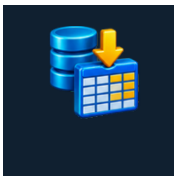
Acceptance: verify the exact output in the waiting SAP caller, then test a late reply and a reply outside a listener flow.

| Fault | Description |
| --- | --- |
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |

| Fault | Description |
| --- | --- |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# Read Table

Activities / SAP ECC

## Overview

### *Read bounded SAP table rows*

Use for an authorized RFC_READ_TABLE query, not unrestricted SQL or a bulk replication engine.

### *Prerequisites*

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: read_table.

## Configuration

### *Configuration procedure*

Select tableName, explicit fields, ABAP WHERE clauses, delimiter and a bounded rowCount. Use rowSkip for paging and avoid changing source data during paging when consistency matters.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| SAP ECC shared connection<br>resourceId | resource<br>Configuration setting | Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.<br>Connection type: sap |
| SAP table / view<br>tableName | text<br>Configuration setting | Authorized SAP table or view read through RFC_READ_TABLE. This is not a general-purpose database connection. |
| Fields (comma separated)<br>fields | text<br>Configuration setting | For Read Table input, a field-name array or configured comma-separated list. Output fields contains SAP field metadata, not row data. |
| ABAP WHERE clauses<br>where | text<br>Configuration setting | ABAP filter clauses, not SQL. Dynamic input is an array of clause strings. Treat user-supplied values as untrusted and validate them before constructing filters. |
| Rows to skip<br>rowSkip | number<br>Configuration setting | Rows skipped by RFC_READ_TABLE, used with a bounded rowCount for paging. |
| Maximum rows<br>rowCount | number<br>Configuration setting | Maximum rows requested; zero delegates an unbounded read to SAP. Prefer an explicit limit for memory and operational safety. |
| Delimiter<br>delimiter | text<br>Configuration setting | Separator used in RFC_READ_TABLE text rows; choose one absent from column values because the adapter splits by this separator. |

## Input

### *Mapping example*

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
tableName = <authorized-table>
fields = ["<key-field>", "<value-field>"]
where = ["<validated ABAP filter>"]
rowCount = 100
rowSkip = 0
delimiter = |
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| Dynamic filter clauses<br>where | array<br>Optional | ABAP filter clauses, not SQL. Dynamic input is an array of clause strings. Treat user-supplied values as untrusted and validate them before constructing filters. |
| Dynamic field list<br>fields | array<br>Optional | For Read Table input, a field-name array or configured comma-separated list. Output fields contains SAP field metadata, not row data. |
| SAP table / view<br>tableName | string<br>Optional | Authorized SAP table or view read through RFC_READ_TABLE. This is not a general-purpose database connection. |
| Rows to skip<br>rowSkip | number<br>Optional | Rows skipped by RFC_READ_TABLE, used with a bounded rowCount for paging. |
| Maximum rows<br>rowCount | number<br>Optional | Maximum rows requested; zero delegates an unbounded read to SAP. Prefer an explicit limit for memory and operational safety. |
| Delimiter<br>delimiter | string<br>Optional | Separator used in RFC_READ_TABLE text rows; choose one absent from column values because the adapter splits by this separator. |

## Output

### *Illustrative runtime output*

```
{"rows":[{"<key-field>":"value","<value-field>":"text"}],"fields":[],"rowCount":1,"tableName":"<table>"
}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

| Field / key | Type / required | Details |
|---|---|---|
| SAP table rows<br>rows | array<br>Optional | Array of row objects. SAP Read Table returns decoded text values; convert numeric/date columns explicitly before arithmetic or comparison. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Field metadata<br>fields | array<br>Optional | For Read Table input, a field-name array or configured comma-separated list. Output fields contains SAP field metadata, not row data. |

## Advanced

### *Delivery, transactions and performance*

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

### *Implemented behavior and limitations*

The adapter splits DATA.WA by the configured delimiter and returns strings. Embedded delimiters can split values incorrectly. SAP row-width/function restrictions still apply; rowCount=0 is potentially unbounded. A successful connection does not imply table authorization.

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

### *Troubleshooting procedure*

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

### *Acceptance test*

Acceptance: test a small known result, no matches, paging, authorization failure and values containing the delimiter.

| Fault | Description |
| --- | --- |
| SAP_CONNECTION | The SAP JCo/RFC connection could not be established. |
| SAP_RFC | SAP returned an RFC or BAPI exception. |
| SAP_IDOC | The IDoc could not be parsed, posted, confirmed, or acknowledged. |
| SAP_TRANSACTION | The tRFC, qRFC, bgRFC, or transactional context failed. |
| SAP_SCHEMA | The downloaded RFC, table, or IDoc metadata is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs



# JDBC Create

Activities / JDBC

## Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: insert.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| JDBC shared connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: jdbc |
| Timeout (seconds)<br>timeout | number<br>Configuration setting | SQL execution timeout in seconds passed to the selected database driver. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Batch update<br>batchUpdate | boolean<br>Configuration setting | Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete. |
| Allow mapped SQL override<br>overrideSqlStatement | boolean<br>Configuration setting | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
|---|---|---|
| Database server time zone<br>ServerTimeZone | string<br>Optional | Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dynamic timeout<br>timeout | number<br>Optional | SQL execution timeout in seconds passed to the selected database driver. |
| Prepared statement parameters<br>parameters | object<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Batch parameter records<br>records | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Dynamic update SQL<br>SqlUpdateStatement | string<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Batch update<br>batchUpdate | boolean<br>Optional | Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete. |
| Allow mapped SQL override<br>overrideSqlStatement | boolean<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |

## Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Number of updates<br>noOfUpdates | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Affected rows<br>rowCount | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Generated key<br>lastInsertId | integer\|string<br>Optional | Generated identity reported after an insert when supported by the database driver. Do not assume every provider supplies it. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| JDBCConnectionNotFoundException | The selected JDBC shared connection is unavailable. |
| InvalidTimeZoneException | The database server time zone is invalid. |
| JDBCSQLException | The database rejected the SQL or procedure call. |
| LoginTimedOutException | The database login timed out. |
| InvalidSQLTypeException | A prepared parameter type does not match its database column. |
| DuplicatedFieldNameException | The query result contains an unsupported duplicate field name. |
| ActivityTimedOutException | The configured activity timeout was reached. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# JDBC Update

Activities / JDBC

## Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: update.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| JDBC shared connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: jdbc |
| Timeout (seconds)<br>timeout | number<br>Configuration setting | SQL execution timeout in seconds passed to the selected database driver. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Insert if record does not exist<br>insertIfRecordDoesNotExist | boolean<br>Configuration setting | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Batch update<br>batchUpdate | boolean<br>Configuration setting | Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete. |
| Allow mapped SQL override<br>overrideSqlStatement | boolean<br>Configuration setting | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Database server time zone<br>ServerTimeZone | string<br>Optional | Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems. |
| Dynamic timeout<br>timeout | number<br>Optional | SQL execution timeout in seconds passed to the selected database driver. |
| Prepared statement parameters<br>parameters | object<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Batch parameter records<br>records | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Dynamic update SQL<br>SqlUpdateStatement | string<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |
| Insert-if-missing SQL<br>InsertStatement | string<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Insert if record does not exist<br>insertIfRecordDoesNotExist | boolean<br>Optional | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Batch update<br>batchUpdate | boolean<br>Optional | Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete. |
| Allow mapped SQL override<br>overrideSqlStatement | boolean<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |

## Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Number of updates<br>noOfUpdates | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Affected rows<br>rowCount | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Generated key<br>lastInsertId | integer\|string<br>Optional | Generated identity reported after an insert when supported by the database driver. Do not assume every provider supplies it. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| JDBCConnectionNotFoundException | The selected JDBC shared connection is unavailable. |
| InvalidTimeZoneException | The database server time zone is invalid. |
| JDBCSQLException | The database rejected the SQL or procedure call. |
| LoginTimedOutException | The database login timed out. |
| InvalidSQLTypeException | A prepared parameter type does not match its database column. |
| DuplicatedFieldNameException | The query result contains an unsupported duplicate field name. |
| ActivityTimedOutException | The configured activity timeout was reached. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# JDBC Query

Activities / JDBC

## Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: query.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| JDBC shared connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: jdbc |
| Timeout (seconds)<br>timeout | number<br>Configuration setting | SQL execution timeout in seconds passed to the selected database driver. |
| Maximum rows (0 retrieves all)<br>maxRows | number<br>Configuration setting | Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Use nil for SQL NULL<br>useNil | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Interpret invalid XML characters<br>interpretInvalidXmlChars | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Process result in subsets<br>processInSubsets | boolean<br>Configuration setting | Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached. |
| Subset size<br>subsetSize | number<br>Configuration setting | Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached. |
| Allow mapped SQL override<br>overrideSqlStatement | boolean<br>Configuration setting | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Database server time zone<br>ServerTimeZone | string<br>Optional | Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems. |
| Dynamic timeout<br>timeout | number<br>Optional | SQL execution timeout in seconds passed to the selected database driver. |
| Dynamic maximum rows<br>maxRows | number<br>Optional | Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available. |
| Dynamic SQL override<br>SqlStatement | string<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |
| Prepared statement parameters<br>parameters | object<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Row offset when processing subsets<br>rowOffset | integer<br>Optional | Start at row offset zero. For subsequent pages, map the previous nextRowOffset to rowOffset. Use a stable ORDER BY and stop when lastSubset is true. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Use nil for SQL NULL<br>useNil | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Interpret invalid XML characters<br>interpretInvalidXmlChars | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Process result in subsets<br>processInSubsets | boolean<br>Optional | Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached. |
| Subset size<br>subsetSize | number<br>Optional | Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached. |
| Allow mapped SQL override<br>overrideSqlStatement | boolean<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |

## Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JDBC result set<br>resultSet | object<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Result records<br>resultSet.Record | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Result records<br>rows | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Rows returned<br>rowCount | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Last result subset<br>lastSubset | boolean<br>Optional | Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached. |
| Offset for the next subset, when available<br>nextRowOffset | integer<br>Optional | Start at row offset zero. For subsequent pages, map the previous nextRowOffset to rowOffset. Use a stable ORDER BY and stop when lastSubset is true. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| JDBCConnectionNotFoundException | The selected JDBC shared connection is unavailable. |
| InvalidTimeZoneException | The database server time zone is invalid. |
| JDBCSQLException | The database rejected the SQL or procedure call. |
| LoginTimedOutException | The database login timed out. |
| InvalidSQLTypeException | A prepared parameter type does not match its database column. |
| DuplicatedFieldNameException | The query result contains an unsupported duplicate field name. |
| ActivityTimedOutException | The configured activity timeout was reached. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# JDBC Truncate

Activities / JDBC

## Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: truncate.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| JDBC shared connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: jdbc |
| Timeout (seconds)<br>timeout | number<br>Configuration setting | SQL execution timeout in seconds passed to the selected database driver. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
|---|---|---|
| Database server time zone<br>ServerTimeZone | string<br>Optional | Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems. |
| Dynamic timeout<br>timeout | number<br>Optional | SQL execution timeout in seconds passed to the selected database driver. |
| Prepared statement parameters<br>parameters | object<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Batch parameter records<br>records | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dynamic update SQL<br>SqlUpdateStatement | string<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |

## Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Number of updates<br>noOfUpdates | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Affected rows<br>rowCount | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Generated key<br>lastInsertId | integer\|string<br>Optional | Generated identity reported after an insert when supported by the database driver. Do not assume every provider supplies it. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| JDBCConnectionNotFoundException | The selected JDBC shared connection is unavailable. |
| InvalidTimeZoneException | The database server time zone is invalid. |
| JDBCSQLException | The database rejected the SQL or procedure call. |
| LoginTimedOutException | The database login timed out. |
| InvalidSQLTypeException | A prepared parameter type does not match its database column. |
| DuplicatedFieldNameException | The query result contains an unsupported duplicate field name. |
| ActivityTimedOutException | The configured activity timeout was reached. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# JDBC Delete

Activities / JDBC

## Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: delete.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| JDBC shared connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: jdbc |
| Timeout (seconds)<br>timeout | number<br>Configuration setting | SQL execution timeout in seconds passed to the selected database driver. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Batch update<br>batchUpdate | boolean<br>Configuration setting | Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete. |
| Allow mapped SQL override<br>overrideSqlStatement | boolean<br>Configuration setting | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
|---|---|---|
| Database server time zone<br>ServerTimeZone | string<br>Optional | Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dynamic timeout<br>timeout | number<br>Optional | SQL execution timeout in seconds passed to the selected database driver. |
| Prepared statement parameters<br>parameters | object<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Batch parameter records<br>records | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Dynamic update SQL<br>SqlUpdateStatement | string<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Batch update<br>batchUpdate | boolean<br>Optional | Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete. |
| Allow mapped SQL override<br>overrideSqlStatement | boolean<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |

## Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Number of updates<br>noOfUpdates | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Affected rows<br>rowCount | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Generated key<br>lastInsertId | integer\|string<br>Optional | Generated identity reported after an insert when supported by the database driver. Do not assume every provider supplies it. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| JDBCConnectionNotFoundException | The selected JDBC shared connection is unavailable. |
| InvalidTimeZoneException | The database server time zone is invalid. |
| JDBCSQLException | The database rejected the SQL or procedure call. |
| LoginTimedOutException | The database login timed out. |
| InvalidSQLTypeException | A prepared parameter type does not match its database column. |
| DuplicatedFieldNameException | The query result contains an unsupported duplicate field name. |
| ActivityTimedOutException | The configured activity timeout was reached. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Stored Procedure

Activities / JDBC

## Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: call.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| JDBC shared connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: jdbc |
| Procedure schema<br>schema | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. |
| Catalog or package<br>catalog | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. |
| Procedure / function<br>procedure | text<br>Configuration setting | Browse and refresh procedure metadata from the JDBC designer below. |
| Timeout (seconds)<br>timeout | number<br>Configuration setting | SQL execution timeout in seconds passed to the selected database driver. |
| Maximum rows (0 retrieves all)<br>maxRows | number<br>Configuration setting | Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Database server time zone<br>ServerTimeZone | string<br>Optional | Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems. |
| Dynamic timeout<br>timeout | number<br>Optional | SQL execution timeout in seconds passed to the selected database driver. |
| IN and IN/OUT procedure parameters<br>parameters | object<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Procedure schema<br>schema | string<br>Optional | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. |
| Catalog or package<br>catalog | string<br>Optional | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. |
| Procedure / function<br>procedure | string<br>Optional | Browse and refresh procedure metadata from the JDBC designer below. |
| Maximum rows (0 retrieves all)<br>maxRows | number<br>Optional | Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |

## Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Procedure result sets<br>resultSets | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| OUT and IN/OUT parameters<br>outParameters | object<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Unresolved result sets<br>UnresolvedResultSets | array<br>Optional | Additional result sets whose row metadata was not resolved at design time. Inspect the actual procedure response before mapping child fields. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| JDBCConnectionNotFoundException | The selected JDBC shared connection is unavailable. |
| InvalidTimeZoneException | The database server time zone is invalid. |
| JDBCSQLException | The database rejected the SQL or procedure call. |
| LoginTimedOutException | The database login timed out. |
| InvalidSQLTypeException | A prepared parameter type does not match its database column. |
| DuplicatedFieldNameException | The query result contains an unsupported duplicate field name. |
| ActivityTimedOutException | The configured activity timeout was reached. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Dynamic SQL

Activities / JDBC

## Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: dynamic.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| JDBC shared connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: jdbc |
| Timeout (seconds)<br>timeout | number<br>Configuration setting | SQL execution timeout in seconds passed to the selected database driver. |
| Maximum rows (0 retrieves all)<br>maxRows | number<br>Configuration setting | Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Configuration setting | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Use nil for SQL NULL<br>useNil | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Interpret invalid XML characters<br>interpretInvalidXmlChars | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Process result in subsets<br>processInSubsets | boolean<br>Configuration setting | Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached. |
| Subset size<br>subsetSize | number<br>Configuration setting | Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached. |
| Allow mapped SQL override<br>overrideSqlStatement | boolean<br>Configuration setting | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Database server time zone<br>ServerTimeZone | string<br>Optional | Timezone used for database or scheduling date/time interpretation. Match the source timezone and explicitly convert timestamps crossing systems. |
| Dynamic timeout<br>timeout | number<br>Optional | SQL execution timeout in seconds passed to the selected database driver. |
| Dynamic maximum rows<br>maxRows | number<br>Optional | Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available. |
| Dynamic SQL override<br>SqlStatement | string<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |
| Prepared statement parameters<br>parameters | object<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Row offset when processing subsets<br>rowOffset | integer<br>Optional | Start at row offset zero. For subsequent pages, map the previous nextRowOffset to rowOffset. Use a stable ORDER BY and stop when lastSubset is true. |
| Override transaction behavior<br>overrideTransactionBehavior | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Override JDBC connection<br>overrideJdbcConnection | boolean<br>Optional | Runtime override for the selected JDBC connection or transaction behavior. Use only the override supported by this operation and coordinate with any enclosing transaction group. |
| Use nil for SQL NULL<br>useNil | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Interpret invalid XML characters<br>interpretInvalidXmlChars | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Process result in subsets<br>processInSubsets | boolean<br>Optional | Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached. |
| Subset size<br>subsetSize | number<br>Optional | Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached. |
| Allow mapped SQL override<br>overrideSqlStatement | boolean<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |

## Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

| Field / key | Type / required | Details |
| --- | --- | --- |
| JDBC result set<br>resultSet | object<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Result records<br>resultSet.Record | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Result records<br>rows | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Rows returned<br>rowCount | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Last result subset<br>lastSubset | boolean<br>Optional | Enable result subsets and set the size of each page. The lastSubset output is true on the final page or when the maximum row limit is reached. |
| Offset for the next subset, when available<br>nextRowOffset | integer<br>Optional | Start at row offset zero. For subsequent pages, map the previous nextRowOffset to rowOffset. Use a stable ORDER BY and stop when lastSubset is true. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| JDBCConnectionNotFoundException | The selected JDBC shared connection is unavailable. |
| InvalidTimeZoneException | The database server time zone is invalid. |
| JDBCSQLException | The database rejected the SQL or procedure call. |
| LoginTimedOutException | The database login timed out. |
| InvalidSQLTypeException | A prepared parameter type does not match its database column. |
| DuplicatedFieldNameException | The query result contains an unsupported duplicate field name. |
| ActivityTimedOutException | The configured activity timeout was reached. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

# Snowflake Insert

Activities / Snowflake

## Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: insert.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Snowflake JDBC connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: snowflake |
| Entity<br>entity | snowflake_entity<br>Configuration setting | Tables and views retrieved in the Snowflake connection Schema section. |
| Activity timeout (seconds)<br>timeout | number<br>Configuration setting | Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout. |
| Override database name<br>overrideDatabaseName | text<br>Configuration setting | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Override schema name<br>overrideSchemaName | text<br>Configuration setting | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Create table from XSD<br>createTableFromXsd | boolean<br>Configuration setting | Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use. |
| Table XSD/schema<br>schemaId | text<br>Configuration setting | Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities. |
| Table name<br>tableName | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. |
| Values columns<br>valueColumns | text<br>Configuration setting | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Batch size<br>batchSize | number<br>Configuration setting | Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Fault on batch failure<br>faultOnBatchFailure | boolean<br>Configuration setting | Batch failure policy/results. With faultOnBatchFailure enabled, failed batches fault the activity; inspect batchFailures for affected records and provider errors. |
| Merge existing records<br>merge | boolean<br>Configuration setting | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Merge on columns<br>mergeOnColumns | text<br>Configuration setting | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Rows to insert<br>records | array<br>Required | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Activity timeout (seconds)<br>timeout | number<br>Optional | Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout. |
| Override database name<br>overrideDatabaseName | string<br>Optional | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Override schema name<br>overrideSchemaName | string<br>Optional | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Create table from XSD<br>createTableFromXsd | boolean<br>Optional | Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use. |
| Table XSD/schema<br>schemaId | string<br>Optional | Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities. |
| Table name<br>tableName | string<br>Optional | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. |
| Values columns<br>valueColumns | string<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Batch size<br>batchSize | number<br>Optional | Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete. |
| Fault on batch failure<br>faultOnBatchFailure | boolean<br>Optional | Batch failure policy/results. With faultOnBatchFailure enabled, failed batches fault the activity; inspect batchFailures for affected records and provider errors. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Merge existing records<br>merge | boolean<br>Optional | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Merge on columns<br>mergeOnColumns | string<br>Optional | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |

## Output

Entity/table metadata adds actual input record and result row columns.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Rows attempted<br>rowsAttempted | integer<br>Optional | Number of rows submitted for processing; compare with rowsAffected and batchFailures to detect partial success. |
| Rows affected<br>rowsAffected | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Batch failure details<br>batchFailures | array<br>Optional | Batch failure policy/results. With faultOnBatchFailure enabled, failed batches fault the activity; inspect batchFailures for affected records and provider errors. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| SNOWFLAKE_CONNECTION | The Snowflake shared resource or connection pool is unavailable. |
| SNOWFLAKE_DATABASE_JDBC-500005 | Snowflake failed to create a prepared statement. |
| SNOWFLAKE_DATABASE_JDBC-500007 | Snowflake failed to bind prepared-statement parameters. |
| SNOWFLAKE_DATABASE_JDBC-500009 | Snowflake failed to execute the generated query. |
| SNOWFLAKE_DATABASE_JDBC-500013 | The activity could not generate its structured output. |
| SNOWFLAKE_DATABASE_JDBC-500016 | One or more insert batches failed. |
| SNOWFLAKE_DATABASE_JDBC-500022 | Schema or table creation failed. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Snowflake Query

Activities / Snowflake

## Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: query.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Snowflake JDBC connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: snowflake |
| Entity<br>entity | snowflake_entity<br>Configuration setting | Tables and views retrieved in the Snowflake connection Schema section. |
| Activity timeout (seconds)<br>timeout | number<br>Configuration setting | Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout. |
| Override database name<br>overrideDatabaseName | text<br>Configuration setting | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Override schema name<br>overrideSchemaName | text<br>Configuration setting | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Snowflake SELECT statement<br>statement | textarea<br>Configuration setting | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |
| Prepared parameters [name:type]<br>preparedParameters | text<br>Configuration setting | Named or positional operation arguments. Match names, order and datatypes to the selected SQL statement, function or method contract. |
| Maximum rows (0 retrieves all)<br>maximumRows | number<br>Configuration setting | Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Prepared statement parameters<br>parameters | object<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Activity timeout (seconds)<br>timeout | number<br>Optional | Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout. |
| Override database name<br>overrideDatabaseName | string<br>Optional | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Override schema name<br>overrideSchemaName | string<br>Optional | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Snowflake SELECT statement<br>statement | string<br>Optional | SQL statement executed by the database activity. Use bound parameters for values and match the statement to the selected operation/connection. |
| Prepared parameters [name:type]<br>preparedParameters | string<br>Optional | Named or positional operation arguments. Match names, order and datatypes to the selected SQL statement, function or method contract. |
| Maximum rows (0 retrieves all)<br>maximumRows | number<br>Optional | Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available. |

## Output

Entity/table metadata adds actual input record and result row columns.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Query result records<br>rows | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Rows returned<br>rowCount | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| SNOWFLAKE_CONNECTION | The Snowflake shared resource or connection pool is unavailable. |
| SNOWFLAKE_DATABASE_JDBC-500005 | Snowflake failed to create a prepared statement. |
| SNOWFLAKE_DATABASE_JDBC-500007 | Snowflake failed to bind prepared-statement parameters. |
| SNOWFLAKE_DATABASE_JDBC-500009 | Snowflake failed to execute the generated query. |
| SNOWFLAKE_DATABASE_JDBC-500013 | The activity could not generate its structured output. |
| SNOWFLAKE_DATABASE_JDBC-500014 | Maximum Rows is negative. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Snowflake Update

Activities / Snowflake

## Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: update.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Snowflake JDBC connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: snowflake |
| Entity<br>entity | snowflake_entity<br>Configuration setting | Tables and views retrieved in the Snowflake connection Schema section. |
| Activity timeout (seconds)<br>timeout | number<br>Configuration setting | Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout. |
| Override database name<br>overrideDatabaseName | text<br>Configuration setting | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Override schema name<br>overrideSchemaName | text<br>Configuration setting | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| SET / Values columns<br>valueColumns | text<br>Configuration setting | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| WHERE / Parameter columns<br>parameterColumns | text<br>Configuration setting | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Insert when record does not exist<br>merge | boolean<br>Configuration setting | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Merge on columns<br>mergeOnColumns | text<br>Configuration setting | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Create table if none exists<br>createTableIfNoneExists | boolean<br>Configuration setting | Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Rows containing values and parameters<br>records | array<br>Required | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Activity timeout (seconds)<br>timeout | number<br>Optional | Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout. |
| Override database name<br>overrideDatabaseName | string<br>Optional | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Override schema name<br>overrideSchemaName | string<br>Optional | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| SET / Values columns<br>valueColumns | string<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| WHERE / Parameter columns<br>parameterColumns | string<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Insert when record does not exist<br>merge | boolean<br>Optional | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Merge on columns<br>mergeOnColumns | string<br>Optional | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Create table if none exists<br>createTableIfNoneExists | boolean<br>Optional | Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use. |

## Output

Entity/table metadata adds actual input record and result row columns.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Rows affected<br>rowsAffected | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| SNOWFLAKE_CONNECTION | The Snowflake shared resource or connection pool is unavailable. |
| SNOWFLAKE_DATABASE_JDBC-500005 | Snowflake failed to create a prepared statement. |
| SNOWFLAKE_DATABASE_JDBC-500007 | Snowflake failed to bind prepared-statement parameters. |
| SNOWFLAKE_DATABASE_JDBC-500009 | Snowflake failed to execute the generated query. |
| SNOWFLAKE_DATABASE_JDBC-500013 | The activity could not generate its structured output. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Snowflake Delete

Activities / Snowflake

## Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: delete.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Snowflake JDBC connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: snowflake |
| Entity<br>entity | snowflake_entity<br>Configuration setting | Tables and views retrieved in the Snowflake connection Schema section. |
| Activity timeout (seconds)<br>timeout | number<br>Configuration setting | Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout. |
| Override database name<br>overrideDatabaseName | text<br>Configuration setting | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Override schema name<br>overrideSchemaName | text<br>Configuration setting | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| WHERE / Parameter columns<br>parameterColumns | text<br>Configuration setting | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Merge when record does not exist<br>merge | boolean<br>Configuration setting | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Merge on columns<br>mergeOnColumns | text<br>Configuration setting | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Create table if none exists<br>createTableIfNoneExists | boolean<br>Configuration setting | Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Rows containing delete parameters<br>records | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Activity timeout (seconds)<br>timeout | number<br>Optional | Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout. |
| Override database name<br>overrideDatabaseName | string<br>Optional | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Override schema name<br>overrideSchemaName | string<br>Optional | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| WHERE / Parameter columns<br>parameterColumns | string<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| Merge when record does not exist<br>merge | boolean<br>Optional | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Merge on columns<br>mergeOnColumns | string<br>Optional | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Create table if none exists<br>createTableIfNoneExists | boolean<br>Optional | Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use. |

## Output

Entity/table metadata adds actual input record and result row columns.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Rows affected<br>rowsAffected | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| SNOWFLAKE_CONNECTION | The Snowflake shared resource or connection pool is unavailable. |
| SNOWFLAKE_DATABASE_JDBC-500005 | Snowflake failed to create a prepared statement. |
| SNOWFLAKE_DATABASE_JDBC-500007 | Snowflake failed to bind prepared-statement parameters. |
| SNOWFLAKE_DATABASE_JDBC-500009 | Snowflake failed to execute the generated query. |
| SNOWFLAKE_DATABASE_JDBC-500013 | The activity could not generate its structured output. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Snowflake Bulk Load

Activities / Snowflake

## Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: bulk_load.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Snowflake JDBC connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: snowflake |
| Entity<br>entity | snowflake_entity<br>Configuration setting | Tables and views retrieved in the Snowflake connection Schema section. |
| Activity timeout (seconds)<br>timeout | number<br>Configuration setting | Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout. |
| Override database name<br>overrideDatabaseName | text<br>Configuration setting | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Override schema name<br>overrideSchemaName | text<br>Configuration setting | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Configuration setting | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Create table from XSD<br>createTableFromXsd | boolean<br>Configuration setting | Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use. |
| Table XSD/schema<br>schemaId | text<br>Configuration setting | Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities. |
| Table name<br>tableName | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. |
| Stage type<br>stageType | select<br>Configuration setting | Snowflake staging/COPY configuration. Select the stage, file format and load options matching the uploaded data and target table.<br>Choices: UserStage, TableStage, NamedStage, AmazonS3 |
| Named stage / storage integration<br>stage<br>namedStage | text<br>Configuration setting | Snowflake staging/COPY configuration. Select the stage, file format and load options matching the uploaded data and target table. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Amazon S3 file format<br>fileFormat | select<br>Configuration setting | Snowflake staging/COPY configuration. Select the stage, file format and load options matching the uploaded data and target table.<br>Choices: DelimitedFiles, JSON, AVRO, ORC, PARQUET, XML |
| Validation mode<br>validationMode | boolean<br>Configuration setting | Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery. |
| Purge stage files<br>purgeStageFiles | boolean<br>Configuration setting | Remove staged files after a successful load where configured. Retain files when replay or investigation requires the original input. |
| Compress internal-stage data<br>compressData | boolean<br>Configuration setting | Compression selection for file or staging content. Match decompression settings at the receiving operation; compression changes the wire bytes. |
| On error<br>onError | select<br>Configuration setting | Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery.<br>Choices: CONTINUE, SKIP_FILE, SKIP_FILE_<num>, SKIP_FILE_<num>%, ABORT_STATEMENT |
| Skip file at error count<br>skipFileErrorCount | number<br>Configuration setting | Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery. |
| Skip file at error percentage<br>skipFileErrorPercentage | number<br>Configuration setting | Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery. |
| Merge loaded records<br>merge | boolean<br>Configuration setting | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Merge on columns<br>mergeOnColumns | text<br>Configuration setting | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Rows for an internal stage<br>records | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Local file for an internal stage<br>filePath | string<br>Optional | File path on the runtime host. Ensure the runtime account can read the file and that the path exists in the deployment environment. |
| Stage load options<br>loadOptions | object<br>Optional | Additional provider settings as name/value entries. Use exact supported provider option names; invalid options may be rejected at connection or execution time. |
| Activity timeout (seconds)<br>timeout | number<br>Optional | Statement execution timeout in seconds; a positive value configures the Snowflake session statement timeout. |
| Override database name<br>overrideDatabaseName | string<br>Optional | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Override schema name<br>overrideSchemaName | string<br>Optional | Per-operation database/schema override. The runtime identity must have access to the resolved namespace. |
| Interpret empty string as NULL<br>interpretEmptyStringAsNull | boolean<br>Optional | Null/serialization policy for database values. Empty string, null and XML nil have different meanings; choose the policy required by the target schema/provider. |
| Create table from XSD<br>createTableFromXsd | boolean<br>Optional | Allow the Snowflake activity to create a missing table from its configured contract. Requires creation privileges; review generated column types before production use. |
| Table XSD/schema<br>schemaId | string<br>Optional | Selected schema/contract for the request, response or record structure. Refresh mappings after changing schema fields, types or cardinalities. |
| Table name<br>tableName | string<br>Optional | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. |
| Stage type<br>stageType | string<br>Optional | Snowflake staging/COPY configuration. Select the stage, file format and load options matching the uploaded data and target table. |
| Named stage / storage integration stage<br>namedStage | string<br>Optional | Snowflake staging/COPY configuration. Select the stage, file format and load options matching the uploaded data and target table. |
| Amazon S3 file format<br>fileFormat | string<br>Optional | Snowflake staging/COPY configuration. Select the stage, file format and load options matching the uploaded data and target table. |
| Validation mode<br>validationMode | boolean<br>Optional | Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery. |
| Purge stage files<br>purgeStageFiles | boolean<br>Optional | Remove staged files after a successful load where configured. Retain files when replay or investigation requires the original input. |
| Compress internal-stage data<br>compressData | boolean<br>Optional | Compression selection for file or staging content. Match decompression settings at the receiving operation; compression changes the wire bytes. |
| On error<br>onError | string<br>Optional | Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery. |
| Skip file at error count<br>skipFileErrorCount | number<br>Optional | Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery. |
| Skip file at error percentage<br>skipFileErrorPercentage | number<br>Optional | Snowflake COPY error/validation policy. Select abort, continuation or skip thresholds deliberately; skipped records/files are not successful business delivery. |
| Merge loaded records<br>merge | boolean<br>Optional | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |
| Merge on columns<br>mergeOnColumns | string<br>Optional | Insert/merge policy. Identify the stable matching key columns for merge/update operations and verify uniqueness before loading records. |

## Output

Entity/table metadata adds actual input record and result row columns.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Per-file load results<br>loadResults | array<br>Optional | Snowflake COPY result rows. Inspect status, parsed/loaded row counts and first-error fields for each input file. |
| Source file<br>loadResults.FILE | string<br>Optional | Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data. |
| Load status<br>loadResults.STATUS | string<br>Optional | Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data. |
| Rows parsed<br>loadResults.ROWS_PARSED | integer<br>Optional | Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data. |
| Rows loaded<br>loadResults.ROWS_LOADED | integer<br>Optional | Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data. |
| Errors seen<br>loadResults.ERRORS_SEEN | integer<br>Optional | Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data. |
| First error<br>loadResults.FIRST_ERROR | string<br>Optional | Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data. |
| First error line<br>loadResults.FIRST_ERROR_LINE | integer<br>Optional | Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data. |
| First error character<br>loadResults.FIRST_ERROR_CHARACTER | integer<br>Optional | Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data. |
| First error column<br>loadResults.FIRST_ERROR_COLUMN_NAME | string<br>Optional | Snowflake per-file load result: file/status, counts, or location/message of the first load error, according to this field. Use these values to identify failed or skipped data. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| SNOWFLAKE_CONNECTION | The Snowflake shared resource or connection pool is unavailable. |
| SNOWFLAKE_DATABASE_JDBC-500005 | Snowflake failed to create a prepared statement. |
| SNOWFLAKE_DATABASE_JDBC-500007 | Snowflake failed to bind prepared-statement parameters. |
| SNOWFLAKE_DATABASE_JDBC-500009 | Snowflake failed to execute the generated query. |
| SNOWFLAKE_DATABASE_JDBC-500013 | The activity could not generate its structured output. |
| SNOWFLAKE_DATABASE_JDBC-500018 | Uploading data to the Snowflake stage failed. |
| SNOWFLAKE_DATABASE_JDBC-500020 | Bulk-load validation found invalid data. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# AMQP Receive Message

Activities / AMQP

## Overview

Sends, receives, gets, or dead-letters AMQP messages.

The shared connection supports RabbitMQ AMQP 0.9.1/1.0 and AMQP 1.0 broker profiles. Destinations, subscriptions, typed messages, properties, settlement handles, recovery, SSL, sessions, and Azure dead-letter behavior are configured in MINA.

Activity type: amqp; operation: receive.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| AMQP connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: amqp |
| Queue name<br>queueName | text<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Entity type<br>entityType | select<br>Configuration setting | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.<br>Choices: Queue, Topic |
| Entity name<br>entityName | text<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Subscription name<br>subscriptionName | text<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Durable subscription<br>durableSubscription | boolean<br>Configuration setting | Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider. |
| Shared subscription<br>sharedSubscription | boolean<br>Configuration setting | Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider. |
| Message type<br>messageType | select<br>Configuration setting | Message representation/media type. Match text, bytes or structured content to what the destination consumer expects.<br>Choices: TextMessage, BytesMessage, Simple, Any |
| Maximum messages<br>maxMessages | number<br>Configuration setting | Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Acknowledge mode<br>acknowledgeMode | select<br>Configuration setting | Message acknowledgement policy. Manual/client modes require explicit confirmation after successful processing; automatic/provider modes follow connector behavior.<br>Choices: Auto, Client, Client or Dead Letter Queue |
| Azure receiver mode<br>receiverMode | select<br>Configuration setting | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.<br>Choices: PeekLock, ReceiveAndDelete |
| Get correlation ID<br>getCorrelationID | boolean<br>Configuration setting | Include the provider message/correlation identifier in receive results when supported. These identifiers assist tracing and request/reply matching. |
| Azure session enabled<br>sessionEnabled | boolean<br>Configuration setting | Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider. |
| Session receive type<br>receiveType | select<br>Configuration setting | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.<br>Choices: FirstAvailableSession, SpecificSession, AllAvailableSessions |
| Maximum concurrent sessions<br>maxConcurrentSessions | number<br>Configuration setting | Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit. |
| Specific session ID<br>sessionId | text<br>Configuration setting | Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider. |
| Prefetch count / credits<br>prefetchCount | number<br>Configuration setting | Maximum number of messages prefetched before processing/settlement. Bound it to worker capacity to avoid excessive in-flight memory. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dynamic queue<br>queueName | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Dynamic topic<br>topicName | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Dynamic subscription<br>subscriptionName | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Entity type<br>entityType | string<br>Optional | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. |
| Entity name<br>entityName | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Durable subscription<br>durableSubscription | boolean<br>Optional | Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider. |
| Shared subscription<br>sharedSubscription | boolean<br>Optional | Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider. |
| Message type<br>messageType | string<br>Optional | Message representation/media type. Match text, bytes or structured content to what the destination consumer expects. |
| Maximum messages<br>maxMessages | number<br>Optional | Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity. |
| Acknowledge mode<br>acknowledgeMode | string<br>Optional | Message acknowledgement policy. Manual/client modes require explicit confirmation after successful processing; automatic/provider modes follow connector behavior. |
| Azure receiver mode<br>receiverMode | string<br>Optional | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. |
| Get correlation ID<br>getCorrelationID | boolean<br>Optional | Include the provider message/correlation identifier in receive results when supported. These identifiers assist tracing and request/reply matching. |
| Azure session enabled<br>sessionEnabled | boolean<br>Optional | Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider. |
| Session receive type<br>receiveType | string<br>Optional | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. |
| Maximum concurrent sessions<br>maxConcurrentSessions | number<br>Optional | Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit. |
| Specific session ID<br>sessionId | string<br>Optional | Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider. |
| Prefetch count / credits<br>prefetchCount | number<br>Optional | Maximum number of messages prefetched before processing/settlement. Bound it to worker capacity to avoid excessive in-flight memory. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Message received<br>received | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |
| User properties<br>UserProperties | object<br>Optional | Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| AMQP message properties<br>MessageProperties | object<br>Optional | Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system. |
| Message body<br>body | string\|binary<br>Optional | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Client/dead-letter settlement token<br>settlementToken | string<br>Optional | Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle. |
| Confirm Message handle<br>ackId | string<br>Optional | Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| AMQPPluginException | The AMQP operation failed or its message configuration is invalid. |
| AMQPConnectionException | The broker connection failed or could not be recovered. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# AMQP Get Message

Activities / AMQP

## Overview

Sends, receives, gets, or dead-letters AMQP messages.

The shared connection supports RabbitMQ AMQP 0.9.1/1.0 and AMQP 1.0 broker profiles. Destinations, subscriptions, typed messages, properties, settlement handles, recovery, SSL, sessions, and Azure dead-letter behavior are configured in MINA.

Activity type: amqp; operation: get.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| AMQP connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: amqp |
| Queue name<br>queueName | text<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Entity type<br>entityType | select<br>Configuration setting | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.<br>Choices: Queue, Topic |
| Entity name<br>entityName | text<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Subscription name<br>subscriptionName | text<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Durable subscription<br>durableSubscription | boolean<br>Configuration setting | Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider. |
| Shared subscription<br>sharedSubscription | boolean<br>Configuration setting | Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider. |
| Message type<br>messageType | select<br>Configuration setting | Message representation/media type. Match text, bytes or structured content to what the destination consumer expects.<br>Choices: TextMessage, BytesMessage, Simple |
| Maximum messages<br>maxMessages | number<br>Configuration setting | Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Acknowledge mode<br>acknowledgeMode | select<br>Configuration setting | Message acknowledgement policy. Manual/client modes require explicit confirmation after successful processing; automatic/provider modes follow connector behavior.<br>Choices: Auto, Client |
| Azure receiver mode<br>receiverMode | select<br>Configuration setting | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.<br>Choices: PeekLock, ReceiveAndDelete |
| Get correlation ID<br>getCorrelationID | boolean<br>Configuration setting | Include the provider message/correlation identifier in receive results when supported. These identifiers assist tracing and request/reply matching. |
| Azure session enabled<br>sessionEnabled | boolean<br>Configuration setting | Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider. |
| Session receive type<br>receiveType | select<br>Configuration setting | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.<br>Choices: FirstAvailableSession, SpecificSession, AllAvailableSessions |
| Maximum concurrent sessions<br>maxConcurrentSessions | number<br>Configuration setting | Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit. |
| Specific session ID<br>sessionId | text<br>Configuration setting | Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider. |
| Prefetch count / credits<br>prefetchCount | number<br>Configuration setting | Maximum number of messages prefetched before processing/settlement. Bound it to worker capacity to avoid excessive in-flight memory. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dynamic queue<br>queueName | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Dynamic topic<br>topicName | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Dynamic subscription<br>subscriptionName | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Entity type<br>entityType | string<br>Optional | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. |
| Entity name<br>entityName | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Durable subscription<br>durableSubscription | boolean<br>Optional | Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider. |
| Shared subscription<br>sharedSubscription | boolean<br>Optional | Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider. |
| Message type<br>messageType | string<br>Optional | Message representation/media type. Match text, bytes or structured content to what the destination consumer expects. |
| Maximum messages<br>maxMessages | number<br>Optional | Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity. |
| Acknowledge mode<br>acknowledgeMode | string<br>Optional | Message acknowledgement policy. Manual/client modes require explicit confirmation after successful processing; automatic/provider modes follow connector behavior. |
| Azure receiver mode<br>receiverMode | string<br>Optional | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. |
| Get correlation ID<br>getCorrelationID | boolean<br>Optional | Include the provider message/correlation identifier in receive results when supported. These identifiers assist tracing and request/reply matching. |
| Azure session enabled<br>sessionEnabled | boolean<br>Optional | Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider. |
| Session receive type<br>receiveType | string<br>Optional | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection. |
| Maximum concurrent sessions<br>maxConcurrentSessions | number<br>Optional | Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit. |
| Specific session ID<br>sessionId | string<br>Optional | Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider. |
| Prefetch count / credits<br>prefetchCount | number<br>Optional | Maximum number of messages prefetched before processing/settlement. Bound it to worker capacity to avoid excessive in-flight memory. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Message received<br>received | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |
| User properties<br>UserProperties | object<br>Optional | Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| AMQP message properties<br>MessageProperties | object<br>Optional | Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system. |
| Message body<br>body | string\|binary<br>Optional | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Client/dead-letter settlement token<br>settlementToken | string<br>Optional | Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle. |
| Confirm Message handle<br>ackId | string<br>Optional | Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| AMQPPluginException | The AMQP operation failed or its message configuration is invalid. |
| AMQPConnectionException | The broker connection failed or could not be recovered. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# AMQP Send Message

Activities / AMQP

## Overview

Sends, receives, gets, or dead-letters AMQP messages.

The shared connection supports RabbitMQ AMQP 0.9.1/1.0 and AMQP 1.0 broker profiles. Destinations, subscriptions, typed messages, properties, settlement handles, recovery, SSL, sessions, and Azure dead-letter behavior are configured in MINA.

Activity type: amqp; operation: send.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| AMQP connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: amqp |
| RabbitMQ destination type<br>destinationType | select<br>Configuration setting | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.<br>Choices: Queue, Exchange |
| Queue name<br>queueName | text<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Entity type<br>entityType | select<br>Configuration setting | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.<br>Choices: Queue, Topic |
| Entity name<br>entityName | text<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Exchange type<br>exchangeType | select<br>Configuration setting | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.<br>Choices: direct, topic, fanout, headers |
| Exchange name<br>exchangeName | text<br>Configuration setting | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. |
| Routing key<br>routingKey | text<br>Configuration setting | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. |
| Message type<br>messageType | select<br>Configuration setting | Message representation/media type. Match text, bytes or structured content to what the destination consumer expects.<br>Choices: TextMessage, BytesMessage, Simple |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Get message ID<br>getMessageID | boolean<br>Configuration setting | Include the provider message/correlation identifier in receive results when supported. These identifiers assist tracing and request/reply matching. |
| Delivery mode<br>deliveryMode | select<br>Configuration setting | Message persistence policy. Persistent delivery asks the provider to retain the message according to its durability configuration; it does not guarantee consumer processing.<br>Choices: Persistent, Non-Persistent |
| Expiration (msec)<br>expiration | number<br>Configuration setting | Message lifetime before expiry, in milliseconds for JMS/EMS. Zero requests no expiration; choose a TTL appropriate for delayed consumers. |
| Priority (0-9)<br>priority | number<br>Configuration setting | Provider message priority. JMS/EMS priorities range from 0 to 9; ordering effects depend on broker scheduling and destination policy. |
| Message type/property<br>type | text<br>Configuration setting | Type reported for this value or selected for this operation. Use the corresponding schema datatype and operation-specific choices. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| User properties<br>userProperties | object<br>Optional | Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system. |
| Dynamic delivery mode<br>deliveryMode | string<br>Optional | Message persistence policy. Persistent delivery asks the provider to retain the message according to its durability configuration; it does not guarantee consumer processing. |
| Message ID<br>messageID | string<br>Optional | Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection. |
| Dynamic expiration<br>expiration | integer<br>Optional | Message lifetime before expiry, in milliseconds for JMS/EMS. Zero requests no expiration; choose a TTL appropriate for delayed consumers. |
| Dynamic priority<br>priority | integer<br>Optional | Provider message priority. JMS/EMS priorities range from 0 to 9; ordering effects depend on broker scheduling and destination policy. |
| Dynamic type<br>type | string<br>Optional | Type reported for this value or selected for this operation. Use the corresponding schema datatype and operation-specific choices. |
| Content type<br>contentType | string<br>Optional | Message representation/media type. Match text, bytes or structured content to what the destination consumer expects. |
| Azure session ID<br>sessionID | string<br>Optional | Messaging/SAP session setting or identifier. Preserve the live session identifier for related operations; session semantics differ by provider. |
| Correlation ID<br>correlationID | string<br>Optional | Correlation identifier for tracing or request/reply matching. Preserve the request value in the reply rather than generating an unrelated identifier. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dynamic exchange<br>exchangeName | string<br>Optional | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. |
| Dynamic routing key<br>routingKey | string<br>Optional | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. |
| Dynamic topic<br>topicName | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Dynamic queue<br>queueName | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Message body<br>body | string\|binary<br>Required | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| RabbitMQ destination type<br>destinationType | string<br>Optional | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. |
| Entity type<br>entityType | string<br>Optional | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. |
| Entity name<br>entityName | string<br>Optional | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |
| Exchange type<br>exchangeType | string<br>Optional | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing. |
| Message type<br>messageType | string<br>Optional | Message representation/media type. Match text, bytes or structured content to what the destination consumer expects. |
| Get message ID<br>getMessageID | boolean<br>Optional | Include the provider message/correlation identifier in receive results when supported. These identifiers assist tracing and request/reply matching. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Send succeeded<br>sendResult | boolean<br>Optional | Provider result of the send attempt. Inspect its status/identifier together with faults; it does not indicate downstream consumption. |
| Broker message ID<br>MessageId | string<br>Optional | Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| AMQPPluginException | The AMQP operation failed or its message configuration is invalid. |
| AMQPConnectionException | The broker connection failed or could not be recovered. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# AMQP Dead Letter Message

Activities / AMQP

## Overview

Sends, receives, gets, or dead-letters AMQP messages.

The shared connection supports RabbitMQ AMQP 0.9.1/1.0 and AMQP 1.0 broker profiles. Destinations, subscriptions, typed messages, properties, settlement handles, recovery, SSL, sessions, and Azure dead-letter behavior are configured in MINA.

Activity type: amqp; operation: dead_letter.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| AMQP connection<br>resourceId | resource<br>Required | Select a saved shared connection of the matching connector family. Connection properties are resolved using the active environment.<br>Connection type: amqp |
| Use retry<br>useRetry | boolean<br>Configuration setting | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. |
| Total timeout (seconds)<br>totalTimeoutSeconds | number<br>Configuration setting | Total timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| Maximum attempts<br>maxAttempts | number<br>Configuration setting | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. |
| Backoff time (msec)<br>backoffTimeMsec | number<br>Configuration setting | Backoff time (msec): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Settlement token<br>settlementToken | string<br>Required | Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle. |
| Dead-letter reason<br>deadLetterReason | string<br>Optional | Reason and diagnostic description attached when rejecting a message to the provider dead-letter destination. Keep descriptions useful and free of credentials. |
| Dead-letter error description<br>deadLetterErrorDescription | string<br>Optional | Reason and diagnostic description attached when rejecting a message to the provider dead-letter destination. Keep descriptions useful and free of credentials. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dynamic retry<br>useRetry | boolean<br>Optional | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. |
| Dynamic total timeout<br>totalTimeoutSeconds | integer<br>Optional | Dynamic total timeout: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| Dynamic attempts<br>maxAttempts | integer<br>Optional | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling. |
| Dynamic backoff<br>backoffTimeMsec | integer<br>Optional | Dynamic backoff: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dead-letter status<br>status | string<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |
| Settlement token<br>settlementToken | string<br>Optional | Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle. |
| Processed message ID<br>messageId | string<br>Optional | Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |
| retryEnabled<br>retryEnabled | boolean expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryEnabled}" |
| retryCount<br>retryCount | number expression<br>Configuration setting | Additional attempts after the first failure, between 0 and 100. Invalid or non-finite settings fail before execution.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | number expression<br>Configuration setting | Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| retryBackoff<br>retryBackoff | string expression<br>Configuration setting | Fixed preserves the base interval. Exponential doubles it after each failure, capped by the maximum interval.<br>Default: "fixed" |
| retryMaxIntervalSeconds<br>retryMaxIntervalSeconds | number expression<br>Configuration setting | Upper bound on each wait, from 0 to 86400 seconds. Blank defaults to 300 seconds or the base interval, whichever is greater.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| retryMaxElapsedSeconds<br>retryMaxElapsedSeconds | number expression<br>Configuration setting | Retry scheduling budget in seconds, up to one week. Zero disables the budget. Another retry is not scheduled if its delay reaches the budget; in-flight work is not interrupted.<br>Default: 0 |
| retryJitter<br>retryJitter | boolean expression<br>Configuration setting | Randomize each wait between half and all of its calculated delay to spread concurrent reconnection attempts.<br>Default: false |
| retryFaultTypes<br>retryFaultTypes | string expression<br>Configuration setting | Optional comma-separated fault types, exception class names or fault codes. Blank retries all ordinary faults. Cancellation is never retried.<br>Default: "" |

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group and connector-level retries to avoid multiplying attempts. An explicit Errors-tab retry policy takes precedence over Advanced retries.

## Errors

| Fault | Description |
| --- | --- |
| AMQPPluginException | The AMQP operation failed or its message configuration is invalid. |
| AMQPConnectionException | The broker connection failed or could not be recovered. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Parse XML

Activities / Data

## Overview

Parses XML text into an XSD-defined tree or renders an XSD-defined tree as XML.

Parse XML accepts serialized XML in Input and defines its published tree in Output Editor. Render XML defines and maps its source tree in Input Editor, then emits xmlString (or binary output when selected).

Activity type: xml; operation: parse.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Input style<br>inputStyle | select<br>Configuration setting | Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing.<br>Choices: Text, Binary, Dynamic |
| Validate parsed output<br>validateOutput | boolean<br>Configuration setting | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected XSD or inline schema determines actual parse output or render input.

| Field / key | Type / required | Details |
|---|---|---|
| XML text<br>xmlString | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Validate parsed output<br>formatOptions.validateOutput | boolean<br>Optional | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| Fallback binary encoding<br>formatOptions.forceEncoding | string<br>Optional | Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters. |
| Legacy binary encoding<br>formatOptions.encoding | string<br>Optional | Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters. |

## Output

The selected XSD or inline schema determines actual parse output or render input.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Parsed XML<br>xml | xml<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| XML text<br>xmlString | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Root element<br>root | string<br>Optional | Qualified root element name of the parsed XML document. Its namespace URI is retained; use the parsed value/tree for element and attribute mappings. |
| Parsed tree<br>value | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| XML_PARSE | The XML input cannot be parsed. |
| XML_RENDER | The value cannot be rendered as XML. |
| SCHEMA_VALIDATION | The data does not conform to the selected schema. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Render XML

Activities / Data

## Overview

Parses XML text into an XSD-defined tree or renders an XSD-defined tree as XML.

Parse XML accepts serialized XML in Input and defines its published tree in Output Editor. Render XML defines and maps its source tree in Input Editor, then emits xmlString (or binary output when selected).

Activity type: xml; operation: render.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Output style<br>outputStyle | select<br>Configuration setting | Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract.<br>Choices: Text, Binary |
| Validate input tree<br>validateInput | boolean<br>Configuration setting | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| Suppress XML declaration<br>suppressXmlDeclaration | boolean<br>Configuration setting | Omit the XML declaration from rendered XML when the receiving interface requires only the document element. |
| Pretty print<br>prettyPrint | boolean<br>Configuration setting | Output formatting. prettyPrint enables readable layout and indent selects indentation width; formatting does not change business values. |
| Use default namespace<br>formatUsingDefaultNamespacePrefix | boolean<br>Configuration setting | Render the root namespace as the default namespace while preserving the namespace of each child and attribute. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected XSD or inline schema determines actual parse output or render input.

| Field / key | Type / required | Details |
|---|---|---|
| XML schema tree<br>value | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |
| Existing XML document<br>xmlString | string<br>Optional | Map an XML document to validate and serialize it directly. |
| Validate input tree<br>formatOptions.validateInput | boolean<br>Optional | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Encoding<br>formatOptions.encoding | string<br>Optional | Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters. |
| Suppress XML declaration<br>formatOptions.suppressXmlDeclaration | boolean<br>Optional | Omit the XML declaration from rendered XML when the receiving interface requires only the document element. |
| Pretty print<br>formatOptions.prettyPrint | boolean<br>Optional | Output formatting. prettyPrint enables readable layout and indent selects indentation width; formatting does not change business values. |
| Use default namespace<br>formatOptions.formatUsingDefaultNamespacePrefix | boolean<br>Optional | Render the root namespace as the default namespace while preserving the namespace of each child and attribute. |
| XSD root override<br>formatOptions.rootElement | string<br>Optional | Optional global element name. The first declared root is used by default. |
| CDATA element paths<br>formatOptions.cdataElements | string<br>Optional | Map a comma-separated list of schema paths, such as order.notes. |

## Output

The selected XSD or inline schema determines actual parse output or render input.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Rendered XML text<br>xmlString | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Rendered content<br>content | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| XML_PARSE | The XML input cannot be parsed. |
| XML_RENDER | The value cannot be rendered as XML. |
| SCHEMA_VALIDATION | The data does not conform to the selected schema. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Parse JSON

Activities / Data

## Overview

Converts between serialized JSON and a schema-defined activity tree.

Parse JSON accepts jsonString and publishes the structure selected in Output Editor. Render JSON maps the structure selected in Input Editor and emits jsonString.

Activity type: json; operation: parse.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Schema type<br>schemaType | select<br>Configuration setting | Choose Native JSON for ordinary JSON data, Generic for XML and JSON conversion without a schema, or XSD for a schema-defined XML structure.<br>Choices: Native JSON, Generic, XSD |
| Validate parsed output<br>validateOutput | boolean<br>Configuration setting | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected JSON schema determines actual parse output or render input.

| Field / key | Type / required | Details |
|---|---|---|
| JSON text<br>jsonString | string<br>Required | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Validate parsed output<br>formatOptions.validateOutput | boolean<br>Optional | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| Duplicate keys<br>formatOptions.duplicateKeyPolicy | string<br>Optional | Allowed values: Last wins, First wins, Error. |

## Output

The selected JSON schema determines actual parse output or render input.

| Field / key | Type / required | Details |
|---|---|---|
| Parsed JSON value<br>value | json<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| JSON_PARSE | The JSON input cannot be parsed. |
| JSON_RENDER | The value cannot be rendered as JSON. |
| SCHEMA_VALIDATION | The data does not conform to the selected schema. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Render JSON

Activities / Data

## Overview

Converts between serialized JSON and a schema-defined activity tree.

Parse JSON accepts jsonString and publishes the structure selected in Output Editor. Render JSON maps the structure selected in Input Editor and emits jsonString.

Activity type: json; operation: render.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Schema type<br>schemaType | select<br>Configuration setting | Choose Native JSON for ordinary JSON data, Generic for XML and JSON conversion without a schema, or XSD for a schema-defined XML structure.<br>Choices: Native JSON, Generic, XSD |
| Validate input tree<br>validateInput | boolean<br>Configuration setting | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| Omit null fields<br>omitNulls | boolean<br>Configuration setting | Omit null-valued properties from rendered output. Confirm that absence and explicit null have the intended meaning in the target contract. |
| Pretty print<br>prettyPrint | boolean<br>Configuration setting | Output formatting. prettyPrint enables readable layout and indent selects indentation width; formatting does not change business values. |
| Escape non-ASCII characters<br>asciiOnly | boolean<br>Configuration setting | Escape non-ASCII characters in rendered JSON when enabled. Disable to retain readable Unicode characters in the output text. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected JSON schema determines actual parse output or render input.

| Field / key | Type / required | Details |
|---|---|---|
| JSON value<br>value | any<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |
| Validate input tree<br>formatOptions.validateInput | boolean<br>Optional | Validate the associated value against the configured contract. A mismatch raises a validation fault; this does not replace validation of business rules. |
| JSON root<br>formatOptions.rootStyle | string<br>Optional | Allowed values: With root, Anonymous. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Omit null fields<br>formatOptions.omitNulls | boolean<br>Optional | Omit null-valued properties from rendered output. Confirm that absence and explicit null have the intended meaning in the target contract. |
| Pretty print<br>formatOptions.prettyPrint | boolean<br>Optional | Output formatting. prettyPrint enables readable layout and indent selects indentation width; formatting does not change business values. |
| Indent spaces<br>formatOptions.indent | integer<br>Optional | Output formatting. prettyPrint enables readable layout and indent selects indentation width; formatting does not change business values. |
| Escape non-ASCII characters<br>formatOptions.asciiOnly | boolean<br>Optional | Escape non-ASCII characters in rendered JSON when enabled. Disable to retain readable Unicode characters in the output text. |

## Output

The selected JSON schema determines actual parse output or render input.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Rendered JSON text<br>jsonString | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Rendered content<br>content | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| JSON_PARSE | The JSON input cannot be parsed. |
| JSON_RENDER | The value cannot be rendered as JSON. |
| SCHEMA_VALIDATION | The data does not conform to the selected schema. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Parse Data

Activities / Data

## Overview

Parses delimited/fixed-width text into records or renders records as formatted text.

The schema contract defines record fields; Configuration controls separators, widths, headers, trimming, and line endings. Parse Data publishes records while Render Data accepts mapped records and emits text.

Activity type: flat; operation: parse.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Input source<br>inputSource | select<br>Configuration setting | Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing.<br>Choices: String, File path |
| Manually specify start record<br>manualStartRecord | boolean<br>Configuration setting | Expose the zero-based start offset in Input. Otherwise a batch size advances the offset within the task. |
| Continue on record errors<br>continueOnError | boolean<br>Configuration setting | Report rejected records in ErrorRows and continue with the remaining records. |
| Shared Data Format<br>dataFormatId | dataFormat<br>Configuration setting | Choose a project JSON schema asset containing a dataFormat object, or configure the format inline. |
| First row is header<br>header | boolean<br>Configuration setting | Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based.<br>Shown when: (c) => !c.dataFormatId |
| Require exact column count<br>strictColumns | boolean<br>Configuration setting | Require record column counts to match the configured flat-data layout; inconsistent records fault instead of being silently accepted.<br>Shown when: (c) => !c.dataFormatId |
| Trim parsed values<br>trimValues | boolean<br>Configuration setting | Trim leading/trailing whitespace from parsed values. Disable when spaces are significant in the source contract.<br>Shown when: (c) => !c.dataFormatId |
| Append final line separator<br>includeFinalLineSeparator | boolean<br>Configuration setting | Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record.<br>Shown when: (c) => !c.dataFormatId |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The record schema determines child fields under records.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Formatted data string<br>text | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Header characters to skip<br>skipHeaderCharacters | integer<br>Optional | Number of leading characters to ignore before reading the header and records. |
| Records to parse (-1 for all)<br>noOfRecords | integer<br>Optional | Maximum input records to read in this batch, including rejected records. Use -1 to read to the end. |
| Manually specify start record<br>formatOptions.manualStartRecord | boolean<br>Optional | Expose the zero-based start offset in Input. Otherwise a batch size advances the offset within the task. |
| Continue on record errors<br>formatOptions.continueOnError | boolean<br>Optional | Report rejected records in ErrorRows and continue with the remaining records. |
| Output XML root element<br>formatOptions.rootElement | string<br>Optional | XML element name used for the document root or repeating record. Match the target XSD and namespace requirements. |
| Output XML record element<br>formatOptions.recordElement | string<br>Optional | XML element name used for the document root or repeating record. Match the target XSD and namespace requirements. |
| Format<br>formatOptions.format | string<br>Optional | Allowed values: delimited, fixed. |
| Column separator<br>formatOptions.delimiter | string<br>Optional | Separator used to split/join record fields. Choose a separator consistent with the data contract and account for quoting or embedded delimiters. |
| Multi-character separator rule<br>formatOptions.separatorRule | string<br>Optional | Allowed values: single, any-character. |
| Line separator<br>formatOptions.lineSeparator | string<br>Optional | Allowed values: Auto, LF, CRLF, CR. |
| First row is header<br>formatOptions.header | boolean<br>Optional | Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based. |
| Field names<br>formatOptions.fields | string<br>Optional | Comma-separated names used when the input has no header. An attached schema supplies these automatically. |
| Field data types<br>formatOptions.fieldTypes | string<br>Optional | Optional comma-separated XML schema types such as string, integer, decimal, and boolean. |
| Fixed field widths<br>formatOptions.widths | string<br>Optional | Fixed-width layout: widths defines each field length and fillCharacter supplies padding. Ensure lengths match the sender/receiver record specification. |
| Fixed field offsets<br>formatOptions.fieldOffsets | string<br>Optional | Optional zero-based start:end offsets (end excluded), one per field. Overrides widths. |
| Fixed line length including separator<br>formatOptions.lineLength | integer<br>Optional | Total fixed record length, including the configured line separator. It must cover all field offsets. |
| Fixed-width fill character<br>formatOptions.fillCharacter | string<br>Optional | Fixed-width layout: widths defines each field length and fillCharacter supplies padding. Ensure lengths match the sender/receiver record specification. |
| Require exact column count<br>formatOptions.strictColumns | boolean<br>Optional | Require record column counts to match the configured flat-data layout; inconsistent records fault instead of being silently accepted. |
| Trim parsed values<br>formatOptions.trimValues | boolean<br>Optional | Trim leading/trailing whitespace from parsed values. Disable when spaces are significant in the source contract. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Blank lines<br>formatOptions.blankLinePolicy | string<br>Optional | Allowed values: Skip, Stop, Keep. |
| Append final line separator<br>formatOptions.includeFinalLineSeparator | boolean<br>Optional | Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record. |
| Custom line ending<br>formatOptions.lineEnding | string<br>Optional | Map a custom line ending when no line separator is selected. |
| Skip blank records<br>formatOptions.skipBlankLines | boolean<br>Optional | Ignore empty input lines/rows. Disable when blank records carry a business meaning or positional layout. |

## Output

The record schema determines child fields under records.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Parsed data XML<br>xml | xml<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| XML text<br>xmlString | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Root element<br>root | string<br>Optional | Qualified root element name of the parsed XML document. Its namespace URI is retained; use the parsed value/tree for element and attribute mappings. |
| Parsed tree<br>value | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |
| Typed records<br>records | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Record count<br>recordCount | integer<br>Optional | Number of parsed or rendered data records, excluding the header row. Use this integer to validate expected row counts or report processing totals. |
| Field names<br>fields | array<br>Optional | Ordered field names from the parsed header, configured field list or selected flat-record schema. The order determines record column positions. |
| Parsed row structure<br>Rows.schema | object<br>Optional | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions. |
| Rejected records<br>ErrorRows.error | array<br>Optional | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |
| Record number<br>ErrorRows.error.recordNumber | integer<br>Optional | One-based record number after the preamble and header, for the rejected record. |
| Error message<br>ErrorRows.error.message | string<br>Optional | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Raw record<br>ErrorRows.error.raw | string<br>Optional | Input values from the rejected record, before conversion to the configured field types. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| End of input<br>done | boolean<br>Optional | True when no more records are available from this input. |
| Next record offset<br>nextRecord | integer<br>Optional | Zero-based offset for the next batch when manually choosing the start record. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| FLAT_PARSE | The FLAT input cannot be parsed. |
| FLAT_RENDER | The value cannot be rendered as FLAT. |
| SCHEMA_VALIDATION | The data does not conform to the selected schema. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Render Data

Activities / Data

## Overview

Parses delimited/fixed-width text into records or renders records as formatted text.

The schema contract defines record fields; Configuration controls separators, widths, headers, trimming, and line endings. Parse Data publishes records while Render Data accepts mapped records and emits text.

Activity type: flat; operation: render.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Shared Data Format<br>dataFormatId | dataFormat<br>Configuration setting | Choose a project JSON schema asset containing a dataFormat object, or configure the format inline. |
| First row is header<br>header | boolean<br>Configuration setting | Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based.<br>Shown when: (c) => !c.dataFormatId |
| Require exact column count<br>strictColumns | boolean<br>Configuration setting | Require record column counts to match the configured flat-data layout; inconsistent records fault instead of being silently accepted.<br>Shown when: (c) => !c.dataFormatId |
| Trim parsed values<br>trimValues | boolean<br>Configuration setting | Trim leading/trailing whitespace from parsed values. Disable when spaces are significant in the source contract.<br>Shown when: (c) => !c.dataFormatId |
| Append final line separator<br>includeFinalLineSeparator | boolean<br>Configuration setting | Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record.<br>Shown when: (c) => !c.dataFormatId |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The record schema determines child fields under records.

| Field / key | Type / required | Details |
|---|---|---|
| Data records<br>records | array<br>Required | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Format<br>formatOptions.format | string<br>Optional | Allowed values: delimited, fixed. |
| Column separator<br>formatOptions.delimiter | string<br>Optional | Separator used to split/join record fields. Choose a separator consistent with the data contract and account for quoting or embedded delimiters. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Multi-character separator rule<br>formatOptions.separatorRule | string<br>Optional | Allowed values: single, any-character. |
| Line separator<br>formatOptions.lineSeparator | string<br>Optional | Allowed values: Auto, LF, CRLF, CR. |
| First row is header<br>formatOptions.header | boolean<br>Optional | Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based. |
| Field names<br>formatOptions.fields | string<br>Optional | Comma-separated names used when the input has no header. An attached schema supplies these automatically. |
| Field data types<br>formatOptions.fieldTypes | string<br>Optional | Optional comma-separated XML schema types such as string, integer, decimal, and boolean. |
| Fixed field widths<br>formatOptions.widths | string<br>Optional | Fixed-width layout: widths defines each field length and fillCharacter supplies padding. Ensure lengths match the sender/receiver record specification. |
| Fixed field offsets<br>formatOptions.fieldOffsets | string<br>Optional | Optional zero-based start:end offsets (end excluded), one per field. Overrides widths. |
| Fixed line length including separator<br>formatOptions.lineLength | integer<br>Optional | Total fixed record length, including the configured line separator. It must cover all field offsets. |
| Fixed-width fill character<br>formatOptions.fillCharacter | string<br>Optional | Fixed-width layout: widths defines each field length and fillCharacter supplies padding. Ensure lengths match the sender/receiver record specification. |
| Require exact column count<br>formatOptions.strictColumns | boolean<br>Optional | Require record column counts to match the configured flat-data layout; inconsistent records fault instead of being silently accepted. |
| Trim parsed values<br>formatOptions.trimValues | boolean<br>Optional | Trim leading/trailing whitespace from parsed values. Disable when spaces are significant in the source contract. |
| Append final line separator<br>formatOptions.includeFinalLineSeparator | boolean<br>Optional | Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record. |
| Custom line ending<br>formatOptions.lineEnding | string<br>Optional | Map a custom line ending when no line separator is selected. |
| Skip blank records<br>formatOptions.skipBlankLines | boolean<br>Optional | Ignore empty input lines/rows. Disable when blank records carry a business meaning or positional layout. |

## Output

The record schema determines child fields under records.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Rendered FLAT text<br>text | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Rendered content<br>content | string<br>Optional | Textual document/content. Supply valid XML/JSON for parsers, or map rendered text for a wire-format destination. Encoding controls conversion to and from bytes. |
| Formatted data text<br>Text | string<br>Optional | Formatted records as text, including the configured column and line separators. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Record count<br>recordCount | integer<br>Optional | Number of parsed or rendered data records, excluding the header row. Use this integer to validate expected row counts or report processing totals. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| FLAT_PARSE | The FLAT input cannot be parsed. |
| FLAT_RENDER | The value cannot be rendered as FLAT. |
| SCHEMA_VALIDATION | The data does not conform to the selected schema. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Read Excel Workbook

Activities / Data

## Overview

Reads an Excel workbook and publishes every selected worksheet as nested JSON.

Leave Worksheet blank to read every tab. The configured header row becomes field names; dotted headers such as customer.address.city create nested objects. Formula cells can return their cached calculated values.

Activity type: excel; operation: read.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Workbook file path<br>filePath | text<br>Configuration setting | Reads .xlsx, .xlsm, or legacy .xls workbooks at runtime. |
| Worksheet name<br>sheetName | text<br>Configuration setting | Leave blank to return every worksheet/tab. |
| Header row<br>headerRow | number<br>Configuration setting | Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based. |
| First data row<br>startRow | number<br>Configuration setting | Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based. |
| Maximum rows per sheet (0 = all)<br>maximumRows | number<br>Configuration setting | Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available. |
| Return calculated cell values<br>dataOnly | boolean<br>Configuration setting | Read stored Excel formula results instead of formula text. Results depend on the workbook having been calculated and saved by a spreadsheet application. |
| Convert dotted headers into nested JSON fields<br>nestedHeaders | boolean<br>Configuration setting | Interpret dotted Excel header names as nested object paths when enabled; otherwise retain literal column names. |
| Skip blank rows<br>skipBlankRows | boolean<br>Configuration setting | Ignore empty input lines/rows. Disable when blank records carry a business meaning or positional layout. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Dynamic workbook path<br>filePath | string<br>Optional | File path on the runtime host. Ensure the runtime account can read the file and that the path exists in the deployment environment. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dynamic worksheet/tab name<br>sheetName | string<br>Optional | Excel worksheet name. Supply an exact existing sheet name, or leave empty to read the workbook sheets supported by this operation. |
| Dynamic header row<br>headerRow | integer<br>Optional | Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based. |
| Dynamic first row<br>startRow | integer<br>Optional | Header/starting-row selection for tabular data. Row-number fields use the activity-defined indexing; Excel headerRow/startRow are one-based. |
| Dynamic maximum rows<br>maximumRows | integer<br>Optional | Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available. |
| Return calculated cell values<br>dataOnly | boolean<br>Optional | Read stored Excel formula results instead of formula text. Results depend on the workbook having been calculated and saved by a spreadsheet application. |
| Convert dotted headers into nested JSON fields<br>nestedHeaders | boolean<br>Optional | Interpret dotted Excel header names as nested object paths when enabled; otherwise retain literal column names. |
| Skip blank rows<br>skipBlankRows | boolean<br>Optional | Ignore empty input lines/rows. Disable when blank records carry a business meaning or positional layout. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Workbook result<br>workbook | object<br>Optional | Workbook result metadata and worksheet collection. Each sheet includes its name, headers and row objects. |
| Workbook file name<br>workbook.fileName | string<br>Optional | File metadata: name, full path, directory, modification time, access flags or size in bytes, according to this field. Values describe the runtime filesystem. |
| Worksheet count<br>workbook.sheetCount | integer<br>Optional | Workbook result metadata and worksheet collection. Each sheet includes its name, headers and row objects. |
| Worksheets/tabs<br>workbook.sheets | array<br>Optional | Workbook result metadata and worksheet collection. Each sheet includes its name, headers and row objects. |
| Worksheet name<br>workbook.sheets.name | string<br>Optional | Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it. |
| Column headers<br>workbook.sheets.headers | array<br>Optional | Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings. |
| Nested JSON row data<br>workbook.sheets.rows | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}" |

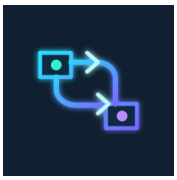
This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| EXCEL_READ | The workbook, requested worksheet, or cell data could not be read. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Mapper

Activities / General

## Overview

Maps execution-path data into a selected target schema.

In Input, drop a repeating complex source onto a repeating complex target. Copy-Of copies the whole collection and hides child bindings. For-Each creates editable child bindings for matching names; SmartMap also matches names that differ by underscores or hyphens. The loop shows the collection expression once and child expressions relative to the current item. Run Map & Test to check the output.

Activity type: mapper; operation: map.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Target schema<br>targetSchemaId | schema selection<br>Configuration setting | Select a schema from project Resources. Studio shows its name and derives the Input/Output trees, datatypes and cardinalities from its content. Select Inline schema to author a contract locally. |
| Inline target schema<br>targetSchemaText | JSON Schema / XSD<br>Configuration setting | Paste the target JSON Schema or XSD when Inline schema is selected. Update affected mapping paths after changing fields or repeating structures. |
| Mapping dialect<br>language | select<br>Configuration setting | Select the expression dialect presented by the editor. Use MINA-supported mapping expressions and functions; a compatibility selection does not supply a complete external XPath/XSLT processor.<br>Choices: XPath 2.0 / functions, XPath 1.0 compatibility, JSONPath / functions, XSLT 2.0 compatibility<br>Default: "XPath 2.0 / functions" |
| Null and missing values<br>nullPolicy | select<br>Configuration setting | Choose whether an unresolved/null mapped value omits its target, preserves null, emits an empty string or uses Default null value. Individual mapping rules can override the policy.<br>Choices: omit, preserve, empty-string, default<br>Default: "omit" |
| Default null value<br>defaultValue | value<br>Configuration setting | Fallback used for null/missing mapped values under the default policy. Supply a value compatible with the target datatype.<br>Shown when: nullPolicy = default |
| Type coercion<br>typeCoercion | select<br>Configuration setting | Strict enforces target types; safe allows supported conversions; off leaves mapped values uncoerced. Target-output validation can still reject values that do not match the schema.<br>Choices: strict, safe, off<br>Default: "safe" |
| Mapping error behavior<br>onMappingError | select<br>Configuration setting | Choose whether a field expression/conversion error fails the activity, omits that field or emits null. Continuing can produce incomplete output, so retain target validation where required.<br>Choices: fail, skip-field, use-null<br>Default: "fail" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Maximum output size (KB)<br>maxOutputSizeKb | integer<br>Configuration setting | Bound the generated output size in kilobytes. Zero disables this bound; use a positive limit to reject oversized mapped documents.<br>Default: 0 |
| Validate target output<br>validateOutput | boolean<br>Configuration setting | Check the completed mapped result against the selected target contract. A schema mismatch fails validation even when individual expression evaluation succeeded.<br>Default: true |
| Trim mapped strings<br>trimStrings | boolean<br>Configuration setting | Remove leading/trailing whitespace from mapped strings before output. Leave disabled when whitespace carries business meaning.<br>Default: false |
| Remove empty structures<br>removeEmptyStructures | boolean<br>Configuration setting | Clean empty containers from the completed mapped result. Check required schema structures after enabling this policy.<br>Default: false |
| Copy nil semantics<br>copyNil | boolean<br>Configuration setting | Retain explicit nil/null semantics from mapped sources. Disable to omit nil values; combine with the null policy and target schema requirements.<br>Default: true |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected target schema and mapping rules determine actual output fields.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Source document<br>source | object<br>Required | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Output

The selected target schema and mapping rules determine actual output fields.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Mapped document<br>result | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| MAPPING | A mapping expression failed. |
| SCHEMA_VALIDATION | The mapped output violates its target schema. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Transform

Activities / General

## Overview

Transforms payloads with MINA's executable DataWeave-compatible language subset.

Scripts support payload/attributes/vars selectors, objects, arrays, defaults, conditionals, concatenation, common coercion/string/collection functions, map/filter/groupBy/orderBy/distinctBy, JSON, XML, and text output. The editor validates and runs the same engine used at runtime.

Activity type: dataweave; operation: transform.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| DataWeave payload<br>payload | object<br>Required | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |
| Input attributes<br>attributes | object<br>Optional | Name/value metadata associated with the message. Keep business content in the body and map only properties understood by the receiving system. |
| Transform variables<br>variables | object<br>Optional | Environment/variable values for this operation. Use defaults only where a missing value is acceptable; variable scope is distinct from shared resource configuration. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Transformed payload<br>result | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Advanced

| Field / key | Type / required | Details |
|---|---|---|
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| DATAWEAVE_SYNTAX | The transform script could not be parsed. |
| DATAWEAVE_EXECUTION | A selector, function, coercion, or collection operation failed. |
| DATAWEAVE_OUTPUT | The transformed value cannot be rendered in the requested MIME type. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Log

Activities / General

## Overview

Writes a structured application log event.

Message, level, and payload accept constants or dynamic mappings. Advanced automatic payload logging can be used when a dedicated Log activity is unnecessary.

Activity type: log; operation: write.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Log level<br>level | select<br>Configuration setting | Logging configuration. Select an appropriate level; includePayload adds mapped content to logs and should be disabled for sensitive or large documents.<br>Choices: DEBUG, INFO, WARN, ERROR |
| Message template<br>message | text<br>Configuration setting | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Include payload<br>includePayload | boolean<br>Configuration setting | Logging configuration. Select an appropriate level; includePayload adds mapped content to logs and should be disabled for sensitive or large documents. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Message expression<br>message | string<br>Optional | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Payload text<br>payload | string<br>Optional | Map a value or enter a text/expression; objects are rendered into the log as structured JSON. |
| Log level<br>level | string<br>Optional | Logging configuration. Select an appropriate level; includePayload adds mapped content to logs and should be disabled for sensitive or large documents. |
| Include payload<br>includePayload | boolean<br>Optional | Logging configuration. Select an appropriate level; includePayload adds mapped content to logs and should be disabled for sensitive or large documents. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Unchanged payload<br>payload | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| LOGGING | The runtime logger rejected the event. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Catch Exception

Activities / General

## Overview

Runs Catch Exception on the task execution path.

Activity type: catch; operation: catch.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Catch all unhandled exceptions<br>catchAll | boolean<br>Configuration setting | Exception selection for Catch. catchAll handles any unhandled fault; otherwise select the fault type/code emitted by the relevant process activity. |
| Exception type<br>errorType | text<br>Configuration setting | Used when Catch All is false; for example XMLParseException. |
| Error code filter<br>errorCode | text<br>Configuration setting | Exception selection for Catch. catchAll handles any unhandled fault; otherwise select the fault type/code emitted by the relevant process activity. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Catch all unhandled exceptions<br>catchAll | boolean<br>Optional | Exception selection for Catch. catchAll handles any unhandled fault; otherwise select the fault type/code emitted by the relevant process activity. |
| Exception type<br>errorType | string<br>Optional | Used when Catch All is false; for example XMLParseException. |
| Error code filter<br>errorCode | string<br>Optional | Exception selection for Catch. catchAll handles any unhandled fault; otherwise select the fault type/code emitted by the relevant process activity. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Exception type<br>type | string<br>Optional | Type reported for this value or selected for this operation. Use the corresponding schema datatype and operation-specific choices. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Error code<br>code | string<br>Optional | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |
| Error message<br>message | string<br>Optional | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Stack trace<br>stackTrace | string<br>Optional | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |
| Failed activity ID<br>activityId | string<br>Optional | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |
| Fault details<br>details | object<br>Optional | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |
| Original cause<br>cause | object<br>Optional | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Throw Exception

Activities / General

## Overview

Runs Throw Exception on the task execution path.

Activity type: throw; operation: throw.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Exception type<br>errorType | text<br>Configuration setting | Exception selection for Catch. catchAll handles any unhandled fault; otherwise select the fault type/code emitted by the relevant process activity. |
| Error code<br>code | text<br>Configuration setting | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |
| Error message<br>message | text<br>Configuration setting | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Fault details (JSON)<br>details | textarea<br>Configuration setting | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Error message<br>message | string<br>Required | Message or response content. Map the business document rather than a previous publisher receipt. Objects retain their structure; string values carry serialized text. |
| Error code<br>code | string<br>Optional | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |
| Exception type<br>type | string<br>Optional | Type reported for this value or selected for this operation. Use the corresponding schema datatype and operation-specific choices. |
| Stack trace<br>stackTrace | string<br>Optional | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |
| Fault details<br>details | object<br>Optional | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Exception type<br>errorType | string<br>Optional | Exception selection for Catch. catchAll handles any unhandled fault; otherwise select the fault type/code emitted by the relevant process activity. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| UserDefinedException | The configured business fault is raised to the nearest matching handler. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Rethrow Exception

Activities / General

## Overview

Runs Rethrow Exception on the task execution path.

Activity type: rethrow; operation: rethrow.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Current caught exception<br>error | object<br>Optional | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| RethrowException | The current caught fault is propagated to the next available handler or calling task. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Confirm Message

Activities / General

## Overview

Acknowledges one or more client-managed messages.

Map the acknowledgement handle emitted by an EMS, Kafka, Pub/Sub, or compatible future receiver. The runtime dispatches confirmation to the originating connector.

Activity type: confirm; operation: acknowledge.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Acknowledgement handle<br>ackId | text<br>Configuration setting | Map a broker acknowledgement handle, or place this activity after an SAP IDoc Parser to confirm the SAP tRFC early. |
| Fail when handle is missing<br>failIfMissing | boolean<br>Configuration setting | Fault confirmation when no acknowledgement handle is available. Keep enabled when missing confirmation would otherwise hide an incorrectly mapped receive path. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Acknowledgement handle<br>ackId | string<br>Optional | Optional for SAP IDoc early confirmation; otherwise map the client acknowledgement handle. |
| Multiple acknowledgement handles<br>ackIds | array<br>Optional | Acknowledgement/settlement handle from a received message. Confirm only after required downstream work succeeds; preserve the original handle. |
| Fail when handle is missing<br>failIfMissing | boolean<br>Optional | Fault confirmation when no acknowledgement handle is available. Keep enabled when missing confirmation would otherwise hide an incorrectly mapped receive path. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Confirmation status<br>confirmed | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |
| Confirmed message count<br>count | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Confirmed transports technologies | array<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| ACKNOWLEDGEMENT_NOT_FOUND | The acknowledgement handle is missing, expired, or was already confirmed. |
| SAP_EARLY_CONFIRM_FAILED | SAP rejected the early IDoc confirmation. |
| ACKNOWLEDGEMENT_FAILED | The transport rejected the message confirmation. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Java Activity

Activities / General

## Overview

Runs Java Activity on the task execution path.

Activity type: java; operation: invoke.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Source mode<br>mode | select<br>Configuration setting | Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing.<br>Choices: JAR / class, Inline source |
| JAR, .class, or .java<br>artifactPath | artifact<br>Configuration setting | Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names. |
| Fully qualified Java class<br>className | text<br>Configuration setting | Java invocation settings/result. Provide the available class/method, select instance creation when required, and map the returned value using its actual datatype. |
| Method<br>method | text<br>Configuration setting | HTTP verb or invoked method name, according to this activity. For HTTP select the verb required by the API, such as GET, POST or PUT. |
| Instantiate class<br>instantiate | boolean<br>Configuration setting | Java invocation settings/result. Provide the available class/method, select instance creation when required, and map the returned value using its actual datatype. |
| Inline Java source<br>sourceCode | textarea<br>Configuration setting | Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments. |
| Timeout seconds<br>timeout | number<br>Configuration setting | Maximum time allowed for the selected operation. Configure the provider-specific wait and whole-activity timeout separately where both are exposed. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Method parameters<br>parameters | array<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| JSON input<br>payload | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |
| Source mode<br>mode | string<br>Optional | Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing. |
| JAR, .class, or .java<br>artifactPath | string<br>Optional | Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names. |
| Fully qualified Java class<br>className | string<br>Optional | Java invocation settings/result. Provide the available class/method, select instance creation when required, and map the returned value using its actual datatype. |
| Method<br>method | string<br>Optional | HTTP verb or invoked method name, according to this activity. For HTTP select the verb required by the API, such as GET, POST or PUT. |
| Instantiate class<br>instantiate | boolean<br>Optional | Java invocation settings/result. Provide the available class/method, select instance creation when required, and map the returned value using its actual datatype. |
| Inline Java source<br>sourceCode | string<br>Optional | Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments. |
| Timeout seconds<br>timeout | number<br>Optional | Maximum time allowed for the selected operation. Configure the provider-specific wait and whole-activity timeout separately where both are exposed. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Method return value<br>methodReturnValue | object<br>Optional | Java invocation settings/result. Provide the available class/method, select instance creation when required, and map the returned value using its actual datatype. |
| Invoked class<br>className | string<br>Optional | Java invocation settings/result. Provide the available class/method, select instance creation when required, and map the returned value using its actual datatype. |
| Invoked method<br>method | string<br>Optional | HTTP verb or invoked method name, according to this activity. For HTTP select the verb required by the API, such as GET, POST or PUT. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| JAVA_CLASS_NOT_FOUND | The configured class or JAR cannot be loaded. |
| JAVA_INVOCATION | The Java method threw an exception. |
| JAVA_TIMEOUT | The worker exceeded its timeout. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Python Invoke

Activities / General

## Overview

Runs Python Invoke on the task execution path.

Activity type: python; operation: invoke.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Source mode<br>mode | select<br>Configuration setting | Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing.<br>Choices: Python file / package, Inline source |
| .py, .zip, or .whl<br>artifactPath | artifact<br>Configuration setting | Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names. |
| Module name<br>moduleName | text<br>Configuration setting | Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments. |
| Function<br>function | text<br>Configuration setting | Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments. |
| Inline Python source<br>sourceCode | textarea<br>Configuration setting | Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments. |
| Timeout seconds<br>timeout | number<br>Configuration setting | Maximum time allowed for the selected operation. Configure the provider-specific wait and whole-activity timeout separately where both are exposed. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
|---|---|---|
| Function parameters<br>parameters | array<br>Optional | SQL/procedure parameters or mapped columns. Match parameter names and types to metadata and use parameter binding rather than constructing SQL from values. |
| JSON input<br>payload | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Source mode<br>mode | string<br>Optional | Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing. |
| .py, .zip, or .whl<br>artifactPath | string<br>Optional | Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names. |
| Module name<br>moduleName | string<br>Optional | Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments. |
| Function<br>function | string<br>Optional | Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments. |
| Inline Python source<br>sourceCode | string<br>Optional | Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments. |
| Timeout seconds<br>timeout | number<br>Optional | Maximum time allowed for the selected operation. Configure the provider-specific wait and whole-activity timeout separately where both are exposed. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Python result<br>result | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |
| Loaded module<br>module | string<br>Optional | Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments. |
| Invoked function<br>function | string<br>Optional | Python module/function or inline source to invoke. Deploy the module and its dependencies, and match the function signature to configured input arguments. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| PYTHON_IMPORT | The module or package could not be imported. |
| PYTHON_INVOCATION | The selected Python function raised an exception. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# External Command

Activities / General

## Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: external_command.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
|---|---|---|
| Command to execute<br>command | textarea<br>Configuration setting | Executable and arguments. Shell pipes are intentionally not evaluated. |
| Provide command output<br>provideCommandOutput | boolean<br>Configuration setting | External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable. |
| Remove parameter quotes<br>removeParameterQuotes | boolean<br>Configuration setting | External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable. |
| Output filename<br>outputFile | text<br>Configuration setting | External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable. |
| Output line splitting<br>outputLineSplitting | select<br>Configuration setting | External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable.<br>Choices: None, AtOperatingSystemLineEnd, AtSpecifiedToken |
| Specified split token<br>splitToken | text<br>Configuration setting | Token used to split command/text output into values. Choose a delimiter consistent with the producer output. |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Environment variables [NAME=value,...]<br>environment | text<br>Configuration setting | Environment/variable values for this operation. Use defaults only where a missing value is acceptable; variable scope is distinct from shared resource configuration. |
| Replace inherited environment<br>replaceEnvironment | boolean<br>Configuration setting | External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable. |
| Timeout seconds<br>timeoutSeconds | number<br>Configuration setting | Timeout seconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| Output encoding<br>encoding | text<br>Configuration setting | Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dynamic command<br>command | string<br>Optional | External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable. |
| Standard input<br>input | string<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |
| Dynamic output file<br>outputFile | string<br>Optional | External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable. |
| Dynamic environment<br>environment | object\|string<br>Optional | Environment/variable values for this operation. Use defaults only where a missing value is acceptable; variable scope is distinct from shared resource configuration. |
| Dynamic working directory<br>workingDirectory | string<br>Optional | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names. |
| Dynamic split token<br>splitToken | string<br>Optional | Token used to split command/text output into values. Choose a delimiter consistent with the producer output. |
| Provide command output<br>provideCommandOutput | boolean<br>Optional | External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable. |
| Remove parameter quotes<br>removeParameterQuotes | boolean<br>Optional | External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable. |
| Output line splitting<br>outputLineSplitting | string<br>Optional | External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable. |
| Replace inherited environment<br>replaceEnvironment | boolean<br>Optional | External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable. |
| Timeout seconds<br>timeoutSeconds | number<br>Optional | Timeout seconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |
| Output encoding<br>encoding | string<br>Optional | Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Process return code<br>returnCode | integer<br>Optional | External process exit code. Zero conventionally indicates success; inspect command output and the executable-specific exit-code contract. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Standard output<br>output | string\|array<br>Optional | Post-operation action or returned output. Use the activity-specific operation/contract to determine whether the value is content, metadata or a completion result. |
| Standard error<br>error | string\|array<br>Optional | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |
| Written output file<br>outputFile | string<br>Optional | External command execution/output policy. Supply the executable and arguments, choose captured/file output and environment handling, and retain quoting required by the executable. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| CommandExecutionError | The operating system command could not start or timed out. |
| FileIOError | Command output could not be written to the configured file. |
| InvalidInputException | The command or line-splitting configuration is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Assign

Activities / General

## Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: assign.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Process variable<br>variable | text<br>Configuration setting | Name stored in process variables. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Assigned value<br>value | object<br>Required | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |
| Process variable<br>variable | string<br>Optional | Name stored in process variables. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Variable name<br>name | string<br>Optional | Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it. |
| Assigned value<br>value | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| VALIDATION | Required input or mapped data is invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Checkpoint

Activities / General

## Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: checkpoint.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Checkpoint name<br>checkpointName | text<br>Configuration setting | Optional logical name stored in runtime execution metadata. |
| Include process state<br>includeProcessState | boolean<br>Configuration setting | Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Checkpoint name<br>checkpointName | string<br>Optional | Optional logical name stored in runtime execution metadata. |
| Include process state<br>includeProcessState | boolean<br>Optional | Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Checkpoint ID<br>checkpointId | string<br>Optional | Checkpoint identifier/content/output location. Process-state capture can include business payloads; protect generated artifacts and use stable checkpoint names. |
| Checkpoint timestamp<br>timestamp | dateTime<br>Optional | Time reported by the operation/provider. Preserve the original representation when tracing; epoch values and formatted date/time strings need explicit conversion. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| CheckpointException | The process state could not be persisted. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

# Deduplicate Message

Activities / General

## Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: deduplicate.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Durable state file<br>stateFile | text<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Message namespace<br>namespace | text<br>Required | Use the same namespace for claims, receipts and replay. |
| State capacity (MB)<br>maxStateSizeMb | number<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Claim lease (seconds)<br>leaseSeconds | number<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Stable message key<br>messageKey | string<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Message payload<br>payload | any<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Durable state file<br>stateFile | string<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Message namespace<br>namespace | string<br>Required | Use the same namespace for claims, receipts and replay. |
| State capacity (MB)<br>maxStateSizeMb | number<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Claim lease (seconds)<br>leaseSeconds | number<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Claim accepted<br>accepted | boolean<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |
| Duplicate delivery<br>duplicate | boolean<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Message key<br>messageKey | string<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Claim token<br>claimToken | string<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Claim status<br>status | string<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |
| Message payload<br>payload | any<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| MessageStateException | Durable state could not be read or written. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

# Message Receipt

Activities / General

## Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: message_receipt.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Durable state file<br>stateFile | text<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Message namespace<br>namespace | text<br>Required | Use the same namespace for claims, receipts and replay. |
| State capacity (MB)<br>maxStateSizeMb | number<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Receipt action<br>action | select<br>Configuration setting | Complete after downstream work; fail retains the message for explicit replay.<br>Choices: complete, fail |
| Completed-key retention (seconds)<br>retentionSeconds | number<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Message key<br>messageKey | string<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Claim token<br>claimToken | string<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Failure reason<br>error | string<br>Optional | Fault/debug metadata identifying the activity, error code, underlying cause, diagnostic details or stack trace. Use it to diagnose the failing operation. |
| Durable state file<br>stateFile | string<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Message namespace<br>namespace | string<br>Required | Use the same namespace for claims, receipts and replay. |
| State capacity (MB)<br>maxStateSizeMb | number<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Receipt action<br>action | string<br>Optional | Complete after downstream work; fail retains the message for explicit replay. |
| Completed-key retention (seconds)<br>retentionSeconds | number<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Message key<br>messageKey | string<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Receipt status<br>status | string<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| MessageStateException | Claim token is missing, stale or expired. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

# Replay Messages

Activities / General

## Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: replay_messages.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Durable state file<br>stateFile | text<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Message namespace<br>namespace | text<br>Required | Use the same namespace for claims, receipts and replay. |
| State capacity (MB)<br>maxStateSizeMb | number<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Replay action<br>action | select<br>Configuration setting | List failed/expired messages or explicitly requeue one key. No downstream work runs automatically.<br>Choices: list, requeue, purge |
| Maximum replay records<br>limit | number<br>Configuration setting | Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Key to requeue<br>messageKey | string<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Durable state file<br>stateFile | string<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Message namespace<br>namespace | string<br>Required | Use the same namespace for claims, receipts and replay. |
| State capacity (MB)<br>maxStateSizeMb | number<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Replay action<br>action | string<br>Optional | List failed/expired messages or explicitly requeue one key. No downstream work runs automatically. |
| Maximum replay records<br>limit | number<br>Optional | Upper bound on returned rows/items. Use a positive bound for large datasets and page/process subsequent batches where available. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Replay candidates<br>messages | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |
| Candidate count<br>count | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.<br>Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| MessageStateException | Message is not eligible for replay or state is unavailable. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

# Aggregate Messages

Activities / General

## Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: aggregate_messages.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Durable state file<br>stateFile | text<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Message namespace<br>namespace | text<br>Required | Use the same namespace for claims, receipts and replay. |
| State capacity (MB)<br>maxStateSizeMb | number<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Aggregation action<br>action | select<br>Configuration setting | Append a message; flush emits a partial group only after its timeout.<br>Choices: append, flush |
| Completion count<br>expectedCount | number<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Maximum group records<br>maxGroupItems | number<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Group timeout (seconds)<br>timeoutSeconds | number<br>Configuration setting | Group timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Correlation key<br>correlationKey | string<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Stable message ID<br>messageId | string<br>Optional | Message identifier returned by the provider or runtime. Retain it for tracing; use a separate stable business key for duplicate detection. |
| Message payload<br>payload | any<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |
| Durable state file<br>stateFile | string<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Message namespace<br>namespace | string<br>Required | Use the same namespace for claims, receipts and replay. |
| State capacity (MB)<br>maxStateSizeMb | number<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Aggregation action<br>action | string<br>Optional | Append a message; flush emits a partial group only after its timeout. |
| Completion count<br>expectedCount | number<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Maximum group records<br>maxGroupItems | number<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Group timeout (seconds)<br>timeoutSeconds | number<br>Optional | Group timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Group ready for delivery<br>ready | boolean<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Completed group records<br>records | array<br>Optional | Collection returned or supplied by the operation. Map repeating child records with For Each; row/record fields follow selected metadata or schema. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Group count<br>count | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |
| Group timed out<br>timedOut | boolean<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Group status<br>status | string<br>Optional | Operation outcome returned by the runtime. Inspect error output as well; provider acceptance does not establish processing by a downstream consumer. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| MessageStateException | Correlation, completion settings or capacity are invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

# Split Records

Activities / General

## Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: split_records.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Records per batch<br>batchSize | number<br>Configuration setting | Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Records<br>payload | array<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |
| Records per batch<br>batchSize | number<br>Optional | Batch execution size/policy. Bound batches to control memory and transaction duration; inspect per-batch failures before considering all records complete. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Record batches<br>batches | array<br>Optional | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing. |
| Record count<br>count | integer<br>Optional | Number of items returned or affected by this operation. Use zero-result handling where no matching data is a valid outcome. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| InvalidInputException | Records or batch size are invalid. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Sleep

Activities / General

## Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: sleep.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Duration<br>duration | number<br>Configuration setting | Sleep duration and its selected unit, or actual reported sleep in milliseconds. Sleeping pauses this branch, not unrelated parallel branches. |
| Unit<br>unit | select<br>Configuration setting | Sleep duration and its selected unit, or actual reported sleep in milliseconds. Sleeping pauses this branch, not unrelated parallel branches.<br>Choices: milliseconds, seconds, minutes |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Dynamic duration<br>duration | number<br>Optional | Sleep duration and its selected unit, or actual reported sleep in milliseconds. Sleeping pauses this branch, not unrelated parallel branches. |
| Unit<br>unit | string<br>Optional | Sleep duration and its selected unit, or actual reported sleep in milliseconds. Sleeping pauses this branch, not unrelated parallel branches. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Elapsed delay<br>sleptMilliseconds | integer<br>Optional | Sleep duration and its selected unit, or actual reported sleep in milliseconds. Sleeping pauses this branch, not unrelated parallel branches. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

| Fault | Description |
| --- | --- |
| INTERRUPTED | The wait was interrupted or cancelled. |

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Get Shared Variable

Activities / General

## Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: get_shared_variable.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Shared variable name<br>name | text<br>Configuration setting | Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Default value<br>default | object<br>Optional | Environment/variable values for this operation. Use defaults only where a missing value is acceptable; variable scope is distinct from shared resource configuration. |
| Shared variable name<br>name | string<br>Optional | Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Variable name<br>name | string<br>Optional | Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it. |
| Shared value<br>value | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx



# Set Shared Variable

Activities / General

## Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: set_shared_variable.

## Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor controls; additional connection requirements are documented under Shared connections.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Shared variable name<br>name | text<br>Configuration setting | Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it. |

## Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Shared value<br>value | object<br>Required | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |
| Shared variable name<br>name | string<br>Optional | Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it. |

## Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Variable name<br>name | string<br>Optional | Name identifying this entry. Keep it stable wherever downstream mappings or matching rules refer to it. |
| Shared value<br>value | object<br>Optional | The business value at this activity boundary. Map an explicit source field or document; use the selected schema to choose child fields and datatypes. |

## Advanced

| Field / key | Type / required | Details |
| --- | --- | --- |
| logPayload<br>logPayload | boolean expression<br>Configuration setting | Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: "${properties.advanced.logPayload}" |

This operation does not expose outbound retry in the common Advanced settings.

## Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

# If

Groups

## Configuration

Evaluates condition before entry; false skips the group.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

| Field / key | Type / required | Details |
| --- | --- | --- |
| condition<br>condition | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |

## Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

## Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

## Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

## Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

# While True

Groups

## Configuration

Evaluates condition before each iteration, including the first.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

| Field / key | Type / required | Details |
| --- | --- | --- |
| condition<br>condition | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |
| maxIterations<br>maxIterations | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "${properties.advanced.groupMaxIterations}" |
| indexVariable<br>indexVariable | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "index" |

## Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

## Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

## Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

## Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

# For Each

Groups

## Configuration

Resolves the collection at entry and runs once per item. Range mode uses inclusive start/end with a nonzero increment.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

| Field / key | Type / required | Details |
|---|---|---|
| iterationMode<br>iterationMode | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "collection" |
| source<br>source | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |
| start<br>start | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |
| end<br>end | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |
| increment<br>increment | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |
| currentElementName<br>currentElementName | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "currentElement" |
| itemVariable<br>itemVariable | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "currentElement" |
| indexVariable<br>indexVariable | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "index" |
| maxIterations<br>maxIterations | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "${properties.advanced.groupMaxIterations}" |
| accumulateOutput<br>accumulateOutput | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |
| accumulatorVariable<br>accumulatorVariable | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |

## Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.records}
${vars.currentElement}
${vars.currentIndex}
```

## Output

The configured current element and one-based index are visible inside the group. Optional accumulation records each successful group-exit result under accumulatorVariable.

## Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

## Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

# Iterate

Groups

## Configuration

Resolves source once at entry and iterates its elements.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

| Field / key | Type / required | Details |
| --- | --- | --- |
| source<br>source | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |
| currentElementName<br>currentElementName | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "currentElement" |
| itemVariable<br>itemVariable | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "currentElement" |
| indexVariable<br>indexVariable | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "index" |
| maxIterations<br>maxIterations | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "${properties.advanced.groupMaxIterations}" |
| accumulateOutput<br>accumulateOutput | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |
| accumulatorVariable<br>accumulatorVariable | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |

## Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.records}
${vars.currentElement}
${vars.currentIndex}
```

## Output

The configured current element and one-based index are visible inside the group. Optional accumulation records each successful group-exit result under accumulatorVariable.

## Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

## Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

# Repeat Until True

Groups

## Configuration

Runs once, checks condition at exit and repeats until true.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

| Field / key | Type / required | Details |
| --- | --- | --- |
| condition<br>condition | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |
| maxIterations<br>maxIterations | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "${properties.advanced.groupMaxIterations}" |
| indexVariable<br>indexVariable | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "index" |

## Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

## Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

## Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

## Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

# Repeat on Error Until True

Groups

## Configuration

Retries the whole group after an unhandled fault while stopCondition is false. A true stop condition or exhausted retry count propagates the fault.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

| Field / key | Type / required | Details |
| --- | --- | --- |
| stopCondition<br>stopCondition | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |
| retryCount<br>retryCount | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "${properties.advanced.retryCount}" |
| retryIntervalSeconds<br>retryIntervalSeconds | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "${properties.advanced.retryIntervalSeconds}" |
| indexVariable<br>indexVariable | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.<br>Default: "index" |

## Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${context.error}
${properties.advanced.retryCount}
${properties.advanced.retryIntervalSeconds}
```

## Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

## Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

## Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

# Scope

Groups

## Configuration

Runs the enclosed graph once. Nested groups retain their own semantics.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

No additional fields declared for this section.

## Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

## Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

## Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

## Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

# None

Groups

## Configuration

Runs the enclosed graph once; this is an execution boundary, not a separate transaction.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

No additional fields declared for this section.

## Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

## Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

## Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

## Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

# Critical Section

Groups

## Configuration

Holds a runtime-local named lock until completion, failure or cancellation. This is not a distributed lock across machines or separate runtime processes.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

| Field / key | Type / required | Details |
| --- | --- | --- |
| lockName<br>lockName | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |

## Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

## Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

## Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

## Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

# JDBC Transaction

Groups

## Configuration

Uses one database connection for matching JDBC operations, commits on successful exit and rolls back on an escaping fault or cancellation. External non-JDBC side effects are not rolled back.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

| Field / key | Type / required | Details |
| --- | --- | --- |
| resourceId<br>resourceId | expression / setting<br>Configuration setting | See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. |

## Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

## Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

## Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

## Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

# Pick First

Groups

## Configuration

Chooses the first eligible transition in graph order. This is first-match routing, not concurrent event racing or fastest-response selection.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

No additional fields declared for this section.

## Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

## Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

## Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

## Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

# concat

Functions / String

## Overview

Concatenate two or more values.

## Signature

```
concat(value, ...values)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
concat($value, "")
```

## Output

Concatenate two or more values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# trim

Functions / String

## Overview

Remove leading and trailing whitespace.

## Signature

```
trim(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
trim($value)
```

## Output

Remove leading and trailing whitespace.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# normalizeSpace

Functions / String

## Overview

Trim and collapse repeated whitespace.

## Signature

```
normalizeSpace(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
normalizeSpace($value)
```

## Output

Trim and collapse repeated whitespace.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# upperCase

Functions / String

## Overview

Convert text to uppercase.

## Signature

```
upperCase(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
upperCase($value)
```

## Output

Convert text to uppercase.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# lowerCase

Functions / String

## Overview

Convert text to lowercase.

## Signature

```
lowerCase(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
lowerCase($value)
```

## Output

Convert text to lowercase.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# capitalize

Functions / String

## Overview

Uppercase the first character.

## Signature

```
capitalize(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
capitalize($value)
```

## Output

Uppercase the first character.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# stringLength

Functions / String

## Overview

Return the number of characters.

## Signature

```
stringLength(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
stringLength($value)
```

## Output

Return the number of characters.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# substring

Functions / String

## Overview

Extract text using one-based positions.

## Signature

```
substring(value, start, length?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
substring($value, 1, 10)
```

## Output

Extract text using one-based positions.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# substringBefore

Functions / String

## Overview

Return text before the first token.

## Signature

```
substringBefore(value, token)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
substringBefore($value, "-")
```

## Output

Return text before the first token.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# substringAfter

Functions / String

## Overview

Return text after the first token.

## Signature

```
substringAfter(value, token)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
substringAfter($value, "-")
```

## Output

Return text after the first token.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# replace

Functions / String

## Overview

Replace text using a regular expression.

## Signature

```
replace(value, pattern, replacement)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
replace($value, "old", "new")
```

## Output

Replace text using a regular expression.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# matches

Functions / String

## Overview

Test text against a regular expression.

## Signature

```
matches(value, pattern)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
matches($value, "^[A-Z]+$")
```

## Output

Test text against a regular expression.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# startsWith

Functions / String

## Overview

Test whether text starts with a prefix.

## Signature

```
startsWith(value, prefix)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
startsWith($value, "PRE")
```

## Output

Test whether text starts with a prefix.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# endsWith

Functions / String

## Overview

Test whether text ends with a suffix.

## Signature

```
endsWith(value, suffix)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
endsWith($value, "END")
```

## Output

Test whether text ends with a suffix.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# contains

Functions / String

## Overview

Test whether text contains a token.

## Signature

```
contains(value, token)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
contains($value, "token")
```

## Output

Test whether text contains a token.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# compare

Functions / String

## Overview

Compare two strings and return -1, 0, or 1.

## Signature

```
compare(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
compare($value, "other")
```

## Output

Compare two strings and return -1, 0, or 1.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# translate

Functions / String

## Overview

Translate characters by position.

## Signature

```
translate(value, from, to)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
translate($value, "abc", "ABC")
```

## Output

Translate characters by position.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# padLeft

Functions / String

## Overview

Left-pad text to a fixed width.

## Signature

```
padLeft(value, width, fill?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
padLeft($value, 10, "0")
```

## Output

Left-pad text to a fixed width.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# padRight

Functions / String

## Overview

Right-pad text to a fixed width.

## Signature

```
padRight(value, width, fill?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
padRight($value, 10, " ")
```

## Output

Right-pad text to a fixed width.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# tokenize

Functions / String

## Overview

Split text using a regular expression.

## Signature

```
tokenize(value, pattern)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
tokenize($value, ",")
```

## Output

Split text using a regular expression.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# currentDate

Functions / Date & Time

## Overview

Current UTC date in ISO format.

## Signature

```
currentDate()
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
currentDate()
```

## Output

Current UTC date in ISO format.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# currentTime

Functions / Date & Time

## Overview

Current UTC time in ISO format.

## Signature

```
currentTime()
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
currentTime()
```

## Output

Current UTC time in ISO format.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# currentDateTime

Functions / Date & Time

## Overview

Current UTC timestamp in ISO format.

## Signature

```
currentDateTime()
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
currentDateTime()
```

## Output

Current UTC timestamp in ISO format.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# parseDate

Functions / Date & Time

## Overview

Parse an ISO date or timestamp.

## Signature

```
parseDate(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
parseDate($value)
```

## Output

Parse an ISO date or timestamp.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# formatDate

Functions / Date & Time

## Overview

Format a date with yyyy/MM/dd-style patterns.

## Signature

```
formatDate(value, pattern)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
formatDate($value, "yyyy-MM-dd")
```

## Output

Format a date with yyyy/MM/dd-style patterns.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# formatDateTime

Functions / Date & Time

## Overview

Format a timestamp.

## Signature

```
formatDateTime(value, pattern)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
formatDateTime($value, "yyyy-MM-dd HH:mm:ss")
```

## Output

Format a timestamp.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# addDays

Functions / Date & Time

## Overview

Add days to a date or timestamp.

## Signature

```
addDays(value, days)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
addDays($value, 1)
```

## Output

Add days to a date or timestamp.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# addMonths

Functions / Date & Time

## Overview

Add calendar months safely.

## Signature

```
addMonths(value, months)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
addMonths($value, 1)
```

## Output

Add calendar months safely.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# addHours

Functions / Date & Time

## Overview

Add hours to a timestamp.

## Signature

```
addHours(value, hours)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
addHours($value, 1)
```

## Output

Add hours to a timestamp.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# addMinutes

Functions / Date & Time

## Overview

Add minutes to a timestamp.

## Signature

```
addMinutes(value, minutes)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
addMinutes($value, 30)
```

## Output

Add minutes to a timestamp.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# addSeconds

Functions / Date & Time

## Overview

Add seconds to a timestamp.

## Signature

```
addSeconds(value, seconds)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
addSeconds($value, 30)
```

## Output

Add seconds to a timestamp.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# dateDifference

Functions / Date & Time

## Overview

Return the difference in days.

## Signature

```
dateDifference(from, to)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
dateDifference($value, currentDateTime())
```

## Output

Return the difference in days.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# year

Functions / Date & Time

## Overview

Extract the year component.

## Signature

```
year(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
year($value)
```

## Output

Extract the year component.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# month

Functions / Date & Time

## Overview

Extract the month component.

## Signature

```
month(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
month($value)
```

## Output

Extract the month component.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# day

Functions / Date & Time

## Overview

Extract the day component.

## Signature

```
day(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
day($value)
```

## Output

Extract the day component.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# hour

Functions / Date & Time

## Overview

Extract the hour component.

## Signature

```
hour(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
hour($value)
```

## Output

Extract the hour component.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# minute

Functions / Date & Time

## Overview

Extract the minute component.

## Signature

```
minute(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
minute($value)
```

## Output

Extract the minute component.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# second

Functions / Date & Time

## Overview

Extract the second component.

## Signature

```
second(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
second($value)
```

## Output

Extract the second component.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# timezoneFromDateTime

Functions / Date & Time

## Overview

Return the timestamp timezone offset.

## Signature

```
timezoneFromDateTime(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
timezoneFromDateTime($value)
```

## Output

Return the timestamp timezone offset.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# equal

Functions / Comparison

## Overview

Compare values using =.

## Signature

```
equal(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
equal($value, 0)
```

## Output

Compare values using =.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# notEqual

Functions / Comparison

## Overview

Compare values using !=.

## Signature

```
notEqual(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
notEqual($value, 0)
```

## Output

Compare values using !=.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# greaterThan

Functions / Comparison

## Overview

Compare values using >.

## Signature

```
greaterThan(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
greaterThan($value, 0)
```

## Output

Compare values using >.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# lessThan

Functions / Comparison

## Overview

Compare values using <.

## Signature

```
lessThan(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
lessThan($value, 0)
```

## Output

Compare values using <.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# greaterOrEqual

Functions / Comparison

## Overview

Compare values using >=.

## Signature

```
greaterOrEqual(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
greaterOrEqual($value, 0)
```

## Output

Compare values using >=.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# lessOrEqual

Functions / Comparison

## Overview

Compare values using <=.

## Signature

```
lessOrEqual(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
lessOrEqual($value, 0)
```

## Output

Compare values using <=.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# add

Functions / Number

## Overview

Add numeric values.

## Signature

```
add(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
add($value, 2)
```

## Output

Add numeric values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# subtract

Functions / Number

## Overview

Subtract numeric values.

## Signature

```
subtract(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
subtract($value, 2)
```

## Output

Subtract numeric values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# multiply

Functions / Number

## Overview

Multiply numeric values.

## Signature

```
multiply(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
multiply($value, 2)
```

## Output

Multiply numeric values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# divide

Functions / Number

## Overview

Divide numeric values.

## Signature

```
divide(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
divide($value, 2)
```

## Output

Divide numeric values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# mod

Functions / Number

## Overview

Mod numeric values.

## Signature

```
mod(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
mod($value, 2)
```

## Output

Mod numeric values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# number

Functions / Number

## Overview

Convert a value to a number.

## Signature

```
number(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
number($value)
```

## Output

Convert a value to a number.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# integer

Functions / Number

## Overview

Convert a value to an integer.

## Signature

```
integer(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
integer($value)
```

## Output

Convert a value to an integer.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# round

Functions / Number

## Overview

Round to an optional decimal precision.

## Signature

```
round(value, precision?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
round($value, 2)
```

## Output

Round to an optional decimal precision.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# roundHalfToEven

Functions / Number

## Overview

Banker's rounding for financial values.

## Signature

```
roundHalfToEven(value, precision?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
roundHalfToEven($value, 2)
```

## Output

Banker's rounding for financial values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# floor

Functions / Number

## Overview

Round down to an integer.

## Signature

```
floor(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
floor($value)
```

## Output

Round down to an integer.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# ceiling

Functions / Number

## Overview

Round up to an integer.

## Signature

```
ceiling(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
ceiling($value)
```

## Output

Round up to an integer.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# abs

Functions / Number

## Overview

Return the absolute value.

## Signature

```
abs(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
abs($value)
```

## Output

Return the absolute value.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# sqrt

Functions / Number

## Overview

Return the square root.

## Signature

```
sqrt(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
sqrt($value)
```

## Output

Return the square root.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# power

Functions / Number

## Overview

Raise a value to a power.

## Signature

```
power(value, exponent)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
power($value, 2)
```

## Output

Raise a value to a power.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# modulo

Functions / Number

## Overview

Return the division remainder.

## Signature

```
modulo(value, divisor)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
modulo($value, 2)
```

## Output

Return the division remainder.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# clamp

Functions / Number

## Overview

Constrain a number to a range.

## Signature

```
clamp(value, minimum, maximum)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
clamp($value, 0, 100)
```

## Output

Constrain a number to a range.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# min

Functions / Number

## Overview

Min collection values.

## Signature

```
min(values)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
min($value)
```

## Output

Min collection values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# max

Functions / Number

## Overview

Max collection values.

## Signature

```
max(values)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
max($value)
```

## Output

Max collection values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# sum

Functions / Number

## Overview

Sum collection values.

## Signature

```
sum(values)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
sum($value)
```

## Output

Sum collection values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# average

Functions / Number

## Overview

Average collection values.

## Signature

```
average(values)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
average($value)
```

## Output

Average collection values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# count

Functions / Number

## Overview

Count collection values.

## Signature

```
count(values)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
count($value)
```

## Output

Count collection values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# lookup

Functions / Collection

## Overview

Look up a key in an object table.

## Signature

```
lookup(value, table, default?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
lookup($value, ${input.table}, "Unknown")
```

## Output

Look up a key in an object table.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# lookupTable

Functions / Collection

## Overview

Look up a key in a named table saved with this Mapper.

## Signature

```
lookupTable(value, name, default?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
lookupTable($value, "countries", "Unknown")
```

## Output

Look up a key in a named table saved with this Mapper.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# enrich

Functions / Collection

## Overview

Merge enrichment fields into a record; enrichment replaces matching fields.

## Signature

```
enrich(record, fields)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
enrich($value, ${input.fields})
```

## Output

Merge enrichment fields into a record; enrichment replaces matching fields.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# chunks

Functions / Collection

## Overview

Split records into bounded batches.

## Signature

```
chunks(records, size)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
chunks($value, 100)
```

## Output

Split records into bounded batches.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# joinBy

Functions / Collection

## Overview

Join records by matching keys; every match produces a merged record.

## Signature

```
joinBy(left, right, leftKey, rightKey)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
joinBy($value, ${input.right}, "id", "id")
```

## Output

Join records by matching keys; every match produces a merged record.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# group

Functions / Collection

## Overview

Group records by a field path, or group equal values. Returns an array of groups.

## Signature

```
group(values, path?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
group($value, "product")
```

## Output

Group records by a field path, or group equal values. Returns an array of groups.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# top

Functions / Collection

## Overview

Return the highest values in descending order (default: one).

## Signature

```
top(values, count?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
top($value, 10)
```

## Output

Return the highest values in descending order (default: one).

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# distinctValues

Functions / Collection

## Overview

Remove duplicate values while preserving order.

## Signature

```
distinctValues(values)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
distinctValues($value)
```

## Output

Remove duplicate values while preserving order.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# sort

Functions / Collection

## Overview

Sort collection values.

## Signature

```
sort(values, descending?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
sort($value)
```

## Output

Sort collection values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# reverse

Functions / Collection

## Overview

Reverse a collection or string.

## Signature

```
reverse(values)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
reverse($value)
```

## Output

Reverse a collection or string.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# first

Functions / Collection

## Overview

Return the first value.

## Signature

```
first(values)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
first($value)
```

## Output

Return the first value.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# last

Functions / Collection

## Overview

Return the last value.

## Signature

```
last(values)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
last($value)
```

## Output

Return the last value.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# indexOf

Functions / Collection

## Overview

Return the zero-based position of an item.

## Signature

```
indexOf(values, item)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
indexOf($value, "item")
```

## Output

Return the zero-based position of an item.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# subsequence

Functions / Collection

## Overview

Return part of a collection.

## Signature

```
subsequence(values, start, end?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
subsequence($value, 0, 10)
```

## Output

Return part of a collection.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# flatten

Functions / Collection

## Overview

Flatten one level of nested arrays.

## Signature

```
flatten(values)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
flatten($value)
```

## Output

Flatten one level of nested arrays.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# join

Functions / Collection

## Overview

Join collection values into text.

## Signature

```
join(values, separator)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
join($value, ",")
```

## Output

Join collection values into text.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# split

Functions / Collection

## Overview

Split text into a collection.

## Signature

```
split(value, pattern)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
split($value, ",")
```

## Output

Split text into a collection.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# deepEqual

Functions / Collection

## Overview

Compare nested values structurally.

## Signature

```
deepEqual(value, other)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
deepEqual($value, $input)
```

## Output

Compare nested values structurally.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# string

Functions / Conversion

## Overview

Convert a value to text.

## Signature

```
string(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
string($value)
```

## Output

Convert a value to text.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# boolean

Functions / Conversion

## Overview

Convert common true/false representations.

## Signature

```
boolean(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
boolean($value)
```

## Output

Convert common true/false representations.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# jsonParse

Functions / Conversion

## Overview

Parse JSON text.

## Signature

```
jsonParse(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
jsonParse($value)
```

## Output

Parse JSON text.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# jsonRender

Functions / Conversion

## Overview

Serialize a value as JSON.

## Signature

```
jsonRender(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
jsonRender($value)
```

## Output

Serialize a value as JSON.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# render-xml

Functions / Conversion

## Overview

Convert an XML object (including a parsed XML value) into an XML string. Use @name for attributes and #text for element text.

## Signature

```
render-xml(value, rootName?, pretty?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
render-xml($value)
```

## Output

Convert an XML object (including a parsed XML value) into an XML string. Use @name for attributes and #text for element text.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# coalesce

Functions / General

## Overview

Return the first non-null/non-empty value.

## Signature

```
coalesce(value, ...fallbacks)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
coalesce($value, "fallback")
```

## Output

Return the first non-null/non-empty value.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# default

Functions / General

## Overview

Apply a fallback to null or empty values.

## Signature

```
default(value, fallback)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
default($value, "fallback")
```

## Output

Apply a fallback to null or empty values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# exists

Functions / General

## Overview

Test whether a value exists.

## Signature

```
exists(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
exists($value)
```

## Output

Test whether a value exists.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# empty

Functions / General

## Overview

Test whether a value is empty.

## Signature

```
empty(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
empty($value)
```

## Output

Test whether a value is empty.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# not

Functions / General

## Overview

Negate a boolean value.

## Signature

```
not(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
not($value)
```

## Output

Negate a boolean value.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# ifThenElse

Functions / General

## Overview

Choose between two values.

## Signature

```
ifThenElse(condition, trueValue, falseValue)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
ifThenElse($value, "yes", "no")
```

## Output

Choose between two values.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# uuid

Functions / General

## Overview

Generate a UUID.

## Signature

```
uuid()
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
uuid()
```

## Output

Generate a UUID.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# base64Encode

Functions / Encoding

## Overview

Encode UTF-8 text as Base64.

## Signature

```
base64Encode(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
base64Encode($value)
```

## Output

Encode UTF-8 text as Base64.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# base64Decode

Functions / Encoding

## Overview

Decode Base64 into UTF-8 text.

## Signature

```
base64Decode(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
base64Decode($value)
```

## Output

Decode Base64 into UTF-8 text.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# hexEncode

Functions / Encoding

## Overview

Encode UTF-8 text as hexadecimal.

## Signature

```
hexEncode(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
hexEncode($value)
```

## Output

Encode UTF-8 text as hexadecimal.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# hexDecode

Functions / Encoding

## Overview

Decode hexadecimal into UTF-8 text.

## Signature

```
hexDecode(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
hexDecode($value)
```

## Output

Decode hexadecimal into UTF-8 text.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# urlEncode

Functions / Encoding

## Overview

Percent-encode text for a URL.

## Signature

```
urlEncode(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
urlEncode($value)
```

## Output

Percent-encode text for a URL.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# urlDecode

Functions / Encoding

## Overview

Decode URL-encoded text.

## Signature

```
urlDecode(value)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
urlDecode($value)
```

## Output

Decode URL-encoded text.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# hash

Functions / Encoding

## Overview

Hash text with SHA-256 or another supported algorithm.

## Signature

```
hash(value, algorithm?)
```

## Input / example

Insertion template: replace $value and other placeholders with data expressions or typed constants for your mapping context.

```
hash($value, "sha256")
```

## Output

Hash text with SHA-256 or another supported algorithm.

## Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: frontend/src/mapper-functions.ts

# JDBC shared connection

Shared connections

## Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Database<br>driver | select<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Choices: sqlite, postgresql, mysql, mariadb, sqlserver, oracle, db2, snowflake, databricks<br>Default: "postgresql" |
| Connection runtime<br>connectionMode | select<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Choices: jdbc, python<br>Default: "python"<br>Shown when: (config) => ["sqlserver", "mssql", "<br>oracle"].includes(config.driver) |
| JDBC driver JAR directory<br>driverDirectory | text<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: ""<br>Shown when: (config) => ["sqlserver", "mssql", "<br>oracle"].includes(config.driver) && config.connectionMode !== "<br>python"<br>Example / hint: Blank uses the MINA driver directory |
| JDBC driver class (blank = vendor default)<br>driverClass | text<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: ""<br>Shown when: (config) => ["sqlserver", "mssql", "<br>oracle"].includes(config.driver) && config.connectionMode !== "<br>python" |
| JDBC URL<br>url | text<br>Configuration setting | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.<br>Default: "jdbc:postgresql://localhost:5432/integration" |
| Host<br>host | text<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: "localhost" |
| Port<br>port | text<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: 5432 |
| Database name<br>database | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "integration" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Schema<br>schema | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "public" |
| PostgreSQL TLS mode<br>postgresSslMode | select<br>Configuration setting | PostgreSQL libpq TLS mode. Leave blank to use the connection URL or libpq default. Verify-full verifies the certificate chain and hostname.<br>Choices: , disable, allow, prefer, require, verify-ca, verify-full<br>Default: ""<br>Shown when: (config) => config.driver === "postgresql" |
| PostgreSQL trusted root certificate<br>postgresSslRootCert | text<br>Configuration setting | Path to the PostgreSQL root certificate on the runtime host for TLS verification.<br>Default: ""<br>Shown when: (config) => config.driver === "postgresql" |
| Username<br>username | text<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| Password<br>password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "" |
| SQL Server ODBC driver (legacy Python mode)<br>odbcDriver | text<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: ""<br>Shown when: (config) => ["sqlserver", "mssql"].includes(config.driver) && config.connectionMode === "python" |
| SQL Server authentication<br>authentication | select<br>Configuration setting | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Choices: SQL Server Authentication, Windows Integrated<br>Default: "SQL Server Authentication"<br>Shown when: (config) => ["sqlserver", "mssql"].includes(config.driver) |
| Encrypt SQL Server connection<br>encrypt | select<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Choices: true, false<br>Default: "true"<br>Shown when: (config) => ["sqlserver", "mssql"].includes(config.driver) |
| Trust SQL Server certificate<br>trustServerCertificate | select<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Choices: false, true<br>Default: "false"<br>Shown when: (config) => ["sqlserver", "mssql"].includes(config.driver) |
| Oracle service name<br>serviceName | text<br>Configuration setting | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.<br>Default: ""<br>Shown when: (config) => config.driver === "oracle" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Oracle SID (when service name is not used)<br>sid | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: ""<br>Shown when: (config) => config.driver === "oracle" |
| MySQL/MariaDB CA certificate file<br>sslCaFile | text<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: ""<br>Shown when: (config) => ["mysql", "mariadb"].includes(config.driver) |
| Databricks server hostname<br>serverHostname | text<br>Configuration setting | Provider endpoint override. Supply the reachable hostname/port or HTTP path expected by the chosen transport and the configured certificate identity.<br>Default: ""<br>Shown when: (config) => config.driver === "databricks" |
| Databricks HTTP path<br>httpPath | text<br>Configuration setting | Provider endpoint override. Supply the reachable hostname/port or HTTP path expected by the chosen transport and the configured certificate identity.<br>Default: ""<br>Shown when: (config) => config.driver === "databricks" |
| Databricks authentication<br>authentication | select<br>Configuration setting | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Choices: Personal Access Token, OAuth M2M, OAuth U2M<br>Default: "SQL Server Authentication"<br>Shown when: (config) => config.driver === "databricks" |
| Databricks access token<br>accessToken | password<br>Configuration setting | Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required.<br>Default: ""<br>Shown when: (config) => config.driver === "databricks" && config.authentication === "Personal Access Token" |
| Databricks OAuth client ID<br>clientId | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => config.driver === "databricks" && config.authentication === "OAuth M2M" |
| Databricks OAuth client secret<br>clientSecret | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => config.driver === "databricks" && config.authentication === "OAuth M2M" |
| Databricks catalog<br>catalog | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: ""<br>Shown when: (config) => config.driver === "databricks" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Databricks CloudFetch<br>useCloudFetch | select<br>Configuration setting | Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode.<br>Choices: true, false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: true<br>Shown when: (config) => config.driver === "databricks" |
| Connection timeout (seconds)<br>timeoutSeconds | text<br>Configuration setting | Connection timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30 |
| Minimum pool size<br>minimumPoolSize | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 1 |
| Maximum pool size<br>maximumPoolSize | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 10 |

## Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

## Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

# SNOWFLAKE shared connection

Shared connections

## Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

| Field / key | Type / required | Details |
|---|---|---|
| Runtime mode<br>mode | select<br>Configuration setting | Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing.<br>Choices: external, mock<br>Default: "external" |
| Authentication type<br>authenticationType | select<br>Configuration setting | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Choices: Username/Password, Federated Authentication and SSO, OAuth, Key Pair Authentication<br>Default: "Username/Password" |
| Authentication provider<br>provider | select<br>Configuration setting | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.<br>Choices: Snowflake, Okta<br>Default: "Snowflake" |
| Account [account.region.platform]<br>account | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "" |
| Snowflake username<br>username | text<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| Snowflake password<br>password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "Username/Password" |
| Okta token endpoint<br>oktaTokenEndpoint | text<br>Configuration setting | Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "OAuth" && config.provider === "Okta" |
| Okta username<br>oktaUsername | text<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "Federated Authentication and SSO" && config.provider === "Okta" |
| Okta password<br>oktaPassword | password<br>Configuration setting | Identity-provider password. Bind protected deployment configuration rather than embedding it in the project or logs. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: ""<br>Shown when: (config) => config.authenticationType === "<br>Federated Authentication and SSO" && config.provider === "Okta" |
| Okta endpoint URL<br>oktaEndpointUrl | text<br>Configuration setting | Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "<br>Federated Authentication and SSO" && config.provider === "Okta" |
| OAuth client ID<br>clientId | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "OAuth" |
| OAuth client secret<br>clientSecret | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "OAuth" |
| OAuth scope [session:role:<role>]<br>scope | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "OAuth" |
| Authorization code<br>authorizationCode | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "OAuth" && config.provider === "Snowflake" |
| OAuth redirect URI<br>redirectUri | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "OAuth" && config.provider === "Snowflake" |
| OAuth access token<br>accessToken | password<br>Configuration setting | Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "OAuth" |
| Private key file<br>privateKeyFile | text<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "Key Pair Authentication" |
| Field / key | Type / required | Details |
| Private key passphrase<br>privateKeyPassphrase | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: ""<br>Shown when: (config) => config.authenticationType === "Key Pair Authentication" |
| Login timeout (seconds)<br>loginTimeoutSeconds | text<br>Configuration setting | Login timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 60 |
| Warehouse<br>warehouse | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "" |
| Default database<br>database | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "" |
| Default schema<br>schema | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "PUBLIC" |
| Default role<br>role | text<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "" |
| Other properties [Name=Value;]<br>otherProperties | text<br>Configuration setting | Additional provider settings as name/value entries. Use exact supported provider option names; invalid options may be rejected at connection or execution time.<br>Default: "" |
| Minimum connections<br>minimumConnections | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 2 |
| Maximum connections<br>maximumConnections | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 8 |
| Maximum connection wait (seconds)<br>maximumConnectionWaitSeconds | text<br>Configuration setting | Provider lifetime/wait setting in seconds, as named by this field. Set it above normal work/latency while retaining a bounded recovery time.<br>Default: 300 |
| Service number of threads<br>serviceThreads | text<br>Configuration setting | Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit.<br>Default: 8 |

## Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

## Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

# AMQP shared connection

Shared connections

## Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

| Field / key | Type / required | Details |
|---|---|---|
| Broker type<br>brokerType | select<br>Configuration setting | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.<br>Choices: Qpid-1-0, RabbitMQ, ActiveMQ-1-0, AzureSB-1-0, AMQ-1-0, ActiveMQ-Artemis-1-0<br>Default: "RabbitMQ" |
| AMQP version<br>amqpVersion | select<br>Configuration setting | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.<br>Choices: AMQP-0-9-1, AMQP-1-0<br>Default: "AMQP-0-9-1"<br>Shown when: (config) => config.brokerType === "RabbitMQ" |
| Host:Port (comma-separated for failover)<br>hostPort | text<br>Required | Provider endpoint override. Supply the reachable hostname/port or HTTP path expected by the chosen transport and the configured certificate identity.<br>Default: "localhost:5672"<br>Shown when: (config) => config.brokerType !== "AzureSB-1-0" |
| Virtual host<br>virtualHost | text<br>Configuration setting | Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace.<br>Default: "/"<br>Shown when: (config) => ["RabbitMQ", "<br>Qpid-1-0"].includes(config.brokerType) |
| Username<br>username | text<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "guest"<br>Shown when: (config) => config.brokerType !== "AzureSB-1-0" |
| Password<br>password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "guest"<br>Shown when: (config) => config.brokerType !== "AzureSB-1-0" |
| Client ID<br>clientId | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => !["RabbitMQ", "<br>AzureSB-1-0"].includes(config.brokerType) |
| Azure authentication<br>authenticationType | select<br>Configuration setting | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Choices: SAS, OAuth, ManagedIdentity<br>Default: "SAS"<br>Shown when: (config) => config.brokerType === "AzureSB-1-0" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Azure Service Bus endpoint / connection string<br>connectionString | text<br>Required | Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace.<br>Default: ""<br>Shown when: (config) => config.brokerType === "AzureSB-1-0" |
| Azure tenant ID<br>tenantId | text<br>Required | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: ""<br>Shown when: (config) => config.brokerType === "AzureSB-1-0" && config.authenticationType === "OAuth" |
| Azure application/client ID<br>azureClientId | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => config.brokerType === "AzureSB-1-0" && ["OAuth", "ManagedIdentity"].includes(config.authenticationType) |
| Azure client secret<br>clientSecret | password<br>Required | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => config.brokerType === "AzureSB-1-0" && config.authenticationType === "OAuth" |
| Shared access key name<br>sharedAccessKeyName | text<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: ""<br>Shown when: (config) => config.brokerType === "AzureSB-1-0" && config.authenticationType === "SAS" |
| Shared access key<br>sharedAccessKey | password<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: ""<br>Shown when: (config) => config.brokerType === "AzureSB-1-0" && config.authenticationType === "SAS" |
| Default entity type<br>entityType | select<br>Configuration setting | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.<br>Choices: Queue, Topic<br>Default: "Queue"<br>Shown when: (config) => config.brokerType === "AzureSB-1-0" |
| Default entity name<br>entityName | text<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.<br>Default: ""<br>Shown when: (config) => config.brokerType === "AzureSB-1-0" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Default subscription<br>entitySubscriberName | text<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: ""<br>Shown when: (config) => config.brokerType === "AzureSB-1-0" && config.entityType === "Topic" |
| Connection timeout (msec)<br>connectionTimeoutMsec | text<br>Configuration setting | Connection timeout (msec): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000 |
| RabbitMQ session count (1-20)<br>sessionCount | text<br>Configuration setting | Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit.<br>Default: 1<br>Shown when: (config) => config.brokerType === "RabbitMQ" |
| Idle timeout (msec)<br>idleTimeoutMsec | text<br>Configuration setting | Idle timeout (msec): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 0<br>Shown when: (config) => config.brokerType === "RabbitMQ" && config.amqpVersion === "AMQP-1-0" |
| Connection recovery<br>connectionRecovery | select<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Choices: true, false<br>Default: "true" |
| Retry interval (msec)<br>retryIntervalMsec | text<br>Configuration setting | Retry interval (msec): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 3000<br>Shown when: (config) => config.brokerType !== "RabbitMQ" |
| Retry attempts<br>retryAttempts | text<br>Configuration setting | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.<br>Default: 20<br>Shown when: (config) => config.brokerType !== "RabbitMQ" |
| RabbitMQ recovery interval (msec)<br>networkRecoveryIntervalMsec | text<br>Configuration setting | RabbitMQ recovery interval (msec): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 5000<br>Shown when: (config) => config.brokerType === "RabbitMQ" |
| SSL confidentiality<br>sslEnabled | select<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Choices: false, true<br>Default: "false" |
| Trusted CA file<br>caFile | text<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: ""<br>Shown when: (config) => String(config.sslEnabled) === "true" |
| Client certificate<br>clientCertificateFile | text<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: ""<br>Shown when: (config) => String(config.sslEnabled) === "true" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Client private key<br>clientKeyFile | text<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: ""<br>Shown when: (config) => String(config.sslEnabled) === "true" |
| Private key password<br>clientKeyPassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: ""<br>Shown when: (config) => String(config.sslEnabled) === "true" |

## Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

## Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

# HTTP shared connection

Shared connections

## Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Connection role<br>connectorMode | select<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Choices: client, server, both<br>Default: "both" |
| MINA API Blueprint contract (JSON / YAML)<br>openApiDocument | text<br>Configuration setting | API Blueprint stores one validated contract on this dedicated server connection. Operation tasks select their operation IDs. Re-import when adding operations; editing a contract does not replace existing business mappings.<br>Default: ""<br>Shown when: (config) => !!config.apiBlueprintId |
| Protocol<br>scheme | select<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Choices: http, https<br>Default: "http" |
| Default host<br>host | text<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: "localhost" |
| Default port<br>port | text<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: 8080 |
| Base path<br>basePath | text<br>Configuration setting | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.<br>Default: "" |
| Host Swagger documentation<br>apiDocsEnabled | select<br>Configuration setting | Serve browser API documentation at /_mina/docs, the OpenAPI document at /_mina/openapi.json, and a downloadable Postman collection at /_mina/postman.json. Enabled by default for HTTP and REST listeners; /_mina is reserved while enabled.<br>Choices: true, false<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| API documentation access<br>apiDocsAccess | select<br>Configuration setting | Authenticated documentation uses the listener's configured authentication for its OpenAPI document and Postman download. Public exposes those documents without credentials. API operations always retain their own authentication.<br>Choices: authenticated, public<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Public API base URL<br>apiPublicBaseUrl | text<br>Configuration setting | Optional externally reachable HTTP or HTTPS URL, including a gateway prefix. Used by generated API clients. Run and Debug |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | show the live listener address separately. Do not include credentials, query strings, or fragments.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Outbound base URL<br>baseUrl | text<br>Configuration setting | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.<br>Default: "https://api.example.com"<br>Shown when: (config) => config.connectorMode !== "server" |
| HTTP protocol version<br>httpVersion | select<br>Configuration setting | HTTP protocol version reported for the response, for example HTTP/1.1.<br>Choices: 1.1, 2, 3<br>Default: "1.1" |
| Authentication<br>authentication | select<br>Configuration setting | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Choices: None, Basic, Digest, NTLM, Bearer, OAuth2, JWT, HMAC, LDAP, Certificate<br>Default: "None" |
| Username<br>username | text<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: ""<br>Shown when: (config) => ["Basic", "Digest", "<br>NTLM"].includes(config.authentication) |
| Password<br>password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: ""<br>Shown when: (config) => ["Basic", "Digest", "<br>NTLM"].includes(config.authentication) |
| Non-preemptive Basic authentication<br>nonPreemptiveAuthentication | select<br>Configuration setting | Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported.<br>Choices: false, true<br>Default: ""<br>Shown when: (config) => config.authentication === "Basic" && config.connectorMode !== "server" |
| Bearer access token<br>bearerToken | password<br>Configuration setting | Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required.<br>Default: ""<br>Shown when: (config) => ["Bearer"].includes(config.authentication) |
| Default confidentiality / system trust<br>defaultConfidentiality | select<br>Configuration setting | Require HTTPS, verify the provider certificate and hostname, and use system trust plus any configured trusted roots. A separate client certificate is needed only when the provider requires mutual TLS.<br>Choices: false, true<br>Default: "false"<br>Shown when: (config) => config.connectorMode !== "server" |
| Custom confidentiality<br>confidentiality | select<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Choices: false, true<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Enable TLS<br>tlsEnabled | select<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Choices: false, true<br>Default: "false" |
| Verify server certificate and hostname<br>verifyTls | select<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Choices: true, false<br>Default: "true"<br>Shown when: (config) => config.connectorMode !== "server" |
| Minimum TLS version<br>tlsVersion | select<br>Configuration setting | TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider.<br>Choices: TLSv1.2, TLSv1.3<br>Default: "TLSv1.2" |
| Maximum TLS version<br>maximumTlsVersion | select<br>Configuration setting | TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider.<br>Choices: , TLSv1.2, TLSv1.3<br>Default: "" |
| TLS 1.2 cipher list<br>cipherSuites | text<br>Configuration setting | TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider.<br>Default: "" |
| Trusted root certificate folder (chain certificates optional)<br>trustedCertificateFolder | text<br>Configuration setting | Place trusted root CA certificates in this folder. Intermediates are optional: the provider normally sends them during TLS negotiation. Do not copy the provider leaf certificate or private keys into the folder. A provider that omits an intermediate must correct its chain or supply the optional intermediate.<br>Default: ""<br>Example / hint: Folder path or ${properties.connections.http.trustedCertificateFolder} |
| Trust store format<br>trustStoreType | select<br>Configuration setting | Trust-store file and format used by the selected provider. Supply trusted issuers in the supported store format; a certificate folder is an alternative where the connector supports it.<br>Choices: PEM, PKCS12, JKS<br>Default: "" |
| Trust store / CA certificate path<br>trustStoreFile | text<br>Configuration setting | Trust-store file and format used by the selected provider. Supply trusted issuers in the supported store format; a certificate folder is an alternative where the connector supports it.<br>Default: "" |
| Trust store password<br>trustStorePassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "" |
| Identity store format<br>identityStoreType | select<br>Configuration setting | Identity store containing the local certificate/private key. Supply its runtime path, password and supported format; this is distinct from the peer CA trust store.<br>Choices: PKCS12, JKS<br>Default: "" |
| Identity store path<br>identityStoreFile | text<br>Configuration setting | Identity store containing the local certificate/private key. Supply its runtime path, password and supported format; this is distinct from the peer CA trust store.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Identity store password<br>identityStorePassword | password<br>Configuration setting | Identity store containing the local certificate/private key. Supply its runtime path, password and supported format; this is distinct from the peer CA trust store.<br>Default: "" |
| Identity key alias (JKS)<br>keyAlias | text<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |
| PEM certificate chain path<br>certificateFile | text<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |
| PEM private key path<br>privateKeyFile | text<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |
| PEM private key password<br>privateKeyPassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "" |
| Inbound client certificates<br>clientAuthentication | select<br>Configuration setting | TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider.<br>Choices: none, optional, required<br>Default: "none"<br>Shown when: (config) => config.connectorMode !== "client" |
| PEM certificate revocation list<br>crlFile | text<br>Configuration setting | TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider.<br>Default: "" |
| Connection timeout (ms; 0 unlimited)<br>connectionTimeoutMs | text<br>Configuration setting | Connection timeout (ms; 0 unlimited): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000<br>Shown when: (config) => config.connectorMode !== "server" |
| Socket / read timeout (ms; 0 unlimited)<br>socketTimeoutMs | text<br>Configuration setting | Socket / read timeout (ms; 0 unlimited): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 60000<br>Shown when: (config) => config.connectorMode !== "server" |
| Write timeout (ms; 0 unlimited)<br>writeTimeoutMs | text<br>Configuration setting | Write timeout (ms; 0 unlimited): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 60000<br>Shown when: (config) => config.connectorMode !== "server" |
| Connection pool wait timeout (ms; 0 unlimited)<br>poolTimeoutMs | text<br>Configuration setting | Connection pool wait timeout (ms; 0 unlimited): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000<br>Shown when: (config) => config.connectorMode !== "server" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Maximum total client connections<br>maximumTotalConnections | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 200<br>Shown when: (config) => config.connectorMode !== "server" |
| Maximum client connections per host<br>maximumConnectionsPerHost | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 20<br>Shown when: (config) => config.connectorMode !== "server" |
| Disable client pooling<br>disableConnectionPooling | select<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Choices: false, true<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Persistent connections<br>usePersistentConnections | select<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Choices: true, false<br>Default: "" |
| Idle connection timeout (ms)<br>idleConnectionTimeoutMs | text<br>Configuration setting | Idle connection timeout (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 40000 |
| Follow redirects<br>followRedirects | select<br>Configuration setting | Follow HTTP redirect responses. Disable when the flow must inspect or reject redirects explicitly.<br>Choices: false, true<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Connection retry count<br>retryCount | text<br>Configuration setting | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Connection retry delay (ms)<br>retryDelayMs | text<br>Configuration setting | Connection retry delay (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Retry response status codes<br>retryStatusCodes | text<br>Configuration setting | Comma-separated HTTP statuses eligible for bounded retries. Disabled when empty. Non-idempotent methods require an Idempotency-Key or the explicit retry override.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server"<br>Example / hint: 429,502,503,504 |
| Maximum retry delay (ms)<br>maxRetryDelayMs | text<br>Configuration setting | Caps exponential backoff and Retry-After waits. All attempts share the activity timeout budget.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Field / key | Type / required | Details |
| Randomize retry backoff<br>retryJitter | select<br>Configuration setting | Spread simultaneous retries using bounded random backoff to reduce load spikes.<br>Choices: true, false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Respect Retry-After<br>respectRetryAfter | select<br>Configuration setting | Honor a server retry delay or HTTP date, bounded by Maximum retry delay and the remaining activity budget.<br>Choices: true, false<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Allow retries of non-idempotent methods<br>retryNonIdempotent | select<br>Configuration setting | Enable only when the downstream operation safely deduplicates repeated POST/PATCH calls. Prefer a mapped Idempotency-Key.<br>Choices: false, true<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Circuit breaker failures (0 disabled)<br>circuitFailureThreshold | text<br>Configuration setting | Consecutive connectivity/server failures before opening this runtime's outbound circuit. A half-open probe tests recovery.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Circuit recovery delay (seconds)<br>circuitResetSeconds | text<br>Configuration setting | Wait before permitting one half-open probe. Success closes the circuit; failure reopens it.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Allowed browser origins<br>corsAllowedOrigins | text<br>Configuration setting | Space/comma-separated exact origins. Empty disables CORS. Wildcard cannot be combined with credentialed requests.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Allow credentialed CORS<br>corsAllowCredentials | select<br>Configuration setting | Permit browser credentials only for explicitly listed origins; never combine with a wildcard origin.<br>Choices: false, true<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Allowed CORS request headers<br>corsAllowedHeaders | text<br>Configuration setting | Comma-separated header names permitted by preflight. Defaults to content-type and authorization.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Response headers exposed to browsers<br>corsExposeHeaders | text<br>Configuration setting | Comma-separated response header names that browser code may read.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| CORS preflight cache (seconds)<br>corsMaxAgeSeconds | text<br>Configuration setting | Browser preflight cache duration, bounded to 86400 seconds. Default 600.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Requests per client/window (0 disabled)<br>rateLimitRequests | text<br>Configuration setting | Fixed-window limit keyed by the actual remote IP and listener route. Excess requests return 429 with Retry-After.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Rate limit window (seconds)<br>rateLimitWindowSeconds | text<br>Configuration setting | Window duration for per-client throttling. Use Redis policy storage when limits must span replicas.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Field / key | Type / required | Details |
| Replay protection / rate limit storage<br>policyStore | select<br>Configuration setting | Memory applies within one runtime. Redis uses atomic counters and nonce claims across replicas and fails closed if unavailable.<br>Choices: memory, redis |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Redis policy connection URL<br>policyRedisUrl | password<br>Configuration setting | Property-bound redis:// or rediss:// connection URL. Use rediss:// for encrypted connections and protect credentials.<br>Default: ""<br>Shown when: (config) => config.policyStore === "redis" |
| Redis policy timeout (seconds)<br>policyTimeoutSeconds | text<br>Configuration setting | Bound Redis connection and socket waits. Default 3 seconds; failures reject protected requests with 503.<br>Default: ""<br>Shown when: (config) => config.policyStore === "redis" |
| Inbound body storage<br>requestBodyMode | select<br>Configuration setting | File mode streams uploads to a temporary file; mappings receive fileName/size. The file is removed after the request flow finishes. Schema body validation requires memory mode.<br>Choices: memory, file<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Temporary upload directory<br>uploadDirectory | text<br>Configuration setting | Existing writable runtime directory for file-backed request bodies. Uploaded files are automatically cleaned on completion, failure and disconnect.<br>Default: ""<br>Shown when: (config) => config.requestBodyMode === "file" |
| Accept and resend cookies<br>enableCookies | select<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Choices: false, true<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Default headers JSON<br>headers | text<br>Configuration setting | Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Default query parameters JSON<br>query | text<br>Configuration setting | HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; encode values rather than concatenating untrusted text.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Proxy URL<br>proxyUrl | text<br>Configuration setting | Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Proxy host<br>proxyHost | text<br>Configuration setting | Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Proxy port<br>proxyPort | text<br>Configuration setting | Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported.<br>Default: 8080<br>Shown when: (config) => config.connectorMode !== "server" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Proxy username<br>proxyUsername | text<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Proxy password<br>proxyPassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Minimum QTP-equivalent authentication workers<br>minimumQtpThreads | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 10<br>Shown when: (config) => config.connectorMode !== "client" |
| Maximum HTTP workers / concurrent executions<br>maximumQtpThreads | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 75<br>Shown when: (config) => config.connectorMode !== "client" |
| Maximum queued HTTP connections<br>maxQueueSize | text<br>Configuration setting | Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work.<br>Default: 100<br>Shown when: (config) => config.connectorMode !== "client" |
| Execution queue timeout (ms)<br>queueTimeoutMs | text<br>Configuration setting | Execution queue timeout (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Socket accept backlog<br>acceptQueueSize | text<br>Configuration setting | Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work.<br>Default: 128<br>Shown when: (config) => config.connectorMode !== "client" |
| Inbound read timeout (ms)<br>readTimeoutMs | text<br>Configuration setting | Inbound read timeout (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000<br>Shown when: (config) => config.connectorMode !== "client" |
| Maximum request header bytes<br>maxHeaderBytes | text<br>Configuration setting | Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work.<br>Default: 65536<br>Shown when: (config) => config.connectorMode !== "client" |
| Maximum request body bytes<br>maxRequestBodyBytes | text<br>Configuration setting | Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work.<br>Default: 67108864 |
| Maximum response body bytes<br>maxResponseBodyBytes | text<br>Configuration setting | Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work.<br>Default: 67108864 |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Allowed client IPs / CIDRs<br>allowedIps | text<br>Configuration setting | Inbound client IP allowlist for the hosted service. Configure the addresses visible to the runtime, including proxy/network translation where applicable.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| OpenAPI API key secrets<br>openApiApiKeySecrets | password<br>Configuration setting | Bind a protected property or enter a JSON object that maps OpenAPI security scheme names to API key secrets. Archived projects require the secrets at deployment.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Trusted proxy IPs / CIDRs<br>trustedProxyCidrs | text<br>Configuration setting | Proxy IPs or CIDRs allowed to supply X-Forwarded-For. Leave empty to use the transport peer for all requests.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Listener error response format<br>httpErrorFormat | select<br>Configuration setting | Use problem to return RFC 9457 Problem Details for listener-generated errors. The json option keeps the original error response shape.<br>Choices: json, problem<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Conditional GET and HEAD<br>conditionalResponses | select<br>Configuration setting | Return 304 for matching GET or HEAD validators using mapped ETag or Last-Modified response headers. The workflow still runs.<br>Choices: false, true<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "client" |
| Propagate HTTP trace context<br>propagateTraceContext | select<br>Configuration setting | Enable W3C version 00 trace context for this connection. Enable it on both the listener and outbound connection to follow a request across calls.<br>Choices: false, true<br>Default: "" |
| Shared public response cache<br>sharedHttpCache | select<br>Configuration setting | Share fresh, explicitly public GET responses within the runtime process. Credentialed and conditional requests bypass this cache.<br>Choices: false, true<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Allowed callback HTTPS origins<br>callbackAllowedOrigins | text<br>Configuration setting | Explicit HTTPS origins trusted to receive callback requests. Separate origins with commas or spaces.<br>Default: ""<br>Shown when: (config) => config.connectorMode !== "server" |
| Total activity / listener timeout (seconds)<br>activityTimeoutSeconds | text<br>Configuration setting | Total activity / listener timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 180 |
| OAuth2 grant<br>oauthGrantType | select<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Choices: client_credentials, authorization_code, refresh_token<br>Default: ""<br>Shown when: (config) => ["OAuth2"].includes(config.authentication) |
| Field / key | Type / required | Details |
| OAuth2 token URL<br>oauthTokenUrl | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | grant.<br>Default: ""<br>Shown when: (config) => ["OAuth2"].includes(config.authentication) |
| OAuth2 client ID<br>oauthClientId | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => ["OAuth2"].includes(config.authentication) |
| OAuth2 client secret<br>oauthClientSecret | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => config.authentication === "OAuth2" && config.oauthClientAuthentication !== "private_key_jwt" |
| OAuth2 client authentication<br>oauthClientAuthentication | select<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Choices: basic, body, private_key_jwt<br>Default: ""<br>Shown when: (config) => config.authentication === "OAuth2" |
| OAuth client signing key file<br>oauthClientPrivateKeyFile | text<br>Required | Path on the runtime host to the PEM signing key registered for the OAuth client. Provide the key file separately from the archive.<br>Default: ""<br>Shown when: (config) => config.authentication === "OAuth2" && config.oauthClientAuthentication === "private_key_jwt" |
| OAuth client assertion algorithm<br>oauthClientAssertionAlgorithm | select<br>Configuration setting | Asymmetric algorithm registered for the OAuth client. Defaults to RS256.<br>Choices: RS256, RS384, RS512, ES256, ES384, ES512<br>Default: ""<br>Shown when: (config) => config.authentication === "OAuth2" && config.oauthClientAuthentication === "private_key_jwt" |
| OAuth client assertion key ID<br>oauthClientAssertionKeyId | text<br>Configuration setting | Optional kid included in the signed OAuth client assertion so the provider can select a verification key.<br>Default: ""<br>Shown when: (config) => config.authentication === "OAuth2" && config.oauthClientAuthentication === "private_key_jwt" |
| OAuth2 scopes<br>oauthScope | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => ["OAuth2"].includes(config.authentication) |
| OAuth2 authorization code<br>oauthAuthorizationCode | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => ["OAuth2"].includes(config.authentication) |
| Field / key | Type / required | Details |
| OAuth2 redirect URI<br>oauthRedirectUri | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant. |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | Default: ""<br>Shown when: (config) => ["OAuth2"].includes(config.authentication) |
| OAuth2 PKCE verifier<br>oauthCodeVerifier | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => ["OAuth2"].includes(config.authentication) |
| OAuth2 refresh token<br>oauthRefreshToken | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => ["OAuth2"].includes(config.authentication) |
| OAuth2 endpoint timeout (seconds)<br>oauthTimeoutSeconds | text<br>Configuration setting | OAuth2 endpoint timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30<br>Shown when: (config) => ["OAuth2"].includes(config.authentication) |
| OAuth2 inbound token introspection URL<br>oauthIntrospectionUrl | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => ["OAuth2"].includes(config.authentication) |
| Required inbound OAuth2 scopes<br>oauthRequiredScopes | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Shown when: (config) => ["OAuth2"].includes(config.authentication) |
| Outbound preissued JWT<br>jwtToken | password<br>Configuration setting | Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required.<br>Default: ""<br>Shown when: (config) => ["JWT"].includes(config.authentication) |
| JWT signature algorithm<br>jwtAlgorithm | select<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Choices: HS256, HS384, HS512, RS256, RS384, RS512, ES256, ES384<br>Default: ""<br>Shown when: (config) => ["JWT"].includes(config.authentication) |
| JWT HMAC secret<br>jwtSecret | password<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: ""<br>Shown when: (config) => ["JWT"].includes(config.authentication) |
| JWT signing private key path<br>jwtPrivateKeyFile | text<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: ""<br>Shown when: (config) => ["JWT"].includes(config.authentication) |

| Field / key | Type / required | Details |
| --- | --- | --- |
| JWT verification public key path<br>jwtPublicKeyFile | text<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: ""<br>Shown when: (config) => ["JWT"].includes(config.authentication) |
| JWT verification JWKS URL (HTTPS)<br>jwtJwksUrl | text<br>Configuration setting | Trusted issuer's explicit HTTPS JWKS endpoint for RSA/EC verification. Tokens never select the URL. Configure the expected issuer, audience and asymmetric algorithm.<br>Default: ""<br>Shown when: (config) => config.authentication === "JWT" && config.connectorMode !== "client" |
| JWKS cache lifetime (seconds)<br>jwtJwksCacheSeconds | text<br>Configuration setting | Lifetime of verified discovery keys before refresh. Default 300 seconds. Expired cache does not bypass verification on discovery failure.<br>Default: ""<br>Shown when: (config) => config.authentication === "JWT" && ! !config.jwtJwksUrl |
| JWKS refresh cooldown (seconds)<br>jwtJwksRefreshSeconds | text<br>Configuration setting | Minimum interval between unknown-key refreshes; limits discovery traffic from attacker-controlled key IDs. Default 10 seconds.<br>Default: ""<br>Shown when: (config) => config.authentication === "JWT" && ! !config.jwtJwksUrl |
| JWKS fetch timeout (seconds)<br>jwtJwksTimeoutSeconds | text<br>Configuration setting | Maximum wait for key discovery. Default 5 seconds. Responses are bounded to 1 MiB and 128 keys.<br>Default: ""<br>Shown when: (config) => config.authentication === "JWT" && ! !config.jwtJwksUrl |
| JWT issuer<br>jwtIssuer | text<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: ""<br>Shown when: (config) => ["JWT"].includes(config.authentication) |
| JWT audience<br>jwtAudience | text<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: ""<br>Shown when: (config) => ["JWT"].includes(config.authentication) |
| Outbound JWT claims JSON<br>jwtClaims | text<br>Configuration setting | Additional JWT claims as an object. Keep issuer/audience/expiry consistent with verification settings and avoid placing secrets in readable token claims.<br>Default: ""<br>Shown when: (config) => ["JWT"].includes(config.authentication) |
| JWT lifetime (seconds)<br>jwtLifetimeSeconds | text<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: 300<br>Shown when: (config) => ["JWT"].includes(config.authentication) |
| JWT clock skew (seconds)<br>jwtClockSkewSeconds | text<br>Configuration setting | Allowed time difference in seconds when checking signed request/token timestamps. Use a small tolerance and synchronize runtime clocks.<br>Default: 30<br>Shown when: (config) => ["JWT"].includes(config.authentication) |
| Field / key | Type / required | Details |
| Request HMAC algorithm<br>hmacAlgorithm | select<br>Configuration setting | HMAC request signing/verification setting. Both endpoints must use the same signing algorithm, protected secret and signature |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | header convention.<br>Choices: SHA256, SHA384, SHA512<br>Default: ""<br>Shown when: (config) => ["HMAC"].includes(config.authentication) |
| Request HMAC key ID<br>hmacKeyId | text<br>Configuration setting | Identifier selecting the HMAC signing key. It identifies the key without exposing the shared secret.<br>Default: "default"<br>Shown when: (config) => ["HMAC"].includes(config.authentication) |
| Request HMAC secret<br>hmacSecret | password<br>Configuration setting | HMAC request signing/verification setting. Both endpoints must use the same signing algorithm, protected secret and signature header convention.<br>Default: ""<br>Shown when: (config) => ["HMAC"].includes(config.authentication) |
| Request HMAC clock skew (seconds)<br>hmacClockSkewSeconds | text<br>Configuration setting | Allowed time difference in seconds when checking signed request/token timestamps. Use a small tolerance and synchronize runtime clocks.<br>Default: 300<br>Shown when: (config) => ["HMAC"].includes(config.authentication) |
| LDAP / LDAPS URL<br>ldapUrl | text<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: ""<br>Shown when: (config) => ["LDAP"].includes(config.authentication) |
| Require LDAP StartTLS<br>ldapStartTls | select<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Choices: true, false<br>Default: ""<br>Shown when: (config) => ["LDAP"].includes(config.authentication) |
| LDAP trusted CA PEM path<br>ldapCaFile | text<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: ""<br>Shown when: (config) => ["LDAP"].includes(config.authentication) |
| LDAP operation timeout (seconds)<br>ldapTimeoutSeconds | text<br>Configuration setting | LDAP operation timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 10<br>Shown when: (config) => ["LDAP"].includes(config.authentication) |
| LDAP user DN template ({username})<br>ldapUserDnTemplate | text<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: ""<br>Shown when: (config) => ["LDAP"].includes(config.authentication) |
| LDAP search bind DN<br>ldapBindDn | text<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: ""<br>Shown when: (config) => ["LDAP"].includes(config.authentication) |

| Field / key | Type / required | Details |
| --- | --- | --- |
| LDAP search bind password<br>ldapBindPassword | password<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: ""<br>Shown when: (config) => ["LDAP"].includes(config.authentication) |
| LDAP user search base DN<br>ldapBaseDn | text<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: ""<br>Shown when: (config) => ["LDAP"].includes(config.authentication) |
| LDAP user filter ({username})<br>ldapSearchFilter | text<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: "(uid={username})"<br>Shown when: (config) => ["LDAP"].includes(config.authentication) |
| Required LDAP memberOf group DN<br>ldapRequiredGroup | text<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: ""<br>Shown when: (config) => ["LDAP"].includes(config.authentication) |

## Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

## Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

# FTP shared connection

Shared connections

## Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

| Field / key | Type / required | Details |
|---|---|---|
| Host<br>host | text<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: "ftp.example.com" |
| Port<br>port | text<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: 21 |
| Username<br>username | text<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| Password<br>password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "" |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.<br>Default: "/" |
| Passive mode<br>passiveMode | select<br>Configuration setting | Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode.<br>Choices: true, false<br>Default: true |
| Timeout (seconds)<br>timeoutSeconds | text<br>Configuration setting | Timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 60 |

## Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

## Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

# SFTP shared connection

Shared connections

## Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

| Field / key | Type / required | Details |
|---|---|---|
| Host<br>host | text<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: "sftp.example.com" |
| Port<br>port | text<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: 22 |
| Username<br>username | text<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| Password<br>password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "" |
| Working directory<br>workingDirectory | text<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.<br>Default: "/" |
| Private key file<br>privateKeyFile | text<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |
| Private key passphrase<br>privateKeyPassphrase | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "" |
| Known hosts file<br>knownHostsFile | text<br>Configuration setting | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.<br>Default: "" |
| Strict host-key checking<br>strictHostKeyChecking | select<br>Configuration setting | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.<br>Choices: true, false<br>Default: true |
| Timeout (seconds)<br>timeoutSeconds | text<br>Configuration setting | Timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 60 |

## Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

## Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

# EMS shared connection

Shared connections

## Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

| Field / key | Type / required | Details |
|---|---|---|
| JMS connection URL<br>serverUrl | text<br>Required | Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace.<br>Default: "tcp://localhost:7222"<br>Example / hint: tcp://ems-host:7222 |
| EMS/JMS driver JAR directory<br>driverDirectory | text<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: ""<br>Example / hint: Blank uses C:\ProgramData\MINA Studio\drivers\jms |
| Native connection factory class<br>connectionFactoryClass | text<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: ""<br>Example / hint: Fully qualified class name from your driver documentation |
| Connection factory type<br>connectionFactoryType | select<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Choices: Direct, JNDI<br>Default: "Direct" |
| Messaging style<br>messagingStyle | select<br>Configuration setting | Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.<br>Choices: Generic, Queue/Topic<br>Default: "Generic" |
| Username<br>username | text<br>Required | SAP logon user with authorization for the selected RFCs and metadata APIs.<br>Default: "" |
| Password<br>password | password<br>Required | SAP logon secret. Bind a secret/environment property; do not put credentials into examples, source control or payload logs.<br>Default: "" |
| Client ID<br>clientId | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Example / hint: Generated automatically when blank |

| Field / key | Type / required | Details |
|---|---|---|
| JNDI connection factory<br>connectionFactory | select<br>Required | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Choices: ConnectionFactory, QueueConnectionFactory, |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | TopicConnectionFactory<br>Default: "ConnectionFactory"<br>Shown when: (config) => config.connectionFactoryType === "JNDI" |
| Queue connection factory<br>queueConnectionFactory | text<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "QueueConnectionFactory"<br>Shown when: (config) => config.connectionFactoryType === "JNDI" && config.messagingStyle === "Queue/Topic" |
| Topic connection factory<br>topicConnectionFactory | text<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "TopicConnectionFactory"<br>Shown when: (config) => config.connectionFactoryType === "JNDI" && config.messagingStyle === "Queue/Topic" |
| JNDI context factory<br>jndiContextFactory | text<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: ""<br>Shown when: (config) => config.connectionFactoryType === "JNDI"<br>Example / hint: Leave blank to use the connector default |
| JNDI provider URL<br>jndiProviderUrl | text<br>Required | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "tcp://localhost:7222"<br>Shown when: (config) => config.connectionFactoryType === "JNDI" |
| JNDI username<br>jndiUsername | text<br>Required | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: ""<br>Shown when: (config) => config.connectionFactoryType === "JNDI" |
| JNDI password<br>jndiPassword | password<br>Required | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: ""<br>Shown when: (config) => config.connectionFactoryType === "JNDI" |
| Use XA connection factory<br>useXa | select<br>Configuration setting | Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode.<br>Choices: false, true<br>Default: false |
| Use EMS unshared failover<br>useUfo | select<br>Configuration setting | Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode.<br>Choices: false, true<br>Default: false |
| Enable SSL<br>sslEnabled | select<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Choices: false, true<br>Default: false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| SSL trusted certificates<br>sslTrustedCertificates | text<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: "" |
| Reconnect attempts<br>reconnectAttempts | text<br>Configuration setting | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.<br>Default: 3 |
| Reconnect delay (ms)<br>reconnectDelayMs | text<br>Configuration setting | Reconnect delay (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 5000 |
| Outgoing heartbeat (ms)<br>heartbeatOutgoingMs | text<br>Configuration setting | STOMP outgoing heartbeat interval in milliseconds. Zero disables that direction; the broker and client negotiate the effective heartbeat interval.<br>Default: 0 |
| Incoming heartbeat (ms)<br>heartbeatIncomingMs | text<br>Configuration setting | STOMP incoming heartbeat interval in milliseconds. Zero disables that direction; the broker and client negotiate the effective heartbeat interval.<br>Default: 0 |
| Connection timeout (seconds)<br>connectionTimeoutSeconds | text<br>Configuration setting | Connection timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30 |

## Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

## Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Direct connections use JMS URL, username and password. JNDI settings become required only when JNDI is enabled. Optional connection IDs are generated by the implementation.

Reference source: frontend/src/main.tsx

# JMS shared connection

Shared connections

## Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

| Field / key | Type / required | Details |
|---|---|---|
| JMS provider<br>provider | text<br>Configuration setting | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.<br>Default: "Generic JMS 2.0" |
| JMS connection URL<br>serverUrl | text<br>Required | Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace.<br>Default: "tcp://localhost:61613"<br>Example / hint: tcp://jms-host:61613 |
| JMS provider JAR directory<br>driverDirectory | text<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: ""<br>Example / hint: Blank uses C:\ProgramData\MINA Studio\drivers\jms |
| Connection factory class<br>connectionFactoryClass | text<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: "" |
| Connection factory type<br>connectionFactoryType | select<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Choices: Direct, JNDI<br>Default: "Direct" |
| Username<br>username | text<br>Required | SAP logon user with authorization for the selected RFCs and metadata APIs.<br>Default: "" |
| Password<br>password | password<br>Required | SAP logon secret. Bind a secret/environment property; do not put credentials into examples, source control or payload logs.<br>Default: "" |
| Client ID<br>clientId | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: ""<br>Example / hint: Generated automatically when blank |
| JNDI connection factory<br>connectionFactory | text<br>Required | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "ConnectionFactory"<br>Shown when: (config) => config.connectionFactoryType === "JNDI" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| JNDI initial context factory<br>jndiContextFactory | text<br>Required | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: ""<br>Shown when: (config) => config.connectionFactoryType === "JNDI" |
| JNDI provider URL<br>jndiProviderUrl | text<br>Required | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: ""<br>Shown when: (config) => config.connectionFactoryType === "JNDI" |
| JNDI username<br>jndiUsername | text<br>Required | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: ""<br>Shown when: (config) => config.connectionFactoryType === "JNDI" |
| JNDI password<br>jndiPassword | password<br>Required | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: ""<br>Shown when: (config) => config.connectionFactoryType === "JNDI" |
| Enable SSL<br>sslEnabled | select<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Choices: false, true<br>Default: false |
| Trusted certificates<br>sslTrustedCertificates | text<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: "" |
| Reconnect attempts<br>reconnectAttempts | text<br>Configuration setting | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.<br>Default: 3 |
| Reconnect delay (ms)<br>reconnectDelayMs | text<br>Configuration setting | Reconnect delay (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 5000 |
| Connection timeout (seconds)<br>connectionTimeoutSeconds | text<br>Configuration setting | Connection timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30 |

## Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

## Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Direct connections use JMS URL, username and password. JNDI settings become required only when JNDI is enabled. Optional connection IDs are generated by the implementation.

Reference source: frontend/src/main.tsx

# KAFKA shared connection

Shared connections

## Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

| Field / key | Type / required | Details |
|---|---|---|
| Bootstrap servers<br>bootstrapServers | text<br>Required | Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace.<br>Default: "localhost:9092" |
| Client ID<br>clientId | text<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| Default consumer group<br>groupId | text<br>Configuration setting | Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups.<br>Default: "mina" |
| Security protocol<br>securityProtocol | select<br>Configuration setting | Kafka transport/authentication selection. SASL_SSL combines TLS and SASL; configure credentials and CA verification for the selected SASL mechanism.<br>Choices: PLAINTEXT, SASL_PLAINTEXT, SASL_SSL, SSL<br>Default: "PLAINTEXT" |
| Authentication type<br>authenticationType | select<br>Configuration setting | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Choices: None, API Key / Secret, PLAIN, SCRAM-SHA-256, SCRAM-SHA-512, GSSAPI, OAUTHBEARER<br>Default: "None" |
| SASL mechanism<br>saslMechanism | select<br>Configuration setting | Kafka transport/authentication selection. SASL_SSL combines TLS and SASL; configure credentials and CA verification for the selected SASL mechanism.<br>Choices: PLAIN, SCRAM-SHA-256, SCRAM-SHA-512, GSSAPI, OAUTHBEARER<br>Default: "PLAIN"<br>Shown when: (config) => String(config.securityProtocol \|\| "").includes("SASL") && config.authenticationType !== "None" |
| Username / API key<br>username | text<br>Configuration setting | SAP logon user with authorization for the selected RFCs and metadata APIs.<br>Default: ""<br>Shown when: (config) => ["API Key / Secret", "PLAIN", "SCRAM-SHA-256", "SCRAM-SHA-512"].includes(config.authenticationType) && String(config.securityProtocol \|\| "").includes("SASL") |
| Password / API secret<br>password | password<br>Configuration setting | SAP logon secret. Bind a secret/environment property; do not put credentials into examples, source control or payload logs.<br>Default: ""<br>Shown when: (config) => ["API Key / Secret", "PLAIN", "SCRAM-SHA-256", "SCRAM-SHA-512"].includes(config.authenticationType) && String(config.securityProtocol \|\| "").includes("SASL") |

| Field / key | Type / required | Details |
| --- | --- | --- |
| SSL CA location<br>sslCaLocation | text<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: ""<br>Shown when: (config) => ["SSL", "SASL_SSL"].includes(config.securityProtocol) |
| SSL certificate location<br>sslCertificateLocation | text<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: ""<br>Shown when: (config) => ["SSL", "SASL_SSL"].includes(config.securityProtocol) |
| SSL key location<br>sslKeyLocation | text<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: ""<br>Shown when: (config) => ["SSL", "SASL_SSL"].includes(config.securityProtocol) |
| SSL key password<br>sslKeyPassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: ""<br>Shown when: (config) => ["SSL", "SASL_SSL"].includes(config.securityProtocol) |
| Schema Registry URL<br>schemaRegistryUrl | text<br>Configuration setting | Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.<br>Default: "" |
| Schema Registry username<br>schemaRegistryUsername | text<br>Configuration setting | Kafka schema-registry configuration. Select the registry endpoint/vendor and authentication; the Avro runtime limitations are documented on the Kafka operation pages.<br>Default: "" |
| Schema Registry password<br>schemaRegistryPassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "" |
| Schema Registry vendor<br>schemaRegistryVendor | select<br>Configuration setting | Kafka schema-registry configuration. Select the registry endpoint/vendor and authentication; the Avro runtime limitations are documented on the Kafka operation pages.<br>Choices: Confluent, Other, Apicurio<br>Default: "Confluent" |
| Use Kerberos ticket cache<br>useTicketCache | select<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Choices: false, true<br>Default: false<br>Shown when: (config) => config.authenticationType === "GSSAPI" |
| Kerberos keytab file<br>keytabFile | text<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "GSSAPI" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Kerberos principal<br>principalName | text<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "GSSAPI" |
| OAuth JAAS config file<br>jaasConfigFile | text<br>Configuration setting | JAAS login configuration file on the runtime host for the selected authentication mechanism. Protect credentials contained in the file.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "OAUTHBEARER" |
| OAuth login callback handler<br>loginCallbackHandler | text<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "OAUTHBEARER" |
| Request timeout (ms)<br>requestTimeoutMilliseconds | text<br>Configuration setting | Request timeout (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000 |
| Connection timeout (ms)<br>connectionTimeoutMilliseconds | text<br>Configuration setting | Connection timeout (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 10000 |
| Reconnect backoff (ms)<br>reconnectBackoffMilliseconds | text<br>Configuration setting | Reconnect backoff (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 50 |
| Retry backoff (ms)<br>retryBackoffMilliseconds | text<br>Configuration setting | Retry backoff (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 100 |
| Advanced client properties (JSON)<br>clientProperties | text<br>Configuration setting | Additional provider settings as name/value entries. Use exact supported provider option names; invalid options may be rejected at connection or execution time.<br>Default: "{}" |

## Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

## Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

# PUBSUB shared connection

Shared connections

## Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Authentication<br>authenticationType | select<br>Required | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Choices: Service Account JSON, Application Default Credentials, Emulator<br>Default: "Service Account JSON" |
| Service account JSON (inline)<br>serviceAccountJson | text<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "Service Account JSON" && !String(config.credentialsFile \|\| "").trim() |
| Service account JSON file<br>credentialsFile | text<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "Service Account JSON" && !String(config.serviceAccountJson \|\| "").trim()<br>Example / hint: Environment property resolving to the complete .json file path |
| GCP project ID<br>projectId | text<br>Configuration setting | Google Cloud project containing the Pub/Sub topic or subscription. Supply the project ID, not its display name.<br>Default: ""<br>Shown when: (config) => config.authenticationType !== "Service Account JSON"<br>Example / hint: Derived automatically from service-account JSON |
| Service endpoint<br>endpoint | text<br>Configuration setting | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.<br>Default: "pubsub.googleapis.com:443"<br>Shown when: (config) => config.authenticationType === " Application Default Credentials" |
| Corporate CA certificate file (optional)<br>caCertificateFile | text<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: ""<br>Shown when: (config) => config.authenticationType !== "Emulator"<br>Example / hint: Only required when TLS inspection uses an internal CA |
| Emulator host<br>emulatorHost | text<br>Configuration setting | Local Pub/Sub emulator address for testing. Leave empty for Google Cloud provider access and keep emulator configuration out of production environments.<br>Default: ""<br>Shown when: (config) => config.authenticationType === "Emulator"<br>Example / hint: localhost:8085 |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Ack deadline (seconds)<br>ackDeadlineSeconds | text<br>Configuration setting | Provider lifetime/wait setting in seconds, as named by this field. Set it above normal work/latency while retaining a bounded recovery time.<br>Default: 60 |
| Connection timeout (seconds)<br>connectionTimeoutSeconds | text<br>Configuration setting | Connection timeout (seconds): bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30 |
| Maximum inbound message bytes<br>maxInboundMessageBytes | text<br>Configuration setting | Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work.<br>Default: 20971520 |
| Keep-alive time (seconds)<br>keepAliveSeconds | text<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Default: 60 |
| Publisher batch maximum messages<br>batchMaxMessages | text<br>Configuration setting | Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency.<br>Default: 100 |
| Publisher batch maximum bytes<br>batchMaxBytes | text<br>Configuration setting | Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency.<br>Default: 1048576 |
| Publisher batch delay threshold (ms)<br>batchDelayThresholdMilliseconds | text<br>Configuration setting | Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency.<br>Default: 10 |
| Publisher flow-control message limit<br>flowControlMaxMessages | text<br>Configuration setting | Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity.<br>Default: 1000 |
| Publisher flow-control byte limit<br>flowControlMaxBytes | text<br>Configuration setting | Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity.<br>Default: 10485760 |

## Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

## Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Service Account JSON supplies Google authentication. Verify project/topic/subscription access and IAM permissions on the target environment.

Reference source: frontend/src/main.tsx

# SAP shared connection

Shared connections

## Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

| Field / key | Type / required | Details |
| --- | --- | --- |
| Runtime adapter<br>mode | select<br>Configuration setting | Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing.<br>Choices: mock, external<br>Default: "external" |
| SAP JCo driver directory<br>driverDirectory | text<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: ""<br>Example / hint: Blank uses C:\ProgramData\MINA Studio\drivers\sap |
| JCo destination name<br>destinationName | text<br>Configuration setting | Connection routing mode or named provider destination. Select the mode that matches the server configuration and supply its corresponding connection fields.<br>Default: ""<br>Example / hint: mina-sap |
| SAP release<br>release | select<br>Configuration setting | SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.<br>Choices: current, 720, 730<br>Default: "current" |
| Connection type<br>connectionType | select<br>Configuration setting | Connection routing mode: direct application server, message-server logon group, SNC variants or WebSocket configuration. Supply the corresponding shared-connection fields.<br>Choices: dedicated, logongroup, snc, sncwithlogongroup, websocket<br>Default: "dedicated" |
| Application server host<br>applicationServerHost | text<br>Configuration setting | Application-server hostname for direct connection mode; must resolve from the runtime host.<br>Default: "sap-ecc.example.com" |
| System number<br>systemNumber | text<br>Configuration setting | SAP instance/system number for a direct application-server connection, normally a two-character string.<br>Default: "00" |
| Client number<br>client | text<br>Configuration setting | SAP client identifier. Keep leading zeros; provide it as a string.<br>Default: "100" |
| Language<br>language | text<br>Configuration setting | SAP logon language key.<br>Default: "EN" |
| Username<br>username | text<br>Configuration setting | SAP logon user with authorization for the selected RFCs and metadata APIs.<br>Default: "" |
| Password<br>password | password<br>Configuration setting | SAP logon secret. Bind a secret/environment property; do not put credentials into examples, source control or payload logs.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Message server host<br>messageServerHost | text<br>Configuration setting | Message-server hostname for logon-group mode.<br>Default: "" |
| System ID<br>systemId | text<br>Configuration setting | SAP system ID used with message-server/logon-group routing.<br>Default: "" |
| Logon group<br>logonGroup | text<br>Configuration setting | SAP logon-group name configured by the SAP administrator.<br>Default: "PUBLIC" |
| SAP Router<br>sapRouter | text<br>Configuration setting | Optional SAProuter routing string supplied by the network/SAP administrator.<br>Default: "" |
| SNC partner name<br>sncPartnerName | text<br>Configuration setting | SNC partner identity expected by the target SAP system.<br>Default: ""<br>Shown when: (config) => ["snc", "sncwithlogongroup"].includes(config.connectionType) |
| SNC library path<br>sncLibraryPath | text<br>Configuration setting | Native SNC library path on the runtime host, matching OS and architecture.<br>Default: ""<br>Shown when: (config) => ["snc", "sncwithlogongroup"].includes(config.connectionType) |
| SNC own name<br>sncMyName | text<br>Configuration setting | Optional client SNC identity.<br>Default: ""<br>Shown when: (config) => ["snc", "sncwithlogongroup"].includes(config.connectionType) |
| SNC quality of protection<br>sncQop | select<br>Configuration setting | SNC quality-of-protection setting accepted by the target system.<br>Choices: , 1, 2, 3, 8, 9<br>Default: ""<br>Shown when: (config) => ["snc", "sncwithlogongroup"].includes(config.connectionType) |
| Program ID (inbound)<br>programId | text<br>Configuration setting | Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions.<br>Default: "" |
| Gateway host<br>gatewayHost | text<br>Configuration setting | SAP gateway hostname override for inbound RFC registration.<br>Default: "" |
| Gateway service<br>gatewayService | text<br>Configuration setting | Gateway service/port override for inbound RFC registration.<br>Default: "" |
| Maximum connections<br>maximumConnections | text<br>Configuration setting | Listener connection/concurrency limit. Size it with SAP gateway capacity and downstream processing, not just CPU count.<br>Default: 8 |
| Persistent outbound JVM workers<br>outboundWorkerCount | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 2 |
| JCo destination pool capacity<br>poolCapacity | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 1 |
| JCo destination peak limit<br>peakLimit | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 1 |

| Field / key | Type / required | Details |
| --- | --- | --- |
| Idle connection expiration (ms)<br>connectionExpirationMilliseconds | text<br>Configuration setting | Lifetime of an idle SAP pooled connection in milliseconds before it is eligible for expiration. Match this to the SAP server connection policy.<br>Default: 600000 |
| Outbound pool idle timeout (seconds)<br>outboundPoolIdleSeconds | text<br>Configuration setting | Provider lifetime/wait setting in seconds, as named by this field. Set it above normal work/latency while retaining a bounded recovery time.<br>Default: 1800 |
| Maximum distinct outbound pools<br>maximumOutboundPools | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 16 |
| Maximum buffered SAP requests<br>maxPendingEvents | text<br>Configuration setting | Bound on buffered listener events; prevents unbounded in-memory accumulation while downstream work is slow.<br>Default: 16 |
| JCo bridge initial heap (MB)<br>jvmInitialHeapMb | text<br>Configuration setting | Java bridge heap size in megabytes. Size it for concurrent payloads and driver overhead; keep the initial heap at or below the maximum.<br>Default: 64 |
| JCo bridge maximum heap (MB)<br>jvmMaximumHeapMb | text<br>Configuration setting | Java bridge heap size in megabytes. Size it for concurrent payloads and driver overhead; keep the initial heap at or below the maximum.<br>Default: 512 |
| Startup timeout (ms)<br>timeoutMilliseconds | text<br>Configuration setting | Startup timeout (ms): bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000 |
| Inbound processing/reply timeout (seconds)<br>ackTimeoutSeconds | text<br>Configuration setting | Maximum listener reply/acknowledgement wait in seconds. Set it above expected flow processing time and test timeout/redelivery behavior.<br>Default: 300 |
| Maximum dynamic sessions<br>maximumDynamicSessions | text<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 1024 |

## Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

## Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

# SAP_TID shared connection

Shared connections

## Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

| Field / key | Type / required | Details |
| --- | --- | --- |
| TID management mode | select<br>Configuration setting | Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing.<br>Choices: active, disabled<br>Default: "active" |
| Transaction ID storage file<br>storageFile | text<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.<br>Default: "data/sap-tids.properties"<br>Example / hint: Durable path shared by the deployed runtime |

## Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

## Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

# Environment property defaults

Shared connections

## Defaults

New projects start with core runtime properties. Adding a shared connection or a protocol activity automatically creates the missing required and optional protocol properties in each environment. Existing values are preserved. Use the property picker to bind connection fields to those properties.

The reference below lists the Studio templates. New connection settings supply non-secret values for new properties in the active environment. Other environments receive template defaults. Set password values separately for each environment.

| Field / key | Type / required | Details |
| --- | --- | --- |
| runtime.logDirectory<br>runtime.logDirectory | string<br>Configuration setting | Directory on the runtime host for execution logs. Ensure write permissions and configure retention to bound disk usage.<br>Default: "" |
| advanced.logPayload<br>advanced.logPayload | boolean<br>Configuration setting | Environment-controlled payload logging. Disable for confidential or large messages; log business identifiers and execution timings instead.<br>Default: false |
| advanced.retryEnabled<br>advanced.retryEnabled | boolean<br>Configuration setting | Enable outbound activity retries in this environment. Enable only when repeated external operations are safe or deduplicated.<br>Default: false |
| advanced.retryCount<br>advanced.retryCount | integer<br>Configuration setting | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.<br>Default: 3 |
| advanced.retryIntervalSeconds<br>advanced.retryIntervalSeconds | integer<br>Configuration setting | advanced.retryIntervalSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 60 |
| advanced.groupMaxIterations<br>advanced.groupMaxIterations | integer<br>Configuration setting | Maximum iterations allowed for a repeating group. Bounds accidental endless loops; choose a limit appropriate for the expected collection size.<br>Default: 10000 |
| connections.jdbc.driver<br>connections.jdbc.driver | string<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: "postgresql" |
| connections.jdbc.url<br>connections.jdbc.url | string<br>Configuration setting | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.<br>Default: "jdbc:postgresql://localhost:5432/integration" |
| connections.jdbc.host<br>connections.jdbc.host | string<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: "localhost" |
| connections.jdbc.port<br>connections.jdbc.port | integer<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: 5432 |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.jdbc.database<br>connections.jdbc.database | string<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "integration" |
| connections.jdbc.schema<br>connections.jdbc.schema | string<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "public" |
| connections.jdbc.username<br>connections.jdbc.username | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| connections.jdbc.password<br>connections.jdbc.password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.jdbc.connectionMode<br>connections.jdbc.connectionMode | string<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Default: "python" |
| connections.jdbc.driverDirectory<br>connections.jdbc.driverDirectory | string<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: "" |
| connections.jdbc.driverClass<br>connections.jdbc.driverClass | string<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: "" |
| connections.jdbc.odbcDriver<br>connections.jdbc.odbcDriver | string<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: "" |
| connections.jdbc.authentication<br>connections.jdbc.authentication | string<br>Configuration setting | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Default: "SQL Server Authentication" |
| connections.jdbc.encrypt<br>connections.jdbc.encrypt | boolean<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Default: true |
| connections.jdbc.trustServerCertificate<br>connections.jdbc.trustServerCertificate | boolean<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Default: false |
| connections.jdbc.serverHostname<br>connections.jdbc.serverHostname | string<br>Configuration setting | Provider endpoint override. Supply the reachable hostname/port or HTTP path expected by the chosen transport and the configured certificate identity.<br>Default: "" |
| connections.jdbc.httpPath<br>connections.jdbc.httpPath | string<br>Configuration setting | Provider endpoint override. Supply the reachable hostname/port or HTTP path expected by the chosen transport and the configured certificate identity.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.jdbc.accessToken<br>connections.jdbc.accessToken | password<br>Configuration setting | Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required.<br>Default: "[supply securely]" |
| connections.jdbc.clientId<br>connections.jdbc.clientId | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.jdbc.clientSecret<br>connections.jdbc.clientSecret | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "[supply securely]" |
| connections.jdbc.catalog<br>connections.jdbc.catalog | string<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "" |
| connections.jdbc.useCloudFetch<br>connections.jdbc.useCloudFetch | boolean<br>Configuration setting | Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode.<br>Default: true |
| connections.jdbc.serviceName<br>connections.jdbc.serviceName | string<br>Configuration setting | SOAP contract/operation settings. Select the service and operation from the WSDL; soapAction identifies the operation and envelope contains the SOAP XML request.<br>Default: "" |
| connections.jdbc.sid<br>connections.jdbc.sid | string<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "" |
| connections.jdbc.sslCaFile<br>connections.jdbc.sslCaFile | string<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: "" |
| connections.jdbc.timeoutSeconds<br>connections.jdbc.timeoutSeconds | integer<br>Configuration setting | connections.jdbc.timeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30 |
| connections.jdbc.minimumPoolSize<br>connections.jdbc.minimumPoolSize | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 1 |
| connections.jdbc.maximumPoolSize<br>connections.jdbc.maximumPoolSize | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 10 |
| connections.jdbc.postgresSslMode<br>connections.jdbc.postgresSslMode | string<br>Configuration setting | PostgreSQL libpq TLS mode. Leave blank to use the connection URL or libpq default. Verify-full verifies the certificate chain and hostname.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.jdbc.postgresSslRootCert<br>connections.jdbc.postgresSslRootCert | string<br>Configuration setting | Path to the PostgreSQL root certificate on the runtime host for TLS verification.<br>Default: "" |
| connections.snowflake.mode<br>connections.snowflake.mode | string<br>Configuration setting | Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing.<br>Default: "external" |
| connections.snowflake.authenticationType<br>connections.snowflake.authenticationType | string<br>Configuration setting | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Default: "Username/Password" |
| connections.snowflake.provider<br>connections.snowflake.provider | string<br>Configuration setting | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.<br>Default: "Snowflake" |
| connections.snowflake.account<br>connections.snowflake.account | string<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "" |
| connections.snowflake.username<br>connections.snowflake.username | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| connections.snowflake.password<br>connections.snowflake.password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.snowflake.warehouse<br>connections.snowflake.warehouse | string<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "" |
| connections.snowflake.database<br>connections.snowflake.database | string<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "" |
| connections.snowflake.schema<br>connections.snowflake.schema | string<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "PUBLIC" |
| connections.snowflake.role<br>connections.snowflake.role | string<br>Configuration setting | Database namespace/connection identifier. Match the selected provider account, database, schema, warehouse, role or table and the runtime user permissions.<br>Default: "" |
| connections.snowflake.loginTimeoutSeconds<br>connections.snowflake.loginTimeoutSeconds | integer<br>Configuration setting | connections.snowflake.loginTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 60 |
| connections.snowflake.minimumConnections<br>connections.snowflake.minimumConnections | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 2 |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.snowflake.maximumConnections<br>connections.snowflake.maximumConnections | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 8 |
| connections.snowflake.maximumConnectionWaitSeconds<br>connections.snowflake.maximumConnectionWaitSeconds | integer<br>Configuration setting | Provider lifetime/wait setting in seconds, as named by this field. Set it above normal work/latency while retaining a bounded recovery time.<br>Default: 300 |
| connections.snowflake.serviceThreads<br>connections.snowflake.serviceThreads | integer<br>Configuration setting | Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit.<br>Default: 8 |
| connections.snowflake.oktaTokenEndpoint<br>connections.snowflake.oktaTokenEndpoint | string<br>Configuration setting | Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required.<br>Default: "" |
| connections.snowflake.oktaUsername<br>connections.snowflake.oktaUsername | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| connections.snowflake.oktaPassword<br>connections.snowflake.oktaPassword | password<br>Configuration setting | Identity-provider password. Bind protected deployment configuration rather than embedding it in the project or logs.<br>Default: "[supply securely]" |
| connections.snowflake.oktaEndpointUrl<br>connections.snowflake.oktaEndpointUrl | string<br>Configuration setting | Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required.<br>Default: "" |
| connections.snowflake.clientId<br>connections.snowflake.clientId | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.snowflake.clientSecret<br>connections.snowflake.clientSecret | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "[supply securely]" |
| connections.snowflake.scope<br>connections.snowflake.scope | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.snowflake.authorizationCode<br>connections.snowflake.authorizationCode | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "[supply securely]" |
| connections.snowflake.redirectUri<br>connections.snowflake.redirectUri | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.snowflake.accessToken<br>connections.snowflake.accessToken | password<br>Configuration setting | Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required.<br>Default: "[supply securely]" |
| connections.snowflake.privateKeyFile<br>connections.snowflake.privateKeyFile | string<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |
| connections.snowflake.privateKeyPassphrase<br>connections.snowflake.privateKeyPassphrase | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.snowflake.otherProperties<br>connections.snowflake.otherProperties | string<br>Configuration setting | Additional provider settings as name/value entries. Use exact supported provider option names; invalid options may be rejected at connection or execution time.<br>Default: "" |
| connections.amqp.brokerType<br>connections.amqp.brokerType | string<br>Configuration setting | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.<br>Default: "RabbitMQ" |
| connections.amqp.amqpVersion<br>connections.amqp.amqpVersion | string<br>Configuration setting | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.<br>Default: "AMQP-0-9-1" |
| connections.amqp.hostPort<br>connections.amqp.hostPort | string<br>Configuration setting | Provider endpoint override. Supply the reachable hostname/port or HTTP path expected by the chosen transport and the configured certificate identity.<br>Default: "localhost:5672" |
| connections.amqp.virtualHost<br>connections.amqp.virtualHost | string<br>Configuration setting | Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace.<br>Default: "/" |
| connections.amqp.username<br>connections.amqp.username | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "guest" |
| connections.amqp.password<br>connections.amqp.password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.amqp.clientId<br>connections.amqp.clientId | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.amqp.authenticationType<br>connections.amqp.authenticationType | string<br>Configuration setting | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Default: "SAS" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.amqp.connectionString<br>connections.amqp.connectionString | password<br>Configuration setting | Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | broker host:port endpoints and AMQP virtualHost selects the broker namespace.<br>Default: "[supply securely]" |
| connections.amqp.tenantId<br>connections.amqp.tenantId | string<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: "" |
| connections.amqp.azureClientId<br>connections.amqp.azureClientId | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.amqp.clientSecret<br>connections.amqp.clientSecret | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "[supply securely]" |
| connections.amqp.sharedAccessKeyName<br>connections.amqp.sharedAccessKeyName | string<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: "" |
| connections.amqp.sharedAccessKey<br>connections.amqp.sharedAccessKey | password<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: "[supply securely]" |
| connections.amqp.entityType<br>connections.amqp.entityType | string<br>Configuration setting | AMQP routing configuration. entityType selects queue/topic where supported; exchanges route messages using the routing key and exchange type. Verify bindings before publishing.<br>Default: "Queue" |
| connections.amqp.entityName<br>connections.amqp.entityName | string<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.<br>Default: "" |
| connections.amqp.entitySubscriberName<br>connections.amqp.entitySubscriberName | string<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.<br>Default: "" |
| connections.amqp.connectionTimeoutMsec<br>connections.amqp.connectionTimeoutMsec | integer<br>Configuration setting | connections.amqp.connectionTimeoutMsec: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000 |
| connections.amqp.sessionCount<br>connections.amqp.sessionCount | integer<br>Configuration setting | Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit.<br>Default: 1 |
| connections.amqp.idleTimeoutMsec<br>connections.amqp.idleTimeoutMsec | integer<br>Configuration setting | connections.amqp.idleTimeoutMsec: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 0 |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.amqp.connectionRecovery<br>connections.amqp.connectionRecovery | boolean<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Default: true |
| connections.amqp.retryIntervalMsec<br>connections.amqp.retryIntervalMsec | integer<br>Configuration setting | connections.amqp.retryIntervalMsec: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 3000 |
| connections.amqp.retryAttempts<br>connections.amqp.retryAttempts | integer<br>Configuration setting | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.<br>Default: 20 |
| connections.amqp.networkRecoveryIntervalMsec<br>connections.amqp.networkRecoveryIntervalMsec | integer<br>Configuration setting | connections.amqp.networkRecoveryIntervalMsec: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 5000 |
| connections.amqp.sslEnabled<br>connections.amqp.sslEnabled | boolean<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Default: false |
| connections.amqp.caFile<br>connections.amqp.caFile | string<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: "" |
| connections.amqp.clientCertificateFile<br>connections.amqp.clientCertificateFile | string<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |
| connections.amqp.clientKeyFile<br>connections.amqp.clientKeyFile | string<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |
| connections.amqp.clientKeyPassword<br>connections.amqp.clientKeyPassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.http.baseUrl<br>connections.http.baseUrl | string<br>Configuration setting | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.<br>Default: "https://api.example.com" |
| connections.http.host<br>connections.http.host | string<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: "localhost" |
| connections.http.port<br>connections.http.port | integer<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: 8080 |
| connections.http.basePath<br>connections.http.basePath | string<br>Configuration setting | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.http.scheme<br>connections.http.scheme | string<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: "http" |
| connections.http.connectorMode<br>connections.http.connectorMode | string<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Default: "both" |
| connections.http.authentication<br>connections.http.authentication | string<br>Configuration setting | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Default: "None" |
| connections.http.bearerToken<br>connections.http.bearerToken | password<br>Configuration setting | Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required.<br>Default: "[supply securely]" |
| connections.http.tlsEnabled<br>connections.http.tlsEnabled | boolean<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Default: false |
| connections.http.defaultConfidentiality<br>connections.http.defaultConfidentiality | boolean<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Default: false |
| connections.http.certificateFile<br>connections.http.certificateFile | string<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |
| connections.http.privateKeyFile<br>connections.http.privateKeyFile | string<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |
| connections.http.privateKeyPassword<br>connections.http.privateKeyPassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.http.trustedCertificateFolder<br>connections.http.trustedCertificateFolder | string<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: "" |
| connections.http.certificateAuthorityFile<br>connections.http.certificateAuthorityFile | string<br>Configuration setting | Environment property connections.http.certificateAuthorityFile. Bind it with ${properties.connections.http.certificateAuthorityFile}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.clientAuthentication<br>connections.http.clientAuthentication | string<br>Configuration setting | TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider.<br>Default: "none" |
| Field / key | Type / required | Details |
| connections.http.tlsVersion<br>connections.http.tlsVersion | string<br>Configuration setting | TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | or client identity only where supported by the chosen provider.<br>Default: "TLSv1.2" |
| connections.http.connectTimeoutSeconds<br>connections.http.connectTimeoutSeconds | integer<br>Configuration setting | connections.http.connectTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30 |
| connections.http.timeoutSeconds<br>connections.http.timeoutSeconds | integer<br>Configuration setting | connections.http.timeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 60 |
| connections.http.username<br>connections.http.username | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| connections.http.password<br>connections.http.password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.http.proxyHost<br>connections.http.proxyHost | string<br>Configuration setting | Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported.<br>Default: "" |
| connections.http.proxyPort<br>connections.http.proxyPort | integer<br>Configuration setting | Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported.<br>Default: 8080 |
| connections.http.verifyTls<br>connections.http.verifyTls | boolean<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Default: true |
| connections.http.openApiDocument<br>connections.http.openApiDocument | string<br>Configuration setting | OpenAPI definition and selected operation identifier. The operation supplies method/path and request/response contracts; reselect it after replacing the specification.<br>Default: "" |
| connections.http.apiDocsEnabled<br>connections.http.apiDocsEnabled | string<br>Configuration setting | Environment property connections.http.apiDocsEnabled. Bind it with ${properties.connections.http.apiDocsEnabled}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.apiDocsAccess<br>connections.http.apiDocsAccess | string<br>Configuration setting | Environment property connections.http.apiDocsAccess. Bind it with ${properties.connections.http.apiDocsAccess}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.apiPublicBaseUrl<br>connections.http.apiPublicBaseUrl | string<br>Configuration setting | Environment property connections.http.apiPublicBaseUrl. Bind it with ${properties.connections.http.apiPublicBaseUrl}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.http.httpVersion<br>connections.http.httpVersion | string<br>Configuration setting | HTTP protocol version reported for the response, for example HTTP/1.1.<br>Default: "1.1" |
| connections.http.nonPreemptiveAuthentication<br>connections.http.nonPreemptiveAuthentication | string<br>Configuration setting | Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported.<br>Default: "" |
| connections.http.confidentiality<br>connections.http.confidentiality | string<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Default: "" |
| connections.http.maximumTlsVersion<br>connections.http.maximumTlsVersion | string<br>Configuration setting | TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider.<br>Default: "" |
| connections.http.cipherSuites<br>connections.http.cipherSuites | string<br>Configuration setting | TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider.<br>Default: "" |
| connections.http.trustStoreType<br>connections.http.trustStoreType | string<br>Configuration setting | Trust-store file and format used by the selected provider. Supply trusted issuers in the supported store format; a certificate folder is an alternative where the connector supports it.<br>Default: "" |
| connections.http.trustStoreFile<br>connections.http.trustStoreFile | string<br>Configuration setting | Trust-store file and format used by the selected provider. Supply trusted issuers in the supported store format; a certificate folder is an alternative where the connector supports it.<br>Default: "" |
| connections.http.trustStorePassword<br>connections.http.trustStorePassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.http.identityStoreType<br>connections.http.identityStoreType | string<br>Configuration setting | Identity store containing the local certificate/private key. Supply its runtime path, password and supported format; this is distinct from the peer CA trust store.<br>Default: "" |
| connections.http.identityStoreFile<br>connections.http.identityStoreFile | string<br>Configuration setting | Identity store containing the local certificate/private key. Supply its runtime path, password and supported format; this is distinct from the peer CA trust store.<br>Default: "" |
| connections.http.identityStorePassword<br>connections.http.identityStorePassword | password<br>Configuration setting | Identity store containing the local certificate/private key. Supply its runtime path, password and supported format; this is distinct from the peer CA trust store.<br>Default: "[supply securely]" |
| connections.http.keyAlias<br>connections.http.keyAlias | string<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |
| connections.http.crlFile<br>connections.http.crlFile | string<br>Configuration setting | TLS negotiation/client-certificate/revocation configuration. Select protocols and ciphers compatible with the peer and provide a CRL or client identity only where supported by the chosen provider.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.http.connectionTimeoutMs<br>connections.http.connectionTimeoutMs | integer<br>Configuration setting | connections.http.connectionTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000 |
| connections.http.socketTimeoutMs<br>connections.http.socketTimeoutMs | integer<br>Configuration setting | connections.http.socketTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 60000 |
| connections.http.writeTimeoutMs<br>connections.http.writeTimeoutMs | integer<br>Configuration setting | connections.http.writeTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 60000 |
| connections.http.poolTimeoutMs<br>connections.http.poolTimeoutMs | integer<br>Configuration setting | connections.http.poolTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000 |
| connections.http.maximumTotalConnections<br>connections.http.maximumTotalConnections | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 200 |
| connections.http.maximumConnectionsPerHost<br>connections.http.maximumConnectionsPerHost | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 20 |
| connections.http.disableConnectionPooling<br>connections.http.disableConnectionPooling | string<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Default: "" |
| connections.http.usePersistentConnections<br>connections.http.usePersistentConnections | string<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Default: "" |
| connections.http.idleConnectionTimeoutMs<br>connections.http.idleConnectionTimeoutMs | integer<br>Configuration setting | connections.http.idleConnectionTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 40000 |
| connections.http.followRedirects<br>connections.http.followRedirects | string<br>Configuration setting | Follow HTTP redirect responses. Disable when the flow must inspect or reject redirects explicitly.<br>Default: "" |
| connections.http.retryCount<br>connections.http.retryCount | string<br>Configuration setting | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.<br>Default: "" |
| connections.http.retryDelayMs<br>connections.http.retryDelayMs | string<br>Configuration setting | connections.http.retryDelayMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.http.retryStatusCodes<br>connections.http.retryStatusCodes | string<br>Configuration setting | Environment property connections.http.retryStatusCodes. Bind it with ${properties.connections.http.retryStatusCodes}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.maxRetryDelayMs<br>connections.http.maxRetryDelayMs | string<br>Configuration setting | connections.http.maxRetryDelayMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: "" |
| connections.http.retryJitter<br>connections.http.retryJitter | string<br>Configuration setting | Environment property connections.http.retryJitter. Bind it with ${properties.connections.http.retryJitter}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.respectRetryAfter<br>connections.http.respectRetryAfter | string<br>Configuration setting | Environment property connections.http.respectRetryAfter. Bind it with ${properties.connections.http.respectRetryAfter}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.retryNonIdempotent<br>connections.http.retryNonIdempotent | string<br>Configuration setting | Environment property connections.http.retryNonIdempotent. Bind it with ${properties.connections.http.retryNonIdempotent}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.circuitFailureThreshold<br>connections.http.circuitFailureThreshold | string<br>Configuration setting | Environment property connections.http.circuitFailureThreshold. Bind it with ${properties.connections.http.circuitFailureThreshold}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.circuitResetSeconds<br>connections.http.circuitResetSeconds | string<br>Configuration setting | Environment property connections.http.circuitResetSeconds. Bind it with ${properties.connections.http.circuitResetSeconds}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.corsAllowedOrigins<br>connections.http.corsAllowedOrigins | string<br>Configuration setting | Environment property connections.http.corsAllowedOrigins. Bind it with ${properties.connections.http.corsAllowedOrigins}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.corsAllowCredentials<br>connections.http.corsAllowCredentials | string<br>Configuration setting | Environment property connections.http.corsAllowCredentials. Bind it with ${properties.connections.http.corsAllowCredentials}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.corsAllowedHeaders<br>connections.http.corsAllowedHeaders | string<br>Configuration setting | Environment property connections.http.corsAllowedHeaders. Bind it with ${properties.connections.http.corsAllowedHeaders}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| Field / key | Type / required | Details |
| connections.http.corsExposeHeaders<br>connections.http.corsExposeHeaders | string<br>Configuration setting | Environment property connections.http.corsExposeHeaders. Bind it with ${properties.connections.http.corsExposeHeaders}; the active environment supplies its value. Keep its datatype consistent |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.corsMaxAgeSeconds<br>connections.http.corsMaxAgeSeconds | string<br>Configuration setting | Environment property connections.http.corsMaxAgeSeconds. Bind it with ${properties.connections.http.corsMaxAgeSeconds}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.rateLimitRequests<br>connections.http.rateLimitRequests | string<br>Configuration setting | Environment property connections.http.rateLimitRequests. Bind it with ${properties.connections.http.rateLimitRequests}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.rateLimitWindowSeconds<br>connections.http.rateLimitWindowSeconds | string<br>Configuration setting | Environment property connections.http.rateLimitWindowSeconds. Bind it with ${properties.connections.http.rateLimitWindowSeconds}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.policyStore<br>connections.http.policyStore | string<br>Configuration setting | Environment property connections.http.policyStore. Bind it with ${properties.connections.http.policyStore}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.policyRedisUrl<br>connections.http.policyRedisUrl | password<br>Configuration setting | Environment property connections.http.policyRedisUrl. Bind it with ${properties.connections.http.policyRedisUrl}; the active environment supplies its value. Supply the secret through protected deployment configuration.<br>Default: "[supply securely]" |
| connections.http.policyTimeoutSeconds<br>connections.http.policyTimeoutSeconds | string<br>Configuration setting | connections.http.policyTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: "" |
| connections.http.requestBodyMode<br>connections.http.requestBodyMode | string<br>Configuration setting | Environment property connections.http.requestBodyMode. Bind it with ${properties.connections.http.requestBodyMode}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.uploadDirectory<br>connections.http.uploadDirectory | string<br>Configuration setting | Environment property connections.http.uploadDirectory. Bind it with ${properties.connections.http.uploadDirectory}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.enableCookies<br>connections.http.enableCookies | string<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Default: "" |
| connections.http.headers<br>connections.http.headers | string<br>Configuration setting | Header name/value object. Map only headers required by the target protocol; credentials belong in protected authentication settings.<br>Default: "" |
| Field / key | Type / required | Details |
| connections.http.query<br>connections.http.query | string<br>Configuration setting | HTTP query parameters. Use a name/value object for structured parameters or QueryString for an already composed query; |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | encode values rather than concatenating untrusted text.<br>Default: "" |
| connections.http.proxyUrl<br>connections.http.proxyUrl | string<br>Configuration setting | Outbound proxy/authentication policy. Configure the proxy address and credentials when the runtime network requires it; non-preemptive authentication waits for a challenge where supported.<br>Default: "" |
| connections.http.proxyUsername<br>connections.http.proxyUsername | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| connections.http.proxyPassword<br>connections.http.proxyPassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.http.minimumQtpThreads<br>connections.http.minimumQtpThreads | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 10 |
| connections.http.maximumQtpThreads<br>connections.http.maximumQtpThreads | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 75 |
| connections.http.maxQueueSize<br>connections.http.maxQueueSize | integer<br>Configuration setting | Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work.<br>Default: 100 |
| connections.http.queueTimeoutMs<br>connections.http.queueTimeoutMs | string<br>Configuration setting | connections.http.queueTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: "" |
| connections.http.acceptQueueSize<br>connections.http.acceptQueueSize | integer<br>Configuration setting | Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work.<br>Default: 128 |
| connections.http.readTimeoutMs<br>connections.http.readTimeoutMs | integer<br>Configuration setting | connections.http.readTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000 |
| connections.http.maxHeaderBytes<br>connections.http.maxHeaderBytes | integer<br>Configuration setting | Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work.<br>Default: 65536 |
| connections.http.maxRequestBodyBytes<br>connections.http.maxRequestBodyBytes | integer<br>Configuration setting | Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work.<br>Default: 67108864 |
| connections.http.maxResponseBodyBytes<br>connections.http.maxResponseBodyBytes | integer<br>Configuration setting | Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work.<br>Default: 67108864 |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.http.allowedIps<br>connections.http.allowedIps | string<br>Configuration setting | Inbound client IP allowlist for the hosted service. Configure the addresses visible to the runtime, including proxy/network translation where applicable.<br>Default: "" |
| connections.http.openApiApiKeySecrets<br>connections.http.openApiApiKeySecrets | password<br>Configuration setting | Bind a protected property or enter a JSON object that maps OpenAPI security scheme names to API key secrets. Archived projects require the secrets at deployment.<br>Default: "[supply securely]" |
| connections.http.trustedProxyCidrs<br>connections.http.trustedProxyCidrs | string<br>Configuration setting | Proxy IPs or CIDRs allowed to supply X-Forwarded-For. Leave empty to use the transport peer for all requests.<br>Default: "" |
| connections.http.httpErrorFormat<br>connections.http.httpErrorFormat | string<br>Configuration setting | Use problem to return RFC 9457 Problem Details for listener-generated errors. The json option keeps the original error response shape.<br>Default: "" |
| connections.http.conditionalResponses<br>connections.http.conditionalResponses | string<br>Configuration setting | Return 304 for matching GET or HEAD validators using mapped ETag or Last-Modified response headers. The workflow still runs.<br>Default: "" |
| connections.http.propagateTraceContext<br>connections.http.propagateTraceContext | string<br>Configuration setting | Enable W3C version 00 trace context for this connection. Enable it on both the listener and outbound connection to follow a request across calls.<br>Default: "" |
| connections.http.sharedHttpCache<br>connections.http.sharedHttpCache | string<br>Configuration setting | Share fresh, explicitly public GET responses within the runtime process. Credentialed and conditional requests bypass this cache.<br>Default: "" |
| connections.http.callbackAllowedOrigins<br>connections.http.callbackAllowedOrigins | string<br>Configuration setting | Explicit HTTPS origins trusted to receive callback requests. Separate origins with commas or spaces.<br>Default: "" |
| connections.http.activityTimeoutSeconds<br>connections.http.activityTimeoutSeconds | integer<br>Configuration setting | connections.http.activityTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 180 |
| connections.http.oauthGrantType<br>connections.http.oauthGrantType | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.http.oauthTokenUrl<br>connections.http.oauthTokenUrl | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.http.oauthClientId<br>connections.http.oauthClientId | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| Field / key | Type / required | Details |
| connections.http.oauthClientSecret<br>connections.http.oauthClientSecret | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "[supply securely]" |
| connections.http.oauthClientAuthentication<br>connections.http.oauthClientAuthentication | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.http.oauthClientPrivateKeyFile<br>connections.http.oauthClientPrivateKeyFile | string<br>Configuration setting | Path on the runtime host to the PEM signing key registered for the OAuth client. Provide the key file separately from the archive.<br>Default: "" |
| connections.http.oauthClientAssertionAlgorithm<br>connections.http.oauthClientAssertionAlgorithm | string<br>Configuration setting | Asymmetric algorithm registered for the OAuth client. Defaults to RS256.<br>Default: "" |
| connections.http.oauthClientAssertionKeyId<br>connections.http.oauthClientAssertionKeyId | string<br>Configuration setting | Optional kid included in the signed OAuth client assertion so the provider can select a verification key.<br>Default: "" |
| connections.http.oauthScope<br>connections.http.oauthScope | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.http.oauthAuthorizationCode<br>connections.http.oauthAuthorizationCode | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "[supply securely]" |
| connections.http.oauthRedirectUri<br>connections.http.oauthRedirectUri | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.http.oauthCodeVerifier<br>connections.http.oauthCodeVerifier | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "[supply securely]" |
| connections.http.oauthRefreshToken<br>connections.http.oauthRefreshToken | password<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "[supply securely]" |
| connections.http.oauthTimeoutSeconds<br>connections.http.oauthTimeoutSeconds | integer<br>Configuration setting | connections.http.oauthTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30 |
| Field / key | Type / required | Details |
| connections.http.oauthIntrospectionUrl<br>connections.http.oauthIntrospectionUrl | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | grant.<br>Default: "" |
| connections.http.oauthRequiredScopes<br>connections.http.oauthRequiredScopes | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.http.jwtToken<br>connections.http.jwtToken | password<br>Configuration setting | Token or identity-provider endpoint for the selected authentication flow. Tokens expire; use the supported acquisition/refresh settings where required.<br>Default: "[supply securely]" |
| connections.http.jwtAlgorithm<br>connections.http.jwtAlgorithm | string<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: "" |
| connections.http.jwtSecret<br>connections.http.jwtSecret | password<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: "[supply securely]" |
| connections.http.jwtPrivateKeyFile<br>connections.http.jwtPrivateKeyFile | string<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: "" |
| connections.http.jwtPublicKeyFile<br>connections.http.jwtPublicKeyFile | string<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: "" |
| connections.http.jwtJwksUrl<br>connections.http.jwtJwksUrl | string<br>Configuration setting | Environment property connections.http.jwtJwksUrl. Bind it with ${properties.connections.http.jwtJwksUrl}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.jwtJwksCacheSeconds<br>connections.http.jwtJwksCacheSeconds | string<br>Configuration setting | Environment property connections.http.jwtJwksCacheSeconds. Bind it with ${properties.connections.http.jwtJwksCacheSeconds}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.jwtJwksRefreshSeconds<br>connections.http.jwtJwksRefreshSeconds | string<br>Configuration setting | Environment property connections.http.jwtJwksRefreshSeconds. Bind it with ${properties.connections.http.jwtJwksRefreshSeconds}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.http.jwtJwksTimeoutSeconds<br>connections.http.jwtJwksTimeoutSeconds | string<br>Configuration setting | connections.http.jwtJwksTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: "" |
| connections.http.jwtIssuer<br>connections.http.jwtIssuer | string<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.http.jwtAudience<br>connections.http.jwtAudience | string<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: "" |
| connections.http.jwtClaims<br>connections.http.jwtClaims | string<br>Configuration setting | Additional JWT claims as an object. Keep issuer/audience/expiry consistent with verification settings and avoid placing secrets in readable token claims.<br>Default: "" |
| connections.http.jwtLifetimeSeconds<br>connections.http.jwtLifetimeSeconds | integer<br>Configuration setting | JWT signing/validation setting. Match algorithm, issuer/audience and signing/verification key to the counterpart; lifetime is in seconds and secrets/private keys require protected storage.<br>Default: 300 |
| connections.http.jwtClockSkewSeconds<br>connections.http.jwtClockSkewSeconds | integer<br>Configuration setting | Allowed time difference in seconds when checking signed request/token timestamps. Use a small tolerance and synchronize runtime clocks.<br>Default: 30 |
| connections.http.hmacAlgorithm<br>connections.http.hmacAlgorithm | string<br>Configuration setting | HMAC request signing/verification setting. Both endpoints must use the same signing algorithm, protected secret and signature header convention.<br>Default: "" |
| connections.http.hmacKeyId<br>connections.http.hmacKeyId | string<br>Configuration setting | Identifier selecting the HMAC signing key. It identifies the key without exposing the shared secret.<br>Default: "default" |
| connections.http.hmacSecret<br>connections.http.hmacSecret | password<br>Configuration setting | HMAC request signing/verification setting. Both endpoints must use the same signing algorithm, protected secret and signature header convention.<br>Default: "[supply securely]" |
| connections.http.hmacClockSkewSeconds<br>connections.http.hmacClockSkewSeconds | integer<br>Configuration setting | Allowed time difference in seconds when checking signed request/token timestamps. Use a small tolerance and synchronize runtime clocks.<br>Default: 300 |
| connections.http.ldapUrl<br>connections.http.ldapUrl | string<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: "" |
| connections.http.ldapStartTls<br>connections.http.ldapStartTls | string<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: "" |
| connections.http.ldapCaFile<br>connections.http.ldapCaFile | string<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: "" |
| connections.http.ldapTimeoutSeconds<br>connections.http.ldapTimeoutSeconds | integer<br>Configuration setting | connections.http.ldapTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 10 |
| Field / key | Type / required | Details |
| connections.http.ldapUserDnTemplate<br>connections.http.ldapUserDnTemplate | string<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: "" |
| connections.http.ldapBindDn<br>connections.http.ldapBindDn | string<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: "" |
| connections.http.ldapBindPassword<br>connections.http.ldapBindPassword | password<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: "[supply securely]" |
| connections.http.ldapBaseDn<br>connections.http.ldapBaseDn | string<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: "" |
| connections.http.ldapSearchFilter<br>connections.http.ldapSearchFilter | string<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: "(uid={username})" |
| connections.http.ldapRequiredGroup<br>connections.http.ldapRequiredGroup | string<br>Configuration setting | LDAP authentication setting. Configure the directory URL, search base/filter or user DN template, bind identity and required group; use TLS/StartTLS and the directory trust certificate where applicable.<br>Default: "" |
| connections.ftp.host<br>connections.ftp.host | string<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: "ftp.example.com" |
| connections.ftp.port<br>connections.ftp.port | integer<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: 21 |
| connections.ftp.username<br>connections.ftp.username | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| connections.ftp.password<br>connections.ftp.password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.ftp.workingDirectory<br>connections.ftp.workingDirectory | string<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.<br>Default: "/" |
| connections.ftp.passiveMode<br>connections.ftp.passiveMode | boolean<br>Configuration setting | Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode.<br>Default: true |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.ftp.timeoutSeconds<br>connections.ftp.timeoutSeconds | integer<br>Configuration setting | connections.ftp.timeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 60 |
| connections.sftp.host<br>connections.sftp.host | string<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: "sftp.example.com" |
| connections.sftp.port<br>connections.sftp.port | integer<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: 22 |
| connections.sftp.username<br>connections.sftp.username | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| connections.sftp.password<br>connections.sftp.password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.sftp.workingDirectory<br>connections.sftp.workingDirectory | string<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.<br>Default: "/" |
| connections.sftp.privateKeyFile<br>connections.sftp.privateKeyFile | string<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |
| connections.sftp.privateKeyPassphrase<br>connections.sftp.privateKeyPassphrase | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.sftp.knownHostsFile<br>connections.sftp.knownHostsFile | string<br>Configuration setting | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.<br>Default: "" |
| connections.sftp.strictHostKeyChecking<br>connections.sftp.strictHostKeyChecking | boolean<br>Configuration setting | SSH host identity verification. Supply a trusted known-hosts file for SFTP; disabling verification permits connecting to an unverified server.<br>Default: true |
| connections.sftp.timeoutSeconds<br>connections.sftp.timeoutSeconds | integer<br>Configuration setting | connections.sftp.timeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 60 |
| connections.ems.host<br>connections.ems.host | string<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: "localhost" |
| Field / key | Type / required | Details |
| connections.ems.port<br>connections.ems.port | integer<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | https.<br>Default: 7222 |
| connections.ems.serverUrl<br>connections.ems.serverUrl | string<br>Configuration setting | Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace.<br>Default: "tcp://localhost:7222" |
| connections.ems.driverDirectory<br>connections.ems.driverDirectory | string<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: "" |
| connections.ems.connectionFactoryClass<br>connections.ems.connectionFactoryClass | string<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: "" |
| connections.ems.connectionTimeoutSeconds<br>connections.ems.connectionTimeoutSeconds | integer<br>Configuration setting | connections.ems.connectionTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30 |
| connections.ems.destination<br>connections.ems.destination | string<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.<br>Default: "orders" |
| connections.ems.sessionCount<br>connections.ems.sessionCount | integer<br>Configuration setting | Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit.<br>Default: 1 |
| connections.ems.flowLimit<br>connections.ems.flowLimit | integer<br>Configuration setting | Provider/listener concurrency setting. Coordinate it with downstream capacity and the operation-specific runtime support; it is not a whole-workflow parallel-branch limit.<br>Default: 0 |
| connections.ems.receiveTimeoutMs<br>connections.ems.receiveTimeoutMs | integer<br>Configuration setting | connections.ems.receiveTimeoutMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000 |
| connections.ems.username<br>connections.ems.username | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| connections.ems.password<br>connections.ems.password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.ems.clientId<br>connections.ems.clientId | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| Field / key | Type / required | Details |
| connections.ems.connectionFactory<br>connections.ems.connectionFactory | string<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | factory/provider URL and registered factory name.<br>Default: "ConnectionFactory" |
| connections.ems.reconnectAttempts<br>connections.ems.reconnectAttempts | integer<br>Configuration setting | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.<br>Default: 3 |
| connections.ems.connectionFactoryType<br>connections.ems.connectionFactoryType | string<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "Direct" |
| connections.ems.messagingStyle<br>connections.ems.messagingStyle | string<br>Configuration setting | Subscription/destination policy. Durable subscriptions retain eligible messages for an offline named subscriber; shared subscriptions distribute eligible deliveries where supported by the provider.<br>Default: "Generic" |
| connections.ems.queueConnectionFactory<br>connections.ems.queueConnectionFactory | string<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "QueueConnectionFactory" |
| connections.ems.topicConnectionFactory<br>connections.ems.topicConnectionFactory | string<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "TopicConnectionFactory" |
| connections.ems.jndiContextFactory<br>connections.ems.jndiContextFactory | string<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "" |
| connections.ems.jndiProviderUrl<br>connections.ems.jndiProviderUrl | string<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "tcp://localhost:7222" |
| connections.ems.jndiUsername<br>connections.ems.jndiUsername | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| connections.ems.jndiPassword<br>connections.ems.jndiPassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.ems.useXa<br>connections.ems.useXa | boolean<br>Configuration setting | Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode.<br>Default: false |
| connections.ems.useUfo<br>connections.ems.useUfo | boolean<br>Configuration setting | Provider-specific connection feature. Enable only when the selected driver/server supports it; check the connector procedure and vendor prerequisites for that mode.<br>Default: false |
| connections.ems.sslEnabled<br>connections.ems.sslEnabled | boolean<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Default: false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.ems.sslTrustedCertificates<br>connections.ems.sslTrustedCertificates | string<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: "" |
| connections.ems.reconnectDelayMs<br>connections.ems.reconnectDelayMs | integer<br>Configuration setting | connections.ems.reconnectDelayMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 5000 |
| connections.ems.heartbeatOutgoingMs<br>connections.ems.heartbeatOutgoingMs | integer<br>Configuration setting | STOMP outgoing heartbeat interval in milliseconds. Zero disables that direction; the broker and client negotiate the effective heartbeat interval.<br>Default: 0 |
| connections.ems.heartbeatIncomingMs<br>connections.ems.heartbeatIncomingMs | integer<br>Configuration setting | STOMP incoming heartbeat interval in milliseconds. Zero disables that direction; the broker and client negotiate the effective heartbeat interval.<br>Default: 0 |
| connections.jms.provider<br>connections.jms.provider | string<br>Configuration setting | Provider/protocol or receive mode selection. Choose the broker implementation and delivery/settlement behavior supported by the configured connection.<br>Default: "Generic JMS 2.0" |
| connections.jms.serverUrl<br>connections.jms.serverUrl | string<br>Configuration setting | Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the broker namespace.<br>Default: "tcp://localhost:61613" |
| connections.jms.driverDirectory<br>connections.jms.driverDirectory | string<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: "" |
| connections.jms.connectionFactoryClass<br>connections.jms.connectionFactoryClass | string<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: "" |
| connections.jms.connectionTimeoutSeconds<br>connections.jms.connectionTimeoutSeconds | integer<br>Configuration setting | connections.jms.connectionTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30 |
| connections.jms.host<br>connections.jms.host | string<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: "localhost" |
| connections.jms.port<br>connections.jms.port | integer<br>Configuration setting | Server address component: host is the resolvable hostname, port the numeric listener port, and scheme the protocol such as http or https.<br>Default: 61613 |
| connections.jms.username<br>connections.jms.username | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.jms.password<br>connections.jms.password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.jms.clientId<br>connections.jms.clientId | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.jms.connectionFactoryType<br>connections.jms.connectionFactoryType | string<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "Direct" |
| connections.jms.connectionFactory<br>connections.jms.connectionFactory | string<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "ConnectionFactory" |
| connections.jms.jndiContextFactory<br>connections.jms.jndiContextFactory | string<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "" |
| connections.jms.jndiProviderUrl<br>connections.jms.jndiProviderUrl | string<br>Configuration setting | JMS connection-factory selection. Direct and JNDI modes require different provider configuration; JNDI uses the context factory/provider URL and registered factory name.<br>Default: "" |
| connections.jms.jndiUsername<br>connections.jms.jndiUsername | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| connections.jms.jndiPassword<br>connections.jms.jndiPassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.jms.sslEnabled<br>connections.jms.sslEnabled | boolean<br>Configuration setting | Transport security setting. Enable TLS for confidential transport and verify server identity/certificates; disabling verification or trusting an unverified server weakens connection authentication.<br>Default: false |
| connections.jms.sslTrustedCertificates<br>connections.jms.sslTrustedCertificates | string<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: "" |
| connections.jms.reconnectAttempts<br>connections.jms.reconnectAttempts | integer<br>Configuration setting | Retry attempt policy. A retry can repeat an accepted external write; use stable identifiers and destination-side duplicate handling.<br>Default: 3 |
| connections.jms.reconnectDelayMs<br>connections.jms.reconnectDelayMs | integer<br>Configuration setting | connections.jms.reconnectDelayMs: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 5000 |
| Field / key | Type / required | Details |
| connections.kafka.bootstrapServers<br>connections.kafka.bootstrapServers | string<br>Configuration setting | Provider connection address/namespace. Use the provider URI/connection-string syntax; Kafka bootstrapServers accepts broker host:port endpoints and AMQP virtualHost selects the |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | broker namespace.<br>Default: "localhost:9092" |
| connections.kafka.topic<br>connections.kafka.topic | string<br>Configuration setting | Broker entity name used for this operation. Bind the correct environment property and verify the runtime identity has send/receive access.<br>Default: "" |
| connections.kafka.clientId<br>connections.kafka.clientId | string<br>Configuration setting | OAuth/client identity setting. Match client ID/secret, grant, token endpoint, redirect URI and scopes to the registered client; use code/verifier or refresh-token fields only for the corresponding grant.<br>Default: "" |
| connections.kafka.groupId<br>connections.kafka.groupId | string<br>Configuration setting | Kafka consumer-group identifier. Consumers in the same group share partitions and offsets; use a separate test group to avoid changing production consumption state.<br>Default: "mina" |
| connections.kafka.securityProtocol<br>connections.kafka.securityProtocol | string<br>Configuration setting | Kafka transport/authentication selection. SASL_SSL combines TLS and SASL; configure credentials and CA verification for the selected SASL mechanism.<br>Default: "PLAINTEXT" |
| connections.kafka.saslMechanism<br>connections.kafka.saslMechanism | string<br>Configuration setting | Kafka transport/authentication selection. SASL_SSL combines TLS and SASL; configure credentials and CA verification for the selected SASL mechanism.<br>Default: "PLAIN" |
| connections.kafka.username<br>connections.kafka.username | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |
| connections.kafka.password<br>connections.kafka.password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.kafka.requestTimeoutMilliseconds<br>connections.kafka.requestTimeoutMilliseconds | integer<br>Configuration setting | connections.kafka.requestTimeoutMilliseconds: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000 |
| connections.kafka.connectionTimeoutMilliseconds<br>connections.kafka.connectionTimeoutMilliseconds | integer<br>Configuration setting | connections.kafka.connectionTimeoutMilliseconds: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 10000 |
| connections.kafka.sslCaLocation<br>connections.kafka.sslCaLocation | string<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: "" |
| connections.kafka.sslCertificateLocation<br>connections.kafka.sslCertificateLocation | string<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |
| connections.kafka.sslKeyLocation<br>connections.kafka.sslKeyLocation | string<br>Configuration setting | Client/server certificate or private-key location/selection. Deploy matching certificate and key material and grant read access only to the runtime identity.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.kafka.sslKeyPassword<br>connections.kafka.sslKeyPassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.kafka.schemaRegistryUrl<br>connections.kafka.schemaRegistryUrl | string<br>Configuration setting | Kafka schema-registry configuration. Select the registry endpoint/vendor and authentication; the Avro runtime limitations are documented on the Kafka operation pages.<br>Default: "" |
| connections.kafka.schemaRegistryUsername<br>connections.kafka.schemaRegistryUsername | string<br>Configuration setting | Kafka schema-registry configuration. Select the registry endpoint/vendor and authentication; the Avro runtime limitations are documented on the Kafka operation pages.<br>Default: "" |
| connections.kafka.schemaRegistryPassword<br>connections.kafka.schemaRegistryPassword | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.kafka.reconnectBackoffMilliseconds<br>connections.kafka.reconnectBackoffMilliseconds | integer<br>Configuration setting | connections.kafka.reconnectBackoffMilliseconds: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 50 |
| connections.kafka.retryBackoffMilliseconds<br>connections.kafka.retryBackoffMilliseconds | integer<br>Configuration setting | connections.kafka.retryBackoffMilliseconds: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 100 |
| connections.kafka.authenticationType<br>connections.kafka.authenticationType | string<br>Configuration setting | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Default: "None" |
| connections.kafka.useTicketCache<br>connections.kafka.useTicketCache | boolean<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: false |
| connections.kafka.keytabFile<br>connections.kafka.keytabFile | string<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: "" |
| connections.kafka.principalName<br>connections.kafka.principalName | string<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: "" |
| connections.kafka.jaasConfigFile<br>connections.kafka.jaasConfigFile | string<br>Configuration setting | JAAS login configuration file on the runtime host for the selected authentication mechanism. Protect credentials contained in the file.<br>Default: "" |
| connections.kafka.loginCallbackHandler<br>connections.kafka.loginCallbackHandler | string<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.kafka.schemaRegistryVendor<br>connections.kafka.schemaRegistryVendor | string<br>Configuration setting | Kafka schema-registry configuration. Select the registry endpoint/vendor and authentication; the Avro runtime limitations are documented on the Kafka operation pages.<br>Default: "Confluent" |
| connections.kafka.clientProperties<br>connections.kafka.clientProperties | json<br>Configuration setting | Additional provider settings as name/value entries. Use exact supported provider option names; invalid options may be rejected at connection or execution time.<br>Default: "{}" |
| connections.pubsub.projectId<br>connections.pubsub.projectId | string<br>Configuration setting | Google Cloud project containing the Pub/Sub topic or subscription. Supply the project ID, not its display name.<br>Default: "my-gcp-project" |
| connections.pubsub.authenticationType<br>connections.pubsub.authenticationType | string<br>Configuration setting | Authentication mechanism for this connection. Supply the corresponding credentials/token/key settings and verify the receiving endpoint accepts that mechanism.<br>Default: "Service Account JSON" |
| connections.pubsub.serviceAccountJson<br>connections.pubsub.serviceAccountJson | password<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: "[supply securely]" |
| connections.pubsub.credentialsFile<br>connections.pubsub.credentialsFile | string<br>Configuration setting | Provider credential/identity material. Use the mechanism supported by the selected connection and runtime; keep private keys, access keys and credential files outside shared source packages.<br>Default: "" |
| connections.pubsub.endpoint<br>connections.pubsub.endpoint | string<br>Configuration setting | Endpoint or filesystem path for this operation. For HTTP use the service URI/path; for files use a path accessible to the runtime host, not only the designer machine.<br>Default: "pubsub.googleapis.com:443" |
| connections.pubsub.emulatorHost<br>connections.pubsub.emulatorHost | string<br>Configuration setting | Local Pub/Sub emulator address for testing. Leave empty for Google Cloud provider access and keep emulator configuration out of production environments.<br>Default: "" |
| connections.pubsub.ackDeadlineSeconds<br>connections.pubsub.ackDeadlineSeconds | integer<br>Configuration setting | Provider lifetime/wait setting in seconds, as named by this field. Set it above normal work/latency while retaining a bounded recovery time.<br>Default: 60 |
| connections.pubsub.connectionTimeoutSeconds<br>connections.pubsub.connectionTimeoutSeconds | integer<br>Configuration setting | connections.pubsub.connectionTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30 |
| connections.pubsub.maxInboundMessageBytes<br>connections.pubsub.maxInboundMessageBytes | integer<br>Configuration setting | Size/capacity bound for the named request, response, header or event queue. Byte limits measure encoded content; queue limits bound buffered work.<br>Default: 20971520 |
| connections.pubsub.keepAliveSeconds<br>connections.pubsub.keepAliveSeconds | integer<br>Configuration setting | Connection lifecycle policy. Pooling/persistent connections reduce connection setup; recovery and keepalive apply to provider connections, not transaction-level replay.<br>Default: 60 |
| Field / key | Type / required | Details |
| connections.pubsub.batchMaxMessages | integer<br>Configuration setting | Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.pubsub.batchMaxMessages | | payload sizes and acceptable delivery latency.<br>Default: 100 |
| connections.pubsub.batchMaxBytes<br>connections.pubsub.batchMaxBytes | integer<br>Configuration setting | Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency.<br>Default: 1048576 |
| connections.pubsub.batchDelayThresholdMilliseconds<br>connections.pubsub.batchDelayThresholdMilliseconds | integer<br>Configuration setting | Pub/Sub batching threshold: message count, total bytes or waiting time in milliseconds, according to the field name. Tune with payload sizes and acceptable delivery latency.<br>Default: 10 |
| connections.pubsub.flowControlMaxMessages<br>connections.pubsub.flowControlMaxMessages | integer<br>Configuration setting | Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity.<br>Default: 1000 |
| connections.pubsub.flowControlMaxBytes<br>connections.pubsub.flowControlMaxBytes | integer<br>Configuration setting | Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity.<br>Default: 10485760 |
| connections.pubsub.caCertificateFile<br>connections.pubsub.caCertificateFile | string<br>Configuration setting | Trusted CA certificate location on the runtime host. A certificate folder can contain root certificates and optional chain certificates; configure the resolved deployment path.<br>Default: "" |
| connections.sap.applicationServerHost<br>connections.sap.applicationServerHost | string<br>Configuration setting | Environment property connections.sap.applicationServerHost. Bind it with ${properties.connections.sap.applicationServerHost}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "sap-ecc.example.com" |
| connections.sap.release<br>connections.sap.release | string<br>Configuration setting | Environment property connections.sap.release. Bind it with ${properties.connections.sap.release}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "current" |
| connections.sap.systemNumber<br>connections.sap.systemNumber | string<br>Configuration setting | Environment property connections.sap.systemNumber. Bind it with ${properties.connections.sap.systemNumber}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "00" |
| connections.sap.client<br>connections.sap.client | string<br>Configuration setting | Environment property connections.sap.client. Bind it with ${properties.connections.sap.client}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "100" |
| connections.sap.language<br>connections.sap.language | string<br>Configuration setting | Environment property connections.sap.language. Bind it with ${properties.connections.sap.language}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "EN" |
| connections.sap.username<br>connections.sap.username | string<br>Configuration setting | Identity used to authenticate to the configured service or proxy. Supply an environment-specific account with the required operation permissions.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.sap.password<br>connections.sap.password | password<br>Configuration setting | Secret used by the corresponding authentication/key/store setting. Resolve from protected deployment configuration; never embed it in payloads or logs.<br>Default: "[supply securely]" |
| connections.sap.messageServerHost<br>connections.sap.messageServerHost | string<br>Configuration setting | Environment property connections.sap.messageServerHost. Bind it with ${properties.connections.sap.messageServerHost}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.sap.systemId<br>connections.sap.systemId | string<br>Configuration setting | Environment property connections.sap.systemId. Bind it with ${properties.connections.sap.systemId}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.sap.logonGroup<br>connections.sap.logonGroup | string<br>Configuration setting | Environment property connections.sap.logonGroup. Bind it with ${properties.connections.sap.logonGroup}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "PUBLIC" |
| connections.sap.sapRouter<br>connections.sap.sapRouter | string<br>Configuration setting | Environment property connections.sap.sapRouter. Bind it with ${properties.connections.sap.sapRouter}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.sap.programId<br>connections.sap.programId | string<br>Configuration setting | Environment property connections.sap.programId. Bind it with ${properties.connections.sap.programId}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.sap.gatewayHost<br>connections.sap.gatewayHost | string<br>Configuration setting | Environment property connections.sap.gatewayHost. Bind it with ${properties.connections.sap.gatewayHost}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.sap.gatewayService<br>connections.sap.gatewayService | string<br>Configuration setting | Environment property connections.sap.gatewayService. Bind it with ${properties.connections.sap.gatewayService}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.sap.maximumConnections<br>connections.sap.maximumConnections | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 8 |
| connections.sap.outboundWorkerCount<br>connections.sap.outboundWorkerCount | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 2 |
| connections.sap.poolCapacity<br>connections.sap.poolCapacity | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 1 |
| Field / key | Type / required | Details |
| connections.sap.peakLimit<br>connections.sap.peakLimit | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider |

| Field / key | Type / required | Details |
| --- | --- | --- |
| | | limits and downstream processing capacity.<br>Default: 1 |
| connections.sap.connectionExpirationMilliseconds<br>connections.sap.connectionExpirationMilliseconds | integer<br>Configuration setting | Lifetime of an idle SAP pooled connection in milliseconds before it is eligible for expiration. Match this to the SAP server connection policy.<br>Default: 600000 |
| connections.sap.outboundPoolIdleSeconds<br>connections.sap.outboundPoolIdleSeconds | integer<br>Configuration setting | Provider lifetime/wait setting in seconds, as named by this field. Set it above normal work/latency while retaining a bounded recovery time.<br>Default: 1800 |
| connections.sap.maximumOutboundPools<br>connections.sap.maximumOutboundPools | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 16 |
| connections.sap.maxPendingEvents<br>connections.sap.maxPendingEvents | integer<br>Configuration setting | Bound on received/buffered messages or bytes, according to this field. Keep the bound within available memory and downstream processing capacity.<br>Default: 16 |
| connections.sap.jvmInitialHeapMb<br>connections.sap.jvmInitialHeapMb | integer<br>Configuration setting | Java bridge heap size in megabytes. Size it for concurrent payloads and driver overhead; keep the initial heap at or below the maximum.<br>Default: 64 |
| connections.sap.jvmMaximumHeapMb<br>connections.sap.jvmMaximumHeapMb | integer<br>Configuration setting | Java bridge heap size in megabytes. Size it for concurrent payloads and driver overhead; keep the initial heap at or below the maximum.<br>Default: 512 |
| connections.sap.timeoutMilliseconds<br>connections.sap.timeoutMilliseconds | integer<br>Configuration setting | connections.sap.timeoutMilliseconds: bounded wait/delay in milliseconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 30000 |
| connections.sap.ackTimeoutSeconds<br>connections.sap.ackTimeoutSeconds | integer<br>Configuration setting | connections.sap.ackTimeoutSeconds: bounded wait/delay in seconds. This setting controls the named connection, request, read/write or retry stage; it does not make a failed external operation safe to repeat.<br>Default: 300 |
| connections.sap.maximumDynamicSessions<br>connections.sap.maximumDynamicSessions | integer<br>Configuration setting | Connection/thread/worker pool sizing. Minimums initialize capacity and maximums bound concurrent work; tune against provider limits and downstream processing capacity.<br>Default: 1024 |
| connections.sap.mode<br>connections.sap.mode | string<br>Configuration setting | Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing.<br>Default: "external" |
| connections.sap.driverDirectory<br>connections.sap.driverDirectory | string<br>Configuration setting | Database/JMS driver implementation or driver directory. Install the matching provider library on the runtime host; Java drivers require the bridge and compatible Java runtime.<br>Default: "" |
| connections.sap.destinationName<br>connections.sap.destinationName | string<br>Configuration setting | Connection routing mode or named provider destination. Select the mode that matches the server configuration and supply its corresponding connection fields.<br>Default: "" |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.sap.connectionType<br>connections.sap.connectionType | string<br>Configuration setting | Connection routing mode or named provider destination. Select the mode that matches the server configuration and supply its corresponding connection fields.<br>Default: "dedicated" |
| connections.sap.sncPartnerName<br>connections.sap.sncPartnerName | string<br>Configuration setting | Environment property connections.sap.sncPartnerName. Bind it with ${properties.connections.sap.sncPartnerName}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.sap.sncLibraryPath<br>connections.sap.sncLibraryPath | string<br>Configuration setting | Environment property connections.sap.sncLibraryPath. Bind it with ${properties.connections.sap.sncLibraryPath}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.sap.sncMyName<br>connections.sap.sncMyName | string<br>Configuration setting | Environment property connections.sap.sncMyName. Bind it with ${properties.connections.sap.sncMyName}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.sap.sncQop<br>connections.sap.sncQop | string<br>Configuration setting | Environment property connections.sap.sncQop. Bind it with ${properties.connections.sap.sncQop}; the active environment supplies its value. Keep its datatype consistent with the activity or shared-connection field that references it.<br>Default: "" |
| connections.sapTid.storageFile<br>connections.sapTid.storageFile | string<br>Configuration setting | Durable mediation state setting/result. Use a stable message/correlation key and persistent state file; expectedCount controls aggregation, leases bound ownership and retention bounds completed-state lifetime. Complete claims only after successful processing.<br>Default: "data/sap-tids.json" |
| connections.sapTid.mode<br>connections.sapTid.mode | string<br>Configuration setting | Input/execution mode selected from the available choices. Match inline text, file or structured input to the actual source; connection modes such as mock/memory are for local testing.<br>Default: "active" |
| connections.file.path<br>connections.file.path | string<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.<br>Default: "" |
| connections.file.destination<br>connections.file.destination | string<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.<br>Default: "" |
| connections.file.encoding<br>connections.file.encoding | string<br>Configuration setting | Character encoding for text conversion, for example UTF-8. Use the sender encoding when parsing; a wrong encoding can corrupt non-ASCII characters.<br>Default: "utf-8" |
| connections.file.pattern<br>connections.file.pattern | string<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.<br>Default: "*" |
| connections.file.pollInterval<br>connections.file.pollInterval | integer<br>Configuration setting | Polling/scheduling interval for repeated execution. Size the interval to expected arrival rate and processing duration; select the unit where the activity exposes one.<br>Default: 5 |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.file.recursive<br>connections.file.recursive | boolean<br>Configuration setting | Include subdirectories when traversing/listing matching files. Restrict the starting directory to avoid scanning unrelated trees.<br>Default: false |
| connections.file.includeTimestamp<br>connections.file.includeTimestamp | boolean<br>Configuration setting | Include timestamp metadata where the operation supports it, to distinguish or trace generated/polled content.<br>Default: false |
| connections.file.readAs<br>connections.file.readAs | string<br>Configuration setting | Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract.<br>Default: "Text" |
| connections.file.excludeFileContent<br>connections.file.excludeFileContent | boolean<br>Configuration setting | Omit file content from polling output when only file metadata is needed. Downstream Read File can fetch content explicitly.<br>Default: false |
| connections.file.writeAs<br>connections.file.writeAs | string<br>Configuration setting | Representation used for reading, writing or returning the value. Select the mode matching the actual document and downstream consumer contract.<br>Default: "Text" |
| connections.file.append<br>connections.file.append | boolean<br>Configuration setting | Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved.<br>Default: false |
| connections.file.overwrite<br>connections.file.overwrite | boolean<br>Configuration setting | Existing-file policy. Append adds content to the existing file; overwrite replaces existing content. Disable replacement where existing output must be preserved.<br>Default: true |
| connections.file.createDirectories<br>connections.file.createDirectories | boolean<br>Configuration setting | Create missing parent directories for the output path. The runtime account still needs directory creation/write permission.<br>Default: false |
| connections.file.addLineSeparator<br>connections.file.addLineSeparator | boolean<br>Configuration setting | Line separator handling for text output. Choose the delimiter expected by the consumer; includeFinalLineSeparator adds a separator after the last record.<br>Default: false |
| connections.file.compression<br>connections.file.compression | string<br>Configuration setting | Compression selection for file or staging content. Match decompression settings at the receiving operation; compression changes the wire bytes.<br>Default: "None" |
| connections.file.listType<br>connections.file.listType | string<br>Configuration setting | File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned.<br>Default: "Files and Directories" |
| connections.file.sortBy<br>connections.file.sortBy | string<br>Configuration setting | File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned.<br>Default: "Name" |
| connections.file.sortOrder<br>connections.file.sortOrder | string<br>Configuration setting | File listing selection/order. sortBy chooses the metadata key, sortOrder its direction, and listType limits the kinds of entries returned.<br>Default: "Ascending" |
| connections.file.ignoreMissing<br>connections.file.ignoreMissing | boolean<br>Configuration setting | Allow a missing file/path where supported instead of faulting. Use only when absence is an expected business outcome.<br>Default: false |

| Field / key | Type / required | Details |
| --- | --- | --- |
| connections.file.preserveAttributes<br>connections.file.preserveAttributes | boolean<br>Configuration setting | Request preservation of supported file attributes during the operation. Availability depends on filesystem and runtime permissions.<br>Default: false |
| connections.file.eventType<br>connections.file.eventType | string<br>Configuration setting | Type of file event selected/reported by the poller, such as a created or changed file.<br>Default: "Created" |
| connections.file.includeExisting<br>connections.file.includeExisting | boolean<br>Configuration setting | Include files already present when polling begins, rather than only later arrivals. Test restart behavior to avoid unintended reprocessing.<br>Default: false |
| connections.file.postAction<br>connections.file.postAction | string<br>Configuration setting | Post-operation action or returned output. Use the activity-specific operation/contract to determine whether the value is content, metadata or a completion result.<br>Default: "None" |
| connections.file.moveTo<br>connections.file.moveTo | string<br>Configuration setting | Path or destination used by the selected file/transfer operation. Relative paths depend on the runtime working directory; patterns filter matching names.<br>Default: "" |

# MINA documentation site and PDF

Getting started

## Guide

### *MINA documentation site and PDF*

Open Help > MINA Documentation in Studio, or visit `/help/index.html` on the same host and port as Studio. No separate documentation service is required. Installed desktop Studio uses its existing local runtime address; do not assume a fixed port.

The documentation includes a searchable, expandable index and deep links to individual topics and tabs. Use Download PDF for the complete offline manual, or Print topic for the selected topic, including all its tabs. Light and dark display modes are supported.

### *Coverage and accuracy*

Activity Configuration, Input, Output, Advanced and Errors sections are generated from Studio's base contracts and documentation definitions. Tasks, groups, mapping, functions, shared connections, deployment, debugging, administration and security guides are included. Project-selected schemas, provider modes and JDBC parameters can change the fields shown in Studio: the reference does not substitute for those project-specific schemas. Provider prerequisites and production qualification still apply.

Only explicitly selected product source files and guides are read. The generator does not inspect project data, environment credentials or user payloads. Existing implementation limitations are documented, not claimed as implemented.

### *Project Explorer and task canvas*

New projects start with a Sub Task containing Start and End. Use a Starter Task when an external event, such as HTTP, a timer, or a message, starts the workflow. Use Call Sub Task to invoke reusable logic. Starter and Sub Task icons distinguish the two in Project Explorer.

Subtasks and Call Sub Task use the same process-network-and-bent-arrow symbol in Project Explorer, task tabs, creation menus, the activity picker, and the canvas. Starter tasks use the process network with a play symbol.

The canvas, activity palette, and activity picker share one activity icon catalog. Shared connections use the same protocol or vendor artwork in Project Explorer and connection creation menus. Imported legacy Mapper activities retain the Mapper symbol; an unrecognized activity uses a neutral process symbol instead of a Start or End icon.

Tasks, shared connections, and schemas belong to packages. Right-click their category or an existing package and choose New Package to create a top-level or nested folder. Choose the package when creating an item. Default is available for each category. Right-click an existing item and choose Move to Package to organize it without changing its references. Studio replaces spaces with hyphens in object names.

To rename a package, right-click the folder under Tasks, Resources -> Connections, or Schemas and choose Rename Package. Its nested folders and items move with it, keeping task, connection, and schema references intact. Under Packaging, right-click the deployment artifact and choose Rename Package to update its name for future exports while keeping its version and settings. Save the project to keep these changes.

Task, connection, and schema names must be unique within the same package. Studio ignores case and treats spaces as hyphens when checking a name. You can reuse a name in a different package. For dynamic Call Sub Task routing, use the task ID or the qualified name (for example, `Orders/Validate`) if more than one subtask has the same name. Validate Project reports conflicts already present in older projects.

Package contents appear below their folder. If a name is shortened in a narrow Project Explorer, hover over it to see the full name or drag the pane divider to widen it.

When you first add a protocol activity or shared connection, Studio adds the relevant missing environment properties. Connection setup also offers those properties for binding. Review the generated defaults in the active environment before running the task.

Use Magnify orchestrator on the canvas and hover to see a 2× preview. Press Escape to turn it off. Select a transition to set its Line color or choose Reset color to restore the default. Close task tabs individually or use Close All to clear the canvas.

Use the Theme selector to change how Studio looks and feels. Compact Workbench uses a shorter ribbon and denser explorer rows. Blueprint Board uses a drafting grid and framed canvas activities. Reading Room uses larger labels, wider spacing, and a paper-like canvas. Existing themes are still available, and Studio remembers your choice.

The bottom-right Runtime status shows startup stages, current Studio operations, and the activities running in the active project. Click it for the designer and runtime details, including parallel activities and the latest detected slow UI operation on supported browsers. Startup progress counts completed stages; an animated bar means the operation has no reliable percentage. The Windows startup screen also shows the current stage before the designer is ready.

The footer and details use the same live runtime health check. If an older runtime does not provide activity-progress details, Studio reports that limitation separately. It does not mark a healthy runtime as unavailable.

The welcome screen shows packets moving along the integration routes. Use Pause route animation to stop them or Play route animation to start them. Until you choose a setting, the packets follow the Windows or browser reduced-motion preference. Studio remembers your choice.

### *JDBC query and writes*

Use `:name` parameters for Query, Update, and Insert, and map each value under Input -> parameters. To execute multiple records together, enable Batch update and map records. An empty list writes zero rows. For Insert if record does not exist, map an INSERT statement to InsertStatement and provide the parameters used by both statements. MINA runs the INSERT only when the UPDATE affects zero rows. If you need an atomic upsert across concurrent executions, use your database's native upsert statement.

To page through a query, enable Process result in subsets and use a stable `ORDER BY`. Start with Row offset zero. Pass the previous result's nextRowOffset to the next call and stop when lastSubset is true. Maximum rows caps the total rows across those pages. Each call re-executes the query.

### *PostgreSQL samples*

- Run [postgresql-setup.sql](/samples/postgresql-setup.sql) in a disposable PostgreSQL database. It creates `mina_demo.orders` and the `mina_demo.order_total` procedure.

- In Studio Samples, open PostgreSQL CRUD or PostgreSQL Procedure and Paging. Set the JDBC URL, username, and password in the local environment. Use Debug to run the example Sub Task or run it by task ID in an exported Python archive.

- CRUD upserts a demo row, updates it with a version guard, queries the JSONB field, and deletes that row. The other sample reads the first page and calls the procedure with an INOUT numeric argument.

PostgreSQL accepts libpq connection strings and JDBC-style PostgreSQL URLs. Configure PostgreSQL TLS mode and a trusted root certificate for verified TLS. A configured Schema becomes the search path. Use named `:parameter` bindings to avoid ambiguity with PostgreSQL's JSONB question-mark operator. Declare JSON or JSONB, ISO date/time, and decimal types for driver adaptation. To retain exact decimal digits from a mapped value, provide it as a string. Use JDBC Query with `SELECT` for PostgreSQL functions and Call Procedure for `CALL` operations. PostgreSQL statement timeout applies when configured on the activity.

### *Updating the guide*

From the repository root, install the documentation authoring dependencies into your chosen Python environment:

```
python -m pip install -r scripts/documentation-requirements.txt
cd frontend
npm run docs:build
npm run docs:check
node --test scripts/documentation.test.mjs
```

If Python is not on PATH, set `MINA_DOCS_PYTHON` to its executable path before running `docs:build`. The Python dependencies are build-time only, not required to read the site or PDF.

`npm run build` checks that the generated documentation matches the selected source files. After changing those sources, regenerate the guide. The site and PDF are copied from `frontend/public/help` to `frontend/dist/help` by Vite and included in the normal Studio distribution. Rebuild the frontend and installer to deliver updated documentation to installed machines. Keep the whole help directory together when copying it for offline use; opening `index.html` directly also works without a server.

The guide is local to the Studio host. Making it reachable on a shared enterprise URL requires publishing these static assets through your approved web server and access controls; this change does not create a public internet endpoint.

Reference source: docs/DOCUMENTATION.md

# XML JSON and Data parse and render activities

Data

## Guide

### *XML JSON and Data parse and render activities*

All six Parse/Render XML, JSON and Data activities use one implementation in Studio, Direct Python archives and Engine Python archives. Configuration contains design selectors and checkboxes. Map documents, paths, encodings, delimiters, fixed offsets, root names, JSON formatting and other runtime values in Input -> formatOptions. A mapped option takes precedence over the checkbox, activity default or shared Data Format value. Existing text settings appear as editable Input mappings; Studio migrates them when you edit the mappings. Attach project or inline schemas in the Input/Output Editor.

### *XML*

Parse XML accepts text, base64 binary or dynamic input. Map binary input to `xmlBinary.bytes` and optionally supply `xmlBinary.forceEncoding` and `xmlBinary.byteCount`. Without an override, Parse XML uses the byte order mark or XML declaration. Attributes use `@name`, schema-defined repeating elements are arrays even with one occurrence, and ordered mixed text and child elements use `#content`.

Render XML accepts a parsed wrapper or a mapped schema tree. In Configuration, choose text or binary and set the default validation, declaration, indentation and namespace options. In Input, map `byteEncoding` for binary output and `formatOptions.cdataElements` for CDATA. The CDATA picker can insert paths from the selected input schema. Binary output includes `xmlBytes`, plus the existing MINA `binaryContent`, `bytes`, `byteCount` and `encoding` fields.

Validate parsed output and Validate input tree perform XSD validation, including required children, attributes and datatypes. Select the entry XSD as the activity schema. Keep its `xs:include` and `xs:import` dependencies as project schema assets with the relative filenames used by `schemaLocation`. An import without a location resolves only when there is one project XSD for that namespace. The schema tree and both Python archives use the same project assets.

The resolver is offline: URLs, local machine paths, traversal outside the project, DTDs, entities, XSD redefine and override are rejected. XML input is limited to 64 MiB; each XSD to 8 MiB and a dependency graph to 32 MiB. Namespace URIs and mixed text are preserved structurally; serialized prefixes, whitespace and lexical formatting need not match the original bytes.

### *JSON*

Select the conversion mode under Configuration -> Schema type:

- Native JSON is the existing MINA mode. It parses and renders objects, arrays and primitive values directly. It supports offline JSON Schema validation with local references, and can omit null object fields without changing array positions.

- Generic converts between a JSON string and an XML string without a schema. With Badgerfish selected, namespaces use `@xmlns`, attributes use `@name` and text uses `$`. Generic Render JSON treats XML text as strings unless Infer numeric and boolean values is enabled.

- XSD converts between JSON and a schema-defined XML tree. Select an XSD in Parse JSON's Output Editor or Render JSON's Input Editor. Validation is on by default. Ignore additional JSON fields filters unknown input fields when validation is enabled. An array declared by the XSD remains an array even when only one value is present.

New JSON activities default to XSD. Existing activities without a saved mode continue to use Native JSON. The JSON root styles are None, JSON with Root and Anonymous Array. For Anonymous Array in XSD mode, the root must have one repeating child. Use null for empty values represents empty containers as nil when there is an XML container. Parse JSON rejects malformed JSON and non-finite numbers; its duplicate-key policy supports First wins, Last wins and Error. Render JSON can format the result and exposes the serialized output as `jsonString`.

### *Delimited and fixed-width Data*

Parse Data accepts a string or runtime file path with a selected encoding. Map `text` or `fileName` in Input. Use `skipHeaderCharacters` to bypass a preamble, `noOfRecords` to limit a batch (`-1` means all records), and `startRecord` for a zero-based manual offset. If you leave the offset unmapped, each batch advances the same activity's cursor within that task execution. Use `done` to exit a loop and `nextRecord` when supplying your own offset. Continue on record errors

returns invalid field values or column counts in `ErrorRows.error`; an invalid document structure, such as an unterminatedquoted field, still fails.

Configure the format inline or add a JSON schema asset under Project Explorer -> Schemas with a `dataFormat` object. Select it as Shared Data Format on both activities. For example:

```
{"dataFormat":{"format":"delimited","delimiter":",","header":true,"fields":"id,name","fieldTypes":"integer,string","strictColumns":true}}
```

An optional `schemaId` links the shared format to a project row schema. Parse Data exposes typed `records`, `Rows.schema`, record counts and the XML representation. Render Data accepts mapped records and exposes the rendered value as `text`, `Text` and `content`.

To reuse an existing layout, right-click Schemas and choose Import XSD or Data Format. Select a Data Format XML resource (`.dataformat`, `.dataformatresource` or `.dataformatsharedresource`) or a ZIP containing it and its companion XSDs. ZIP entries retain their relative paths. The importer converts delimited and fixed layouts into reusable MINA assets and links supported Parse/Render Data references in imported processes.

A source module property bound to the Data Format column separator becomes a MINA environment property and an editable Input -> formatOptions -> Column separator mapping. Import the ZIP with its `default.substvar` and, if available, `local.substvar`, `dev.substvar`, `qa.substvar`, `pre.substvar` or `production.substvar`. A profile value overrides the default, which falls back to the resource's literal separator if present. If no value is available, set the imported property before running. Select the imported Data Format on both activities to add its mapping without overwriting custom mappings. Unsupported substitution targets and ambiguous or secret profile values are reported during import.

Quoted CSV values retain delimiters, escaped quotes and embedded line breaks. Literal multi-character delimiters and any-character separator sets also support quoting. Render Data exposes both `content` and `text` for mapping into HTTP bodies, files or subsequent activities. Headers, line endings and final separators are configurable.

Fixed-width records use positive widths or zero-based `start:end` offsets (end excluded), so columns can have gaps. The optional fixed line length includes the separator. Numeric fields use left padding and preserve significant trailing zeroes. Strict mode rejects over-width values; permissive mode retains truncation. Single-character delimited and fixed-width files are read record by record; the multi-character delimiter reader has a 64 MiB limit. Use Mapper for hierarchy and complex transformations.

### *Dynamic HTTP and REST invocation settings*

HTTP Request and REST Invoke Configuration keeps the display name and shared connection selector. Request-specific settings are mapped in Input -> RestInputRequest:

```
RestInputRequest
 Config
 host, port, scheme, requestURI, Method, RequestBody, bodyFormat
 QueryParameters, URIParameters, QueryString, FilePath
 timeout, activityTimeoutSeconds, followRedirects, raiseForStatus
 openApiDocument, operationId, openApiBaseUrl
 openApiReferenceDocuments, openApiAllowedRefOrigins
 validateRequest, requestSchema, validateResponse, responseSchema
 Headers
 accept, content-type, Accept-Charset, Accept-Encoding, Cookie, pragma
 CustomHeaders[] -> name, value
 mimeEnvelopeElement
 mimePart[] -> mimeHeaders, binaryContent | textContent | fileName
```

Map constants, properties, upstream values or functions. String literals require quotes; booleans use `true`/`false`. Contract/schema fields accept JSON objects or quoted JSON/YAML/XSD text as appropriate. Operation IDs select the contract operation, whose method/path are authoritative. Empty custom schema overrides use the contract.

Previously saved Configuration values appear as typed Input mappings without a manual migration. Listener routing, server connections and security policies remain startup settings because they must exist before a request arrives. Keep external-reference origins restricted to trusted values; empty disables network reference retrieval.

### *Verification*

The tests cover binary encoding overrides, CDATA, XML namespaces, XSD import/include resolution and access boundaries, Generic and XSD JSON conversion, Data Format import, dynamic Input mappings, rejected records, batching, multiline CSV and fixed offsets. All six activities run in fresh Direct and Engine archives and are compared with Studio Run and Debug.

### *Imported processes*

Import a compatible process ZIP from Studio. MINA can retain XML and JSON Parse/Render steps and their original XSLT input bindings. Selected XSD elements are resolved by namespace and local name, including schemas embedded in the process. When a connector blocks full process conversion, the import can recover supported Mapper and format activities as separate subtasks. Supply each recovered activity's original input variables by name. Studio reports unbound Parse Data resources and empty input bindings.

From `backend`, with its Python environment:

```
python -m unittest tests.test_format_activities tests.test_http_dynamic_contracts
tests.test_ai_exception_data tests.test_raw_python tests.test_engine_export tests.test_activity_packs
-q
```

References: [Python CSV newline handling](https://docs.python.org/3/library/csv.html) and [lxml parsing and validation](https://lxml.de/apidoc/lxml.etree.html).

Reference source: docs/PARSE_RENDER_ACTIVITIES.md

# Transformation and mediation capabilities

Mapping

## Guide

### *Transformation and mediation capabilities*

The Mapper remains the transformation activity. General activities provide durable message composition and recovery without reintroducing Mediation.

### *Mapper schemas and suggestions*

The Mapper supports multiple named views of one source, each with its own schema, optional XSD complex type, and occurrence range. Keep schema files at their project-relative paths when importing or editing the contract. The Mapper resolves imports and includes from project assets. A repeating element stays an array in the structured result even when the generated XML has one occurrence; the XML itself retains the original element structure.

In Visual AI Mapper -> AI mapping options, choose Exact names or a similarity strategy. Select a source or target branch to limit matching to that branch, then set each side's search depth from 1 to 10 (default 4). Leave a side on Entire schema to search all fields on that side. Existing manual mappings are preserved.

### *Drop a repeating structure*

On Mapper Input, drag a repeating complex source onto a repeating complex target and choose Finish after selecting a mapping mode:

- Make a copy of each copies the whole collection. The target shows one [Copy-Of] row with its source expression and hides the children. Existing child mappings in that occurrence are replaced. Clear the copy to show the children again. Use this when the source items already match the target structure.

- For each adds a [For-Each] row for the collection and keeps the item and its children visible. Matching paths are filled in without regard to case. Children display expressions relative to the current item, such as `Name` or `Address.City`, and remain editable.

- SmartMap builds an editable loop and also matches child paths that differ by underscores or hyphens. Ambiguous matches are left unmapped.

The same three choices are available on other activity Input tabs and in Edit mappings when both structures expose repeating records. This includes Call Sub Task inputs and End return mappings when the task has a schema. Drops onto simple fields still map the value directly.

To add another source for a repeating target, right-click the row and choose Duplicate mapping. This works in Mapper and other activity Input editors, including Call Sub Task and End. You can duplicate an unmapped repeating field too. The new occurrence starts with empty bindings. Drop a different collection onto it and choose a mapping mode for complex records. The original keeps its source and child bindings. At runtime the original items come first, followed by the configured duplicates in order. An occurrence with no source adds no items.

Right-click a complex element and choose Delete mapping to clear its binding and all mappings below it, even when only its children are mapped. Nested repeating occurrences within that branch are cleared too. The schema structure stays available for remapping. This works in Mapper and other activity Input editors, including HTTP responses, Call Sub Task and End. Deleting a duplicate occurrence keeps the original and sibling occurrences unchanged.

Use Map & Test to check the generated result before running or exporting the flow.

### *XML expressions and bindings*

In Mapper Configuration, choose the XPath version and expand XML expressions and runtime binding. Set the XML source to a mapping path such as `${last.xmlString}`, or leave it empty to use the previous activity's XML output. Add namespace prefixes and variables as JSON objects. For example, `{"ord":"urn:orders"}` binds a namespace and `{"minimum":"${input.minimum}"}` binds a value to `$minimum`.

In Input, enter an expression such as `xpath('string(/ord:orders/ord:order[@id=$minimum]/@id)')`. The XPath expression builder can test a raw formula against sample XML or a JSON envelope and apply it to the selected target. Choose a function to display its arguments. Drag a sample XML field into an argument, or select the field and use the argument's button. In XPath 2.0, the target cast action wraps the current formula in the target's supported XML Schema constructor. XPath node selections return sequences. Use an XPath scalar function when the target expects one value. Map & Test accepts raw XML or a JSON input envelope. XPath resource loading is disabled. XPath 2.0 regex functions run in an isolated worker with a deadline.

Import an `.xsl` or `.xslt` file or edit the binding directly to select manual XSLT. A manual XSLT 1.0 or 2.0 binding publishes `xmlString` and structured XML values. Select a target XSD to validate the generated XML. Referenced XSDs must be present in project schemas under their declared locations. Stylesheets run with file and network access disabled. XML inputs, stylesheets, and parsed results are limited to 8 MiB per document. XSLT 2.0 runs in a separate SaxonC worker with a scrubbed environment and a deadline. The configured XML variables also supply stylesheet parameters. XSLT 2.0 accepts XML-valued parameters as document nodes.

In Configuration, use Source type overrides to coerce specific source paths before formulas and XML parameter binding. For example, `{"input.order.amount":"decimal"}` converts the input amount to a number.

In Input -> Source coercions and cardinality, create named views of the same source path. Set optional, exactly one, repeating, or at least one. Select a project schema to copy its contract into the view, or enter an inline contract. For XML, a named complex type must be valid for the source element's declaration. For JSON, the selected contract validates each occurrence. Choose a view or one of its fields from Data Sources. Occurrence ranges and type validation apply when a rule reads that view. Source views run in visual mapping mode.

In the XPath builder, use Check XPath 1.0 quick fix or Check XPath 2.0 quick fix to compare the two evaluations. Apply a passing fix to the selected visual mapping. Other mappings keep their existing XPath version. A matching sample result does not establish equivalence for every possible input.

In Input -> Open visual AI Mapper, use Download XSLT to save the active stylesheet or generate XSLT 2.0 from direct rules. Generation supports XML element paths, scalar constants, If, Choose, For-Each, For-Each-Group, nested loops, and duplicate loop occurrences with mapped children. It supports trim, normalize space, and case conversion. A selected XSD must have one root and a uniform element qualification policy. Supply an XML sample with the named document root for testing. Use Open XSLT to restore the editable mappings from a generated file. Use generated XSLT binds execution to the current visual rules. Generated XML binding preferences controls optional field existence checks, suppression when a nil source maps to an optional non-nillable target, and copying `xsi:nil` to nillable targets. Generated conditions accept XML field paths, scalar literals, comparisons, and logical operators. Custom functions and mapping statements outside this subset require a handwritten stylesheet.

The XPath builder provides extension functions such as `ext:trim`, `upper-case`, `lower-case`, `left`, `right`, `concat-sequence`, `xor`, `substring-before-last`, `substring-after-last`, `timestamp`, `base64-to-hex`, and `hex-to-base64`. `project:getModuleProperty` reads scalar MINA project properties supplied to the execution. Java system properties, proprietary extensions, and custom Java XPath functions require a separate migration.

### *Legacy process import*

Open a compatible process file or ZIP through Studio's project import. Supported sequences, simple scopes, conditional XPath links, no-match links, error links, static subtask calls, Mapper bindings, XML render, file activities, and basic outbound HTTP requests become MINA activities. Supported HTTP, JMS and Timer event definitions become native starters. HTTP XML reply bindings become response activities. The importer also reads supported HTTP client and server settings, non-XA JDBC resources and static SQL activities, and JMS text-message activities with direct or JNDI connections. Passwords are represented by empty deployment properties. Project XSDs are included. Check the import messages in Studio for processes and settings that need review. Unsupported processes can contribute standalone Mapper tasks for their recoverable bindings. Supply original activity variables as task input for these standalone tasks. REST/SOAP service descriptors, explicit BPEL join expressions, XA transactions, vendor-specific JDBC/JMS features, and proprietary transport policies still need manual conversion.

### *Shared Smart Mapper learning*

Use Share reviewed mappings in Visual AI Mapper after reviewing direct field mappings. The Studio server stores field paths and schema fingerprints. Recommendations for the same source and target schema rank shared field pairs first. Disabled rules, pending suggestions, formulas, and literal values are excluded from learning. The catalog is stored in the Studio

server's data directory and is available to Studio users connected to that server.

Both Python archive formats include `requirements-mapper.txt` when they use XML bindings. Install those dependencies in the target runtime before running the archive.

### *Mapper tests, templates and contracts*

Open Map & Test and expand Saved transformation tests. Give each case a name, an input JSON value and an independently reviewed expected JSON value. Save the case and run the suite. Failures show field paths, expected/actual values and contract errors in dark red. Tests remain in the activity config and survive project save/export/import. They do not call connectors.

Versioned templates and schema changes exports a `.mtemplate.json` file containing schemas, mapping rules, lookup tables, execution policies, custom functions and test cases. Templates have a format version and content checksum; the checksum detects corruption, not authenticity. Import a template into another Mapper. Project schema IDs are cleared because IDs are project-specific. Local history retains the last 20 exported versions; changed content requires a new version. Restore a version, review its contract and rerun its tests.

Capture a contract baseline before changing the selected schema. Compare the current contract to see added/removed/changed fields, affected target mappings and raw contract changes. This reports changes; it is not a full backward compatibility proof for every schema dialect.

The portable validator now checks JSON `pattern`, `patternProperties`, common formats, conditional contracts (`if`/`then`/`else`), dependent required fields, property names and `contains` counts, in addition to the previous checks. Supported asserted formats are date, date-time, time, IPv4, IPv6, UUID, email, hostname and URI; unknown formats remain annotations. Pattern matching has a 100 ms timeout and requires the `regex` package, declared in deployment qualification when a schema uses patterns. XSD restrictions support enumeration, length bounds, numeric bounds and digit constraints. XSD patterns use the portable regex dialect with full matching; XML Schema-specific regex syntax and complete JSON Schema/XSD conformance are not claimed.

### *Lookups, enrichment and joins*

Save named JSON lookup tables under Lookup tables and execution limits:

```
{"countries": {"US": "United States", "IN": "India"}}
```

Use `lookupTable(${input.country}, "countries", "Unknown")` in Mapper, or append a `lookupTable("countries", "Unknown")` pipeline step. Keys are case-sensitive. `lookup(value, table, default)` accepts an object table directly; `enrich(record, fields)` merges objects with enrichment fields taking precedence; `joinBy(left, right, leftKey, rightKey)` produces matching merged records; `chunks(records, size)` splits an array into batches. Missing join keys do not match. Many-to-many joins stop at 100,000 result records.

### *Large data and cancellation*

Mapper has evaluation-step, execution-time and output-size limits. Defaults are 100,000 evaluation steps and 30 seconds. Execution runs in a worker thread; workflow cancellation signals the evaluator at subsequent evaluation checks. These are cooperative limits, not preemption of every individual function or vendor call. Input has target-field search and collapse/expand controls, so users can focus on a small portion of a large schema without losing mappings.

Both Python archive formats include:

```
python run_tests.py
Get-Content records.jsonl | python transform_jsonl.py --activity main/map --batch-size 100
```

`run_tests.py` runs saved suites and exits nonzero when any case fails or no saved suites exist. `transform_jsonl.py` processes one JSON record per line and prints one mapped JSON record per line. It runs Mapper alone, without connectors. Records are limited to 1 MB; batches default to 16 MB of serialized input/output and mapped records default to a 1 MB output limit. Use a smaller batch if the batch quota is exceeded. Output written before an error remains written; stage the output before publishing if all-or-nothing delivery is needed.

### *Durable message activities*

Use an explicit Message namespace for each integration and the same Durable state file for cooperating activities. `MINA_MESSAGE_STATE_FILE` can provide the file globally; the default is `message-state.sqlite3` under `MINA_DATA_DIR` or the user's `.mina` folder. SQLite transactions coordinate processes sharing the same local file. For containers, mount durable storage. This is not a distributed database for multiple hosts or network filesystems. Payloads are capped at 1 MB, aggregate groups at 64 MB and state capacity defaults to 256 MB. Size accounting is an approximate SQLite page budget.

- Deduplicate Message claims a stable message key and returns `accepted`

and `claimToken`. Route on `accepted`: process accepted messages, skip or acknowledge duplicate deliveries according to the connector policy.

- Message Receipt completes the exact claim token after downstream success

or marks it failed on an error path. Stale/expired tokens cannot complete a newer claim. Acknowledge the broker only after successful work and receipt.

- Replay Messages lists failed/expired candidates. `requeue` makes one

explicit key claimable again; it does not execute downstream activities. Feed the returned stored payload back through the normal task deliberately. `purge` removes expired completed keys and old completed aggregates; it keeps failed and pending messages. Completed keys default to seven-day retention.

- Aggregate Messages appends messages with a stable message ID and

correlation key. Duplicate IDs do not increase the group count. When the expected count is reached, `ready=true` returns the records once; subsequent deliveries do not emit again. Check `ready` before forwarding. After a timeout, `flush` explicitly emits a partial group. A completed group needs a new correlation/window key for the next aggregation.

- Split Records creates record batches from an array for use with the

existing For Each group. It is an in-memory activity; use the JSONL tool for streaming file transformations.

These primitives support reliable processing but cannot guarantee exactly-once external side effects. A crash between an external write and its receipt may require replay; make the downstream write idempotent using the stable key. Live broker/provider qualification remains necessary for production deployment.

### *Visual mapper and generated previews*

Open Visual AI Mapper from Mapper Configuration. The left pane contains editable source test data and available source paths, the middle pane contains saved mappings and the complete selected target schema field list, and the right pane displays generated output after Run mapping test. Named XSD types and local JSON Schema references use the same field parser as the Input tab. Source expressions remain editable, including upstream activity references and functions. Schema text is available under the collapsed schema sections.

Map & Test also displays generated output directly after running. JSON targets show formatted JSON; XSD targets show an XML preview with repeating elements and the target namespace. The preview does not change the Mapper runtime object or exported archive behavior; use Render XML or render-xml when the downstream activity requires a serialized XML value. Test data and the latest output are retained when saving the visual mapper.

Reference source: docs/TRANSFORMATION_CAPABILITIES.md

# Activity debugging and testing

Studio

## Guide

### *Activity debugging and testing*

#### *Job Data tree*

The ribbon's Job Data window shows processes, call activities, and nested subtask activities in an expandable tree (initially expanded). Select a process or activity to inspect its captured input and output. Data is keyed by process and activity, not by a flattened activity ID. Completed calls record the actual called task, including dynamically resolved calls. Unexecuted activities are labelled as not captured. Repeated invocations show the latest captured values for that task/activity; this is not a per-invocation execution-history viewer.

Windows installations must be rebuilt and updated with the new installer to receive these UI/backend changes; restarting an older installation is insufficient.

During an active debug session, a dedicated Debug ribbon appears underneath the Studio ribbon. It provides Continue, Pause, Stop, Step In/Over/Out, Jump In/Out, Run to Selected, Breakpoints, Watches, Variables, Evaluate, Job Data, Edit Payload, Preview Inputs, Test Activity, and Mock Output. Data/tool buttons open the relevant editor section; real tests and mocks still require explicit submission there. Commands are enabled according to session state. The ribbon disappears when the session completes, fails, or stops; it is not shown during ordinary Run mode.

Choose Run -> Start Activity Testing to create a paused debug session and open the test bench. Unlike ordinary Debug, this does not automatically run the flow or start event listeners. The existing Debug Tools window also contains the bench.

Use the searchable process/activity tree on the left of the test bench. Expand a process or subprocess and select its activity; enter the input in the right pane. Each target retains its own input draft while the dialog remains open. Selecting a Call Sub Task row selects the call activity itself; expanding its arrow reveals the called subprocess and its activities. Nested calls can be expanded the same way, and selecting a nested activity targets its owning subprocess. Search includes nested activities. Circular references stop with a notice. Dynamic calls show a configured fallback, if any, labelled as potentially different from the runtime target; unresolved targets are not guessed.

Selecting a process targets its unique entry activity (or Start); Run once still executes only that entry activity, not the entire process. If the entry is ambiguous, select an activity explicitly. Independent tests can target another process in the debug project without moving the paused flow. Mock stepping is restricted to the current paused process and activity.

Select an activity and enter a JSON payload (including arrays, strings, numbers, or null), or choose raw text/XML. The test payload is available as both `input` and `last` for this independent test. Earlier activity outputs and environment properties remain available for mappings.

- Preview inputs resolves configuration and mappings without executing the activity.

- Run once (real) executes the selected activity without advancing the paused

flow. Inspect resolved inputs, output, logs, duration, and failure details.

- Use mock output & step supplies the payload as the current paused activity's

output, skips its actual connector execution, and follows its normal transitions. Use this to simulate a receiver or downstream dependency without contacting it.

- Optional field overrides are a JSON object keyed by the activity's configuration

field names, such as `message` or `topic`. They are applied after mappings as literal values. Preview first to verify the inputs.

- Optional assertions use runtime conditions, for example `${last.count} > 0`.

Results distinguish execution failures from assertion failures.

The variable editor accepts whole `input`, `last`, and `vars` replacements, as well as nested paths such as `input.customer.id`. Replacing `input` does not also replace `last`; edit each explicitly when needed. `vars` must remain an

object.

### *Safety and scope*

Real tests use the configured connections and can publish, acknowledge, write, or call external systems. Use test environments and credentials. They are not an external-side-effect sandbox or an automatic rollback mechanism. Independent tests do not inherit a paused group's database transaction or lock context; use normal flow stepping to test those lifecycles.

Single-activity tests default to a ten-second timeout. A timeout cannot guarantee cancellation of work already submitted to a provider or blocking vendor driver. Mocks bypass the activity, not the surrounding group's execution semantics. Input overrides apply to configuration-driven activity fields; special task or interface mapping behavior still follows the activity's runtime implementation.

The most recent full result and the last 30 result summaries are retained for the debug session. Test payloads and mock results do not modify or export into the project. This bench complements breakpoints, watches, expression evaluation, variable editing, and flow stepping; real-provider integration and load testing still require the target infrastructure.

Reference source: docs/DEBUG_TESTING.md

# MINA Studio debugging

Studio

## Guide

### *MINA Studio debugging*

MINA Studio supports interactive debugging for ordinary tasks and continuous event starters.

### *Execution and logs pane*

Click Run or Debug to open Execution / Debug automatically in the lower pane, replacing the activity Configuration and Input tabs. The canvas remains visible above it. Run logs update while the task executes; debug logs update as you step or continue. Listener endpoints and generated outputs appear beside the logs.

The pane stays open while Debug is paused or a receiver is listening. It closes when execution completes, fails, or is stopped, restoring the selected activity tab. Its splitter resizes the lower pane; the expand and full-screen controls provide more space. To inspect retained logs after a session ends, choose Window -> Execution & Debug.

Click an activity on the canvas to replace the execution pane with that activity's Configuration tab. Its Input and Output tabs remain available for inspecting the mappings and schemas. Debugging continues in its current running or paused state; activity selection does not stop or restart the session. Use Execution Logs on the debug ribbon, or Window -> Execution & Debug, to return to the live logs.

When running Studio in a browser from a source checkout, the runtime serves `frontend/dist`. After updating the source, run `npm.cmd run build` from the `frontend` directory, then reload Studio in the browser (Ctrl+Shift+R in Firefox). A build written to a separate test output directory does not update the browser-served Studio. Short tasks can finish and close the pane immediately; use Window -> Execution & Debug to review their retained logs, or set a breakpoint before debugging to keep the session paused.

### *Execution controls*

- Continue runs until completion, an enabled breakpoint, a handled exception when pause-on-error is enabled, or a manual pause.

- Step In, Step Over, and Step Out navigate activities and Call Sub Task frames.

- Run to selected continues until the activity selected on the canvas is about to execute.

- Stop cancels the debug driver and listener, rolls back open transaction groups, and releases owned native processes.

### *Debug Tools*

Open Debug Tools from the debug bar to use the following features:

- Add an optional condition to any canvas breakpoint. Conditions use the same expression syntax as transitions, for example `${last.amount} > 100` or `contains(${last.status}, "FAIL")`.

- Enable or disable pause-on-error for handled error and retry paths.

- Add watch expressions such as `${input.orderId}`, `${last.customer.id}`, or `${vars.retryCount}`.

- Inspect the current input, previous output, process variables, process context, call stack, group iterations, and last exception.

- Evaluate an expression without modifying the running task.

- While paused, change a value under `input.*`, `last.*`, or `vars.*`. JSON values are typed; other input is stored as text.

Environment properties and connector credentials are intentionally excluded from the live-variable editor.

### *Continuous starters*

An EMS/JMS, Kafka, Pub/Sub, SAP, timer, or file listener displays ready · waiting for event until an event arrives. Breakpoints, watches, and pause-on-error remain active for every event. Stopping the session also stops the listener and prevents it from re-arming.

Reference source: docs/DEBUGGING.md

# Execution analytics

Studio

## Guide

### *Execution analytics*

Open Run -> Execution Analytics, View -> Execution Analytics, or the analytics icon in the Execution / Debug panel.

Select the current debug execution or a completed run. The window provides:

- Invocation and failure counts, summed activity duration and completed-run wall time.

- Recent-run duration graph and slowest-activity bar graph.

- Ordered invocation history, including repeated activity executions.

- Per-activity total, mean, minimum, maximum and nearest-rank P95 durations.

- Search by activity, task or type; click an activity name to open its configuration.

- CSV timing summaries, JSON metadata and a standalone HTML graph report. Open the HTML report in a browser to print or save as PDF.

Reports omit activity inputs, outputs, credentials and configuration. Timings include connector waits and retries. Subprocess durations include child execution, so summed activity times can overlap; they are not process wall time. Debug timings exclude time spent paused at breakpoints. Mocked debug steps measure the mock execution rather than a real connector call.

Execution history is held in memory by the running Studio backend and retains up to 100 runs. A backend restart clears this history; export reports to retain them. Older runs captured before this feature do not contain detailed timing records. These reports describe the Studio runtime and debugger; they do not aggregate remote Control Plane agents.

Reference source: docs/execution-analytics.md

# Data and file utilities

Studio

## Guide

### *Data and file utilities*

Open Window -> XML Viewer, Window -> JSON Viewer, or Window -> Compare Files. These tools use a dedicated Studio workspace above the orchestration editor, so a utility can remain open without changing the active Task.

### *Large-file behavior*

The desktop application gives the UI an opaque file handle rather than unrestricted filesystem access. It reads only the selected 256 KB, 1 MB, or 4 MB window. Previous and Next move through the file without retaining earlier windows. Browser mode applies the same behavior with `Blob.slice()`.

Each active window is editable. Desktop saves stream the untouched prefix and suffix through a temporary file and then replace the original, so even a very large file is not assembled in renderer memory. Navigation is locked while edits are unsaved. Browser mode downloads the edited result because web pages cannot overwrite the selected local file directly.

The utilities do not require an existing file. New XML, New JSON, New left, and New right create scratch documents where content can be typed or pasted directly. Save As converts a scratch document into a filesystem-backed document. Ctrl+S saves the editor that currently has focus.

Complete XML or JSON files up to 16 MB can be validated and pretty-printed as a whole. Larger files use a tolerant pretty printer for the current bounded window, avoiding an unbounded parse tree or full-file string in renderer memory.

### *XML Viewer*

- Opens XML, XSD, and WSDL files.

- Creates blank XML documents for direct typing or clipboard paste.

- Pretty-prints well-formed complete documents.

- Decode < and > changes `<` and `>` into visible angle brackets in the display.

- Wraps long or minified lines by default so payload content stays visible; wrapping can be toggled from the toolbar.

- Searches the current file window.

- Saves edits or formatting back to the selected desktop file and provides Revert for unsaved changes.

### *JSON Viewer*

- Opens JSON and JSON Lines files.

- Creates blank JSON documents for direct typing or clipboard paste.

- Pretty-prints and validates complete JSON documents.

- Uses bounded-window formatting for very large payloads.

- Wraps long or minified lines by default.

- Searches the current file window.

- Saves edits or formatting back to the selected desktop file and provides Revert for unsaved changes.

### *Compare Files*

- Accepts any two file types.

- Allows either side to be a newly created document instead of an imported file.

- Uses side-by-side aligned text rows for text files.

- Marks added, removed, and changed rows separately.

- Automatically displays binary or unknown content as hexadecimal and printable ASCII.

- Supports a differences-only view and synchronized window navigation.

- Allows individual difference rows-or all differences in the current window-to be selected and copied left-to-right or right-to-left.

- Provides a two-pane editing mode and independent Save left and Save right actions.

- Detects external file changes before saving and refuses to overwrite newer content.

- Renders at most 10,000 comparison rows per window to prevent unusually short-line files from creating an unbounded number of UI elements.

Reference source: docs/FILE_UTILITIES.md

# Runtime logging

Operations

## Guide

### *Runtime logging*

Studio records Run, Debug, listener, job, activity, transition, retry, exception, and application lifecycle events in the Execution / Debug panel. The disk button in that panel reloads saved records after a run or after Studio restarts.

Each environment properties file contains the user-configurable property:

`runtime.logDirectory`

Set it independently in `local.properties`, `dev.properties`, `qa.properties`, `pre.properties`, and `production.properties`. Run, Debug, listeners, and Stop use the selected runtime environment and write beneath that environment's configured directory. For example, setting the development value to `D:\IntegrationLogs\dev` produces:

`D:\IntegrationLogs\dev\<project-name>\<project-name>.log`

Relative values are resolved beneath the Mina data directory. Environment variables and `~` are expanded. When the property is blank, each project has an isolated structured log at the default location:

`<Mina data directory>/logs/<project-name>/<project-name>.log`

Log and audit timestamps use Arizona time by default (`America/Phoenix`, shown with the `-07:00` offset). Set `MINA_LOG_TIMEZONE` to another IANA timezone when a deployment requires a different display timezone. Scheduler and token expiry calculations continue to use UTC internally.

The active file automatically rolls at 10 MB. Four numbered archives are retained by default (`<project-name>.log.1` through `<project-name>.log.4`), so an individual file never grows without bound. Each line is UTF-8 JSON and includes the project, timestamp, level, message, and available run, correlation, task, activity, duration, and exception context. Failed package builds also write a `packaging` record with a traceback to this log, and the API returns the log path with the error. The exact resolved path is shown in Studio's Execution / Debug panel after clicking the disk button.

Packaged desktop installations place the Mina data directory under the Studio user-data location configured through `MINA_DATA_DIR`. Administrators may override logging without changing code:

- `MINA_RUNTIME_LOG_DIR`: process-wide fallback root when the active environment's `runtime.logDirectory` property is blank.

- `MINA_PROJECT_LOG_MAX_BYTES`: maximum bytes per active or archived file; default `10485760` (10 MB).

- `MINA_PROJECT_LOG_BACKUP_COUNT`: number of rolling archives; default `4`.

Payload bodies are written only when the activity's Automatic payload logging option is enabled. Passwords and secrets should never be deliberately mapped into log messages.

Reference source: docs/RUNTIME_LOGGING.md

# MINA Control Plane and Data Plane: one-stop guide

Operations

## Guide

### *MINA Control Plane and Data Plane: one-stop guide*

This is the authoritative setup and operations entry point. Use one program, `scripts/linux/mina-setup.py`, with separate INI files for the Control Plane and each Data Plane. The older shell entry points are compatibility aliases. Do not combine instructions from older layouts. No systemd or systemctl is required.

### *1. Provision the Linux hosts*

Ask the Unix team to provide an installation directory owned by the account that will run MINA, for example `/opt/mina`. Run the installation and processes as that account. Each host needs Linux, Python 3.12+ with venv/pip, Bash and network access to the approved Python package mirror, or a complete Linux wheelhouse. `curl` is useful for manual health checks. The Control Plane source build additionally uses PyInstaller, installed by the build script. Build and deployment must use the same Linux architecture and a compatible Linux distribution/glibc.

Data Planes need the Control Plane URL and owner credential for registration and the current remote-agent API. Use an internal HTTPS reverse proxy for remote access; the Control Plane listener itself is HTTP. Allow access to the configured Control Plane port/proxy and to the application connector endpoints. The setup script does not install OS packages, configure firewalls or create a TLS proxy.

For Java/JDBC/SAP/JMS connectors, provide JDK 17+ and licensed vendor JARs plus Linux native libraries where required. Windows `.dll` files and Windows virtual environments are not Linux runtimes. Set `[runtime] build_java_bridge=true` to compile the Java bridge automatically. Native ODBC, AMQP 1.0 and optional DB2 prerequisites must be supplied on hosts whose applications use them.

### *2. Prepare the software on Windows*

From the repository root in PowerShell:

```
cd D:\Integration-tool\IntegrationFabric
py -3 scripts/linux/mina-setup.py bundle --output MinaLinuxSetup.zip
scp .\MinaLinuxSetup.zip minauser@control-plane-host:/tmp/
```

The bundle contains `administrator/`, `backend/`, `scripts/`, Java bridge source, `drivers/` when present, the two INI templates and this guide under `software/`. It excludes local databases, logs, virtual environments, caches, Windows build outputs and release archives. Review vendor driver files before transfer. The bundle sets shell scripts to LF and removes UTF-8 BOMs; the Linux entry also normalizes copied shell scripts automatically. No `dos2unix`, script editing or `chmod +x` step is required to invoke the setup entry or its subscripts.

The Linux Administrator executable is built on Linux. A Windows executable cannot be used. To use a prebuilt Linux release instead, copy the entire `MinaAdministrator-<version>-Linux-x64.tar.gz` separately and set its absolute path in `[setup] administrator_archive`; set `build_administrator=false`.

### *3. Extract and configure the Control Plane*

On the Control Plane host, as the MINA account:

```
mkdir -p /opt/mina
python3.12 -m zipfile -e /tmp/MinaLinuxSetup.zip /opt/mina
cd /opt/mina/software
umask 077
mkdir -p /opt/mina/config
cp scripts/linux/mina-control-plane.ini /opt/mina/config/control-plane.ini
```

Edit only `/opt/mina/config/control-plane.ini`:

- `install_root=/opt/mina`, `source_root=/opt/mina/software` and the available

Python command/path under `[setup]`.

- `version` for the Linux Administrator build/release.

- `[control-plane] host` is a bind address; `0.0.0.0` binds all interfaces. Set

`port`, a strong `api_key`, and a separate stable `secret_key` for encryption. `CHANGE_ME` values are rejected. Preserve the encryption key across upgrades.

- Set `pip_index_url` to the approved package mirror, or `wheelhouse` to an

absolute directory with all required Linux wheels and build dependencies. With a wheelhouse, pip uses `--no-index`. Blank values use the configured pip defaults. The INI contains plain values without shell quoting or interpolation: `#`, `%`, `$` and `;` inside a value are preserved. Put comments on separate lines.

- `start=true` starts and health-checks the server after installation;

`start=false` installs it without starting it.

- Keep `install_local_runtime=false` when applications run on separate Data Planes.

Enable it only when the Control Plane host must also run local applications.

```
chmod 600 /opt/mina/config/control-plane.ini
python3.12 scripts/linux/mina-setup.py control-plane --config /opt/mina/config/control-plane.ini
--check
python3.12 scripts/linux/mina-setup.py control-plane --config /opt/mina/config/control-plane.ini
```

Execute from `/opt/mina/software` as shown. Absolute script/config paths also work from another directory. The default action is `setup`: it builds or reads the Linux archive, installs the complete bundle, retains existing Control Plane data, copies the private INI, starts the listener and waits for `/api/health`. Data Plane registration is a separate step. Do not append `&` to setup: it already starts the persistent process in the background and reports failures. Open the configured reverse-proxy URL, or `http://<control-plane-host>:19080/` on the permitted internal network, to use the Control Plane browser UI.

Default layout:

```
/opt/mina/
 software/ copied source and setup scripts
 config/ private operator-maintained INI files
 control-plane/ Linux executable, internal libraries, installed INI
 control-plane-data/ repository, encrypted secrets and audit state
 runtime/ Data Plane Python venv and runtime adapter (when installed)
 agent/<plane-id>/ isolated packages, agent configuration, PID and logs
 drivers/ operator-supplied Linux vendor drivers
 logs/control-plane/ administrator.log
 logs/runtime/ runtime logs
 run/ Control Plane PID
```

Optional INI keys `home`, `data_dir`, `log_dir`, `pid_dir`, and `runtime_command` under `[control-plane]`, and `data_dir`, `log_dir`, `driver_home`, `java_bridge_home`, `java`, `build_java_bridge`, `install_db2` under `[runtime]` allow explicit paths/options. Use absolute Linux paths. `runtime_command` accepts the existing application/environment placeholders; leave it absent for the generated local adapter. With separate Data Planes, the agent launches applications on its own host, not through the Control Plane's local runtime command.

### *4. Set up each Data Plane separately*

Copy the same software ZIP to the Data Plane host and extract it as in step 3. On that host:

```
cd /opt/mina/software
umask 077
mkdir -p /opt/mina/config
cp scripts/linux/mina-data-plane.ini /opt/mina/config/customer-plane.ini
```

Edit the new INI:

- `[setup] install_root`, `source_root`, `python`, mirror/wheelhouse and `start`.

- `[control-plane] control_plane_url` is the externally reachable URL (HTTP or

HTTPS), `admin_key` matches the Control Plane owner key, and optional `ca_bundle` points to a PEM trust bundle for an internal HTTPS CA. TLS verification remains enabled. Include required CA certificates; never use a verification bypass.

- `[data-plane] id`, `name`, `host`, `namespace`, `region`, `available_capacity`,

`heartbeat_seconds`, `agent_version` and `capability_name`. IDs and namespaces use letters, digits, underscores and hyphens. `host` identifies the runtime host in inventory; the setup must be executed on that host.

- Leave `state_dir` empty to derive `/opt/mina/agent/<id>`. Different planes must

not share state directories. Applications, PIDs and logs are isolated by plane.

- Configure the vendor driver directory and Java bridge options under `[runtime]`.

Copy the Linux vendor libraries into the configured driver directory before starting connector applications. `install_db2=false` omits optional `ibm-db`; set true only when the required DB2 build/runtime prerequisites are available.

- Team/user creation is optional: leave their `id` fields empty to manage them

through the UI. When using `[delivery-team]`, set its `id` and `name`; an empty `scopes_json` adds this plane/namespace to existing scopes. An explicit JSON array replaces the team's full scope list. When using `[user]`, set a unique user ID, `team_id`, role and scope. Empty `resource_id` uses this namespace.

```
chmod 600 /opt/mina/config/customer-plane.ini
python3.12 scripts/linux/mina-setup.py data-plane --config /opt/mina/config/customer-plane.ini --check
python3.12 scripts/linux/mina-setup.py data-plane --config /opt/mina/config/customer-plane.ini
```

This installs the host runtime, registers/updates the Data Plane, provisions its Integration Runtime capability, optionally creates the team/user, and starts its reconciliation/heartbeat agent. It does not install another Control Plane. The agent downloads assigned application archives, starts/stops runtime workers, reports health and handles graceful shutdown. Inspect its agent log and the Control Plane Data Planes view to confirm ONLINE status; successful registration alone is not proof that application deployments are healthy.

For another plane, copy the INI to a different filename and change its ID, name, host and namespace. If `state_dir` is explicit, give it a unique path. Run the same command on that plane's host. Separate planes can share one installed runtime on the same Linux host; stop affected agents before upgrading that shared runtime.

### *5. Lifecycle commands and upgrades*

From `/opt/mina/software`, use the same entry and same INI with `--action`:

```
python3.12 scripts/linux/mina-setup.py control-plane --config /opt/mina/config/control-plane.ini
--action status
python3.12 scripts/linux/mina-setup.py control-plane --config /opt/mina/config/control-plane.ini
--action stop
python3.12 scripts/linux/mina-setup.py control-plane --config /opt/mina/config/control-plane.ini
--action start
python3.12 scripts/linux/mina-setup.py data-plane --config /opt/mina/config/customer-plane.ini
--action status
python3.12 scripts/linux/mina-setup.py data-plane --config /opt/mina/config/customer-plane.ini
--action stop
python3.12 scripts/linux/mina-setup.py data-plane --config /opt/mina/config/customer-plane.ini
--action start
```

`restart` is available for both roles. To change installed software or INI values, stop the relevant process, back up state and the private INI/encryption key, replace the software tree, update the version/configuration and run `setup` again. Setup refuses to overwrite a running installation. Stops are graceful; Data Plane shutdown also stops managed runtime workers. The setup does not delete deployment state or generate a replacement encryption key. Keep the installation root and private configuration paths unchanged unless deliberately migrating the service.

`--check` validates the INI and source location without installing or starting anything; it does not prove that the network, OS libraries or package mirror are ready. API update errors other than 404 stop setup instead of creating duplicate resources. A failure leaves logs and completed setup steps available for diagnosis; correct the INI/prerequisite and rerun after stopping any affected process.

### *6. Troubleshooting and backup*

- Control Plane: `/opt/mina/logs/control-plane/administrator.log` and

`--action status`; check configured listener/proxy and `/api/health`.

- Data Plane: `/opt/mina/agent/<id>/agent.log`, its application logs and

`--action status`. Agent PID checks identify the expected script/configuration rather than signalling an unrelated reused PID.

- 401/403: verify the owner credential and intended permissions; it is never

printed by the setup program. 400/409: inspect conflicting namespaces/scopes or immutable capability identifiers before rerunning.

- Mirror/offline failures: supply Linux wheels matching Python and architecture,

including PyInstaller dependencies for source builds. Windows wheels cannot replace them. The mirror must provide the versions in the shipped requirements.

- Vendor failures: verify Linux native library compatibility, JDK/bridge and

driver locations. OS prerequisites and licensed drivers are not downloaded.

- Protect INIs with mode 600. Back up `control-plane-data/`, the stable encryption

key, private INIs, Data Plane state and any required vendor libraries together. Restore the same key with the data; changing it makes existing secrets unreadable.

### *7. Compatibility entries*

Use the Python entry for Windows-to-Linux transfers. `setup-mina-linux.sh` and `00-setup-and-register.sh` now delegate to Control Plane setup only. They read `MINA_CONFIG_FILE`; edit version and paths in the INI rather than supplying a positional version. `build-administrator.sh` delegates to the Linux build script. Numbered registration helpers and `deploy-application.sh` remain available for advanced operations. They are subscripts, not additional installation guides.

### *8. Control Plane model and operator reference*

#### *Purpose and operating model*

MINA Control Plane is the self-hosted management plane for MINA data planes, capabilities, applications, resources, access assignments, observability, and immutable deployment packages. A data plane represents an on-premises runtime host or Kubernetes runtime boundary. Capabilities are provisioned into a data-plane namespace, while applications are deployed to a selected data plane, namespace, and compatible Integration Runtime capability.

MINA separates control and data planes and provides namespace-scoped capabilities and applications, platform resources, role assignments, health/heartbeat inventory, application lifecycle, audit, and observability dashboards.

#### *Organization and delivery-team isolation*

The built-in Technology Team is the permanent Control Plane owner. It registers data planes, provisions capabilities, configures platform resources, creates delivery teams, assigns principals, issues or revokes delivery credentials, and can govern all assets. Set `MINA_ADMIN_API_KEY` for its production credential.

Each data delivery team must be assigned one or more exclusive `{data plane, namespace}` scopes. The same namespace cannot be assigned to two active delivery teams. Packages, extracted package storage, deployments, environment requirements, encrypted secrets, runtime logs, lifecycle operations, and application observability carry an immutable `teamId`. Backend authorization returns `404` when another delivery team probes an asset identifier, preventing both access and asset discovery.

Technology Team can upload a package on behalf of a delivery team by selecting its name beside Upload package. Delivery automation uses a one-time credential issued from Delivery teams and sends it in `X-Control-Plane-Key`. Only a SHA-256 hash of that credential is stored. An Application Manager token can upload, deploy, manage secrets, run lifecycle operations, and inspect its own team assets. An Application Viewer token is read-only. Delivery credentials cannot access Control Plane overview, data-plane registration, capabilities, resources, team/access administration, global monitoring, or audit APIs.

For production, configure corporate identity-provider groups at the reverse proxy or identity gateway and exchange their authenticated team identity for scoped Control Plane credentials. Never expose the Technology Team key to delivery pipelines.

Control Plane does not rebuild a Studio project. A package already contains the selected starter tasks and every reachable called task, shared connection, XSD/JSON schema, resource, property profile, and deployment descriptor selected during packaging. Cloud packages keep their generated Docker/Kubernetes descriptors and can be assigned only to a Kubernetes data plane. On-premises packages can run locally through the configured command adapter.

### *Configuration*

| Variable | Default | Meaning |
|---|---|---|
| `MINA_ADMIN_HOST` | `0.0.0.0` | HTTP bind address |
| `MINA_ADMIN_PORT` | `9080` | HTTP port |
| `MINA_ADMIN_DATA_DIR` | `administrator/data` | Repository, state, encrypted secrets, logs, and audit location |
| `MINA_ADMIN_API_KEY` | empty | Owner credential sent in `X-Admin-Key` or `X-Control-Plane-Key`. Credential-free owner access is restricted to direct loopback clients and same-origin browser requests. Remote and reverse-proxy clients must supply an owner key or scoped team token. |
| `MINA_ADMIN_SECRET_KEY` | generated local key | Stable encryption passphrase; supply from a secret manager in clustered/production installs |
| `MINA_ADMIN_RUNTIME_COMMAND` | empty | Administrator-approved runtime command template |
| `MINA_ADMIN_MAX_PACKAGE_MB` | `250` | Maximum uploaded archive size |
| `MINA_ADMIN_MAX_EXPANDED_MB` | `1024` | Maximum expanded package size |
| `MINA_ADMIN_MAX_PACKAGE_FILES` | `10000` | Maximum archive members |

The runtime command supports `{application}`, `{package}`, `{environment}`, `{deployment_id}`, and `{instance_id}` placeholders. The desktop installer includes a separate `MINAWorker` executable for this purpose. Example:

```
export MINA_ADMIN_RUNTIME_COMMAND='mina-runtime --application {application} --environment
{environment}'
```

On Windows desktop installations, configure the worker executable rather than the Studio sidecar:

```
$worker = 'C:\Program Files\MINA Studio\resources\runtime\MINAWorker\MINAWorker.exe'
$command = "`\"$worker`\" --application `\"{application}`\" --environment `\"{environment}`\""
[Environment]::SetEnvironmentVariable('MINA_ADMIN_RUNTIME_COMMAND', $command, 'Machine')
```

The runtime receives `MINA_APPLICATION_DIR`, `MINA_ENVIRONMENT`, `MINA_DEPLOYMENT_ID`, `MINA_INSTANCE_ID`, and decrypted deployment secret values in its process environment. The command comes only from trusted Administrator configuration; package contents cannot provide an executable command.

### *Package and deployment workflow*

- In Studio Packaging, select the starter tasks, environments, target, archive format (`.mpkg`, `.zip`, `.tar.gz`, or `.ear`), and deployment files. Only selected starter tasks and their recursively called Sub Tasks are packaged. Legacy `.ifpkg` uploads remain supported during the compatibility period.

- Select Export archive for an offline bundle, or enter the Control Plane URL, credential, team, data plane, namespace, capability, deployment environment, and required secrets and select Deploy to Control Plane. Studio builds the archive once, uploads those exact bytes, creates the deployment, and reports the Control Plane deployment ID/state. Credentials and secret values are transient and are not saved in the project or package.

- Alternatively, upload an exported archive in Applications. Administrator rejects traversal paths, links/devices, duplicate paths, oversized expansion, unsupported formats, missing project/task artifacts, and invalid manifests.

- Review checksum, target, profiles, selected task metadata, and secret requirements.

- For local on-premises deployment, Start uses `MINA_ADMIN_RUNTIME_COMMAND`. Generated packages contain Linux `install.sh`/`deploy.sh` and Windows `install.ps1`/`deploy.ps1`/`start.ps1` assets. If the command adapter is not configured, startup fails visibly rather than reporting a false running state.

- For cloud deployment, the Control Plane records the desired application, environment, replicas, namespace, and capability. The registered Kubernetes data-plane agent applies the generated Dockerfile, ConfigMap, Secret, Deployment, Service, HPA, and Kustomize assets; Studio does not incorrectly invoke the local command adapter for cloud targets.

- Use Details for instance PIDs and logs. Stop performs normal termination; Kill is forced; Restart stops and recreates desired instances; Undeploy removes deployment secrets and the active inventory record.

For a private CA, keep TLS verification enabled and supply the CA PEM path in Studio. Disabling verification is provided only for isolated development systems.

Legal lifecycle transitions are enforced. Package deletion is blocked while any non-undeployed deployment references it.

Application health is evaluated per deployment. For the local command adapter, Control Plane verifies each managed runtime PID. Remote and Kubernetes agents report `HEALTHY`, `DEGRADED`, `UNHEALTHY`, or `UNKNOWN` in the `deploymentHealth` object of their data-plane heartbeat; Control Plane does not infer application health merely because the data plane is online. Health checks can be disabled per deployment.

Starter Task start/stop changes the deployment's desired starter set. A running local deployment is restarted with `MINA_ENABLED_STARTERS` containing only enabled task IDs. Remote and Kubernetes runtime agents reconcile the same desired state and may report their observed task state with the health heartbeat. Whole-application lifecycle state and individual Starter Task state remain separate.

### *Control-plane model and screens*

- Overview: fleet, capability, deployment, request, and recent activity summaries.

- Data planes: local, remote on-premises, and Kubernetes registrations; namespace, capacity, tag, tunnel, heartbeat, and health inventory.

- Capabilities: namespace-scoped capability provisioning and status. Application deployment requires an Integration Runtime capability.

- Applications: searchable package/deployment repository; archive task and Starter Task inventory; health, instance details and logs; deploy, start, stop, configure, redeploy, undeploy, and safe package deletion; deployment and configuration revision history; individual Starter Task desired-state controls.

- Environments & secrets: exportable packaged profiles, JSON profile editing/upload, optional immediate redeployment, and required secret names. Password values are rejected from profile uploads because secret values remain encrypted and deployment-scoped.

- Observability: control-plane request/error totals plus data-plane and application CPU, memory, instance, and state telemetry.

- Resources: reusable global or data-plane-scoped resource definitions. Secret-valued properties are masked in list responses and audit entries.

- Access control: platform and team principals with Owner, Team Admin, Capability Manager, Application Manager, and Application Viewer roles, optionally scoped to a data plane and namespaces.

- Delivery teams: exclusive namespace allocation, asset counts, and one-time scoped automation credentials. Control Plane access remains disabled for delivery teams.

- Audit trail: provisioning, registration, deployment, resource, access, secret, and lifecycle operations with time, outcome, target, and detail.

### *REST API summary*

- `GET /api/health`, `/api/control-plane/overview`, `/api/monitoring`, `/api/observability`, `/api/audit`

- `GET|POST /api/data-planes`; `GET|DELETE /api/data-planes/{id}`; `POST /api/data-planes/{id}/heartbeat`

- `GET|POST /api/capabilities`; `DELETE /api/capabilities/{id}`

- `GET|POST /api/resources`; `DELETE /api/resources/{id}`

- `GET|POST /api/access/principals`

- `GET|POST /api/teams`; `PUT|DELETE /api/teams/{id}`

- `POST /api/teams/{id}/tokens`; `DELETE /api/teams/{id}/tokens/{tokenId}`

- `GET /api/session`

- `GET /api/applications`

- `GET|POST /api/packages`; `GET|DELETE /api/packages/{artifact}/{version}`

- `GET /api/packages/{artifact}/{version}/tasks`

- `GET|PUT /api/packages/{artifact}/{version}/environments/{environment}`

- `GET|POST /api/deployments`; `GET /api/deployments/{id}`

- `PUT /api/deployments/{id}/configuration`; `GET /api/deployments/{id}/health`

- `POST /api/deployments/{id}/starters/{taskId}/{start|stop}`

- `PUT /api/deployments/{id}/secrets`

- `POST /api/deployments/{id}/{start|stop|restart|kill|undeploy}`

- `GET /api/deployments/{id}/logs`

- `GET /api/revisions/{package|deployment}/{id}`

### *Security, recovery, and troubleshooting*

- Put Control Plane behind TLS/reverse proxy and set an API key. Restrict the data directory to the service identity.

- Permission enforcement is performed on every backend API call. UI filtering is informational and is not the security boundary.

- Back up the entire data directory together with the external `MINA_ADMIN_SECRET_KEY`. Losing or changing the key makes stored secrets unreadable.

- Rotate a secret with the secret API while stopped, then restart. The API returns secret names/configuration state only.

- A `FAILED` deployment preserves its error and logs. Correct the adapter or configuration and select Restart.

- `No runtime adapter is configured` means package validation and deployment creation are working, but `MINA_ADMIN_RUNTIME_COMMAND` has not been supplied.

- Control Plane PID reconciliation detects local processes that exited unexpectedly. Data-plane registration and heartbeat APIs provide the management-plane inventory. Remote command execution still requires a trusted data-plane agent/tunnel; this repository does not silently execute remote commands or claim a remote application is running without that adapter.

### *Application operations*

Open Applications to manage deployments or uploaded packages. Existing configuration, starter controls, logs, revision history, and health information remain available through Manage on an application card.

### *Views and finding applications*

- Cards shows application identity, environment, owning team, instance state, and reported health.

- Compact provides denser application cards.

- Status board groups deployments by lifecycle state. It is not a drag-to-deploy board: state changes require explicit lifecycle actions.

- Combine search, lifecycle state, data plane, environment, and team filters. Needs attention selects failed deployments or unhealthy/degraded health reports.

- Star applications and enable Favorites to focus on them. Favorites and view preferences are stored locally in this browser, not shared with other operators.

- Ctrl+K (Command+K on macOS) opens application search, unless a dialog is open.

### *Bulk lifecycle operations*

Select individual deployments or Select visible, then choose Start, Stop, or Restart. Only eligible selected deployments are included in the confirmation dialog. Review the names, environments, and teams before confirming. Stop/restart may interrupt processing.

The client re-fetches each deployment before sending its operation, executes requests sequentially, and reports accepted, skipped, or failed results separately. A failure does not prevent later targets from being attempted. Accepted requests are not proof of healthy execution; inspect updated deployment status and health. Closing the result dialog does not cancel requests already sent. Changing filters clears hidden selections. There is deliberately no bulk delete.

UI eligibility follows the Technology Team/Application Manager model. Every request still uses the existing authenticated API and server-side authorization; client-side controls do not grant permissions.

### *Themes*

Use Theme in the top header: Ocean dark, Slate dark, Light, or System. System follows the operating system preference. The choice persists locally per browser origin and applies to navigation, forms, dialogs, application views, and telemetry surfaces. Windows desktop and browser clients use the same served UI; their local preferences may differ.

### *Updating an installation*

Deploy the updated `administrator/web` assets together, including `index.html`, `operations-workspace.js`, and `operations-workspace.css`. Refresh the browser or reload the Windows Control Plane client. No database migration is required. Do not replace your existing administrator data directory, environment settings, or credentials.

### *Application inspector and operations*

The Control Plane home page summarizes running applications, data-plane connectivity, unhealthy/stopped deployments, and recent operations. The Applications workspace lets an operator search and filter deployments and packages, then inspect an application without leaving the list.

The application inspector exposes lifecycle actions, health and instances, environment/instance/health-check configuration, write-only secret updates, Starter Task controls, runtime logs, revision history and revert, configuration comparison, backup, and restore. Existing package upload, archive inspection, and deployment flows remain available.

The Telemetry page shows CPU, memory, and disk readings reported by agents, with bounded trends (up to 720 samples per data plane, sampled no more than once every 15 seconds). It also shows task outcomes and recent Control Plane request errors. Blank readings mean the agent has not reported that metric; they are not treated as zero.

Operators can create or delete alert rules for offline data planes, failed deployments, unhealthy applications, and CPU/memory/disk thresholds. Resource rules report `NO_DATA` when a host has not sent the selected metric. Rules are evaluated in the Control Plane UI/API; external notification delivery is not implemented by these rules.

Python data-plane agents install `psutil` to report host CPU, memory, and disk. If `psutil` is unavailable, the agent still reports disk space and its local process count. A remote on-premises Python agent forwards a bounded tail of each instance log. Kubernetes pod-log forwarding is not yet implemented; local deployments are read from the Control Plane host.

### *Optional Windows client*

The Linux Control Plane remains the server. The existing browser UI remains available at the same HTTPS address. This client adds a Windows application; it does not install Python, start a local Control Plane or change server permissions.

### *Build on Windows*

From `frontend`, run `npm ci` if dependencies are not installed, then:

```
npm run control-plane:installer
# Or an unpacked build:
npm run control-plane:unpacked
# Override the version without editing application code:
powershell -File ../scripts/build-control-plane-desktop.ps1 -Version 1.2.0
```

Installer: `frontend/release-control-plane/MINAControlPlane-<version>-Setup.exe`. Unpacked executable: `frontend/release-control-plane/win-unpacked/MINA Control Plane.exe`. These are separate from the Studio installer and Administrator server archive. Release signing requires your organization's Windows code-signing configuration; unsigned development installers may trigger Windows SmartScreen warnings.

### *Connect*

- Update the Linux server's `administrator/web/admin.js` from this version (or

redeploy the updated Control Plane). Sign-in and rename dialogs now support both browsers and Electron. Older servers using `window.prompt` are incompatible.

- Expose the existing Control Plane through HTTPS with a certificate trusted by

Windows. A reverse proxy can terminate TLS and forward to the existing service. Serve at the origin root, e.g. `https://mina.company.com`, not `/control-plane`. Enable authentication and preserve existing API routes, uploads and downloads.

- Install and launch MINA Control Plane on Windows, enter that HTTPS address,

then enter your existing Control Plane credential when requested.

- Use Connection -> Reload / reconnect after a connection failure, or

Sign out / change server to end the local session. Closing the app does not stop remote applications or cancel operations already accepted by the server.

### *Security and limitations*

Only the server address is stored in `%APPDATA%/MINAControlPlaneDesktop/server.json`. Remote browsing uses an isolated, non-persistent session. Its cookies, credentials and local storage are not persisted across app restarts; changing server creates a new session. Browser credential storage behavior is unchanged. Local sign-out does not revoke a server-issued credential; revoke credentials through server administration.

Remote pages have no Node.js or desktop IPC access. Sandboxing, context isolation, certificate validation and web security are enabled. HTTP and invalid certificates are rejected; there is no bypass switch. Cross-origin navigation, pop-ups and device permissions are blocked. External identity-provider redirects and cross-origin CDNs are not supported by this initial client. Existing same-origin Control Plane credential authentication, uploads, exports and management APIs remain available.

Network outages interrupt the desktop view, not deployed workloads. Reload when connectivity returns. This is an online client, not an offline administration replica.

### *Development and tests*

`npm run control-plane:desktop` launches from source. `npm run control-plane:test` checks URL policy, isolation settings, independent packaging and dialog compatibility. The entry point is `frontend/control-plane-desktop/main.cjs`; the connection screen is `connect.html`, `connect.js` and `connect.css`. Remote content has no preload. The local screen's narrow IPC bridge is validated against its exact sender/frame. Follow release qualification with a real HTTPS Linux server: sign-in, team isolation, package upload/download, start/stop, network interruption, reconnect and local sign-out.

Reference source: docs/LINUX_SETUP.md

# Python archive modes

Deployment

## Guide

### *Python archive modes*

Studio exposes two distinct Python exports. Both contain Python application files and `manifest.json` for Control Plane package metadata; neither includes project, task, or resource JSON descriptors.

- Export direct Python / Deploy direct Python compiles supported task transitions into async Python state machines. The generated task source contains direct `if`/`elif` flow decisions, inline group conditions/loops, `asyncio.gather` for success fan-out, and typed Python references for property and activity bindings. It does not bundle the MINA workflow engine. The exporter rejects unsupported semantics with activity names instead of silently substituting an engine-backed implementation. Supported paths include Start, End, Scheduler, Log, Mapper (including functions, conditions, and repeating targets), Call Sub Task, global and transition Catch, Throw, Rethrow, General activities (Empty, Assign, Sleep, Checkpoint, shared variables, and External Command), File, FTP/SFTP, hosted and outbound HTTP/REST/SOAP, HTTP Response, XML/JSON/flat-data parse and render, Excel Read, Transform, Python/Java Invoke, JDBC, Snowflake, AMQP, Kafka, Pub/Sub, EMS/JMS send, receive, request/reply and reply, and supported SAP operations and IDoc parsing. Scheduler, Kafka, Pub/Sub, EMS, JMS, SAP, HTTP, REST, and SOAP starters remain active until stopped. SAP IDoc listeners support direct JCo and EMS/JMS/Kafka messaging sources. Single-entry/single-exit If, For Each, Iterate, While, Repeat, Repeat-on-Error, JDBC transaction, and critical-section groups compile to explicit Python control flow and may be nested with mixed group types. Client-acknowledge JMS receivers use a persistent bridge session and Confirm Message can acknowledge an active delivery.

- Python compatibility archive (engine-backed) retains the full Python runtime engine and the existing wider activity/group behavior. It remains deployable but is not a full direct-code conversion. The package manifest records `pythonSource.mode: engine` so consumers can tell which implementation they received.

The standard `.mpkg` and `.ear` exports remain available. Python archives require separately licensed vendor drivers where the underlying connection needs them (for example SAP JCo or JMS/EMS). A Python source package does not replace those vendor libraries. Legacy `.ifpkg` packages remain importable.

### *Parallel branches and joins*

Multiple eligible success transitions start parallel branches. When all branches converge on a shared activity, Run, Debug, and Python exports wait for the branches and execute that shared activity once. Nested forks join at their own convergence before reaching an enclosing join. Partial joins also wait for their participating branches: if two of three branches merge first, that intermediate activity runs once before its result joins the third branch. An unhandled branch failure prevents the shared continuation; Run and Python exports let sibling branches settle before reporting the failure. Debug steps branches individually and applies the same join barrier.

Each branch retains the task input and receives its own flow value and variables. Activity outputs remain available for explicit downstream mappings. The default value at a join comes from the last branch in transition order, independent of completion timing; business objects are not implicitly merged.

For example, connect Kafka and Pub/Sub branches to the same EMS activity to send to EMS after both complete. If EMS is instead a third outgoing branch, it runs in parallel and the common continuation waits for all three. Completion follows each connector's settings: Kafka `waitForDelivery=true` waits for broker acknowledgement; `false` completes after local enqueue. Regenerate deployed Python archives after upgrading Studio's execution behavior.

Engine-backed Python archives select connector modules from the packaged activities and groups instead of copying every engine file. All included Sub Tasks participate in selection, and JDBC transaction groups include the JDBC adapter. The exporter follows adapter imports transitively (including Java bridge dependencies and the JDBC adapter's Snowflake support). Shared scheduling, model, mapping/expression, and timestamp modules remain in every engine-backed archive. Optional connectors load only when executed; unused shared connection definitions do not cause their connector modules to be bundled. Standard JSON-based EAR archives do not contain this Python engine.

Direct archives are capability-linked from the persisted project model and selected task closure. The linker never checks project names, sample identifiers, known canvas layouts, or a catalog of previously exported applications. It records the exact activity operations and group types in `application/capabilities.py`, removes unreachable execution branches, specializes connector functions to the operations used by the project, and omits unreferenced activity modules, connector modules, native adapters, inbound servers, shared resources, schemas, and properties. Referenced shared resources are followed transitively, as are XSD `schemaLocation` and JSON Schema `$ref` dependencies, so dependency pruning does not produce an incomplete package. Its registry is required to cover every activity operation advertised by the direct exporter; adding a future supported Studio activity without linker metadata fails the exporter and its coverage test instead of silently generating an incomplete application. A property-driven resource, schema, or SAP messaging-provider selection conservatively retains compatible candidates because the selected object can change between packaged environments. Packaging never removes something required by a selected environment merely to make the archive smaller.

For a successfully compiled direct archive, run `python exported.pympkg --task TASK_ID --input '{"key":"value"}'` on a standard Python host. No extraction or source edit is needed. Alternatively, extract the archive and use `python run.py` as a Python-script job entry point (including a Databricks Python script task); pass `--environment PROFILE` when multiple profiles are packaged, and `--task TASK_ID` for a single task. Without `--task`, the generated application runs the selected starters; receiver starters keep polling until stopped. If no environment is specified, the first packaged profile is used.

Run `python run.py --check` (or `python exported.pympkg --check`) in the target data plane before deployment. The generated preflight checks only the dependencies used by that application, returns machine-readable JSON, and exits with code `2` when a required package, Java runtime, or external project artifact is missing. The Control Plane manifest also exposes the preflight command and dependency inventory.

For qualification against real providers, run `python run.py --qualify-task TASK_ID --input '{"test":true}' --iterations 1000 --concurrency 10`. Optional thresholds include `--max-error-rate`, `--max-p95-ms`, and `--max-memory-growth-mb`; `--duration-seconds` turns the same command into a bounded soak test. The command exercises the actual exported task and produces JSON evidence containing throughput, success/failure counts, latency percentiles, bounded error samples, and Python memory growth. It exits with code `3` when a threshold fails. Provider failover is tested by running this command while performing an approved broker/database/SAP failover in the test environment.

Real connectors require runtime provisioning, not source edits. Set `MINA_JAVA_BRIDGE_HOME` to the installed Java bridge directory containing `classes`, and `MINA_DRIVER_HOME` to a directory containing `sap/` and/or `jms/` licensed vendor JARs. Rebuild the bridge from this source before deploying client-acknowledge JMS flows. Place the SAP JCo native library in the SAP driver directory or on the platform native library path. A Java runtime must be available via `MINA_JAVA` or `PATH`. Depending on the generated preflight inventory, install `aiokafka`, `google-cloud-pubsub` and `google-auth`, `openpyxl`, `paramiko`, `snowflake-connector-python`, `pika`, or `azure-servicebus`. Provide connection URLs, credentials, and target-network access through the selected environment profile or environment variables. Missing packages and artifacts are reported before application startup.

The direct export is not a blanket compatibility promise for every future Studio feature. Grouped parallel fan-out still requires an explicit top-level branch boundary because a deterministic in-group join is not represented in the saved graph. Live ECC/JCo, EMS/JMS provider, AMQP, Snowflake, failover, and soak results cannot be asserted without those systems; use the generated qualification command in the target test environment and retain its evidence before approving production. The automated suite covers hosted REST, JMS request/reply correlation, SAP-over-Kafka, mixed nested groups, mock/in-memory providers, SQLite transactions, and bridge compilation. The saved `order-integration` project passes direct compiler static validation, which is not itself a live SAP qualification. Export reports unsupported graph behavior instead of generating a package with silently different behavior.

Reference source: docs/PYTHON_EXPORT.md

# MINA build and distribution

Deployment

## Guide

### *MINA build and distribution*

#### *Product outputs*

MINA has three independently distributable components:

- Studio - an Electron desktop IDE for Windows. It embeds the React designer and a packaged Python sidecar.

- Control Plane - a Linux or Windows management plane for data planes, capabilities, applications, resources, access, observability, and package deployment.

- Runtime package - a portable `.mpkg`, `.tar.gz`, or `.ear`-compatible archive produced by Studio for either an on-premises or Kubernetes data plane.

#### *Windows installation progress*

The assisted Windows installers display a checklist during setup. Studio lists the app, local runtime, and background worker. Control Plane lists the desktop app. The installer checks off the bundled components together after extraction, then marks Windows registration and shortcuts complete.

#### *Build Studio on Windows*

Prerequisites:

- Windows 10 or Windows 11 x64

- Node.js satisfying Vite's requirement (`20.19+` or `22.12+`)

- Python 3.11 x64 available through the `py` launcher

- Internet access on the build machine for the first dependency installation

Run from a normal PowerShell or Command Prompt. PowerShell is used internally by the npm build script, but installed Studio users do not need PowerShell, Node.js, or Python.

```
cd D:\Integration-tool\MINA\frontend
npm ci
npm run desktop:installer -- -Version 2.4.0
```

Output:

```
frontend\release\MINAStudio-2.4.0-Setup.exe
```

`-Version` is injected into Electron Builder at build time; `package.json` is not edited. It must be a semantic version. CI can set `$env:MINA_VERSION = '2.4.0'` and run `npm run desktop:installer` without passing the argument. When neither is supplied, the version in `frontend/package.json` is used as the fallback.

The installer is machine-wide so different Windows users can launch Studio in parallel. Each user gets an independent local runtime port and workspace data directory under that user's application-data profile.

Useful development commands:

```
npm run desktop:dev # Vite UI in an Electron window with the Python sidecar
npm run desktop:unpacked # Build without creating the installer
npm run desktop:installer # Build the sidecar and final Setup.exe
```

#### *Corporate TLS certificate or proxy*

`UNABLE_TO_GET_ISSUER_CERT_LOCALLY` from npm and `CERTIFICATE_VERIFY_FAILED` from pip mean the build machine does not trust the CA used by an HTTPS-inspecting corporate proxy. This is download trust, not Windows

executable signing. Obtain the organization's root and intermediate CA chain from IT as one PEM file. Do not commit thecertificate to this repository.

Configure both Node/npm and Python/pip from PowerShell, then open a new terminal:

```
$ca = 'C:\Certificates\company-ca-chain.pem'
npm.cmd config set cafile "$ca" --location=user
npm.cmd config set strict-ssl true --location=user
py -3.11 -m pip config --user set global.cert "$ca"
[Environment]::SetEnvironmentVariable('NODE_EXTRA_CA_CERTS', $ca, 'User')
[Environment]::SetEnvironmentVariable('PIP_CERT', $ca, 'User')
```

For the current terminal, set the values as well:

```
$env:NODE_EXTRA_CA_CERTS = $ca
$env:PIP_CERT = $ca
$env:REQUESTS_CA_BUNDLE = $ca
```

Keep SSL verification enabled. If the company requires an explicit proxy, obtain its URL from IT and additionally configure `npm config set https-proxy <proxy-url> --location=user`, `HTTPS_PROXY`, and `ELECTRON_GET_USE_PROXY=1`.

After a failed `npm ci`, close running Node/Electron processes and delete the incomplete `frontend\node_modules` directory before retrying. The desktop build script now stops at the first failed npm, pip, Electron, or PyInstaller command instead of reporting a misleading successful sidecar build.

The desktop runtime is built with Python 3.11. Install the standard x64 distribution from python.org; a Microsoft Store execution alias that appears in `py -0p` but cannot launch is not sufficient. If `backend\.venv` uses another Python version, the installer build now preserves it and creates a dedicated Python 3.11 environment under `backend\build\.venv311`. The AMQP 1.0 connector uses the prebuilt `python-qpid-proton-wheel` distribution because the upstream `python-qpid-proton` 0.40 source release does not provide a CPython 3.11 Windows wheel and otherwise requires Microsoft Visual C++ Build Tools. Before PyInstaller starts, the build imports every external connector module and fails if a package or native DLL is unavailable.

Database shared connections accept either the Studio host/port/database fields or vendor JDBC URLs and translate them to the corresponding native Python adapter. The packaged runtime includes adapters for SQLite, PostgreSQL, MySQL/MariaDB, Oracle, IBM Db2, Databricks SQL, Snowflake, and SQL Server. SQL Server additionally requires Microsoft ODBC Driver 18 (or 17) to be installed on the runtime machine; the Studio detects installed drivers and reports the available names. Databricks supports personal access tokens, OAuth machine-to-machine, and interactive OAuth user-to-machine authentication with SQL warehouse hostname, HTTP path, catalog, and schema configuration.

### *Real SAP ECC connections*

SAP shared connections use the licensed SAP Java Connector (JCo). New SAP connections use the external adapter by default; the local mock remains available as an explicit design-time option. JCo is not included in the installer. Place `sapjco3.jar` and its matching `sapjco3.dll` in the configured SAP driver directory (or the default `C:\ProgramData\MINA Studio\drivers\sap`). No Python SAP package or separate NW RFC SDK installation is required.

The repository includes `requirements-sap.txt` as a reminder. The Java bridge loads JCo at runtime, so the JCo JAR and DLL remain outside source control and are reported in the connection-test response.

See [SAP_INTEGRATION.md](SAP_INTEGRATION.md) for the supported IDoc/ALE, RFC/BAPI, transaction, lifecycle, and production certification behavior.

Ensure the SDK's native library directory is on `PATH` before starting Studio. For an application-server connection, configure `ashost`, `sysnr`, `client`, `user`, and `passwd`; for a message-server/logon-group connection, configure `mshost`, `sysid`, `group`, `client`, `user`, and `passwd`. SNC connections additionally require the SNC partner name and SAP Cryptographic Library path. The Test Connection action calls `STFC_CONNECTION` and reports the native RFC error when authentication, routing, authorization, or network setup fails.

After PyInstaller finishes, the build launches that exact packaged runtime on an isolated loopback port and requires a successful `/api/health` response before Electron Builder is allowed to create an installer. Installed Studio startup writes the runtime path, port, stdout, stderr, spawn errors, and exit status to `%APPDATA%\MINA Studio\logs\runtime-startup.log`. Startup errors include the log path and its latest output instead of only showing a generic readiness timeout.

The Windows Studio window opens while the local runtime starts and switches to the welcome screen after the runtime passes its health check. Launching Studio again in the same Windows profile brings the existing window forward. If startup is slow, the same log records elapsed time for the startup page, runtime readiness, and Studio page load.

The build compiles the complete backend with Python 3.11 before freezing it. PyInstaller then explicitly collects every module under the backend `app` package and verifies `app.main` both before and after freezing. This catches incompatible Python syntax early and avoids environment-dependent namespace/package discovery producing an executable that fails with `ModuleNotFoundError: app.main`.

The unpacked executable is:

```
frontend\release\win-unpacked\MINA Studio.exe
```

For production distribution, configure a company `.pfx` code-signing certificate through environment variables. A signing certificate is optional for producing an installer, but recommended for publisher identity and SmartScreen reputation:

```
$env:WIN_CSC_LINK = 'C:\Certificates\mina-signing.pfx'
$env:WIN_CSC_KEY_PASSWORD = '<password-from-secret-store>'
npm.cmd run desktop:installer
```

Do not put the certificate or password in `package.json` or source control. Electron Builder automatically discovers `WIN_CSC_LINK`/`WIN_CSC_KEY_PASSWORD` (and supports `CSC_LINK`/`CSC_KEY_PASSWORD` as fallbacks).

### *Build Control Plane on Windows*

```
cd D:\Integration-tool\MINA
.\scripts\build-administrator.ps1 -Version 2.4.0
```

Or from the `frontend` directory:

```
npm run administrator:windows -- -Version 2.4.0
```

Outputs:

```
administrator\dist\MinaAdministrator\MinaAdministrator.exe
administrator\release\MinaAdministrator-2.4.0-Windows-x64.zip
```

The ZIP is a PyInstaller one-directory distribution. Copy or extract the complete `MinaAdministrator` directory, including its `_internal`, `web`, and `bin` content. Copying only `MinaAdministrator.exe` will not work reliably.

Run it:

```
$env:MINA_ADMIN_DATA_DIR = "D:\MINAAdmin\data"
& .\administrator\dist\MinaAdministrator\MinaAdministrator.exe
```

Open `http://localhost:9080`. To listen on a different address or port, set `MINA_ADMIN_HOST` and `MINA_ADMIN_PORT`.

### *Build and install Control Plane on Linux*

Use the [Control Plane and Data Plane one-stop guide](LINUX_SETUP.md) for transfer packaging, INI configuration, installation and lifecycle commands.

### *Run Control Plane as a container*

```
docker build -f Dockerfile.administrator --build-arg MINA_VERSION=2.4.0 -t mina-administrator:2.4.0 .
docker run -d --name mina-admin -p 9080:9080 \
 -v mina-admin-data:/var/lib/mina/administrator \
 mina-administrator:2.4.0
```

### *Create deployment packages in Studio*

Open Packaging in Project Explorer or select Package on the ribbon. Configure:

- Artifact name and version

- Target: On-premises Linux or Cloud / Kubernetes

- Target environment

- Archive: `.mpkg`, `.tar.gz`, or `.ear`

An on-premises package contains an Administrator deployment descriptor. A cloud package contains a Docker build input and Kubernetes deployment manifest. Password property values are removed from both outputs and replaced by deployment-time secret requirements.

The `.mpackage` file is the editable Studio project package. The `.mpkg` file is the immutable deployment artifact. Studio and Control Plane continue accepting legacy `.ifproject`, `.ifpackage`, and `.ifpkg` files during the compatibility period.

### *Vendor Java connector bridge*

Desktop builds require JDK 17 or newer (`javac` and `jlink`). The build bundles a minimal Java runtime automatically; target machines do not need Java installed. EMS, Microsoft SQL Server JDBC, and Oracle JDBC JAR placement is documented in [VENDOR_DRIVERS.md](VENDOR_DRIVERS.md).

Reference source: docs/BUILDING_AND_DISTRIBUTION.md

# Vendor driver installation

Deployment

## Guide

### *Vendor driver installation*

MINA Studio bundles its own minimal Java runtime and connector bridge. Vendor-licensed JARs are intentionally not distributed in the installer.

Vendor native client libraries (`.lib`, `.dll`, `.so`, `.dylib`) and license keys are also external prerequisites. They are ignored by Git under `drivers/`, and artifact verification rejects known SAP/EMS/JMS binaries in release payloads. Keep local drivers in the configured driver folder; source cleanup must not delete customer-owned local copies. See [distribution compliance](../compliance/README.md) for release SBOMs, notices and remaining permission/source-access review.

The default machine-wide driver root on Windows is:

```
C:\ProgramData\MINA Studio\drivers
```

Set `MINA_DRIVER_HOME` to override the complete root, or enter a Driver JAR directory in an individual shared connection. Source checkouts also search `D:\Integration-tool\MINA\drivers` as a development fallback.

### *EMS / JMS*

Place the EMS client files in:

```
C:\ProgramData\MINA Studio\drivers\jms\tibjms.jar
C:\ProgramData\MINA Studio\drivers\jms\jakarta.jms-api-2.0.3.jar
```

For older EMS releases, use the matching `jms-2.0.jar` supplied with that EMS installation. Add `tibjmsufo.jar` for unshared-state failover. Do not mix JARs from different EMS releases.

The EMS direct factory class is supplied by the runtime default. If overriding it, use the exact fully qualified class name from your licensed driver documentation. Java class names are provider identifiers and must not be renamed.

Direct mode uses the EMS URL, username, and password. JNDI mode additionally uses the configured initial-context factory, provider URL, credentials, and connection-factory JNDI name.

### *Microsoft SQL Server*

Download the Microsoft JDBC Driver for SQL Server and place the Java 11-compatible JAR in:

```
C:\ProgramData\MINA Studio\drivers\jdbc\sqlserver\mssql-jdbc-<version>.jre11.jar
```

The driver class is `com.microsoft.sqlserver.jdbc.SQLServerDriver`. SQL Server Authentication is fully Java-based and does not need ODBC. Windows Integrated Authentication additionally requires Microsoft's architecture-matched authentication DLL; username/password authentication is the portable default.

### *Oracle Database*

Place the Oracle JDBC driver in:

```
C:\ProgramData\MINA Studio\drivers\jdbc\oracle\ojdbc11.jar
```

Optional Oracle companion JARs such as `orai18n.jar`, `ucp.jar`, or Oracle PKI libraries can be put in the same directory when the selected Oracle features require them. The driver class is `oracle.jdbc.OracleDriver`. Thin JDBC mode does not require Oracle Instant Client.

### *Build prerequisite*

The installer build machine needs JDK 17 or newer because `desktop:installer` compiles the bridge and creates the bundled Java runtime. End-user machines do not need Java installed.

Build the bridge alone with:

```
powershell -ExecutionPolicy Bypass -File scripts\build-java-bridge.ps1
```

The connection-test response reports `loadedJars`, making it possible to verify exactly which vendor files were loaded.

Reference source: docs/VENDOR_DRIVERS.md

# MINA environment variables

Deployment

## Guide

### *MINA environment variables*

This is the reference list of environment variables recognized by the current MINA source, Studio sidecar, and Control Plane. Variable names are case-sensitive on Linux and Kubernetes. Values containing passwords, private keys, API keys, or tokens must be supplied by the operating system, service manager, or cloud secret store; do not commit them to a project or container image.

### *Runtime and Studio sidecar*

| Variable | Default | Scope and purpose |
|---|---|---|
| `MINA_PORT` | `8787` | Backend HTTP port used by `run_sidecar.py`. Studio normally assigns this automatically. |
| `MINA_LOG_LEVEL` | `info` | Uvicorn process log level, for example `debug`, `info`, `warning`, or `error`. |
| `MINA_DATA_DIR` | `backend/data` | Runtime data root for projects, SAP TID files, and default project logs. |
| `MINA_RUNTIME_LOG_DIR` | `<data-dir>/logs` | Process-wide fallback log root when `runtime.logDirectory` is blank. |
| `MINA_PROJECT_LOG_MAX_BYTES` | `10485760` | Maximum size of each project log file before rotation. |
| `MINA_PROJECT_LOG_BACKUP_COUNT` | `4` | Number of rotated project log files retained. |
| `MINA_LOG_TIMEZONE` | `America/Phoenix` | IANA display timezone for runtime, debugger, and audit timestamps. Arizona is emitted with `-07:00`. |
| `MINA_WORKER_THREADS` | `auto` | Runtime worker-thread setting; otherwise environment property `advanced.workerThreads` or `advanced.threadCount` is used. |
| `MINA_APPLICATION_DIR` | unset | Application directory supplied to a deployed runtime by the Control Plane. |
| `MINA_ENVIRONMENT` | `local` or deployment value | Active runtime environment. |
| `MINA_DEPLOYMENT_ID` | unset | Deployment identifier supplied by the Control Plane. |
| `MINA_INSTANCE_ID` | unset | Runtime instance identifier supplied by the Control Plane. |
| `MINA_BUILD_VERSION` | `source` | Build/version value included in startup diagnostics. |
| `MINA_STATIC_DIR` | bundled static directory | Override location of backend-served frontend assets. |
| `MINA_DRIVER_HOME` | platform search locations | SAP JCo/native driver directory override. Must contain matching `sapjco3.jar` and native library. |
| `MINA_JAVA` | automatic Java lookup | Java executable override for the SAP/JMS Java bridge. |
| `MINA_JAVA_BRIDGE_HOME` | repository/bundled bridge location | Java bridge installation directory override. |
| `MINA_AI_MODEL` | `gpt-5` | AI Builder model name when `OPENAI_API_KEY` is configured. |
| `OPENAI_API_KEY` | unset | Enables the OpenAI AI Builder provider. Store as a secret. |

The Electron Studio sets `MINA_PORT`, `MINA_DATA_DIR`, `MINA_LOG_LEVEL`, and `MINA_BUILD_VERSION` for its child runtime. It also supports these Studio-only process variables:

| Variable | Purpose |
| --- | --- |
| `MINA_PYTHON` | Python executable override for development sidecar startup. |
| `MINA_DEV_URL` | Frontend development URL loaded by Electron instead of the local production URL. |

### *Control Plane*

| Variable | Default | Scope and purpose |
| --- | --- | --- |
| `MINA_ADMIN_HOST` | `0.0.0.0` | Control Plane HTTP bind address. |
| `MINA_ADMIN_PORT` | `9080` | Control Plane HTTP port. |
| `MINA_ADMIN_DATA_DIR` | `administrator/data` | Control Plane state root: packages, deployments, machines, secrets, audit, and logs. Mount this directory persistently. |
| `MINA_ADMIN_API_KEY` | unset | Technology Team API key. When set, protected APIs require `X-Admin-Key` or `X-Control-Plane-Key`. Store in a secret manager. |
| `MINA_ADMIN_SECRET_KEY` | generated local key | Stable encryption key for stored deployment secrets. Required from a durable secret store for production and clustered deployments. |
| `MINA_ADMIN_RUNTIME_COMMAND` | unset | Trusted command template used to start an on-premises runtime. Supports `{application}`, `{package}`, `{environment}`, `{deployment_id}`, and `{instance_id}`. |
| `MINA_ADMIN_MAX_PACKAGE_MB` | `250` | Maximum uploaded package size in megabytes. |
| `MINA_ADMIN_MAX_EXPANDED_MB` | `1024` | Maximum expanded package size in megabytes. |
| `MINA_ADMIN_MAX_PACKAGE_FILES` | `10000` | Maximum archive member count. |
| `MINA_ADMIN_VERSION` | packaged value or `development` | Control Plane version override. |
| `MINA_ADMIN_WEB` | bundled `web` directory | Override location of Control Plane frontend files. |

The Linux service wrapper also recognizes:

| Variable | Default | Purpose |
| --- | --- | --- |
| `MINA_ADMIN_HOME` | `/opt/mina/control-plane` | Installed Control Plane directory. |
| `MINA_ADMIN_PID_DIR` | `/opt/mina/run` | PID file directory used by `bin/minaadmin`. |
| `MINA_ADMIN_LOG_DIR` | `/opt/mina/logs/control-plane` | Service-wrapper stdout/stderr log directory. |

### *Build and packaging variables*

| Variable | Purpose |
| --- | --- |
| `MINA_VERSION` | Default version passed to Studio or Control Plane build scripts when no explicit version argument is supplied. |

| Variable | Purpose |
| --- | --- |
| `WIN_CSC_LINK` / `WIN_CSC_KEY_PASSWORD` | Windows Electron signing certificate and password. Keep both outside source control. |
| `CSC_LINK` / `CSC_KEY_PASSWORD` | Electron Builder fallback names for Windows signing. |

`MINA_VERSION` and signing variables are build-time settings; they are not needed on an installed runtime machine.

### *Automatically injected deployment variables*

When the Control Plane starts an on-premises deployment, it injects `MINA_APPLICATION_DIR`, `MINA_ENVIRONMENT`, `MINA_DEPLOYMENT_ID`, and `MINA_INSTANCE_ID`. It also injects the deployment's decrypted secret values using their configured names. These values should not be manually duplicated in a package or checked into source control.

### *Windows*

For the current PowerShell session:

```
$env:MINA_DATA_DIR = 'C:\ProgramData\MINA\runtime-data'
$env:MINA_RUNTIME_LOG_DIR = 'D:\IntegrationLogs'
$env:MINA_LOG_TIMEZONE = 'America/Phoenix'
$env:MINA_DRIVER_HOME = 'C:\SAP\jco'
python -m uvicorn app.main:app --host 127.0.0.1 --port 8787
```

For a persistent machine-level setting, run PowerShell as Administrator and use `[Environment]::SetEnvironmentVariable('NAME', 'VALUE', 'Machine')`. Restart the service or application after changing environment variables.

### *Linux and systemd*

Create a root-readable environment file such as `/etc/mina/runtime.env`:

```
MINA_DATA_DIR=/opt/mina/runtime/data
MINA_RUNTIME_LOG_DIR=/opt/mina/logs/runtime
MINA_LOG_TIMEZONE=America/Phoenix
MINA_DRIVER_HOME=/opt/mina/drivers
MINA_ENVIRONMENT=dev
```

Reference it from the service unit:

```
[Service]
EnvironmentFile=/etc/mina/runtime.env
```

Protect files containing secrets with `chmod 600` and set the service account as owner. Run `systemctl daemon-reload` and restart the service after changes.

### *Docker and Kubernetes*

Use ordinary configuration values for paths and ports, and use Docker/Kubernetes Secrets for credentials:

```
env:
 - name: MINA_ENVIRONMENT
 value: dev
 - name: MINA_LOG_TIMEZONE
 value: America/Phoenix
 - name: MINA_DATA_DIR
 value: /var/lib/mina/runtime
 - name: MINA_ADMIN_API_KEY
 valueFrom:
 secretKeyRef:
 name: mina-admin-secrets
 key: api-key
 - name: MINA_ADMIN_SECRET_KEY
 valueFrom:
 secretKeyRef:
```

```
name: mina-admin-secrets
key: encryption-key
```

Persist `MINA_DATA_DIR` and `MINA_ADMIN_DATA_DIR` on volumes. For SAP JCo, provide the licensed matching JCo JAR/native library through a protected volume or image layer and set `MINA_DRIVER_HOME` to that directory. Do not print secret environment values in startup diagnostics.

### *Operational notes*

- Environment variables are read during process startup; restart the affected

process after changing them.

- Relative paths are resolved from the process/application data location and

should be replaced with absolute paths in production.

- `MINA_LOG_TIMEZONE` changes display timestamps only. Scheduler and token

expiry calculations continue to use UTC internally.

- Existing log files are not rewritten when the timezone changes.

- To inspect effective non-secret settings on Windows use `Get-ChildItem Env:`;

on Linux use `env` or `systemctl show <service> --property=Environment`.

Reference source: docs/ENVIRONMENT_VARIABLES.md

# SAP ECC integration runtime guide

Connectors

## Guide

### *SAP ECC integration runtime guide*

MINA uses the licensed SAP Java Connector (JCo) supplied by the customer. The connector is not redistributed with Studio. This implementation interoperates with SAP ECC through public RFC/JCo contracts. MINA is an independent product and does not include third-party proprietary connector implementations.

MINA itself is not SAP-certified merely because it uses JCo. SAP permits JCo to connect external applications to SAP in both directions, but the customer must download it from SAP, comply with named-user/solution licensing, and must not redistribute the connector. Record the JCo, Java, OS, and CPU architecture reported by Test Connection as part of every production approval.

### *Supported integration patterns*

| Pattern | Runtime behavior |
|---|---|
| IDoc Listener | Persistent registered JCo server, tRFC TID duplicate protection, bounded delivery queue, one workflow per IDoc in a package, enclosing transaction commit only after every IDoc succeeds |
| RFC/BAPI Listener | Persistent registered JCo server for the selected function, request/reply export and table response mapping, rollback when the task fails or omits Reply |
| Invoke RFC/BAPI | Persistent, bounded JVM worker pool; JCo destination pooling; import, changing, nested structure, and table parameters; request/reply, tRFC, and qRFC transport |
| Post IDoc / IDoc Reader posting | XML or rendered input is converted to EDI_DC40 and EDI_DD40 and sent with `IDOC_INBOUND_ASYNCHRONOUS`; tRFC or qRFC is selected at transport level |
| IDoc Parser / Converter / Renderer | Metadata-driven, byte-position fixed-width conversion with physical E2 to logical E1 normalization, configurable code page, strict/replace invalid-byte policy, and delimiter-safe flat records |
| Read Table | `RFC_READ_TABLE` field, filter, paging, and named-record output |
| Dynamic Connection | A dedicated persistent JVM and thread-bound JCo context across activities; explicit commit/end, rollback on failure/expiry, bounded session count, and duplicate-session rejection |
| IDoc acknowledgment / confirmation | Invokes the customer-configured SAP acknowledgment RFC because this function is solution-specific |

ALE distribution is implemented through the IDoc listener/posting paths. SAP partner profiles, ports, process codes, message types, RFC destinations, and authorizations remain SAP Basis configuration and are not generated by Studio.

### *Required runtime files*

Install a matching x64 JCo distribution in one of these locations:

```
C:\ProgramData\MINA Studio\drivers\sap\sapjco3.jar
C:\ProgramData\MINA Studio\drivers\sap\sapjco3.dll
```

On Linux use `libsapjco3.so` next to `sapjco3.jar`, or select an explicit SAP driver directory in the shared connection. The packaged Java runtime and JCo native library must have the same architecture. Do not mix files from different JCo releases.

Use a currently supported SAP JCo 3.1 patch and a JDK/JRE listed for that patch in SAP Note 2786882. As of September 2026 SAP publishes JCo 3.1.14 and lists Java 8, 11, 17, 21, and 25; re-check SAP's support page when qualifying a build.

### *ECC configuration checklist*

- Create the RFC user with only the required S_RFC and IDoc authorizations.

- Configure SM59 for the Studio Program ID, gateway host, and gateway service.

- Configure WE21, WE20, message types, process codes, and partner profiles for

ALE/IDoc traffic.

- For qRFC, configure and monitor the queue in SMQ1/SMQ2 and set a non-empty

SAP queue name in the activity.

- Make each concurrently deployed registered server identity unambiguous.

- Keep the TID store on durable writable storage. Never share one TID file

between unrelated Program IDs.

- Validate `STFC_CONNECTION`, fetch the actual IDoc/RFC metadata, and save the

selected metadata with the project before packaging.

### *Resource and failure behavior*

- Listener event queues and stderr history are bounded. Backpressure reaches

the JCo callback instead of permitting unbounded Python heap growth.

- Cancellation does not leave blocked Python executor threads. Runtime shutdown

closes native listener processes and removes their temporary descriptors.

- The JCo server waits for the workflow result. Success commits the delivery;

task failure, cancellation, missing RFC Reply, or reply timeout rolls it back so SAP can retry according to its transaction policy.

- TID state is persisted atomically. A previously committed or in-flight TID is

rejected as a duplicate; SAP confirmation removes it.

- Single-call BAPI auto-commit executes `BAPI_TRANSACTION_COMMIT` in the same

JCo context as the BAPI call.

- A Dynamic Connection with `transactional=true` pins all calls carrying its

`sessionID` to one worker/JCo context. `contextEnd=true` closes the context; with `autoCommit=true` it first executes `BAPI_TRANSACTION_COMMIT`. Failure, expiry, or normal termination without `commitOnTerminate=true` rolls back.

- External outbound calls use persistent JVM workers. `outboundWorkerCount`

controls Python-side worker concurrency; `poolCapacity`, `peakLimit`, and `connectionExpirationMilliseconds` configure the JCo destination pool in each worker. Changing any connection setting, including credentials, creates a distinct pool identity without storing plaintext credentials in its key.

### *Flat IDoc and messaging modes*

In JSON mode, the IDoc Parser and Converter expose one schema-shaped document under `SAPIDoc`, for example `SAPIDoc.ARTMAS05.IDOC`. Map `SAPIDoc` to a Kafka JSON payload to send that document once. Older direct runtime mapping paths such as `SAPIDoc.IDOC` and `ARTMAS05.IDOC` remain available as lookup aliases; they are not additional serialized JSON branches. Actual repeating IDocs and segments remain in the payload.

The built-in parser reads canonical EDI_DD40 byte fields (`SEGNAM`, `MANDT`, `DOCNUM`, `SEGNUM`, `PSGNUM`, `HLEVEL`, and `SDATA`) and EDI_DC40 control fields. Set `idocEncoding` to the actual sender code page. Keep `invalidCharacterPolicy=strict` for production unless replacement characters are an accepted business rule. When SDATA may contain CR/LF, configure a unique `idocRecordDelimiter`; that delimiter is authoritative and embedded line breaks remain data. With no delimiter, canonical 1063-byte records are split by width.

If the separately licensed `sapidoc3.jar` is installed, Test Connection reports its presence and version. Selecting the SAP JIDocLib parser engine validates IDoc XML with SAP's repository-backed `IDocXMLProcessor`; flat EDI_DD40 input continues to use the built-in byte parser because SAP documents the public IDoc class-library processor as an XML processor.

IDoc Listener supports these sources:

- `NoMessaging`: the direct persistent JCo RFC server, including durable TID

handling and SAP transaction acknowledgment.

- `EMS` or `JMS`: consumes one queue message through the selected shared

connection and publishes its body to the parser. Client acknowledgment is exposed through the standard Confirm Message activity.

- `Kafka`: consumes one topic record through the selected Kafka resource and

publishes its value to the parser. Manual offset confirmation uses the same Confirm Message contract.

For broker-backed modes, select `messagingResourceId` and `messagingDestination`. SAP-to-broker transaction atomicity is owned by the upstream bridge; MINA does not claim an XA transaction spanning SAP tRFC and an independent EMS/JMS/Kafka broker.

### *Production certification*

Unit and mock tests verify serialization, lifecycle, transaction routing, and bounded queues, but they cannot certify a particular ECC/JCo combination. Before production, run an environment test with the exact JCo version and architecture:

- sustained IDoc batches containing the largest expected SDATA payloads;

- concurrent RFC/BAPI calls at the configured maximum connection count;

- SAP and Studio restart during in-flight tRFC/qRFC deliveries;

- duplicate TID, reply timeout, malformed IDoc, network partition, and expired

credential scenarios;

- a 24-72 hour soak while recording process RSS, JVM heap/GC, thread count,

handle count, queue depth, throughput, and SAP SM58/SMQ1/SMQ2 state.

Do not accept the connector for production until those measurements remain stable and every failed transaction is demonstrably recoverable.

The repository includes a repeatable qualification runner and an intentionally non-secret example configuration:

```
Copy-Item docs\sap-soak-config.example.json sap-soak-config.local.json
# Edit the local copy with QA-only credentials and parameters, then run:
backend\.venv\Scripts\python.exe scripts\sap-production-soak.py `
 --config sap-soak-config.local.json --output sap-soak-report.json
```

Set `terminateWorkerEveryCalls` to a positive number for controlled worker-JVM failure injection. Run separate 24-72 hour profiles for sRFC, transactional BAPI, tRFC/qRFC IDoc posting, and inbound listeners. The generated report holds throughput, failures, latency percentiles, and worker process IDs; correlate those IDs with OS/JVM monitoring and SAP SM58/SMQ1/SMQ2 evidence. Never commit the local configuration because it contains credentials.

Reference source: docs/SAP_INTEGRATION.md

# EMS/JMS sender reuse in Run mode

Connectors

## Guide

### *EMS/JMS sender reuse in Run mode*

Run previously included the unique job execution ID in the persistent sender cache key. Each incoming event therefore opened another Java worker/EMS connection. After eight cached configurations, additional jobs fell back to one-shot JVMs. Debug used a stable session ID and did not normally encounter this problem.

The runtime now gives EMS/JMS an application-scoped sender identity, separate from the job ID. Jobs keep unique IDs for logging and tracing. Subtasks and parallel branches inherit their caller's sender scope; Debug retains its own session scope. Different applications and runtime instances remain isolated. Connection settings still participate in the cache key, so changed credentials/settings are not ignored.

Studio project Stop cancels execution and force-closes that application's senders. Restart can establish a fresh sender. Idle eviction, bounded caching, serialized JMS session access, and the existing no-replay-on-uncertain-send behavior remain.

The fix applies to Windows and Linux execution through WorkflowRuntime, including new engine-based exports. Rebuild/reinstall Studio and regenerate/redeploy engine archives to replace older bundled code. Direct Python connector exports use a different connection path and are not modified by this fix.

Verification uses the actual Java bridge and a local test JMS provider: twelve separate Run jobs retain one sender, application Stop is isolated, restart works, and provider failures are not automatically replayed. Debug, scheduling, runtime, and engine-export regression suites also pass. Real EMS throughput still requires measurement against the target broker; no fixed delivery latency is guaranteed.

For diagnosis inspect `senderReused`, `providerSendMs`, and `publishTiming` (`queueMs`, `bridgeMs`, `totalMs`) in send output. A healthy warm send should reuse its sender. Upstream event rate, broker persistence, network latency, and payload logging can still constrain end-to-end application throughput.

Reference source: docs/EMS_RUN_CONNECTION_REUSE.md

# Kafka publishing performance

Connectors

## Guide

### *Kafka publishing performance*

An explicit Input mapping replaces the selected message field, including an object-valued `${last}` default. It does not merge earlier EMS, Pub/Sub, or Kafka publish receipts into that message. Map a parser's document branch (for example `${IDoc-Parser.SAPIDoc.ARTMAS05}`) to send only the IDOC. Kafka's String serializer encodes mapped objects and arrays as JSON text; existing strings and bytes retain their content. The JSON serializer also encodes objects as JSON.

Studio and engine-backed exports reuse Kafka producers and run publishing on a dedicated four-thread executor. Long-running receivers, SAP/JCo calls, and other default-executor work therefore do not hold up Kafka enqueue operations. Producer cleanup closes this executor with the runtime. Direct Python exports reuse an asynchronous producer per connection/configuration and event loop.

Publish output includes `publishLatencyMs` plus:

- `publisherQueueMs` (Studio/engine): time waiting for a Kafka worker thread.

- `producerSetupMs`: producer lookup/creation time; direct Python includes initial broker connection setup and waiting for the creation lock.

- `sendAndWaitMs`: serialization/enqueue/delivery work after producer lookup; this is not a pure network-latency measurement.

- `producerReused` (direct Python): whether the existing asynchronous producer was reused.

With `waitForDelivery` enabled, completion includes broker acknowledgement. Kafka transactions also wait for delivery and commit in the Studio engine. With it disabled, `queued: true` and `deliveryConfirmed: false` indicate local queue acceptance, not confirmed broker delivery. This change does not alter acknowledgement settings or transaction semantics.

For remaining delays, compare the first publish with subsequent publishes using the same connection. High setup time on the first direct Python publish can involve metadata, DNS, TLS, or authentication. High send/wait time can include configured linger, queue backpressure, broker replication, retries, and network latency. Capture these timing fields alongside the effective producer settings and broker metrics before changing reliability options. Rebuild installed Studio and regenerate exported Python archives to use the updated implementation.

The automated tests cover thread-pool saturation, producer reuse, queued and acknowledged sends, and transaction initialization reuse. They do not establish throughput or latency against a real Kafka broker.

Reference source: docs/KAFKA_PUBLISHING.md

# REST APIs and shared HTTP connections

Connectors

## Guide

### *REST APIs and shared HTTP connections*

Use HTTP Client Connection for HTTP Request / REST Invoke and HTTP Server Connection for HTTP Listener / REST Receiver. These presets create reusable HTTP resources with explicit client/server roles. Existing HTTP resources continue to work; choose a role when editing them. A server resource cannot be used for an outbound request, and a client resource cannot host a listener.

### *MINA API Blueprint: load YAML and host an API*

Open API Blueprint on the Studio ribbon. Load a `.raml` RAML 1.0 API, or a `.yaml`, `.yml` or `.json` OpenAPI contract, or paste its contents. Load example supplies a small two-operation API. Choose Validate YAML to check syntax, contract structure, unique routes/operation IDs, path parameters, schemas and any declared response examples. Missing operation IDs receive stable generated IDs. YAML duplicate keys and aliases are rejected; ordinary YAML strings such as `on`/`off` remain strings. Numeric response keys are normalized to status-code strings.

For a multi-file contract, choose Load API folder, then select Entry API document. The folder's relative paths are preserved, so references such as `models/order.yaml` and local JSON Pointers can be bundled without fetching URLs or reading files outside the selected folder. Import accepts up to 16 supplied source files, each at most 8 MiB, with a 32 MiB upload total and an 8 MiB bundled-contract limit. RAML folders can include RAML/YAML/JSON, XML/XSD examples, text and Markdown. Change remote references to matching local relative paths before importing; the wizard does not download URLs. Switching entry documents invalidates the previous validation and operation selection while retaining edits to previously selected files.

Validation checks every declared request/response media schema, response codes, parameter locations and duplicates, route placeholders, callback structure, and mock examples against their corresponding media schemas. Named/referenced examples, explicit JSON null examples, and schema-level examples are supported. Example response headers use declared examples or schema defaults. Declared 204/205 responses can be mocked without a body. Generated names remain unique even when routes differ only in case or callbacks share a display name. Contracts are bounded to 256 paths, 512 operations, and 1024 generated operation/callback tasks.

Generated mock responses preserve JSON strings and null as JSON values in Studio, Run, Debug, and exported Python applications. Hyphenated URI parameter names, such as `{order-id}`, are routed correctly. Hosted Swagger resolves named examples from imported documents and retains response header declarations; Postman exports preserve explicit null request examples.

Use Find operations, Select all, and Clear selection for larger contracts. Filtering does not deselect hidden operations. Each operation indicates its available mock status, or that it will return 501 until implemented. Creating an API remains separate from implementing its business logic; loading a contract never generates that logic automatically.

Select the operations to implement, then choose a saved HTTP server connection or enter a new bind host, port and protocol. HTTPS hosting requires the server's own certificate/private key or a saved connection with an identity store. The selected connection settings are copied into a dedicated API server resource, preserving property bindings and authentication settings. The contract is stored once on that resource and remains editable there.

Choose initial response behavior explicitly. 501 placeholders is the default: importing an interface does not implement business logic. Declared response examples hosts schema-validated mock examples for testing; operations without examples still return 501. Each selected operation becomes an editable Starter Task containing REST Receiver -> HTTP Response. Replace the placeholder/example response with your integration logic and enable response validation when implementing the operation.

Select API package to choose an existing task package. New package creates a root folder; New subpackage creates a child of the selected package. Names use the same validation as Project Explorer, and spaces become hyphens. Generated operation tasks, contract callback helpers when present, and the API-specific server connection are saved in the selected package. New folders are only saved when the API is created; closing the wizard leaves the project unchanged.

Create & host API saves and starts all selected routes together, showing their actual endpoint URLs. There is no separate Create API tasks button in this wizard. If hosting fails, the saved tasks remain available and Host API retries without duplicating them. Run on any task in that Blueprint hosts its sibling API tasks; Stop releases the listener ports. Unrelated starter tasks do not join the Blueprint. Both Python archive formats host the same generated routes when running the application.

The wizard imports RAML 1.0, OpenAPI 3 and Swagger 2 contracts. The activity contract editor separately supports explicit external-reference configuration. Create new API creates a separate design; Update existing API changes a selected Blueprint in place through review. When retaining multiple API versions, give them distinct routes/base paths or ports so an application-wide Python deployment can host them together. The popup stays within the viewport and scrolls its content independently of its action controls.

### *RAML traits, libraries and resource types*

Choose the API root beginning with `#%RAML 1.0`, rather than a Library/Trait/ResourceType fragment. Load API folder supplies local `!include` documents and `uses` libraries, including nested namespaces. The importer expands parameterized traits (`is`) and resource types (`type`), inheritance, optional methods such as `post?`, reserved `resourcePath`, `resourcePathName` and `methodName` parameters, and standard naming transformations such as `!singularize`, `!pluralize` and case conversions. Explicit declarations override inherited values; inherited collections are merged. An optional method is expanded only when that method exists on the resource or through nonoptional inheritance.

Supported types include objects, arrays, inherited/namespaced types, unions, nullable values, primitive constraints, date/time formats, XML metadata and included JSON schemas. Nested routes, URI/query/header parameters, media types, responses, examples, Basic/OAuth2 security and single-credential Pass Through security convert into the shared runtime contract. Explicit `protocols: [HTTPS]` requires a TLS connection. Configure the actual credentials and TLS identity on the server shared connection; importing security declarations does not generate secrets.

The saved resource contains normalized OpenAPI, so Swagger, Postman, Run, Debug and both Python archive formats use the same expanded contract without a RAML parser at deployment. Keep the original source folder under version control and reload it to update the API; the wizard does not regenerate RAML source from edited OpenAPI. The Studio backend additionally installs the pinned `inflection` dependency from `backend/requirements.txt` for template transformations.

This is a bounded offline RAML importer, not a claim of complete RAML specification certification. Overlays/extensions, whole-object `queryString`, unsupported security mechanisms, custom validation facets and unsupported type facets must be converted before import. Missing files, cyclic expansion, unknown types/templates, YAML aliases/duplicate keys and unsupported declarations produce validation errors. No import reads arbitrary disk paths, fetches URLs or executes custom YAML tags.

### *Safely update an existing API*

- Stop Run/Debug. Open API Blueprint, choose Update existing API and select its API-specific server connection.

- Load the revised contract or source folder, validate it, and select the operations to retain or add. Keep `(operationId)` stable in RAML when renaming a route; otherwise matching falls back to the existing method/path.

- Choose Preview update. The first preview saves pending Studio edits. Inspect added, changed, unchanged and removed operations, conflicts and warnings. Response behavior controls only newly generated operations.

- Acknowledge the review and choose Apply reviewed update. Host API then starts the updated implementation.

Matched tasks retain their IDs, names, packages, activities, transitions, groups, input/output schemas, business logic and mappings. Existing response mappings are preserved rather than replaced with examples. The dedicated connection retains its ID, transport, authentication and property bindings; its normalized contract changes. New operations receive new editable flows. Callback helpers retain stable IDs and custom mappings; obsolete callback helpers are retained with a warning so existing Call Sub Task references remain valid.

Removal of an implemented operation blocks apply instead of silently deleting its flow. Resolve removals explicitly or retain the operation in the contract. Ambiguous matches and object-name collisions also block apply. Schema changes require review and may require manual business-mapping changes; preserving a mapping does not guarantee that it satisfies a changed schema.

Apply is rejected while that project's Run/Debug session is active. A preview token binds the exact contract and selection to the saved project revision. If another editor changes the saved project, apply returns a conflict; Refresh preview reads the latest saved revision without writing an old Studio snapshot over it, and requires review again. A temporary project-directory snapshot restores the prior saved project if saving the update fails. If restoration itself fails, the server reports the retained recovery directory; keep that directory for recovery. This protects ordinary save failures, rather than guaranteeing recovery from every machine/storage failure.

### *Default confidentiality*

On an HTTP client/shared connection, enable Default confidentiality / system trust to require HTTPS with server-certificate and hostname verification. System root trust remains available; a configured trusted-certificate folder augments it. No client identity certificate is needed unless the provider requires mutual TLS. An HTTP URL or disabled certificate verification conflicts with default confidentiality and fails configuration. When no port is configured, HTTPS uses 443; explicit ports are respected. The setting is also available as the boolean environment property `connections.http.defaultConfidentiality`.

This is MINA's Python equivalent of default TLS confidentiality, using the runtime host's system trust rather than a Java `cacerts` file. Certificate-folder trust applies to HTTP Request, REST Invoke and SOAP Request/Reply, in Studio and both Python exports.

### *Design and run a REST API*

### *HTTP Send Response input headers*

Every HTTP Send Response activity exposes these optional fields by default in Input, for both HTTP Listener and REST Receiver flows:

- HTTPHeaders -> Content-Type, complex StatusLine, and Cookies. Content-Type accepts common media types or a custom value. Cookies accepts one complete Set-Cookie string or an array of strings; map an array in Studio, for example `["session=abc; Path=/; HttpOnly", "locale=en; Path=/"]`. Each cookie is emitted separately.

- DynamicHTTPHeaders -> Headers* -> headerName, headerValue. Choose Add header for independent entries, duplicate the repeating Headers structure, map a collection, or use For Each. Each populated entry needs a name and a value; an empty string is a valid value. Completely empty entries are ignored. Header names are case-insensitive. Repeated names produce separate header lines; Content-Type must have a single value.

Legacy `headers`, `contentType` and `statusCode` mappings remain supported. HTTPHeaders overrides matching legacy settings. DynamicHTTPHeaders overrides matching standard/legacy headers, except Set-Cookie values append to Cookies. Unmapped optional fields keep the existing defaults. Response media types must still satisfy the endpoint contract and the caller's Accept header. Header names and values are validated; newline/control-character injection and malformed status lines fail the activity. The transport retains control of Content-Length, Transfer-Encoding and Connection. These mappings run identically in Studio Run/Debug and Direct/Engine Python archives.

Expand HTTPHeaders -> StatusLine in Input to map each optional child independently:

| Field | Type | Example Input expression | Behavior |
|---|---|---|---|
| `httpVersion` | string | `"HTTP/1.0"` or `${input.version}` | Selects the actual response version on a native HTTP/1 listener (1.0 or 1.1). On HTTP/2 or HTTP/3 it must match the negotiated protocol. The `HTTP/` prefix is optional. |
| `statusCode` | integer | `201` or `${Receive.query.status}` | Overrides the legacy statusCode when supplied. Leave unmapped to retain the legacy value or default 200. Numeric literals have no quotes. |
| `reasonPhrase` | string | `"Stored by integration"` or `${input.description}` | Sends a custom reason phrase on native HTTP/1 listeners, including an explicitly empty |

| Field | Type | Example Input expression | Behavior |
| --- | --- | --- | --- |
| | | | string. Omit to use the standardstatus description. Newlines,control characters andnon-Latin-1 text are rejected. |

Map the whole StatusLine object when an upstream activity already supplies these fields. An empty object or omitted children keeps the existing numeric status. Legacy full status-line string mappings, such as `"HTTP/1.0 201 Stored by integration"`, also apply their version and reason rather than discarding them. Mapped settings remain visible in Output -> statusLine and execution/debug data. Hosted final responses require status codes 200-599; 1xx informational responses are not sent by this activity.

`statusCode` controls the actual response status. Native HTTP/1 listeners now apply the mapped version and reason to the wire response. For example, version `"1.0"`, status `201` and reason `"Stored by integration"` send `HTTP/1.0 201 Stored by integration`. This works over HTTP and HTTPS. HTTP/1.0 responses use length-delimited bodies and `Connection: close`; header-only responses retain their normal bodyless semantics. File responses retain their known content length. Each connection/request has independent settings, so custom phrases cannot leak to another response or modify global status tables.

| Listener/connection | Version override | Reason override |
| --- | --- | --- |
| Native HTTP/1 listener (server connection version 1.1), HTTP or HTTPS | HTTP/1.0 or HTTP/1.1 | Custom Latin-1 text or explicitly empty string |
| Negotiated HTTP/2 | Must be HTTP/2 (2.0 alias accepted) | Omit or use empty string; nonempty phrases are rejected |
| Negotiated HTTP/3 | Must be HTTP/3 (3.0 alias accepted) | Omit or use empty string; nonempty phrases are rejected |
| HTTP/1 fallback on a connection configured for HTTP/2 or HTTP/3, or an HTTP/1 ASGI preview/host without MINA's extension | Matching HTTP/1.1 only | Unsupported; omit the field |

HTTP/2 and HTTP/3 carry a numeric `:status`, not a textual reason phrase ([HTTP/2](https://www.rfc-editor.org/rfc/rfc9113.html#section-8.3.2), [HTTP/3](https://www.rfc-editor.org/rfc/rfc9114.html#section-4.3.2)). A response mapping cannot upgrade or downgrade an established HTTP/2/3 connection: configure the server shared connection and client for that protocol instead. Unsupported settings produce an HTTP_RESPONSE 500 before response headers are committed, and the server log records the rejected setting. The workflow may already have run before its response is checked. Protocol-error responses use the transport's normal status line. Studio Run, Debug and both Python archive formats share these rules; ASGI-based previews cannot claim native status-line support unless their host advertises it.

### *Request and listener setup*

For HTTP Request and REST Invoke, map the contract, operation ID, reference documents/origins and request/response validation settings under Input -> RestInputRequest.Config. Existing configuration values are shown as typed mappings. Configuration keeps the display name and shared connection selector. JSON/XSD validation and contract selection can use properties or upstream values; XSD validation uses offline lxml validation with self-contained schemas. Generated archives include the required dependencies. Listener routing stays in startup configuration.

Create a Starter Task and choose HTTP Listener or REST API Receiver. In endpoint setup, select the HTTP server shared connection, methods, route, supported request and response content types, and any request or response schema. Use Create new connection in the shared connection picker if needed. Studio creates the listener or receiver and an HTTP Send Response activity. Schema controls display the selected schema name. Selecting a project schema copies its current definition, so reselect it after editing that definition when the endpoint needs the update.

The receiver starts a task when its configured method and path match a request. Run starts a real listener on the shared connection's host and port, and Stop releases the port. Multiple exported receiver tasks can share a port when their transport

settings agree and their routes do not conflict.

Use HTTP Response on the same workflow path to reply to a REST Receiver's caller. Studio may name the generated node HTTP-Send-Response. Set the status code and map the response body and any headers there.

Select every method you want the receiver to accept. In Request parameters, use URI path to type a placeholder already in the route, such as `{id}` in `/orders/{id}`. Add Query entries for values such as `?details=true` and Custom header entries for values such as `X-Tenant`. Set the type and mark any required entries. The receiver returns HTTP 400 if a required value is missing or a declared value has the wrong type. The workflow receives the values in `pathParameters`, `query`, and `headers`, with header names normalized to lowercase. OpenAPI and Postman include these declarations.

Run and Debug show the live listener address. If the connection has a public API base URL, Studio shows that separately as the deployment address. Selecting HTTP on the shared server connection disables TLS and switches HTTP/2 back to HTTP/1.1.

REST Invoke and HTTP Request support URI parameters, default and activity headers/query parameters, JSON, text/XML, form, binary and file bodies, buffered/chunked transfer, response status validation, and optional request/response schema validation. XML requires a text body and the appropriate media type. REST invocation returns `statusCode`, `headers`, `body`, `elapsedMs`, and `httpVersion`; Untouched legacy HTTP Request activities retain their payload response; new requests and requests edited through the Input tree return the response envelope.

Optionally paste an OpenAPI 3 or Swagger 2 JSON/YAML contract and select its exact operation ID. The operation defines the HTTP method/path, required parameters, media types and request/response schemas. Invocation uses the connection's base URL when supplied, otherwise the contract server URL. A receiver combines its base path with the operation path; without a configured base path it uses the contract server's path. Body validation defaults to enabled. Parameter validation includes required values, arrays, objects, cookies, numeric/string constraints and formats. Operation-level parameters override path-level definitions. Response schemas select exact status, status class (for example `4XX`), then `default`, including error responses before status faults are raised.

Local references work offline. Supply external documents in Supplied external reference documents as a JSON object mapping document URI to document object or YAML text. For network references, configure Contract base URL for relative paths and explicitly list exact HTTPS origins in Allowed external reference HTTPS origins. No origins means no remote retrieval; redirects, URL credentials and implicit file access are forbidden. Retrieval uses the shared client's TLS trust, with document/count/size bounds; schema validation itself never fetches URLs. YAML aliases are rejected. The contract limit is 8 MiB, with at most 16 referenced documents and 32 MiB downloaded references. Mapping expressions do not execute YAML code. MIME bodies continue to use the explicit Input tree; OpenAPI does not automatically manufacture MIME mapping rules or perform browser authorization/consent.

The Studio preview URL uses the same validation and authentication as the real listener. It cannot establish mutual TLS for an individual project; use the configured HTTPS endpoint to test client certificates. Preview signatures use the project-relative listener path. The real endpoint's HTTP/TLS transport settings apply when Run hosts the dedicated listener or an exported application runs.

### *Request and response trees*

Configuration holds the shared HTTP client connection and optional API contract and schema validation settings. Request values belong in Input, under `RestInputRequest`:

- `Config`: host, port, scheme, requestURI, Method, RequestBody, bodyFormat, query and URI parameters, QueryString, FilePath, timeout in milliseconds, and request policy overrides. Unmapped connection settings supply defaults.

- `Headers`: accept, content-type, Accept-Charset, Accept-Encoding, Cookie and pragma.

- `CustomHeaders`: choose Add header to create a row with separate name and value mappings. Add or remove rows independently, or map a source collection with For Each. Repeated custom headers retain all values. Duplicate entries for a nonrepeating standard header fail validation. Header names and values must be ASCII and cannot contain line breaks.

- `mimeEnvelopeElement`: repeating `mimePart` entries containing `mimeHeaders` and exactly one of binaryContent (base64), textContent or fileName. MIME headers include content-type, content-transfer-encoding, content-id and content-disposition. File names refer to files available on the runtime host. Multipart messages are buffered by default; `Config.streamRequest` enables bounded streaming within the configured size limit.

Legacy request configuration values appear as Input mappings when opened; editing saves them into the new structure. String constants require quotes; numbers and booleans use native literals, while expressions supply runtime values. Output exposes `RestOutputResponse` with statusLine (httpVersion, statusCode, reasonPhrase), body, asciiContent, binaryContent, Headers, repeating DynamicHeaders and parsed MIME parts. The runtime also retains the envelope's statusCode, headers, body, elapsedMs and httpVersion fields. Input and Output trees show optional and repeating cardinalities.

The request input uses the `RestInputRequest` root. Studio, Direct Python and Engine Python exports use the same request and MIME implementation.

For a request to `/orders/{id}`, map `Config.URIParameters` to an object such as `{"id":"A/B"}` and `Config.QueryParameters` to an object such as `{"tag":["open","priority"]}`. The slash in the ID is encoded as part of the path value, and both tags are sent. To send `X-Tenant: west`, open CustomHeaders, choose Add header, and map the name to `"X-Tenant"` and the value to `"west"`. A mapped query value overrides the same key from the connection or request URI.

### *Connection configuration*

| Area | Implemented configuration |
|---|---|
| Addressing | Client/server role, protocol, host, port, base URL and base path; activity path and methods |
| Confidentiality | System trust with default confidentiality, custom trust with a configured store, HTTPS enforcement, hostname verification, TLS 1.2/1.3 bounds, TLS 1.2 cipher list |
| Certificates | PEM trust certificates, PEM certificate chain/private key and key password, PKCS12 identity/trust store passwords, JKS store password and identity alias, CRL, optional/required inbound client certificates |
| Client timeouts | Separate connection, socket/read, write and pool wait timeouts in milliseconds; legacy second-based values remain accepted |
| Pooling | Total and per-host limits, persistent connections, idle expiry, disabled pooling, cookie persistence and response buffer size |
| Delivery | Redirect switch, bounded connection retries and retry delay, request/response size limits, expected content/accept types and success status codes |
| Proxy | Proxy URL or host/port and optional credentials |
| Server limits | Minimum/maximum QTP worker settings, concurrency and queue limits, queue timeout, accept backlog, persistent connection idle timeout, body read timeout, header/body size limits and IP/CIDR allowlist |
| Execution | Request execution timeout and contract validation |

Zero client phase timeout means no independent phase deadline, subject to the activity timeout budget. The default activity budget is 180 seconds; synchronous client phase waits and response consumption are bounded, but this is not a hard wall-clock cancellation of a Python thread. Connection failures are retryable; configured status retries additionally require idempotency safeguards. Read failures are not automatically replayed. Host/port settings must refer to an address the runtime can actually bind or reach.

QTP is a Jetty concept. MINA implements its minimum/maximum settings with Python authentication workers, a bounded admission queue and an asynchronous execution gate. It does not embed Jetty or duplicate its scheduling semantics. HTTP/1.1 listeners use Uvicorn. Selecting HTTP version `2` on a server connection enables Hypercorn with HTTPS ALPN negotiation for HTTP/2 and HTTP/1.1. HTTP/2 listeners require HTTPS and use the same trust, identity and mutual-TLS configuration. TLS 1.3 cipher selection follows Python/OpenSSL defaults. Certificate revocation uses configured CRLs; OCSP fetching and Java security-provider configuration are not implemented.

PEM and PKCS12 work in Python. JKS conversion requires Java `keytool` on PATH or under JAVA_HOME. A selected JKS identity key must use the store password; convert stores with a distinct key password to PKCS12 first. Certificate/key paths are external deployment requirements; MINA does not bundle private keys or certificate stores into source archives.

### *Trust certificates from a property-bound folder*

Set the string property `connections.http.trustedCertificateFolder` to your certificate folder, for example `D:\MINA\certificates\trusted` on Windows or `/etc/mina/certificates/trusted` on Linux. In the HTTP shared connection, bind Trusted root / chain certificate folder to `${properties.connections.http.trustedCertificateFolder}` using the property picker. Select that HTTP connection for HTTP Request, REST Invoke or SOAP Request/Reply. Enable HTTPS and keep certificate verification enabled.

Place trusted root certificates directly in the folder. Intermediate chain certificates are optional: normally the HTTPS server supplies them, but you can add them to the same folder if needed. A complete path to a trusted root is still required; an intermediate alone does not bypass root verification. Server/client identity certificates and private keys remain separate settings for hosting HTTPS or mutual TLS.

The folder accepts `.pem`, `.crt`, `.cer` and `.der` files, including PEM bundles and binary DER certificates. It requires no OpenSSL rehash operation. Subfolders are not scanned; unrelated file extensions are ignored. Empty, missing, unreadable or invalid certificate folders produce a configuration error. Keep private keys out of the trust folder. Limits are 256 certificate files, 8 MiB per file and 32 MiB total. Folder trust is added to system trust and any explicitly configured trust store.

New outbound calls detect certificate file additions, removals and file modification times and create a fresh TLS client when needed. Existing requests finish with their original TLS context. Inbound servers load folder trust at startup; restart them after changing client CA certificates. Configure an environment-specific property value and provision the folder on each deployment host. Studio and both Python archive formats resolve the same property and use the same folder-loading implementation; the folder itself remains an external deployment requirement.

### *Authentication*

| Scheme | Client | Server |
|---|---|---|
| Basic | Preemptive or challenge-based username/password | Constant-time credential comparison; empty credentials rejected |
| Digest / NTLM | HTTP challenge authentication | Not implemented |
| Bearer | Supplied access token | Supplied token comparison |
| OAuth2 | Client credentials, supplied authorization code with optional PKCE verifier, refresh token; cached token exchange | HTTPS token introspection; active/expiry checks and required scopes |
| JWT | Supplied JWT, or generate a signed token with claims, issuer, audience and lifetime | Verify configured algorithm/key, expiration, issuer/audience when configured, clock skew |
| HMAC | Sign method, exact request path/query, timestamp, nonce and SHA256 body digest | Signature, timestamp and nonce replay checks |
| LDAP | Not an outbound REST authentication scheme | Basic credentials authenticated by verified LDAPS or StartTLS, service search/user DN template, optional required group |
| Certificate | TLS client identity | Required and verified mutual TLS |

JWT algorithms are explicitly limited to HS256/384/512, RS256/384/512 and ES256/384. Asymmetric signing uses a PEM private key; verification uses a PEM public key. Configure strong shared secrets and the expected issuer/audience for your application.

OAuth token and introspection endpoints require HTTPS. Authorization-code acquisition/consent is performed outside MINA; provide the returned code, redirect URI and verifier when applicable. Token refresh is automatic when the provider supplies a refresh token. LDAP search escapes usernames, rejects anonymous binds and always verifies its TLS connection. LDAP group checking uses `memberOf` and depends on directory support.

HMAC uses MINA's explicit signing format, not AWS SigV4 or a vendor-specific HMAC policy. Send `X-MINA-Key-Id`, `X-MINA-Timestamp`, `X-MINA-Nonce`, and `Authorization: HMAC <base64 signature>`. The signed text is five newline-separated values: uppercase method, exact encoded path/query, Unix timestamp, nonce, and hexadecimal SHA256 body digest. Select SHA256/384/512 for the keyed signature. Signed requests cannot follow redirects. Select policy storage `redis` and bind Redis policy connection URL to a protected property for atomic nonce claims across replicas; `memory` is local to one runtime. Redis failures reject protected requests with 503 rather than silently disabling replay protection. Use `rediss://` for encrypted Redis connections.

### *Browser access, key rotation and throttling*

Configure exact Allowed browser origins on a server connection to enable CORS. Credentialed CORS rejects wildcard origins. Preflight checks the requested method and headers before authentication; actual requests still authenticate normally, and allowed-origin authentication failures include CORS headers. CORS is a browser policy, not an alternative to authentication. Configure required browser headers and exposed response headers explicitly.

For JWT issuer key rotation, set an explicit HTTPS JWT verification JWKS URL, asymmetric signature algorithm, expected issuer and audience. The endpoint comes from configuration, never from token headers. Keys are cached for 300 seconds by default; an unknown `kid` can refresh after a default 10-second cooldown. Discovery is bounded to 1 MiB/128 keys and uses configured TLS trust. Symmetric JWKS algorithms, mismatched algorithms and unsuitable keys are rejected. Missing keys or discovery failures never bypass signature checks.

Set Requests per client/window above zero to enable fixed-window throttling. The key uses the effective client IP and route. Exceeding the limit returns 429 and `Retry-After`. Redis storage shares counts across replicas. Memory storage applies within one runtime. Set Trusted proxy IPs / CIDRs if a trusted proxy supplies client addresses.

### *Retries and circuit breakers*

`retryCount` bounds retries. Retry response status codes is empty by default; explicitly select statuses such as `429,502,503,504` when appropriate. Response retries require an idempotent method, a mapped `Idempotency-Key`, or the explicit non-idempotent retry override. An idempotency header only helps when the destination enforces it. Backoff grows exponentially with optional jitter; `Retry-After` seconds/HTTP dates are honored within the configured maximum delay and remaining activity budget. Read failures are not automatically replayed. All retry attempts share one activity budget.

Set Circuit breaker failures above zero to open a runtime-local circuit after consecutive connectivity/server failures. Circuit recovery delay controls when one half-open probe may run; success closes the circuit and failure reopens it. Configuration/client validation errors do not count as server failures. Circuit state is local, while optional Redis throttling/replay state is shared.

### *Binary bodies and large files*

Listeners preserve binary request bytes and expose base64 `binaryContent`, complete repeated `headerPairs`, and scalar-or-list query/header values. `?tag=a&tag=b` preserves both tags. Ambiguous duplicate Authorization/Host/Content-Type/Content-Length/Origin headers are rejected. Existing routes return 405 with `Allow` for unsupported methods; GET routes support HEAD and listeners answer OPTIONS automatically.

File request bodies (`Config.FilePath`) stream from disk. Set `Config.streamRequest = true` to stream MIME attachments; streaming MIME supports base64/binary/7bit/8bit transfer encoding and bounded nested envelopes. Streamed uploads require authentication that does not internally buffer the body: preemptive Basic, Bearer, OAuth2, JWT, HMAC or certificates. Challenge-authentication modes that require buffering reject streaming requests explicitly.

For downloads, map `Config.responseFile` to a runtime filename. Data streams into a sibling temporary file; successful completion publishes it atomically. Existing destinations are protected unless `Config.overwriteResponseFile = true`. Failures and size-limit errors remove the partial file. Output body contains `fileName` and `size`. Use buffered mode when response-schema validation is required; file mode cannot bypass that validation. Set `Config.responseBodyMode = "binary"` for a buffered byte body; structured output retains base64 `binaryContent`.

For large inbound bodies, select Inbound body storage = file and an existing writable Temporary upload directory on the server connection. The flow receives `fileName`/`size`; consume or move the file during that request. Temporary uploads are deleted on success, failure, cancellation or disconnect. This mode spools the complete wire entity, including a multipart entity, without buffering it or incrementally extracting its parts. Use memory mode for JSON/XSD body validation or automatic MIME-part decoding.

HTTP Response can map either `body`, base64 `binaryContent`, or `fileName` for streamed output. Repeating response-header values such as `Set-Cookie` remain separate headers. Size bounds still apply in all storage modes.

### *Swagger documentation and Postman*

Studio's Samples -> HTTP & REST examples opens twenty editable host/caller projects directly. Each opening creates a new copy. Cards show provider dependencies and setup requirements. The Samples footer downloads the API setup guide and helper kit, including certificate preparation and local OAuth fixtures. For secure examples, prepare the helper kit and set the opened project's local `demo.directory` property to its generated `local` folder. No machine-specific certificate paths or generated private keys are bundled in the gallery.

Hosted HTTP and REST listeners automatically serve MINA API Explorer, with bundled Swagger UI and Try it out, at `http(s)://host:port/_mina/docs`. Run and Debug show the actual listener endpoint, an Open Swagger link and a Download Postman collection button. The download is a Postman v2.1 collection JSON; import it into Postman and supply any credential variables locally. The API Explorer also has a collection download button. No CDN is needed.

The OpenAPI document is at `/_mina/openapi.json` and the collection at `/_mina/postman.json`. These describe the currently hosted routes, including the selected API Blueprint operations, effective base path, parameters, request/response schemas, examples and authentication. Generic HTTP listeners receive generated operation descriptions. Swagger testing invokes the actual integration. In Debug it pauses at configured breakpoints; use Continue/Step in Studio to finish the request. One request can be debugged at a time; additional requests return 503. HTTP activity timeouts still apply; a timed-out debug request stops that debug session so a stale paused job cannot affect another request.

On the HTTP shared connection, Host Swagger documentation disables/enables documentation (enabled by default). `/_mina` is reserved when enabled. API documentation access defaults to authenticated: the specification and collection use the connection's authentication, while the HTML/assets are available to display the access panel. Public documentation access must be selected explicitly and does not bypass API authentication. Public API base URL optionally supplies an externally reachable origin and gateway prefix for generated links and collections. Configure your gateway to forward the documentation paths as well as the API routes.

Basic/LDAP credentials, access tokens and HMAC secrets entered in the API Explorer stay in that tab and are not saved in generated collections. Postman collections use empty credential variables. HMAC signing is supplied for raw JSON/text requests; multipart/file signing requires your own signing client. Browser HMAC signing needs HTTPS or localhost. Configure mutual-TLS client certificates separately in your browser/Postman, and trust the hosting server's TLS certificate. Browser security and CORS policies still apply. Outbound HTTP Request and REST Invoke activities call providers; they do not themselves host Swagger routes.

Swagger UI assets and their Apache-2.0 license/notices are included in both Direct and Engine Python archives. To update bundled assets after upgrading the pinned npm dependency, run `npm run swagger:assets` from `frontend` and rerun the API documentation/export tests.

### *Python deployment details*

Studio, Direct Python archives and Engine Python archives use the same HTTP transport, authentication and listener implementation. Generated dependency checks include HTTPX, Uvicorn and the security libraries required by the configured connections. Provision external certificate/key/store files on the execution host, and supply exported secrets through the deployment environment/property bindings. Export retains endpoint URLs and key-file locations while removing secret values.

Direct archives can run with `python run.py` after extraction or as a Python zip application. Engine archives run with `python -m application.main`. Existing Administrator deployment commands continue to apply. Optional modules include PyJWT/cryptography, ldap3, httpx-ntlm and h2. Test against your own directory, identity provider, certificate chain and proxy before deployment; local automated tests exercise protocol behavior and failures, not access to external enterprise services.

### *Reference documentation and scope*

Use the activity's Documentation tab for its own Configuration, Input, Output, and Errors reference. In Help -> MINA Documentation, search for HTTP and REST activity verification for the supported test matrix and remaining standards gaps.

### *OpenAPI security and multipart forms*

Select a server connection that can enforce the imported operation's security. MINA uses operation-level OpenAPI security, or inherits the top-level declaration if omitted. Configure OpenAPI API key secrets as a protected JSON property keyed by security scheme name. API keys can be read from headers, query parameters or cookies. MINA requires all keys named in a selected alternative, together with its configured Basic/LDAP, Bearer/JWT/OAuth2 or mutual TLS requirements where applicable. The listener refuses startup if it cannot enforce any alternative. A shared connection supports one HTTP Authorization scheme at a time.

For a contract with `multipart/form-data`, map RequestBody by form property name. Arrays generate repeated parts, nested objects are JSON, and `format: binary` fields take base64. OpenAPI `encoding.contentType` controls the part type. Map multipartPartHeaders as an object keyed by form property when the contract declares extra headers. Declared defaults fill omitted outbound headers. MINA validates required headers and their schemas on both sides. Reserved framing and content headers remain controlled by the serializer. The receiving workflow gets typed `body` and `mimeEnvelopeElement`. Contract-driven forms are buffered and bounded.

Outbound query parameters with OpenAPI `allowReserved: true` preserve URI-safe reserved characters and valid percent escapes. Delimiters that could create another query parameter or a URL fragment remain escaped.

### *Shared HTTP policies*

- Set Listener error response format to problem for RFC 9457 `application/problem+json` errors produced by the listener. Existing connections default to the original JSON error shape. Workflow responses still use the body and content type mapped in HTTP Send Response.

- Set Trusted proxy IPs / CIDRs only for proxies you control that maintain the `X-Forwarded-For` chain. MINA checks the immediate peer and walks the chain from right to left. The effective address appears in `clientIp` and applies to IP allowlists and rate limits. The original peer remains in `remote`.

- Enable Propagate HTTP trace context on the inbound and outbound connections to link version 00 `traceparent` headers across a workflow. The listener exposes `traceId`. This propagates identifiers and does not export trace spans.

- Enable Conditional GET and HEAD and map `ETag` or `Last-Modified` response headers to return 304 when the request validators match. The workflow still runs. The option does not cache responses or enforce write preconditions.

- HTTP Write Preconditions checks inbound headers against mapped datastore validators and returns `allowed` and `statusCode` (200, 412, or 428). Branch before any mutation. Use a version predicate or transaction lock in the actual write to prevent a concurrent change between the check and mutation.

- Shared public response cache reuses fresh public HTTP 200 GET responses in the runtime process. It partitions entries by destination and request headers, honors explicit freshness and skips authenticated, conditional, range and file requests. A successful mutation through a cache-enabled connection invalidates that origin's entries. Replicas each have their own cache.

- For OAuth client assertions, select private_key_jwt and set the registered PEM key path, asymmetric algorithm, and optional key ID. The runtime signs a short-lived assertion for the configured HTTPS token or introspection URL. Provide the signing key separately on the runtime host.

### *HTTP/3, event streams, and callbacks*

Select protocol 3 for native HTTP/3. Hosting binds UDP and TCP on the configured port and needs an HTTPS PEM certificate plus unencrypted PEM private key. TCP also serves HTTP/2 and HTTP/1.1. Native outbound HTTP/3 requires direct, verified HTTPS and supports buffered requests and replies. Proxies, file streaming, mutual TLS, custom CRLs and custom cipher suites are outside the HTTP/3 implementation.

Use Read Server-Sent Events to read a bounded batch with an optional Last-Event-ID. Use WebSocket Exchange with a `ws://` or `wss://` URL to send an optional message and collect bounded replies. Binary replies are base64. These activities close their connection when finished. They do not publish SSE streams, host WebSocket endpoints, or reconnect automatically.

API Blueprint imports OpenAPI webhooks as selectable listeners under `/_webhooks/<name>`. Configure authentication on the server connection before hosting them. Callbacks on selected operations generate reusable Sub Tasks. Call one explicitly with `callbackUrl` and `body` and set the `connections.http.callbackAllowedOrigins` property to the approved HTTPS origins. Use the contract expression shown in the task description to map the URL. MINA validates the callback

contract, verifies TLS, and disables redirects. The generated task performs one workflow-managed delivery rather than creating an automatic subscription or durable delivery queue.

### *Deployment qualification*

The activity-level suite `tests.test_http_activity_e2e` exercises HTTP Listener and REST Receiver through both HTTP Request and REST Invoke, returning through HTTP Response on real local listeners. It also covers Studio contract preview, HMAC encoded request targets, HTTPS preview enforcement, multiple content types, strict JSON, bodyless responses and fresh Direct/Engine archives.

REST contracts retain all declared request/response content types and validate against the actual Content-Type, using exact media types before type wildcards and `*/*`. Swagger exposes all declared formats. Set HTTP Response's content type explicitly when returning a representation other than the configured default; the workflow chooses the response representation. Explicit request-schema overrides remain effective. Optional absent request bodies are accepted; required bodies cannot be bypassed with Send Body Mode = NEVER/AUTO.

In HTTP Listener and REST Receiver setup or Configuration, check one or more Request content types and Response content types. You can add custom types alongside the common ones. No selection keeps automatic behavior. With multiple response types, MINA uses the caller's `Accept` header and its quality values; the first checked format is the default. An explicitly mapped HTTP Send Response content type must be one of the offered types and acceptable to the client. For XML output, map an XML document or an object with one root element. With mixed media types, a standalone XSD applies to XML and a standalone JSON Schema applies to JSON; use an OpenAPI contract for separate schemas by media type. Swagger lists each selected format, and the Postman export includes a request for each selected request format.

URL-encoded form fields use declared primitive schema types before validation, including repeated primitive arrays. Direct archives also accept the default HTTP Response operation, matching runtime behavior.

When body format is omitted, outbound text/XML content is sent as text and URL-encoded form content is encoded as form data. Explicit body format settings retain precedence. Text responses such as `42` remain strings. Malformed nonempty JSON and NaN/Infinity are rejected; binary response mode retains raw content. HEAD, 204, 205 and 304 responses skip body schema validation because they have no response content. TRACE is supported as an optional API operation; CONNECT proxy tunneling is not offered as an API method.

Studio preview resolves methods and paths from the selected API contract, including HEAD, OPTIONS and 405 behavior. Signed previews use the actual encoded public request target. A connection requiring HTTPS must use an HTTPS endpoint; plain HTTP preview cannot bypass confidentiality.

An explicit HTTPS server scheme always enables TLS, even if a legacy TLS checkbox value is false. A missing server certificate/key then fails startup instead of silently opening a plaintext listener.

Run `python -m unittest tests.test_http_capabilities tests.test_http_request_tree tests.test_http_production` from `backend` with its virtual environment active. The suite covers local TLS/mTLS, HTTP/2 negotiation, HTTP contracts, binary/repeating values, streaming/file cleanup, policy behavior and Python exports. Redis protocol calls are tested with an injected store; provision and exercise the actual Redis service, issuer/LDAP, certificates, proxy and upstream API in staging.

Export Direct or Engine Python and install the archive's generated dependencies. HTTP contract support requires `jsonschema`, `PyYAML` and `regex`; HTTP/2 hosting adds `hypercorn`/`h2`; distributed policies add `redis`. Keep certificate files and secrets external. Supply the Redis URL through the generated secret binding, never an unprotected source literal.

Use the Direct archive's `--qualify-task <task-id>` runner against the staging API with representative payloads, concurrency, latency/error thresholds and memory limits. Test the Engine archive with the same fixtures and deployment load harness. Repeat under upstream outages, token/key rotation, Redis failure and deployment restart. Record the report with the release. Automated regression success is evidence for the implemented behavior, not a guarantee about an untested production environment or third-party service.

Reference source: docs/HTTP_REST_CAPABILITIES.md

# HTTP and REST activity verification

Connectors

## Guide

### *HTTP and REST activity verification*

The verification covers HTTP Listener, REST Receiver, HTTP Request, REST Invoke and HTTP Response in Studio, native Python listeners, Debug, Swagger/Postman generation and both Python archive formats. Live checks use local providers and generated local certificates. External enterprise services still need validation in their deployment environment.

### *Activity combinations*

`backend/tests/test_http_activity_e2e.py` sends real requests through these four flows. Each receives a JSON request, calls a separate local provider with a mapped body/custom header, and returns the provider response through HTTP Response.

| Inbound activity | Outbound activity | Response activity |
|---|---|---|
| HTTP Listener | HTTP Request | HTTP Response |
| HTTP Listener | REST Invoke | HTTP Response |
| REST Receiver | HTTP Request | HTTP Response |
| REST Receiver | REST Invoke | HTTP Response |

### *Confirmed gaps corrected*

| Gap | Resulting behavior |
|---|---|
| Only one OpenAPI request/response representation retained | Keep all declared representations; choose the schema from actual Content-Type, with exact/type-wildcard/global-wildcard precedence |
| Text/XML bodies implicitly JSON-quoted | Infer text, XML and URL-encoded form serialization when no explicit body format is supplied |
| Numeric-looking plain text parsed as JSON numbers | Preserve text responses as strings |
| HEAD/204/304 rejected by payload schema validation | Skip response-body validation when HTTP forbids response content |
| Optional missing body rejected by an object schema | Validate supplied bodies; enforce presence independently for required bodies |
| URL-encoded numeric/boolean/array fields validated as raw strings | Decode primitive form fields using their declared schema types before validation |
| Required mapped body could be suppressed after validation | Apply Send Body Mode before validating required presence |
| Malformed JSON silently returned as text | Reject malformed nonempty JSON and non-finite JSON numbers; explicit binary mode keeps raw content |
| Non-finite OAuth introspection expiry accepted | Require a finite expiry later than the current time when exp is present |
| Preview ignored contract-derived methods | Resolve effective contract paths/methods and use the shared listener protocol handler |
| Preview did not mirror HEAD/OPTIONS/405 behavior | Use the same method dispatch as native listeners |
| HMAC preview rewrote the signed target | Verify the actual public encoded path/query sent by the client |

| Gap | Resulting behavior |
| --- | --- |
| HTTP preview could bypass an HTTPS connection setting | Reject plain HTTP preview for HTTPS-required connections |
| Explicit HTTPS scheme could be contradicted by a false TLS flag | HTTPS scheme always enables TLS; missing server identity fails startup instead of opening a plaintext listener |
| TRACE exposed inconsistently; CONNECT offered as an API method | Support TRACE through contracts, hosting, invocation and documentation; remove CONNECT tunneling from editor choices |
| Default HTTP Response accepted at runtime but rejected by Direct export | Allow the unambiguous default HTTP Response operation in Direct archives |

Swagger documents every declared content type. The workflow still chooses its response representation: set HTTP Response Content-Type to the representation actually returned. This does not automatically implement Accept-header negotiation or business logic.

### *Capability coverage*

| Capability | Evidence |
| --- | --- |
| Connection/activity/Input-tree configuration resolution | Request-tree tests and the live activity matrix |
| HTTP/HTTPS; root-folder trust; hostname verification; mutual TLS | Local TLS tests and runnable sample checks |
| Basic, Bearer/JWT, OAuth2 and HMAC | Authentication tests; local Basic/JWT/HMAC and OAuth token/introspection sample services |
| Digest, non-preemptive Basic, NTLM/LDAP integration paths | Existing transport tests; LDAP/provider calls use injected test clients |
| Path/query/header serialization and repeated values | Contract, request-tree and production HTTP tests |
| JSON, XML/XSD, plain text, form, binary and MIME | Contract, protocol, streaming and activity-level tests |
| Status/header propagation, HEAD, OPTIONS, method rejection | Listener and preview tests |
| Timeouts, connection limits, queues and cancellation cleanup | Transport/server and production tests |
| File streaming, MIME limits and upload/download cleanup | Production streaming tests |
| Retry limits, idempotency controls, circuit breaking, CORS, IP rules and throttling | Policy/production tests; distributed store calls are injected |
| HTTP/2 over TLS | Live local ALPN/HTTP2 test |
| Swagger, OpenAPI and Postman export | API documentation tests and exported live API tests |
| Live HTTP Debug pause/continue | API documentation Debug test |
| Direct/Engine archive compatibility | Fresh archive subprocesses hosting actual API routes; text contract and offline docs checks |
| Studio Samples | Installed template checks and executable sample verifier |

### *Reproduce the checks*

From `backend`, with its Python environment active:

```
python -m unittest tests.test_http_activity_e2e tests.test_api_docs tests.test_api_blueprint
tests.test_http_capabilities tests.test_http_request_tree tests.test_http_production
tests.test_raw_python tests.test_engine_export tests.test_activity_packs tests.test_debug_testing
tests.test_debug_input_capture -q
```

From the repository root, after sample certificate preparation:

```
backend/.venv/Scripts/python.exe samples/http-rest-examples/verify_examples.py
```

On Linux use the corresponding environment's Python executable. The sample verifier opens loopback ports 9001-9003 and 9443-9449; keep these free. Fixtures are shut down afterward. From `frontend`, run the Node test suite and `npm run build` to verify editor contracts and the distributable Studio assets.

### *Deployment boundaries*

The checks do not certify a particular external OAuth issuer, LDAP directory, proxy, Redis service, certificate/key rotation process or production load profile. Test those with actual staging infrastructure and representative payloads, concurrency, expiry and outage cases.

This is a request/response API implementation. The table below identifies the remaining protocol and contract boundaries. Certificate and authentication settings must be available on the execution host when deploying an exported application.

### *Standards coverage and deployment limits*

The activities cover ordinary HTTP/REST request and response integrations. This table separates protocol defects from capabilities needed by particular API contracts or deployment environments.

| Priority | Gap | Current behavior and impact |
|---|---|---|
| Security scope | Imported OpenAPI security | Hosting rejects unenforceable requirements. A supported alternative can combine multiple OpenAPI API keys with the connection's Authorization scheme and mutual TLS. The connection supports one Authorization scheme at a time. |
| Corrected | HTTP 205 Reset Content | Listeners suppress response content, and clients skip body processing and validation. Generated OpenAPI omits content for 205. |
| Supported for buffered forms | OpenAPI serialization and multipart | Outbound query values honor `allowReserved` for safe wire characters. Contract-driven forms apply and validate declared non-reserved part headers and content types. |
| Implemented as a setting | Problem Details | Opt in to RFC 9457 responses for listener-generated errors. Existing connections retain their legacy JSON format. |
| Implemented as settings | Trace context and proxy trust | Version 00 W3C trace context can follow inbound and outbound activities. Explicit proxy CIDRs enable trusted X-Forwarded-For client attribution. Span export and other forwarded-header formats are outside this feature. |
| Implemented as a setting | OAuth client authentication | The runtime can sign a private-key JWT assertion for the configured HTTPS endpoint. Real provider registration, credential rotation and interoperation remain staging checks. |
| Opt-in | Conditional requests and caching | Conditional GET/HEAD uses mapped validators. A dedicated activity evaluates write preconditions before a datastore write. The bounded process cache serves fresh public GETs and does not perform stale serving or distributed invalidation. |
| Supported with limits | Callbacks and webhooks | Named webhooks become hosted endpoints. Callback Sub Tasks require explicit workflow calls and an HTTPS allowlist. Durable subscriptions and automatic callback delivery remain outside the scaffold. |
| Bounded clients | SSE and WebSocket | Activities read SSE events or exchange WebSocket messages and close after their configured limits. Inbound SSE publishing and WebSocket hosting are outside these activities. |

| Priority | Gap | Current behavior and impact |
| --- | --- | --- |
| Native transport | HTTP/3 | QUIC hosting and direct buffered clients use TLS 1.3. Mutual TLS, custom CRLs or cipher suites, proxies, and outbound file or event streaming are outside HTTP/3 support. |

### *Standards references*

- [OpenAPI media type matching](https://spec.openapis.org/oas/v3.1.2.html#request-body-object)

- [OpenAPI parameter serialization and security requirements](https://spec.openapis.org/oas/v3.1.2.html)

- [HTTP 205 Reset Content and conditional requests](https://www.rfc-editor.org/rfc/rfc9110.html#section-15.3.6)

- [HTTP caching](https://www.rfc-editor.org/rfc/rfc9111.html)

- [HTTP/3](https://www.rfc-editor.org/rfc/rfc9114.html)

- [Problem Details for HTTP APIs](https://www.rfc-editor.org/rfc/rfc9457.html)

- [OAuth JWT client authentication](https://www.rfc-editor.org/rfc/rfc7523.html#section-2.2)

- [W3C Trace Context](https://www.w3.org/TR/trace-context/)

Reference source: docs/HTTP_REST_ACTIVITY_VERIFICATION.md

# MINA security review - 2026-09-22

Security

## Guide

### *MINA security review - 2026-09-22*

#### *Outcome and scope*

Production blockers found, particularly for shared/multi-team deployments. Reviewed source at commit `676c0bf` with the existing working-tree changes present. This is a security-focused review across Studio/runtime, debugger, Control Plane, local/Kubernetes agent, Electron IPC, package handling, connector execution, secret storage, and selected dependencies. It is not an exhaustive certification that all code or dependencies are safe. No application security fixes were applied in this review. Only this report was added.

Checks used static source inspection, mocked in-process authorization checks, synthetic credentials, installed-package metadata, upstream advisories, and `npm audit --json --ignore-scripts`. No real credentials were printed, no live deployments were modified, and no destructive or resource-exhaustion tests ran.

Severity below assumes the affected service is reachable or the stated attacker role exists. Firewall, reverse proxy, OS permissions, and deployment topology can reduce exposure, but do not fix the underlying application boundary.

#### *Findings*

#### *F1 - Critical: unauthenticated Control Plane owner access by default*

Evidence: `administrator/app/main.py:66,141-157` uses an empty API key by default and returns the Technology Team Owner identity when no credential is provided. `administrator/run_admin.py:6` binds to `0.0.0.0:9080` by default.

Impact: Anyone able to reach an unconfigured instance can obtain governance and deployment privileges, including access to agent deployment responses that contain decrypted deployment secrets. The risk is not restricted to localhost.

Validation: With `API_KEY` mocked to empty, an anonymous synthetic request resolved to `roles=['Owner']`. No live server configuration was changed.

Fix: Fail closed when credentials are missing. Restrict initial bootstrap to loopback plus a one-time credential; require authenticated setup before exposing network access. Keep management APIs behind TLS and an authenticated boundary.

#### *F2 - Critical: deployed application code inherits management credentials and host privileges*

Evidence: `administrator/app/main.py:1238-1243` copies the full administrator environment into application subprocesses. `administrator/agent/main.py:173-184` does the same for agent deployments. Agent configuration reads `MINA_AGENT_KEY` from that environment (`administrator/agent/main.py:39`). Raw Python deployments are started under the same OS identity, not a separate tenant security boundary.

Impact: A malicious or compromised deployed application can read management credentials such as the admin API key, secret-encryption key, or agent key when configured via environment. Agent endpoints currently require a Technology Team credential, not a narrowly scoped per-plane identity. Consequently an inherited credential can expose other teams' deployments and secrets. The same OS identity also permits access to other application files accessible to the service account; clearing environment variables alone is insufficient isolation.

Validation: Source-confirmed inheritance and privileged agent authorization. No real credential extraction or malicious deployment was attempted.

Fix: Explicit child-process environment allowlist, per-plane scoped credentials, and isolated low-privilege application identities/containers with separate filesystem access. Never expose management or encryption keys to application code. Treat packages, scripts, and vendor drivers as executable code requiring approval.

#### *F3 - High: deployment move bypasses team namespace authorization*

Evidence: `administrator/app/main.py:1735-1748` checks deployment ownership, the target plane's registered namespaces, and the capability, but omits `team_can_use_namespace`. Initial deployment creation checks this permission at `administrator/app/main.py:1160`.

Impact: An Application Manager can move its stopped deployment to a registered namespace not assigned to its team, bypassing the intended deployment-placement boundary. Availability of valid plane/capability IDs is required.

Validation: A mocked Team A request successfully moved its own deployment to `team-b`; a mocked `team_can_use_namespace` returning false was never called. All persistence/audit functions were mocked; no real deployment moved.

Fix: Apply the same team/namespace/capability authorization to move, redeploy, restore/revert, and start. Recheck immediately before execution as permissions may change after deployment creation. Add negative cross-team route tests.

### *F4 - High: Studio management/runtime APIs have no authentication boundary*

Evidence: `backend/app/main.py:575-617` exposes project listing, creation, editing, and execution without authentication dependencies. The middleware near line 80 controls caching, not access. Workflows can execute Python and commands (`backend/app/runtime.py:2030,2089`).

Impact: A caller who can access the runtime HTTP endpoint can read projects and configure/execute operations with the runtime account's permissions. The packaged sidecar binds to loopback (`backend/run_sidecar.py`), which limits remote exposure but does not authenticate other local users/processes. Exposing this same API for browser access without an authenticated proxy broadens the risk. Cross-origin browser exploitation was not established in this review.

Validation: Unauthenticated in-process requests to project listing and a synthetic memory-only connection test both returned HTTP 200. No shell commands or real connector operations were invoked through these endpoints.

Fix: Per-launch authenticated desktop session credentials, Origin/Host checks, and explicit authenticated user access/RBAC for browser deployments. Keep untrusted workflow code out of the privileged Studio/management process.

### *F5 - Medium: project exports and workspace files contain plaintext secrets*

Evidence: `backend/app/store.py:46-58` serializes resource configurations and property values without encryption. `backend/app/main.py:689-700` writes full project/resource/property models into the project export without redaction.

Impact: Shared `.mpackage` exports and workspace backups can disclose database, EMS, cloud, and other connector credentials. This concerns project exports; it does not imply every deployment archive's secret handling is identical.

Validation: A synthetic password in a mocked project resource was present in the exported `project.json`. No real project secret was read or displayed.

Fix: Redact secret values by default on project export, preserve secret references, and make any sensitive export an explicit protected operation. Use OS-backed or managed secret storage and restrict workspace ACLs.

### *F6 - High: Studio project import has unbounded memory/decompression exposure*

Evidence: `backend/app/main.py:1475-1494` calls `await file.read()` with no application limit and decompresses manifest/project ZIP entries without checking uncompressed size or compression ratios. By contrast, Control Plane and agent archive handlers implement explicit member and expanded-size limits.

Impact: A reachable import endpoint can be used to consume large amounts of memory/CPU using oversized or highly compressed uploads, disrupting Studio and its runtime. A reverse-proxy compressed-body limit alone is insufficient.

Validation: Source-confirmed. No ZIP bomb or large-memory attack was executed.

Fix: Limit upload bytes, entry count, expanded size and per-entry size before materialization; use bounded streaming reads, concurrency limits, and quotas.

### *F7 - Medium: installed Python web dependencies have known advisories*

Evidence: The inspected backend environment has FastAPI `0.115.12`, Starlette `0.46.2`, and python-multipart `0.0.20`. Both backend and administrator requirements pin FastAPI `0.115.12` and python-multipart `0.0.20`.

- Starlette's multipart rollover can block the event loop: CVE-2025-54121,

fixed in `0.47.2`. The installed `0.46.2` is affected. [Maintainer advisory](https://github.com/Kludex/starlette/security/advisories/GHSA-2c2j-9gv5-cj73).

- python-multipart has Content-Disposition parameter interpretation issues

covered by CVE-2026-53537, fixed in `0.0.30`. [Maintainer advisory](https://github.com/Kludex/python-multipart/security/advisories/GHSA-vffw-93wf-4j4q).

- The separate python-multipart arbitrary-file-write advisory CVE-2026-24486

requires non-default `UPLOAD_DIR`/`UPLOAD_KEEP_FILENAME` settings. Those settings were not found in the reviewed application, so this is not being claimed as a demonstrated arbitrary-file-write exploit here. [Advisory](https://github.com/advisories/GHSA-wp53-j4wj-2cfg).

Fix: Upgrade to a mutually compatible, patched FastAPI/Starlette/multipart set and regression-test uploads, forms, and packaged builds. Scan resolved Python dependencies and shipped artifacts in CI rather than only requirements text.

### *Additional hardening observations*

- Electron uses context isolation, disabled Node integration, and sandboxing,

which are positive. However, file-write IPC accepts caller-provided paths (`frontend/electron/main.cjs:158-170`) without sender-frame authorization, and navigation/window-opening restrictions were not found. A compromised renderer could abuse the bridge. A renderer compromise was not demonstrated here.

- Control Plane UI stores its credential in localStorage. Reduce exposure with

scoped/session credentials and a suitable CSP; do not assume HTML escaping alone provides a complete browser security boundary.

- TLS enforcement, firewall rules, Windows ACLs, Unix permissions, Kubernetes

RBAC/network policies, and supply-chain/signing controls need deployment-level verification. They cannot be certified from this source checkout.

- Existing archive path/member checks, Control Plane encrypted secret storage,

hashed team tokens, ownership checks on many routes, and default SSH host-key rejection are useful controls. Their presence does not mitigate the specific bypasses above.

### *Dependency-check results and limitations*

`npm audit --json --ignore-scripts` completed successfully and reported zero known advisories across 478 dependency entries. This is not a claim that the frontend or Electron bridge is vulnerability-free.

`pip-audit` is not installed in the backend environment. No package installation or application dependency mutation was performed. Python checks were limited to installed metadata and selected verified upstream advisories, not a full resolved dependency audit. Vendor SAP/EMS/JDBC JARs, native libraries, bundled Java/Python, and the actual Windows installer were not fully audited.

### *Recommended order*

- Before shared deployment: close anonymous Control Plane access and isolate

application processes from management credentials and filesystem privileges.

- Fix namespace authorization on move and audit every lifecycle route.

- Authenticate Studio APIs and restrict network exposure.

- Patch the web dependency stack and bound import/upload processing.

- Redact/export secrets safely, harden Electron IPC/navigation, and add security

regression tests plus complete Python/JAR/native artifact scans.

No live-server compromise was demonstrated or required. The findings are enough to block a claim that the current product is safe for untrusted multi-team hosting.

Reference source: docs/SECURITY_REVIEW_2026-09-22.md

# Web dependency security update

Security

## Guide

### *Web dependency security update*

The Studio backend, browser runtime, and Administrator now pin the same reviewed web stack: FastAPI 0.141.1, Starlette 1.6.0, and python-multipart 0.0.32. This replaces FastAPI 0.115.12 and python-multipart 0.0.20 and explicitly pins Starlette instead of relying on an indirect dependency.

Upstream references:

- [Multipart upload rollover denial of service](https://github.com/Kludex/starlette/security/advisories/GHSA-2c2j-9gv5-cj73)

- [URL-encoded form limits](https://github.com/Kludex/starlette/security/advisories/GHSA-82w8-qh3p-5jfq)

- [Windows StaticFiles UNC handling](https://github.com/Kludex/starlette/security/advisories/GHSA-wqp7-x3pw-xc5r)

- [Multipart disposition parsing](https://github.com/Kludex/python-multipart/security/advisories/GHSA-vffw-93wf-4j4q)

### *Applying the update*

For source installations, stop the service, install its requirements with its own Python environment, then restart it. From the repository root on Windows:

```
& .\backend\.venv\Scripts\python.exe -m pip install -r backend/requirements.txt
& .\backend\.venv\Scripts\python.exe scripts/verify-web-dependencies.py
& .\administrator\.venv\Scripts\python.exe -m pip install -r administrator/requirements.txt
& .\administrator\.venv\Scripts\python.exe scripts/verify-web-dependencies.py
```

For a browser-only installation use `backend/requirements-browser.txt` instead. For packaged installations, rebuild and redistribute the desktop/Administrator installer. Already installed executables retain their old bundled dependencies. Existing deployment environments must update their own Python dependencies too. Do not disable certificate verification to install dependencies.

Windows packaging scripts now verify the installed web versions before bundling; dependency-install failures stop packaging rather than reusing stale packages.

### *Verification and limits*

Security regressions cover multipart upload functionality, offloaded disk rollover, URL-encoded field count/size enforcement, consistent dependency pins, and stale build rejection. Backend runtime and Administrator tests pass with the new stack. This is not a full security certification or remediation of other audit findings. Request/body size limits, authentication, and deployment hardening remain necessary.

Local environment caveats: the Administrator virtual environment points to a missing Python 3.11 installation, so Administrator tests were run with the updated backend environment. Repair/recreate that environment before building it. The backend Python 3.14 environment also reports platform compatibility issues for httptools, PyYAML, watchfiles, and websockets during `pip check`; these packages were not changed by this update. A clean supported build environment and installer smoke test are still required before release.

Reference source: docs/WEB_DEPENDENCY_SECURITY.md